
ANTSDR 开发手册

GNU Radio 篇

Rev. 1.0



1. 版本记录

版本	时间	描述
Rev. 1.0	2021.6.23	第一版发布

2. 版权声明:

XME/IME/SDR 系列核心模块、评估板及其相关知识产权由微相科技有限公司所有。

本文档由微相科技有限公司版权所有，并保留一切权利。在未经微相公司书面许可的情况下，不得以任何方式或形式来修改、分发或复制本文档的任何部分。

3. 免责声明:

产品中所提供的程序源代码、软件、资料文档等，微相科技不提供任何类型的担保；不论是明确的，还是隐含的，包括但不限于合适特定用途的保证，全部的风险，由使用者来承担。

前 言

软件无线电 SDR 的定义是大多数的物理层功能都使用数字信号处理算法实现，理想的 SDR 接收机的组成部分是很少量的硬件前端、天线和高速 GHz 的采样器，采样器能够捕获大范围的无线信号并转换为数字信号。任何的解调、同步、解码、解密都是用软件来实现，这个设备的处理速度必须非常快。

目前大多数智能手机及类似的设备都拥有多达 8 种无线模块，用于处理不同频段的信号，比如 WiFi(2.4GHz)、LTE(800MHz)、GSM(900MHz)、UMTS(2.1GHz)、GPS(1.5GHz)、蓝牙(2.4GHz)、NFC(13.56MHz)和 FM(100MHz)，还有包括 IoT 和电视频段 UHF(400MHz)。其中的一种解决方案是用一个单一的 SDR 以 GHz 的带宽来采样，并且把这些信号进行数字化，把从基带到 3GHz 的所有信号都直接用软件来解调。

软件无线电的主要特点如下：

具有很强的灵活性：软件无线电可以通过增加软件模块，很容易增加新的功能。可以与其他任何电台通信，并且可以作为其他电台的射频中继。为了减少开支，可以根据所需功能的强弱，取舍选用合适的软件模块。

具有较强的开放性：软件无线电由于采用了标准化模块化的结构，其硬件可以随着器件和技术的发展而更新或扩展，软件也可以随着需要而不断升级。

由于软件无线电这一概念的提出，就受到了全世界无线电领域的广泛关注。由于软件无线电所具有的灵活性开放性等特点，使其不仅在军、民无线通信中获得了使用，在其他领域，比如电子战、雷达、智能家电等领域得到推广。

目 录

1.	版本记录.....	2
2.	版权声明:	3
3.	免责声明:	3
前	言.....	4
第 1 章	GNU Radio 以及 Pluto SDR 驱动安装.....	9
1.1	GNU Radio 简介.....	9
1.2	GNU Radio 安装.....	9
1.2.1	添加源, 更新和安装 GNU Radio.....	10
1.2.2	安装 Pluto SDR 驱动.....	10
第 2 章	GNU Radio 的基本使用.....	13
2.1	“Hello World!” 产生一个单音信号.....	13
2.2	控制单音信号	23
2.3	使用 ANTSDR 收发信号	29
2.4	小结	36
第 3 章	滤波器使用.....	37
3.1	低通滤波器设计	38
3.1.1	滤波器设计基础	38
3.1.2	低通滤波器实现	40
3.2	高通滤波器	44
3.3	根升余弦滤波器	47
3.4	总结	51
第 4 章	GNU Radio 中的数据流标签和控制消息.....	53
4.1	数据流标签 Stream Tags.....	53
4.1.1	在 GNU Radio 当中给数据打上标签.....	53
4.2	消息传递	58
4.2.1	GNU Radio 中消息传递.....	59

4.3 基于消息和流标签构建数据帧	62
4.3.1 使用 GNU Radio 构建数据帧.....	62
4.4 总结	72
第 5 章 AM-DSB-TC 的调制与解调	73
5.1 常规调幅原理	73
5.1.1 单音基带信号调制	73
5.1.2 基带信号的调制	75
5.2 使用 GNU Radio 仿真常规调幅.....	76
5.3 使用常规调幅发送音频信号	78
5.4 常规调幅的非相干解调原理	88
5.4.1 一般情况下的包络检波	88
5.4.2 实数解调的优化	89
5.4.3 复包络检波	90
5.5 常规调幅的非相干解调仿真	91
5.6 解调常规调幅音频信号	93
5.7 总结	96
第 6 章 FM 的调制与解调	97
6.1 FM 调制的数学原理	97
6.1.1 FM 信号产生的过程	97
6.1.2 FM 调制一个单音信号	98
6.1.3 FM 调制一个基带信号	99
6.1.4 宽带调频	99
6.2 使用 GNU Radio 仿真 WFM.....	102
6.3 使用 ANTSDR 发送 FM 信号	105
6.4 复数延迟解调	107
6.5 复数延迟解调仿真	109
6.6 使用 ANTSDR 解调 FM 信号	112

6.7 总结	114
第 7 章 ASK 的调制与解调	115
7.1 ASK 调制	115
7.2 在 GNU Radio 当中生成 OOK 信号	116
7.3 ASK 信号的解调	118
7.4 使用 ASK 发送文本数据	121
7.5 使用 ANTSDR 基于 ASK 发送文字和图片	126
7.6 总结	129
第 8 章 BPSK 的调制与解调	130
8.1 相移键控	130
8.2 在 GNU Rado 中生成 BPSK 信号	132
8.3 BPSK 信号解调	137
8.4 使用 BPSK 发送文本数据	144
8.5 使用 ANTSDR 基于 BPSK 发送文字	148
8.6 总结	151
第 9 章 QPSK 的调制与解调	152
9.1 QPSK 调制的星座图	152
9.2 GNU Radio 当中生成 QPSK 信号	153
9.3 使用 QPSK 发送文本数据	155
9.4 使用 ANTSDR 收发数据	157
9.5 总结	159
第 10 章 FSK 调制与解调	161
10.1 频移键控	161
10.2 GNU Radio 当中生成 FSK 信号	162
10.3 FSK 信号的解调	164
10.4 使用 ANTSDR 收发 FSK 信号	168
10.5 总结	172

第 11 章	使用 GNU Radio 发送视频.....	173
11.1	DVB 系统.....	173
11.2	获取 GNU Radio 源码.....	173
11.3	DVB-S 视频发送.....	177
11.4	DVB-S 视频解调.....	181
11.5	总结	187
第 12 章	使用 GNU Radio 发送 WebCam 视频.....	188
12.1	获取 WebCam 数据.....	188
12.2	创建 GRC 流图.....	189
12.3	DVB-S 数据解调.....	192
12.4	总结	194
总	结.....	195
参	考.....	196
附	录.....	197
1.	VirtualBox 安装.....	197
2.	Ubuntu 虚拟机安装	201

第1章 GNU Radio 以及 Pluto SDR 驱动安装

接下来将正式开始介绍 GNU Radio 这款软件，GNU Radio 是一个开源的软件，其官方的地址如下：<https://www.gnuradio.org/>。

笔者在这里将会在 Ubuntu 实体机上安装 GNU Radio，在实体机上的 GNU Radio 表现正常，在虚拟机下使用 GNU Radio 在和外部 SDR 设备进行数据传输的时候可能会出现问題，在这里推荐使用实体机安装 GNU Radio。

在虚拟机下不使用到外部硬件的时候，GNU Radio 都是可以正常地工作了，若用户想要在 GNU Radio 上进行算法地开发和验证，可以在虚拟机上安装 GNU Radio，安装虚拟机可以参考附录的内容。

用户也可以查看对应的 Markdown 文档进行软件的安装 GNU Radio 和 Pluto SDR 的驱动。

1.1 GNU Radio 简介

GNURadio 是一个免费的软件开发工具包，提供信号处理模块用来实现软件定义的无线电和信号处理系统。它可以与外部 RF 硬件一起使用，以创建软件定义的无线电，或者在类似仿真的环境中无需硬件。它广泛用于业余爱好者，学术和商业环境，以支持无线通信研究和现实世界的无线电系统。

GNURadio 软件提供了构建和运行软件无线电或仅用于通用信号处理应用程序的框架和工具。GNURadio 应用程序本身通常称为“流程图”，它是连接在一起的一系列信号处理块，从而描述了数据流。这些流程图可以用 C++或 Python 编程语言编写。GNURadio 基础结构完全用 C++编写，而许多用户工具都是用 Python 编写的。

与所有软件定义的无线电系统一样，可重配置性是一个关键特性。不为特定硬件而设计，GNU Radio 开放的接口能够轻松地将用户自己地无线电设备接入到 GNU Radio 的生态当中。

1.2 GNU Radio 安装

- 环境及软件版本
 - Linux 系统版本： Ubuntu18.04 LTS
 - GNU Radio 版本： GNU Radio 3.8.2.0

GNU Radio 官方的安装教程链接地址如下：

<https://wiki.gnuradio.org/index.php/InstallingGR>

在官方的 wiki 上，详细地说明了不同 Linux 发行版，不同 GNU Radio 版本的安装说明，用户可以自行参考，在这以笔者自身的硬件平台为参考。

1.2.1 添加源，更新和安装 GNU Radio

在终端下依次输入如下命令：

```
sudo add-apt-repository ppa:gnuradio/gnuradio-releases-3.8
sudo apt update
sudo apt install gnuradio
```

这样就能安装号 GNU Radio，安装完成之后在终端中输入以下命令能够查看 GNU Radio 版本。

```
gnuradio-companion --help
```

```
wcc@wcc-dev:~$ gnuradio-companion -h
usage: gnuradio-companion [-h] [--log {debug,info,warning,error,critical}]
                        [flow_graphs [flow_graphs ...]]

GNU Radio Companion 3.8.2.0 This program is part of GNU Radio GRC comes with
ABSOLUTELY NO WARRANTY. This is free software, and you are welcome to
redistribute it.
gnuradio/gnuradio-releases-3.7

positional arguments:
  flow_graphs

optional arguments:
  -h, --help            show this help message and exit
  --log {debug,info,warning,error,critical}
```

1.2.2 安装 Pluto SDR 驱动

安装 Pluto SDR 的驱动之前，先安装所需要的依赖，输入如下命令：

```
sudo apt install -y gnuradio-dev libxml2 libxml2-dev bison flex cmake git libaio-dev
libboost-all-dev swig
```

接下来编译和安装 **libiio**:

```
git clone https://github.com/analogdevicesinc/libiio.git
cd libiio
mkdir build && cd build
cmake -DPYTHON_BINDINGS=ON -DCMAKE_INSTALL_PREFIX:PATH=/usr ..
make
sudo make install
sudo ldconfig
cd ../..
```

接下来编译和安装 ad9361 的驱动, **libad9361-iio**。

```
git clone https://github.com/analogdevicesinc/libad9361-iio.git
cd libad9361-iio
mkdir build && cd build
cmake -DCMAKE_INSTALL_PREFIX:PATH=/usr ..
make
sudo make install
sudo ldconfig
cd ../..
```

最后编译和安装 GNU Radio 当中的 iio 模块, **gr-iio**。

```
git clone https://github.com/analogdevicesinc/gr-iio.git
cd gr-iio
git checkout upgrade-3.8
mkdir build && cd build
cmake -DCMAKE_INSTALL_PREFIX:PATH=/usr ..
make
sudo make install
```

```
cd ../../  
sudo ldconfig
```

全部安装完成之后，我们可以打开 GNU Radio，在终端中输入：

```
gnuradio-companion
```

打开的 GRC 的界面如图 1-1 所示：

图 1-1 GRC 界面

在右侧搜索栏中输入 iio，可以看到当前的 IIO 模块已经存在与 GNU Radio 当中。其中上方的是 ADI 官方的评估版对应接口模块，下面是官方的 Pluto SDR 对那个的接口模块。

在接下来的实验当中，我们也主要是使用 PlutoSDR 的接口模块来设计，当然，也可以使用上方的 FMComms 的接口来进行设计，在进行诸如 MIMO 类的算法的设计与验证时可以采样上方的模块。

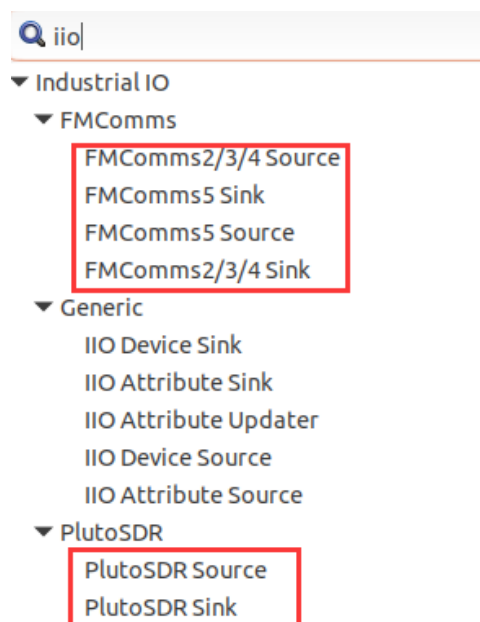


图 1-2 IIO 模块

第2章 GNU Radio 的基本使用

GNU Radio 作为一个开源的软件，向开发者提供了许多开源的模块，因此在使用 GNU Radio 的时候，我们能够像使用 Simulink 一样来搭建自己的仿真模型，来运行仿真，与此同时，还可以加上自己的软件无线电 SDR 设备，完成真实信道当中的信号的传输的设计与验证。

在本章的实验当中，将会简单介绍一下 GNU Radio 当中比较常用的模块。

2.1 “Hello World!” 产生一个单音信号

在数字信号处理中，单音信号最为最基础的一个信号，能够让我们快速地熟悉 GNU Radio 地使用，在一开始的实验当中，我们会生成一个单音信号，并且观察这个信号。

安装好 GNU Radio 之后，在终端中输入，就可以打开 GNU Radio 了。



图 2-1 打开 gnuradio

打开 GNU Radio 之后，可以看到其工作界面如图 2-2 所示，在区域 1 当中，可以将所需要的模块拖放到该区域当中，实现用户自己的功能和算法。在区域 2 中，是 GNU Radio 或者第三方提供的模块，GNU Radio 作为一个开源软件，第三方可以很方便的将自己的算法模块通过 GNU Radio 提供的 OOT 模块集成到 GNU Radio 中，前面我们安装的 gr-iio 模块，就是 ADI 公司提供的关于其软件无线电平

台的输入输出模块。

区域 3 是变量区，在 GRC 中设计的一些变量，可以在区域中进行展示和管理。区域 4 是消息打印窗口，能够打印 GNU Radio 在运行过程当中的一些信息。

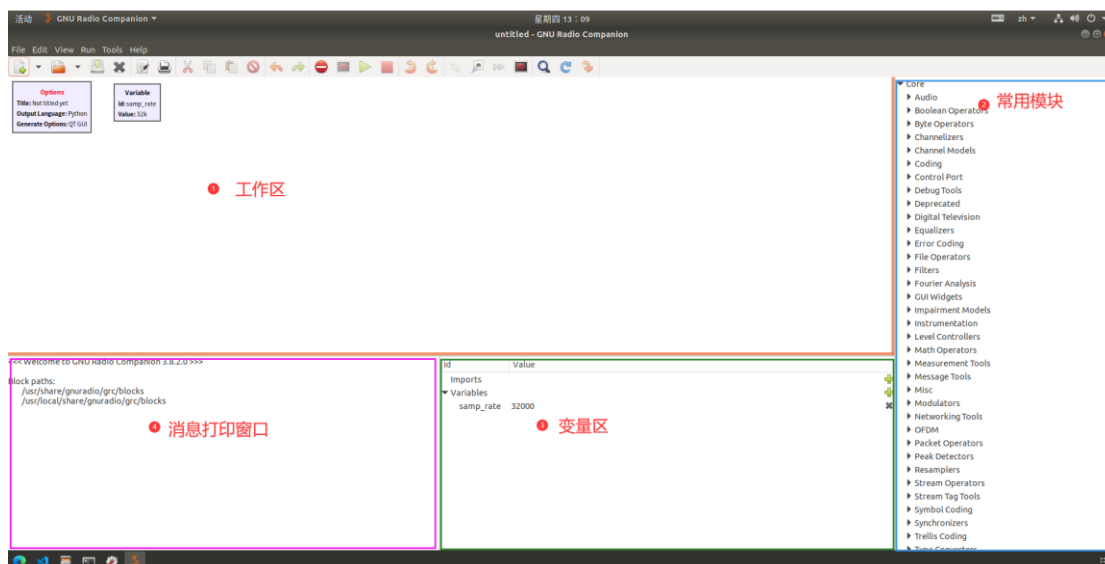


图 2-2 GNU Radio 界面

接下来，我们就可以来实现第一个例子了，首先我们需要看到在新创建一个 GRC 的时候，就会默认为我们创建一个 option 和 sample rate 的两个模块，其中 option 这个模块需要设置一个 ID 用来标识我们创建的工程，然后还需要选择显示的界面是基于 QT 还是其他的选项。

在这里将 ID 设置为 lab1_tone, title 设置为 lab1_tone, author 设置为用户自己的名称这里设置为 wcc, 其余保持默认即可，如图 2-3 所示。

设置完成之后，点击 apply，就完成了模块的设置，然后我们就可以进行下一步的操作了。

然后我们可以观察新建工程时，默认提供的模块，该模块会创建一个变量，其设置如图 2-4 所示，其中 id 也就是这个变量的名称，这个默认时 samp_rate，其变量的值是 32000。这样当我们在其他模块中需要使用到 samp_rate 这个变量的时候，只需要将 samp_rate 这个变量名称填入即可。

在 GNU Radio 的模块当中，这些需要填入值的地方，满足 python 语法的表达式都可以填入。在这里我们先保持默认。

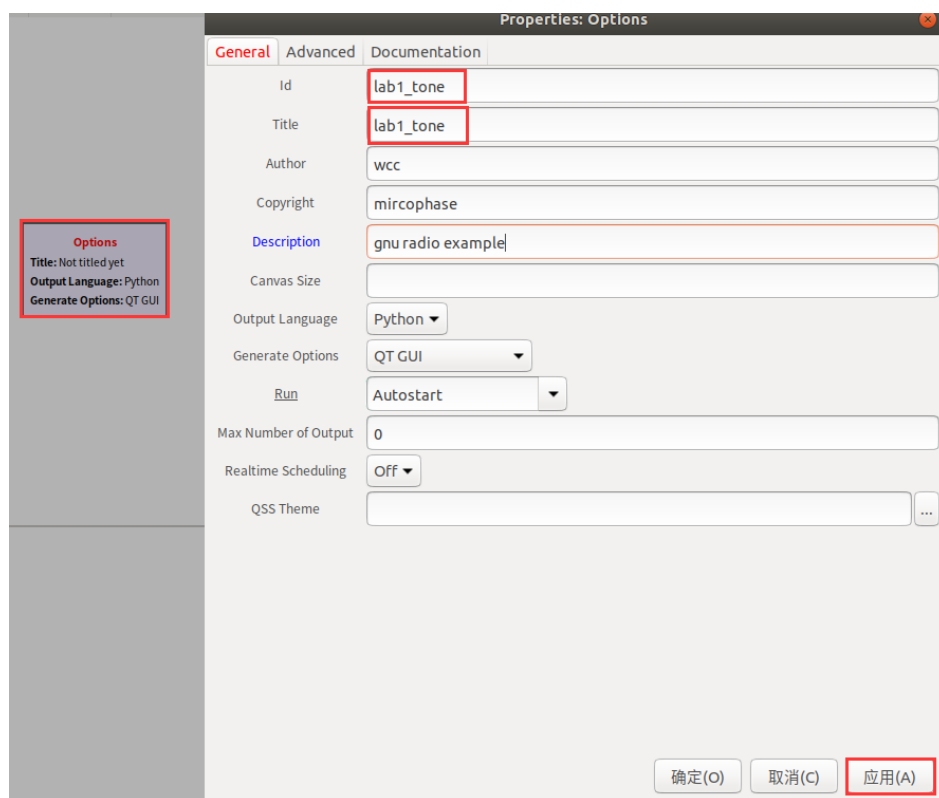


图 2-3 option 模块参数

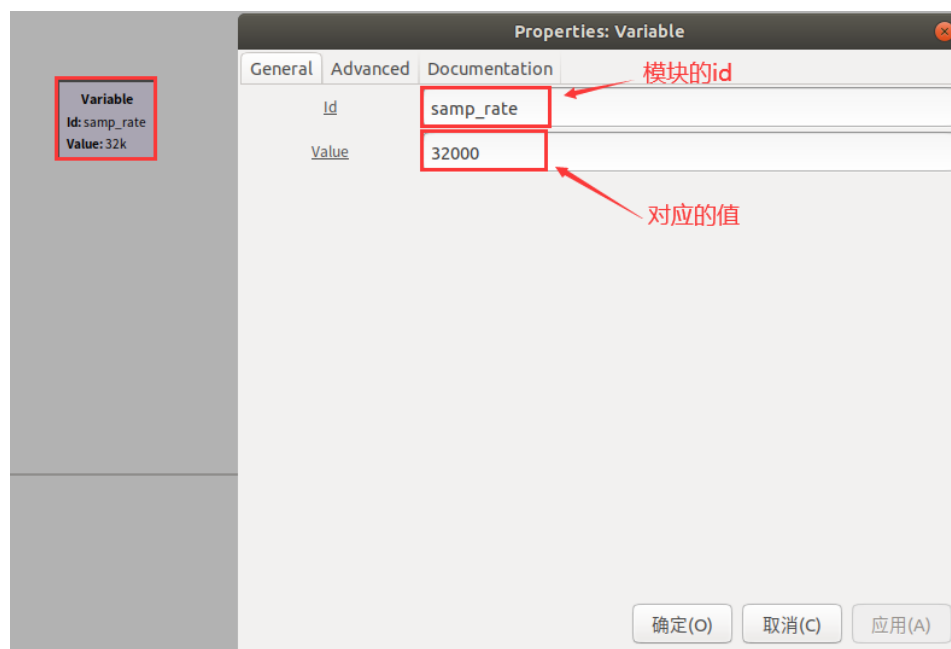


图 2-4 变量模块

然后我们可以添加一个信号源模块，用于产生单音信号。按下 **ctrl+f** 或者左上方的搜索按钮即可搜索想要的模块。在这里输入 **source**，然后双击 **signal source** 模块就可以把这个模块添加到工作区当中，或者将该模块拖动到工作区当中。

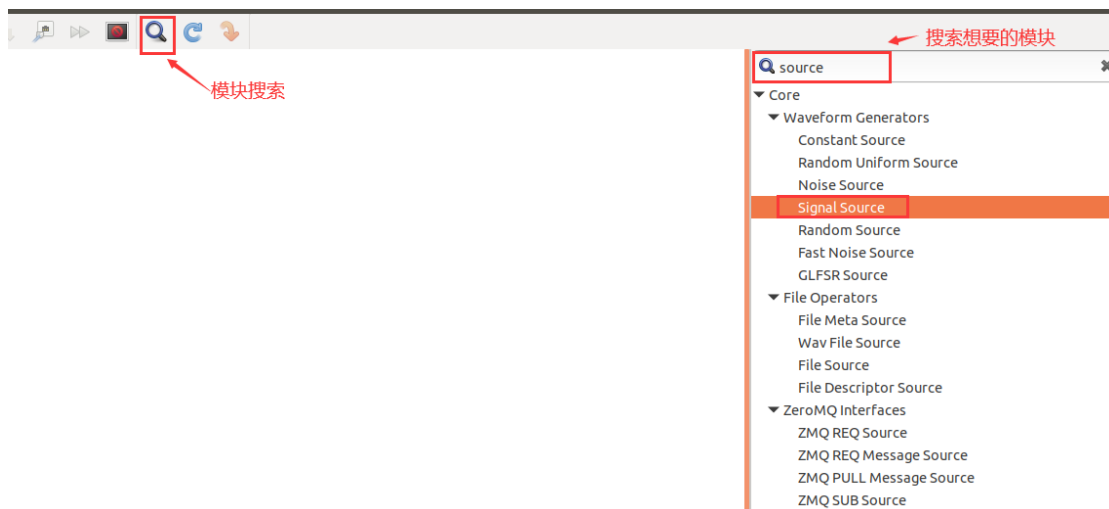


图 2-5 添加模块

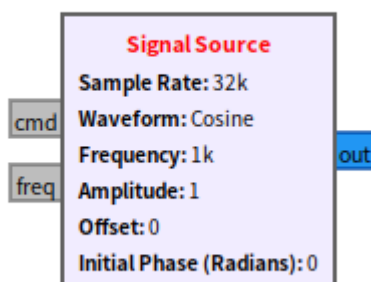


图 2-6 添加的信号源模块

双击添加的模块可以对这个模块进行配置，可以看到该模块具有如下参数：

- **Output Type:** 输出数据的类型，可以是复数或者浮点数等类型。
- **Sample Rate:** 设置该模块的采样率
- **Waveform:** 输出信号类型，可以是正弦波或者余弦波
- **Amplitude:** 整幅
- **Offset:** 直流偏置
- **Initial Phase:** 初始相位

在这里我们设置输出类型为 `flaot` 类型，其余保持默认即可。采样率在这里我们使用前面设置的变量 `samp_rate` 也就是 32K。

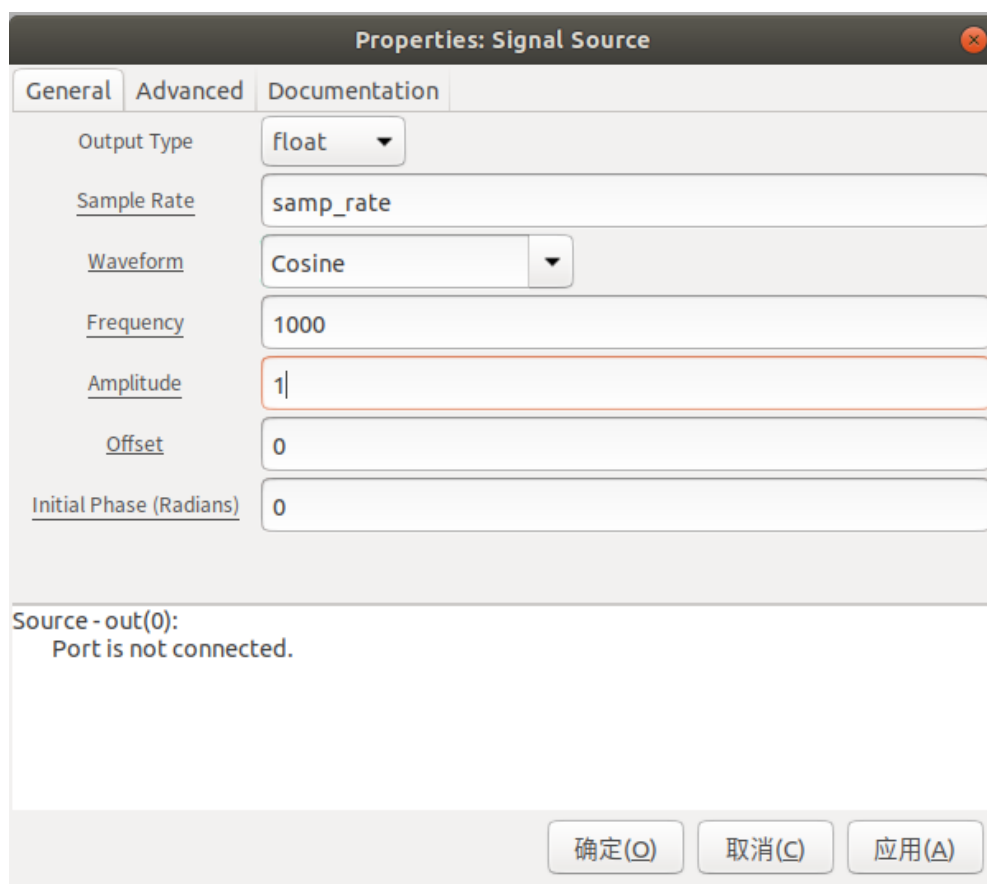


图 2-7 signal source 参数

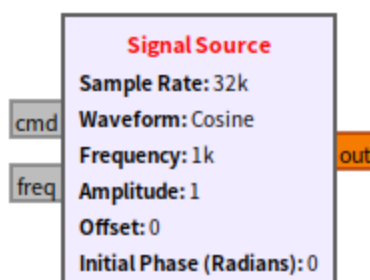


图 2-8 设置完成后的模块

可以和图 2-6 进行比较可以发现,当前这个模块的输出的信号的颜色发生了变化,这是由于我们更改了输出信号的类型导致的,默认这个模块输出的是 **complex** 类型,这里我们改为了 **float** 类型。关于 GNU Radio 支持的数据类型可以通过使用工具栏上方的 **help** 来查看。

在 GNU Radio 当中,不同的类型对应着不同的颜色如图 2-10 所示。

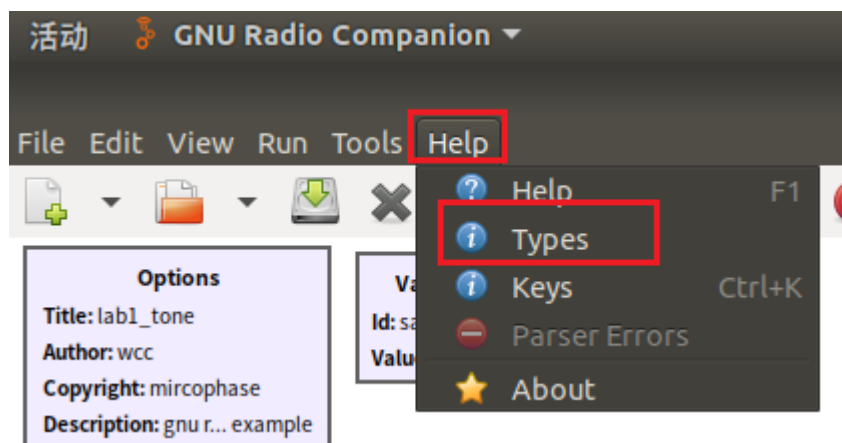


图 2-9 查看支持的数据类型

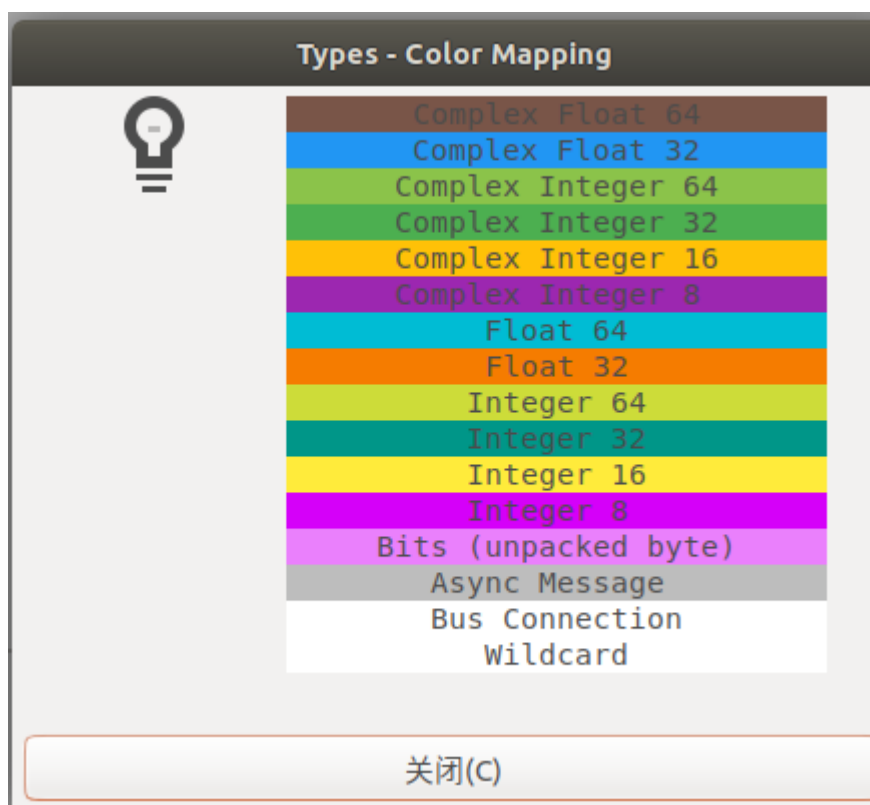


图 2-10 不同类型对应的颜色

然后我们就可以添加一个模块，来观察输出的信号了，我们会观察信号的时域和频域的信息，在这里将 QT GUI Frequency Sink 和 QT GUI Time Sink 添加到工程当中。

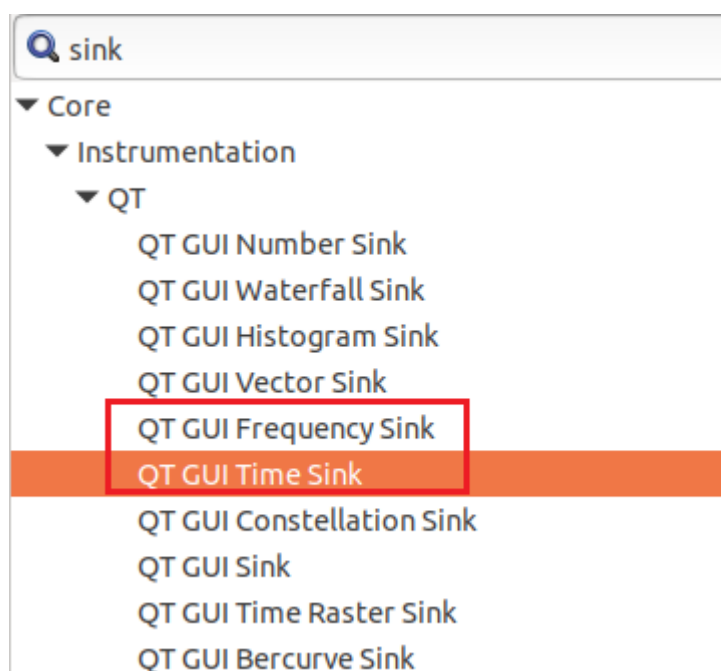


图 2-11 时域和频域分析工具

然后设置 time sink 模块，将其输入的数据类型更改为 float，其余保持默认即可。该模块能够设置要显示的采样点个数，采样频率输入的要显示的信号个数等，在这里其余选项保持默认即可。

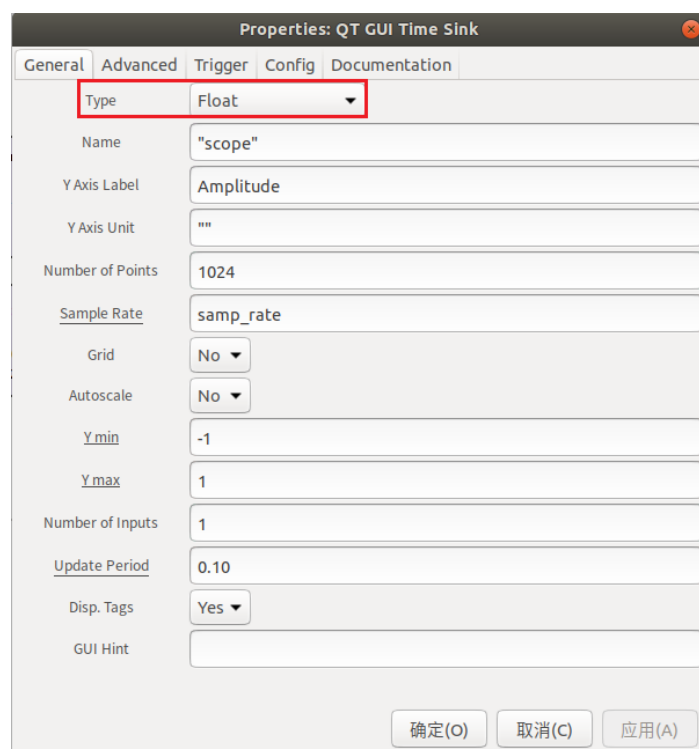


图 2-12 time sink 模块

对于 Frequency Sink 模块的设置同前面设置一样，只需要设置输入的信号类型为 float，其余保持默认即可。

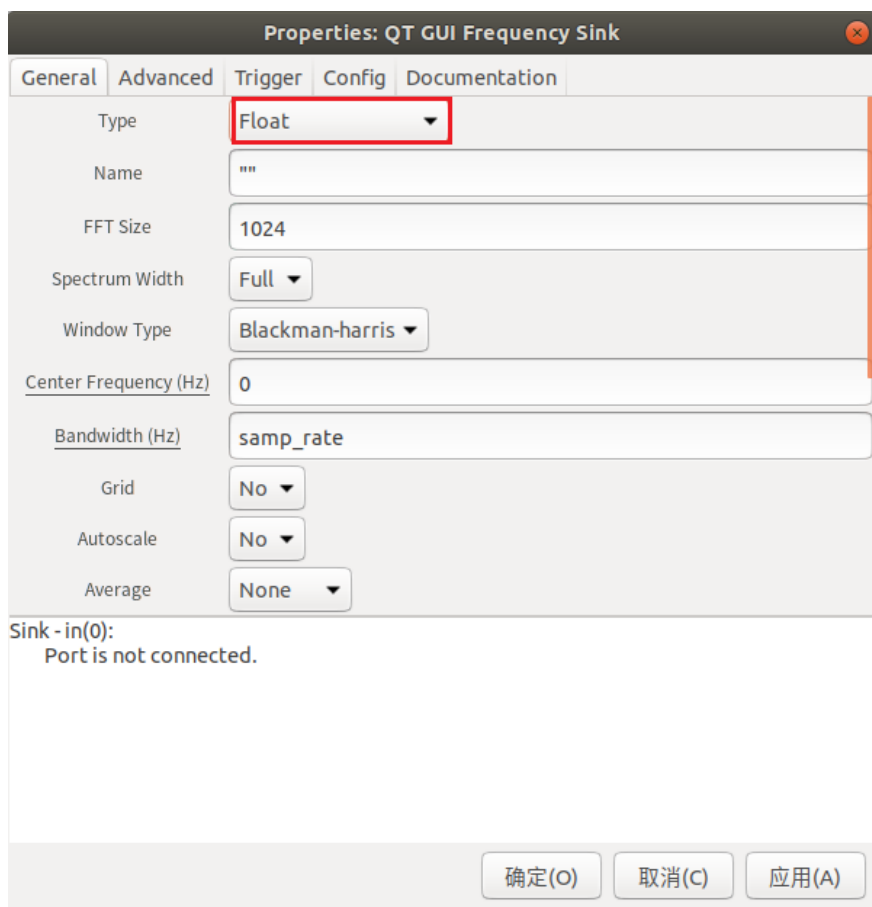


图 2-13 Frequency Sink 模块设置

然后我们会添加 Throttle 模块，该模块的作用是用来控制系统的频率的，由于在本次实验当中，我们还没有使用到具体的硬件，如果在工程中，我们没有添加 Throttle 模块，那么计算机就会以最大的频率来进行仿真，会导致暂用的资源过多，降低计算机的效率。添加这个模块之后，该工程就会以不查过该模块设定的最大的频率进行运行，从而节省 CPU 的效率。

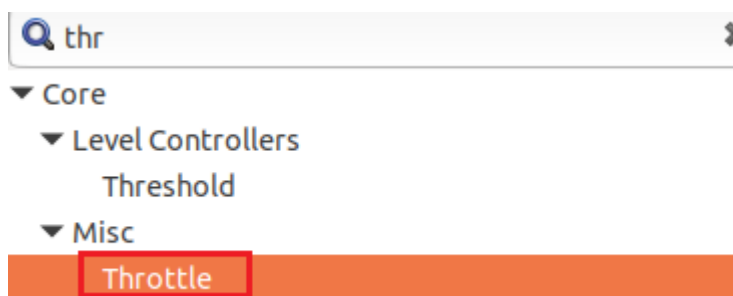


图 2-14 添加 Throttle 模块

在这里我们将 Throttle 模块的 sample rate 设置为前面设置好的变量也就是 32K。

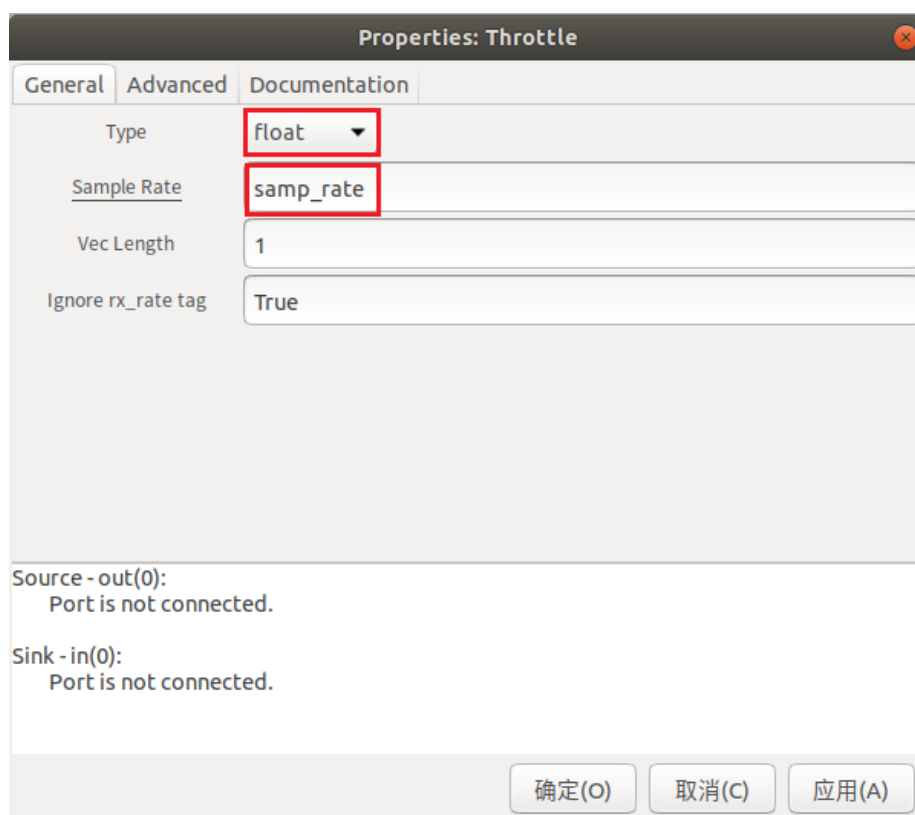


图 2-15 Throttle 模块设置

准备好这些模块之后，我们就可以来进行模块之间的连接了，模块的连接图如下，点击一个模块的输出和另一个模块的输入，就可以在这两个模块之间生成一个连线。

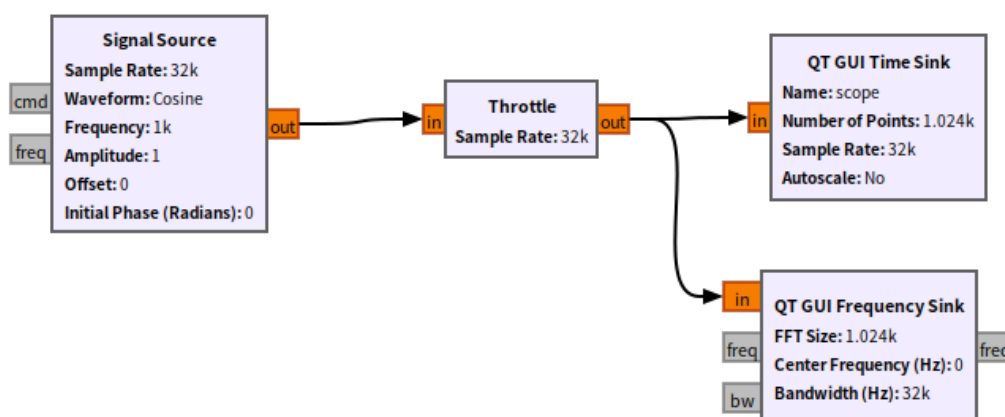


图 2-16 模块之间连接

然后我们可以按下 **ctrl+s** 进行这个工程的保存。

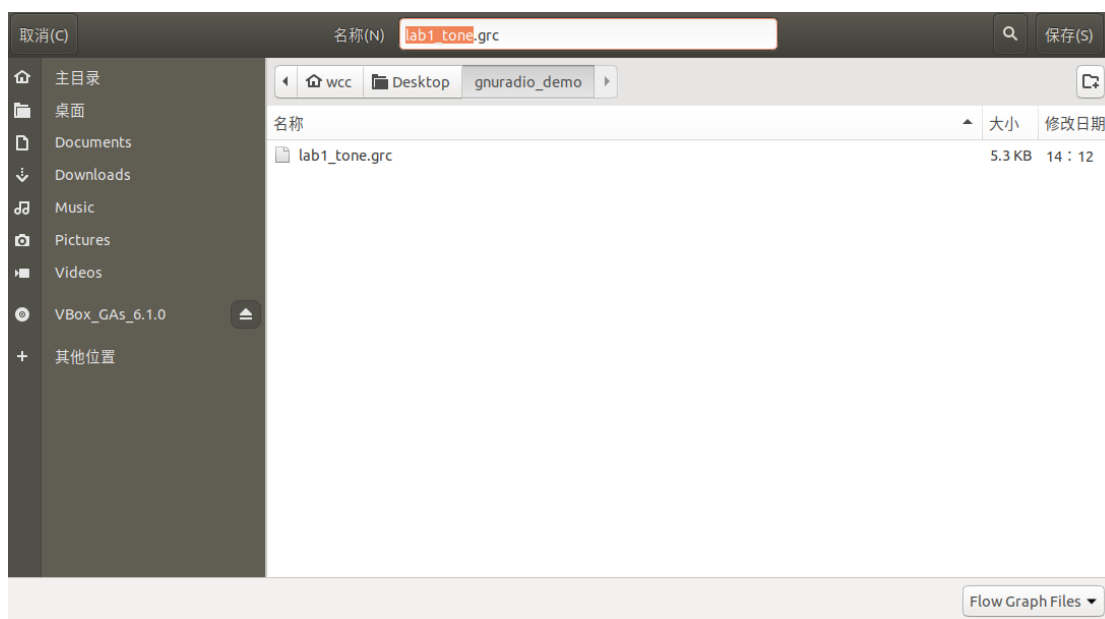


图 2-17 保存工程

接下来就可以点击运行，运行这个工程了。

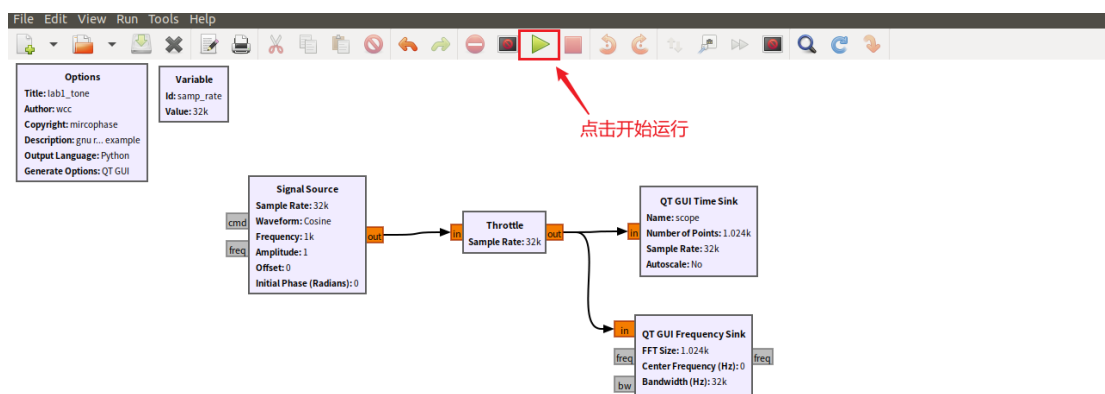


图 2-18 开始运行

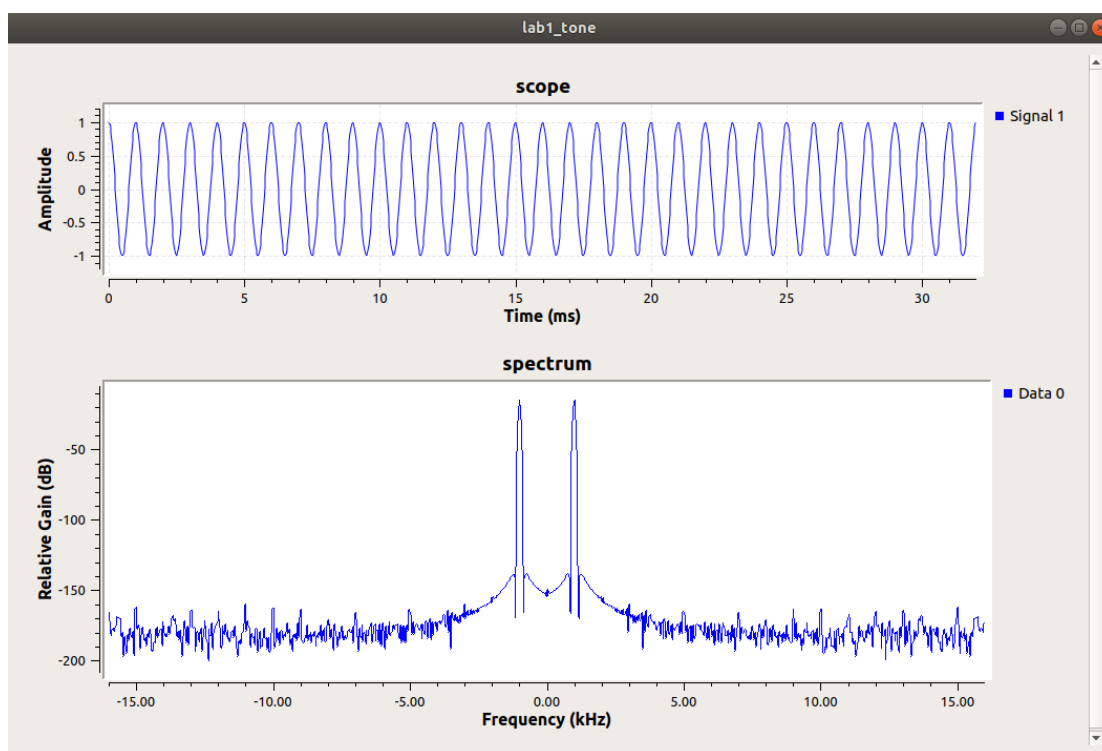


图 2-19 观察时域和频域信息

可以看到，上面是产生的正弦波的信号，下方是频域波形。由于我们输出的 float 型的数据，是一个实数，所以我们可以从频谱上看到对称的两个分量。

到这里我们就完成了一个单音信号的产生了。

2.2 控制单音信号

在前面实验当中，我们使用 GNU Radio 完成了一个十分简单的模型，只产生一个单音信号并对这个信号进行观察。接下来我们会添加一些其他的控制模块，通过这些控制模块能够控制生成的信号的频率和振幅等。

首先新建一个 GRC，保存为 lab2_signal_control.grc。然后和前面的实验一致，先设置 option 这个模块，如图 2-20 所示。

然后将前面实验的流图中使用到的模块，移植到本次实验当中。在本次实验当中，我们会生成两个单音信号，并控制信号的幅度和频率等。

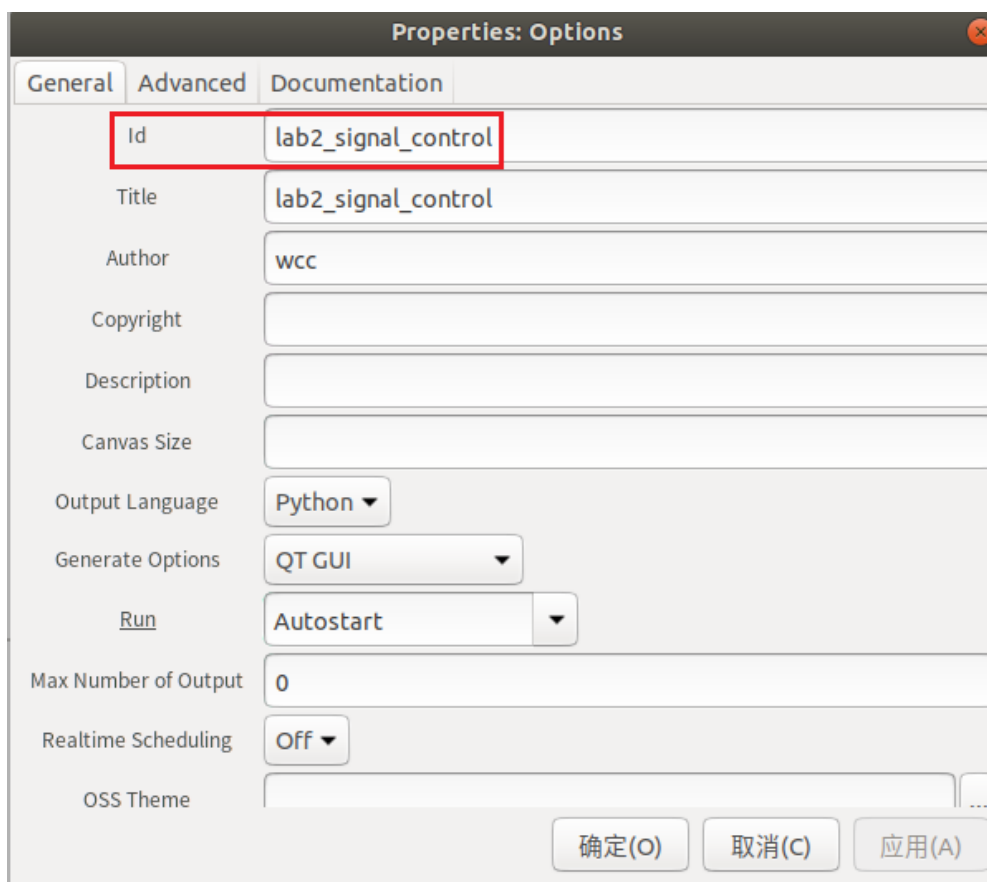


图 2-20

添加两个 QT GUI Range 模块，这个模块能够控制某个变量在一个范围内变化。首先添加两个模块一个对应信号的振幅，一个对应信号的频率。

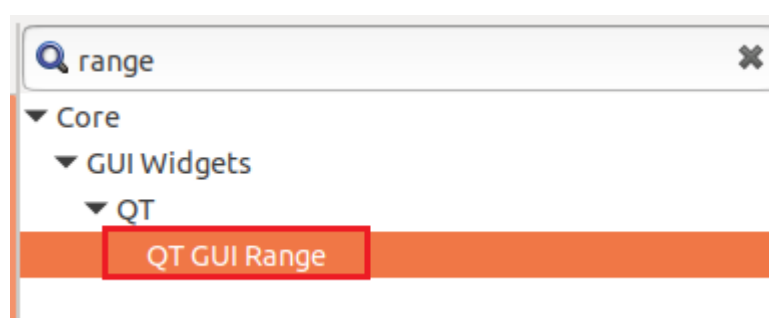


图 2-21 加入 Range 模块

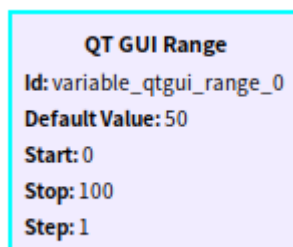


图 2-22 QT GUI Range 模块

对于信号频率的设置如图 2-23 所示，该变量的名称为 freq_0，变量的类型为 int 型，变量的范围为 0~5KHz，变化的步长为 50Hz。

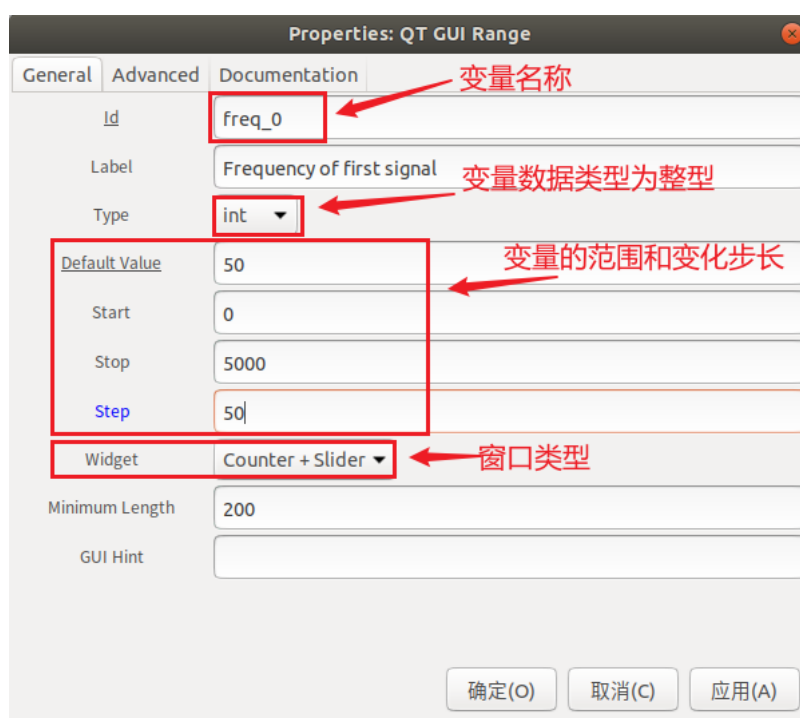


图 2-23 信号频率变化模块设置

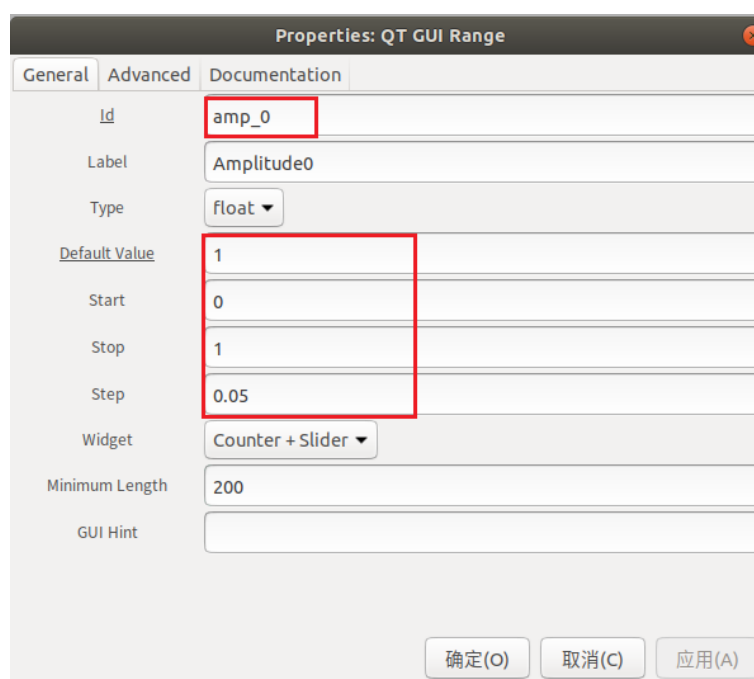


图 2-24 信号振幅变化模块设置

对于信号的振幅，可以采取和前面频率设置模块相似的办法来进行设置，如图 2-24 所示，其振幅的变化范围为 0~1。

然后，我们可以更改生成信号源模块的频率和振幅的参数，将频率和振幅，用前面变量进行代替。



图 2-25 设置信号的频率和振幅

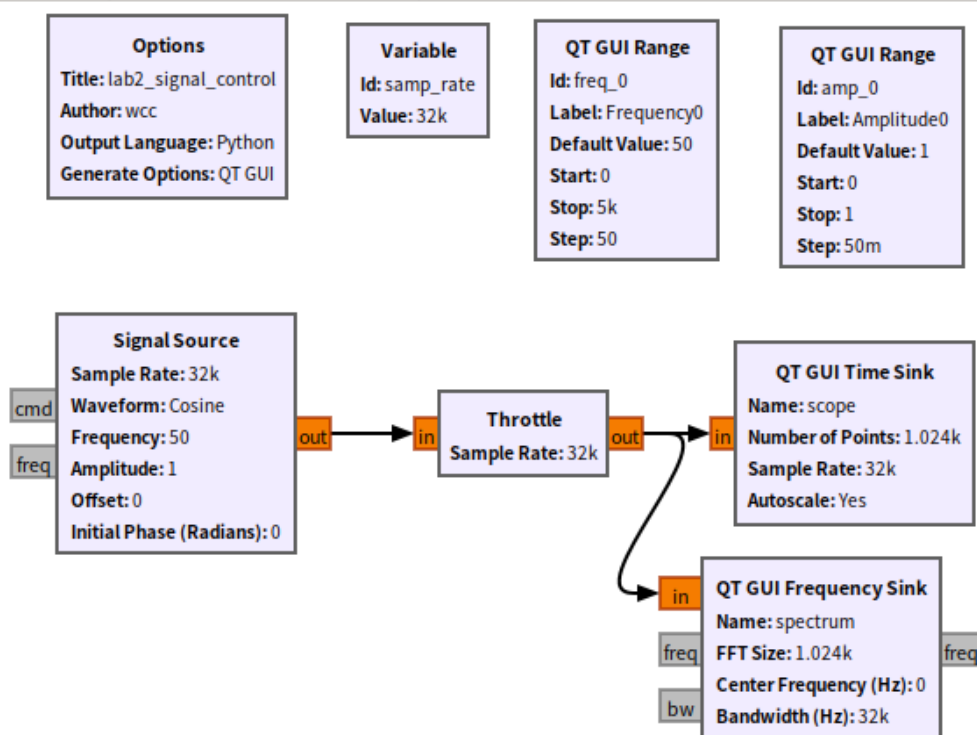


图 2-26 修改之后的 GRC

修改完成之后，点击仿真开始运行这个 GRC。

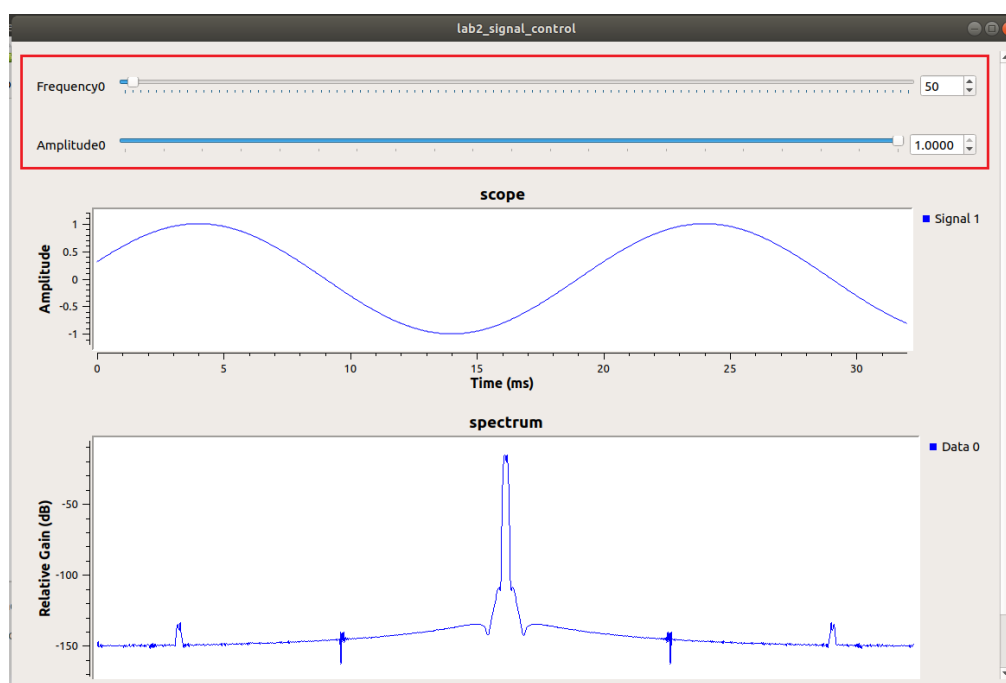


图 2-27 当前的仿真结果

可以看到在波形窗口的上方，有两个可以拖动滑块，通过这个可以调节生成

的单音信号的频率和振幅信息。

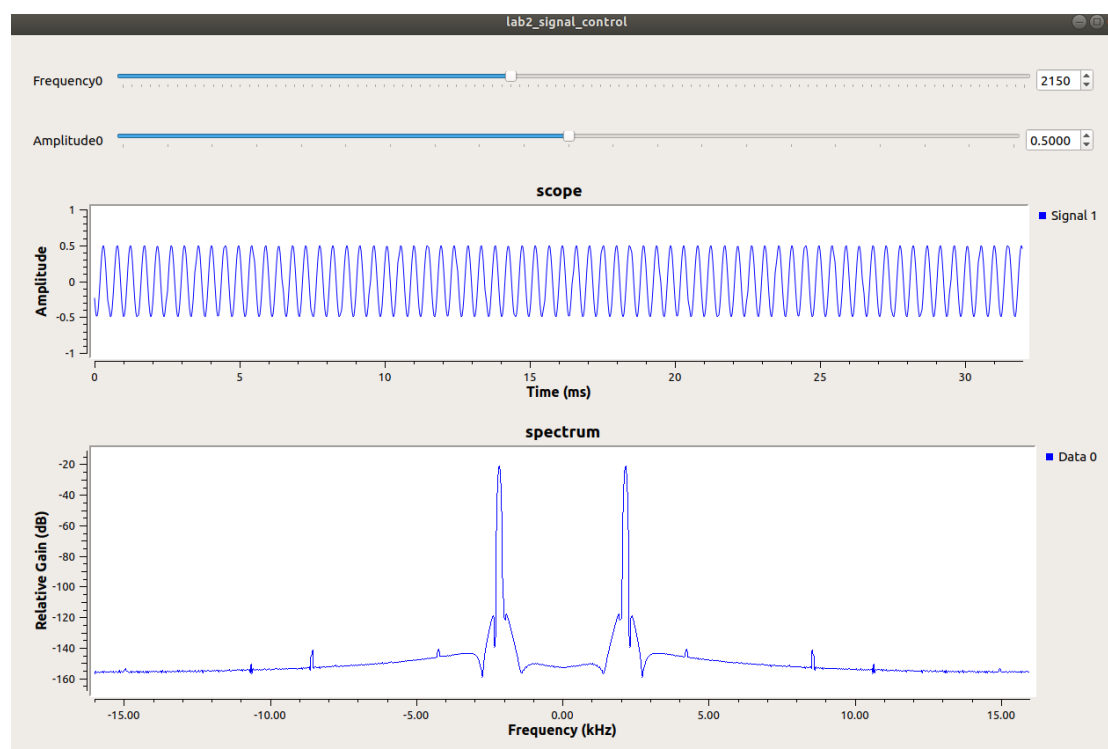


图 2-28 调整频率和振幅之后的结果

可以看到，通过调节上方的频率和振幅的值，可以控制生成的信号的频率和振幅。

接下来，我们可以使用同样的操作，来生成第二个单音信号，只需要将信号产生模块和对应的控制模块进行复制和简单的修改即可。然后将这两个单音信号进行累加。

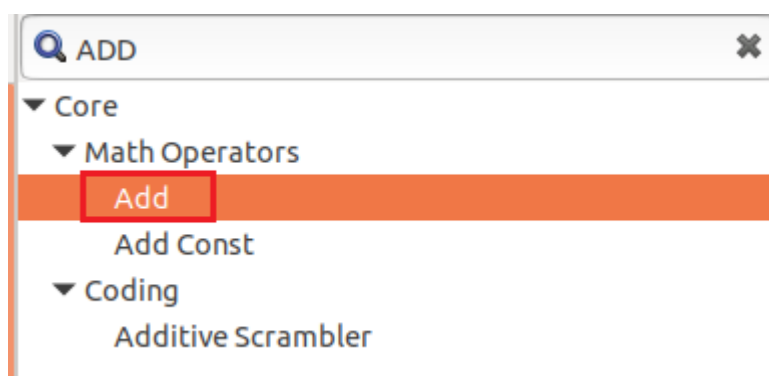


图 2-29 相加模块

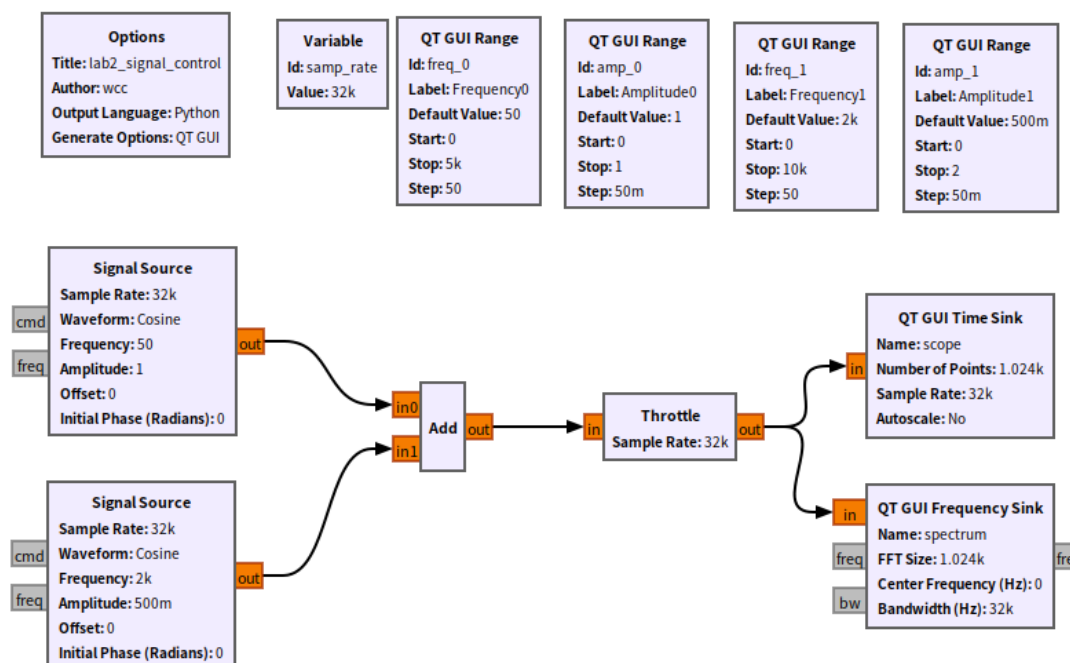


图 2-30 两个单音信号累加的 GRC

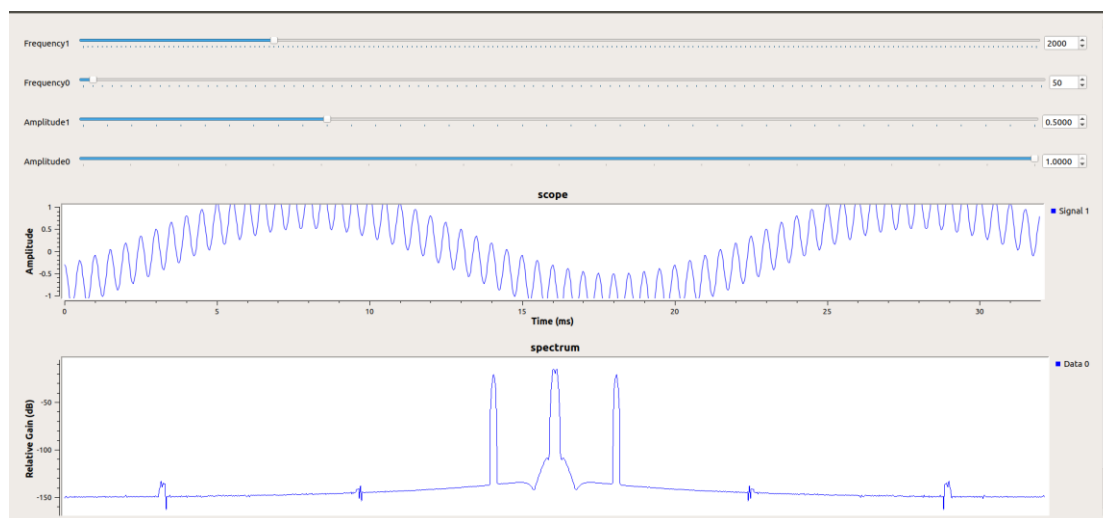


图 2-31 两个单音信号累加的结果

2.3 使用 ANTSDR 收发信号

为了能够直观地显示 SDR 设备在 GNU Radio 当中的使用方法，接下来我们会使用 ANTSDR，基于前面实现的工程来完成单音信号的发送和接收。在 GNU Radio 中使用需要使用到 ADI 提供的 IIO 模块，请在开始下面的实验之前，确认已经成功地在 Ubuntu 系统上装好了对应的驱动和 GNU Radio 所需要的 gr-iio 模块。

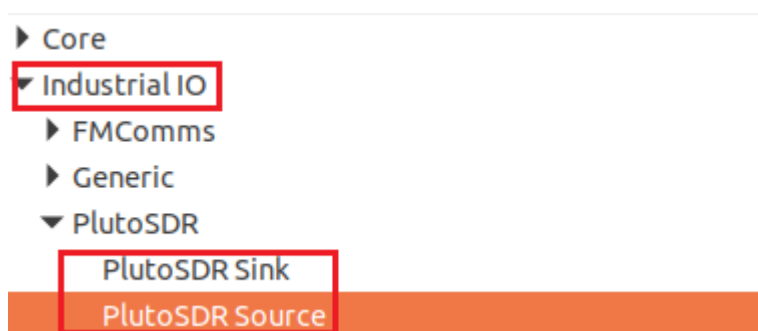


图 2-32 PlutoSDR 输入输出模块

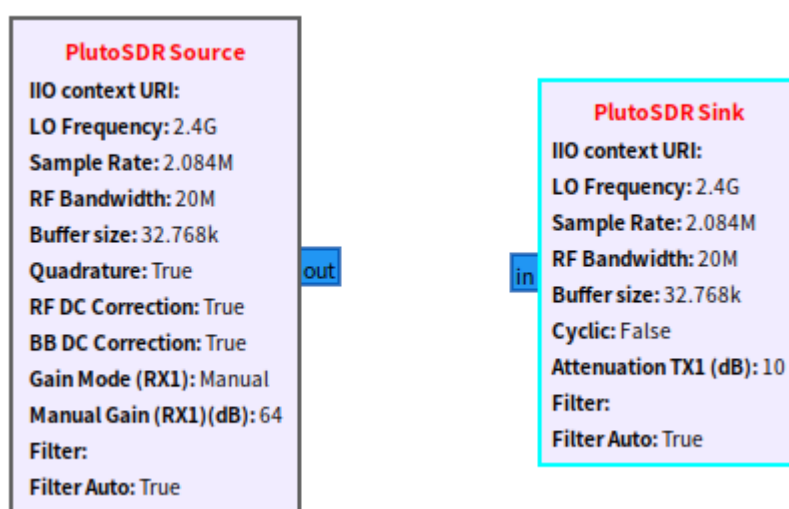


图 2-33 PlutoSDR 模块

在本次实验当中，需要将 `plutosdr` 的输入输出模块添加到工程当中。然后开始进行这两个模块的设置。

添加一个 `Parameter` 模块，用于 GNU Radio 找到 PlutoSDR，其中这个模块的值需要根据用户选择的连接方式进行设置。

如果用户使用的网口连接至 ANTSDR，那么该处的值需要设置为用户以太网端口的值。

ip:192.168.1.10

关于用户网口的 IP 地址如何设置，可以参考 [开箱检测](#) 当中以太网检测部分。

对于 Linux 上的网络设置比较简单，以笔者的 Linux 实体机为例，首先使用 `ifconfig` 命令查看当前电脑的网口信息。

```

eth0: flags=4099<UP,BROADCAST,MULTICAST> mtu 1500
    inet 192.168.1.100 netmask 255.255.255.0 broadcast 192.168.1.255
    inet6 fe80::725a:fff:fe25:ca13 prefixlen 64 scopeid 0x20<link>
    ether 70:5a:0f:25:ca:13 txqueuelen 1000 (Ethernet)
    RX packets 4057989 bytes 5256319606 (5.2 GB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 12867711 bytes 17539374172 (17.5 GB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0x10<host>
    loop txqueuelen 1000 (Local Loopback)
    RX packets 37849 bytes 38804075 (38.8 MB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 37849 bytes 38804075 (38.8 MB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

wlan0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 192.168.3.186 netmask 255.255.255.0 broadcast 192.168.3.255
    inet6 fe80::16e:7166:9c0a:bdb prefixlen 64 scopeid 0x20<link>
    ether 74:df:bf:0c:18:54 txqueuelen 1000 (Ethernet)
    RX packets 25375 bytes 16426393 (16.4 MB)
    RX errors 0 dropped 0 overruns 0 frame 56759
    TX packets 10645 bytes 1110424 (1.1 MB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
    device interrupt 17
  
```

图 2-34 当前主机的网络设备

可以看到 eth0 是笔者 Linux 电脑上的以太网卡，wlan0 是笔者 Linux 电脑上的无线网口。由于 ANTSDR 需要通过网线连接至电脑，因此需要将电脑的 eth0 的 IP 地址设置为与 ANTSDR 处于同一网段。

使用 ifconfig 命令设置 IP 地址为 192.168.1.100.

```
sudo ifconfig eth0 192.168.1.100
```

若用户使用 USB OTG 进行连接到电脑主机，那么需要在参数中输入

```
ip:pluto.local
```

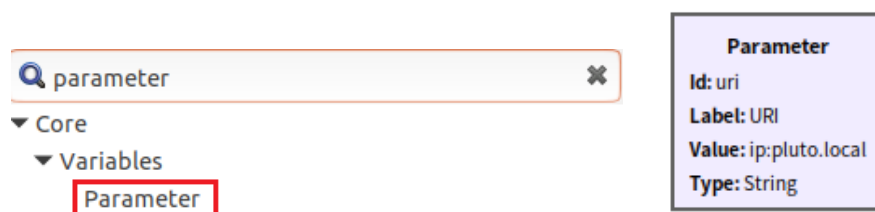


图 2-35 Parameter 模块

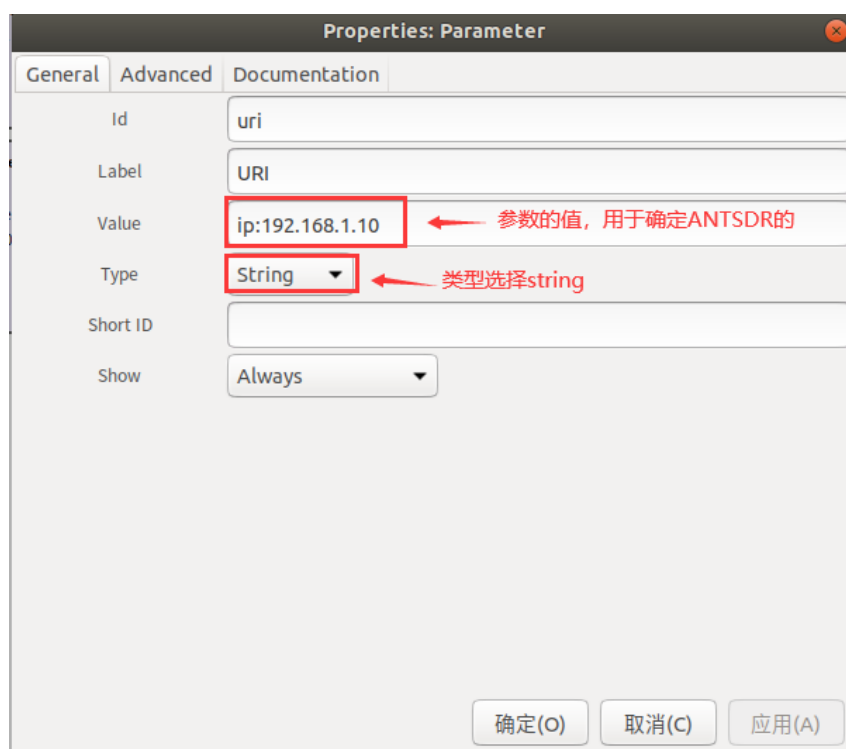


图 2-36 Parameter 模块设置

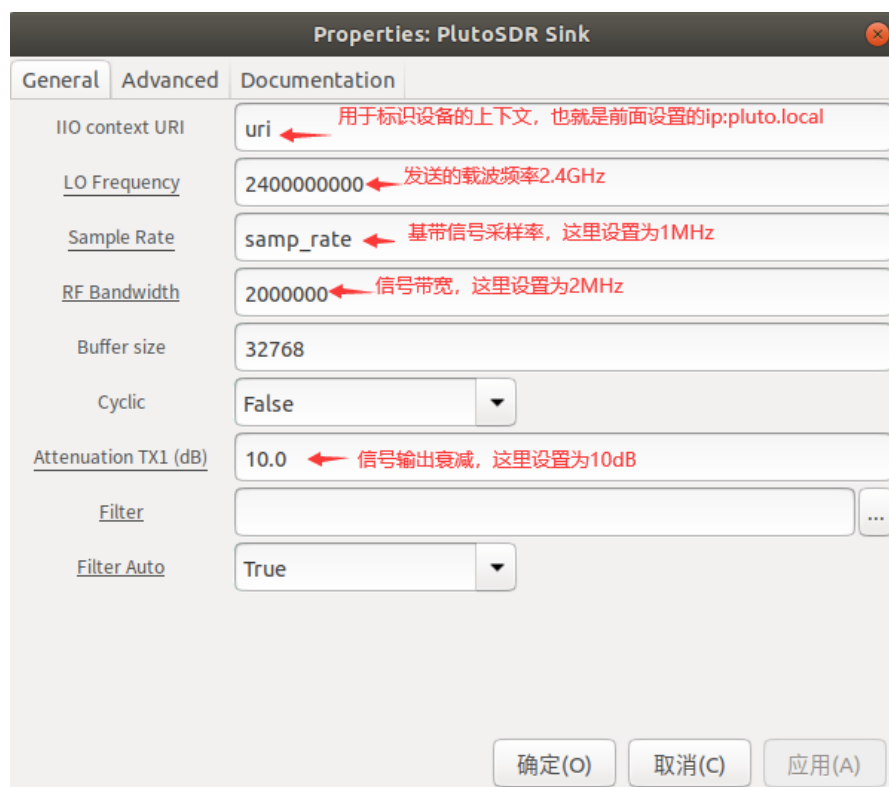


图 2-37 PlutoSDR Sink 模块

然后观察信号输出模块，在这里也就是 PlutoSDR Sink 模块，当信号流入到这个模块的时候，将会进入到软件无线电平台当中，最终经过天线发送出去。其设置如图 2-37 所示。

其中 URI 用于标识 SDR 的设备，这样当 GRC 开始运行的时候，就能够正确地打开 SDR 设备。然后需要设置信号的载波频率和基带的采样率，带宽等。

其中载波设置的载波频率也就是最终希望将信号发送到某个具体的频段上的频率，在这里设置为 2.4GHz。

基带采样率也就是基带信号的采样频率，也就是当前在 GRC 中处理的数据，在数字基带上的采样率，在这里设置为 1MHz。

然后还需要设置信号的带宽，在本次实验发送的是一个单音信号，因此其带宽不会很宽，在之后如果需要发送一个占用频带带宽较宽的信号的时候，这个值需要根据具体得应用来修改。

最后是设置输出得信号衰减，在进行实验得时候，在不影响实验结果的情况下，将输出的衰减设置得大一点，这样输出到信道中的信号就不会带有很大的能量。

对于 PlutoSDR 的输入模块其设置和输出模块的设置基本一致，如图 2-38 所示：

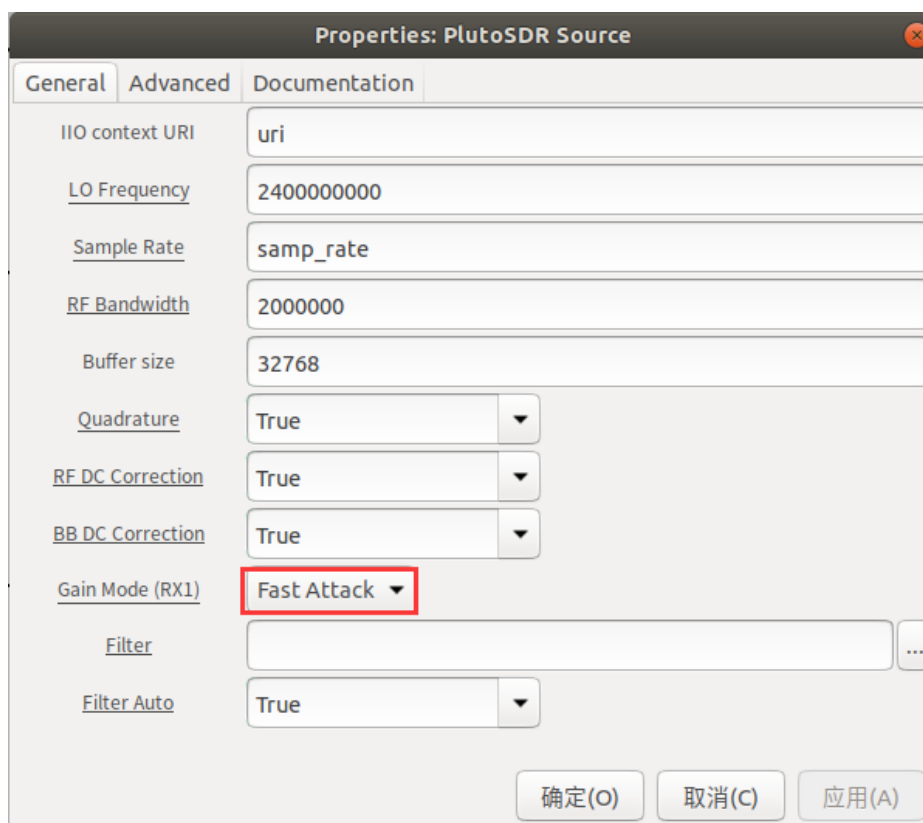


图 2-38 输入模块设置

其中在 **Gain Mode** 当中，可以设置输入信号的增益，可以选择手动设置，也就是 **manual**，也可以选择使用 **Fast Attack** 或者 **Slow Attack** 这两种模式。

然后我们需要考虑如何使用 **PlutoSDR** 来发送设备了，由于 **PlutoSDR** 发送和接收的都是复数基带信号。而在前面的实验当中我们的信号源都是 **float** 类型的数据，因此可以考虑将两个 **float** 类型的信号，组成一个复数基带信号，其中一个信号用作 **I** 路，另外一个信号用作 **Q** 路。因此需要使用到一个模块，**float to complex**，这个模块的作用就是将 **float** 类型的数据转换成 **complex** 类型的数据。

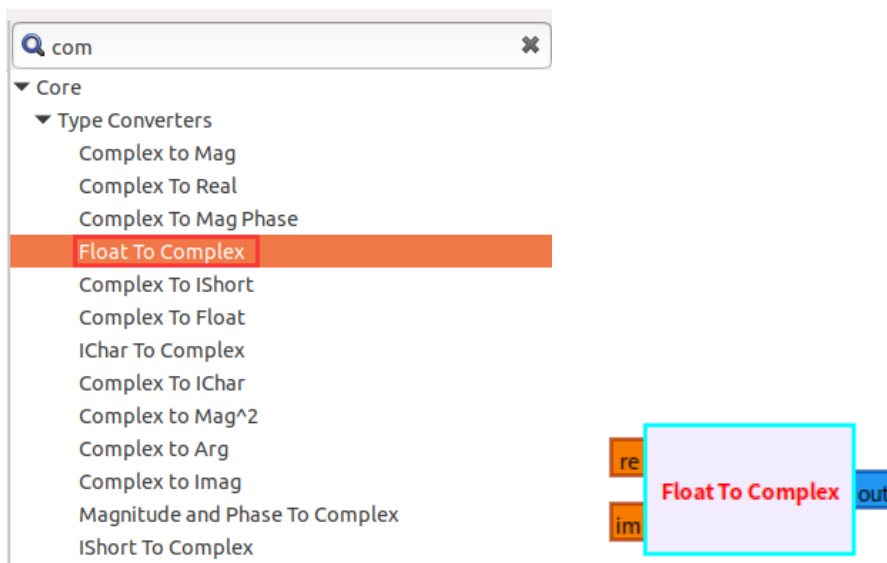


图 2-39 数据类型转换模块

然后将两个单音信号，连接到数据转换模块对应的接口上去了，这样就能够组成一个复数基带信号了。

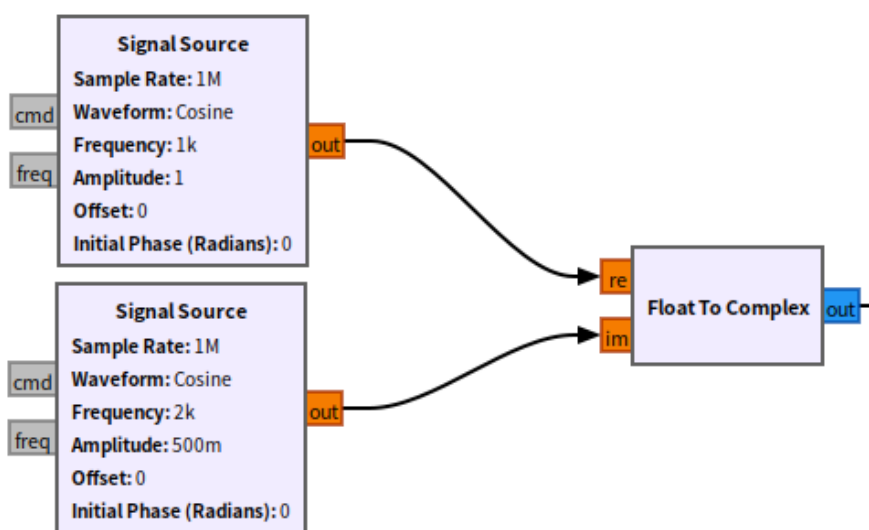


图 2-40 组成复数基带信号

最终，实现的 GRC 流程图如下：

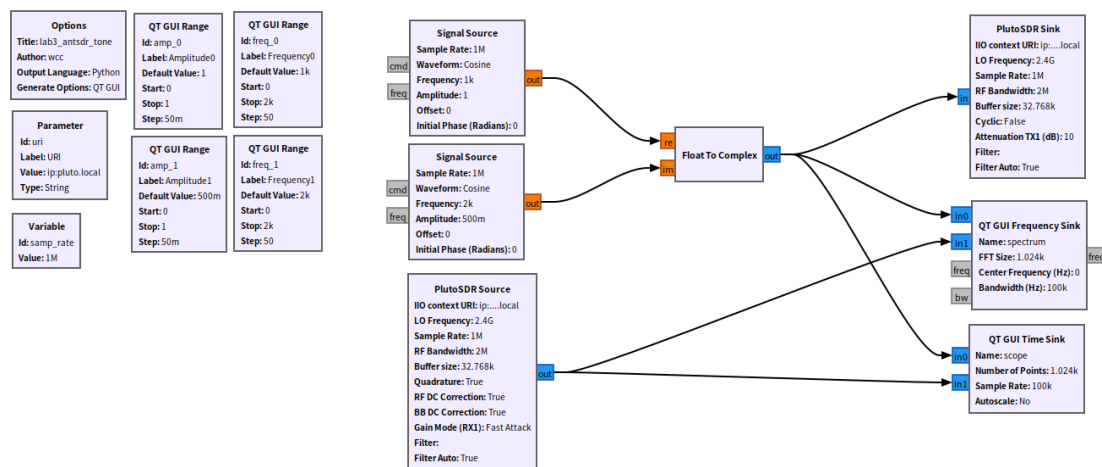


图 2-41 最终的 GRC 流程图

可以看到，由于在这个 GRC 流程图当中，使用到了具体的硬件，因此在这个模块当中，我们没有使用到 **Throttle** 这个模块来控制运行的频率。因为在使用到具体的外部硬件的时候，这时候 GRC 会根据外部硬件的运行速率来确定采样率，而不会是 CPU 一直以最快的速率来运行这个 GRC。

点击开始运行，可以观察发送前的信号和使用 PlutoSDR 接收到的信号，点击图 2-42 中右侧的对应信号的名称，可以选中要显示的信号和不显示的信号。

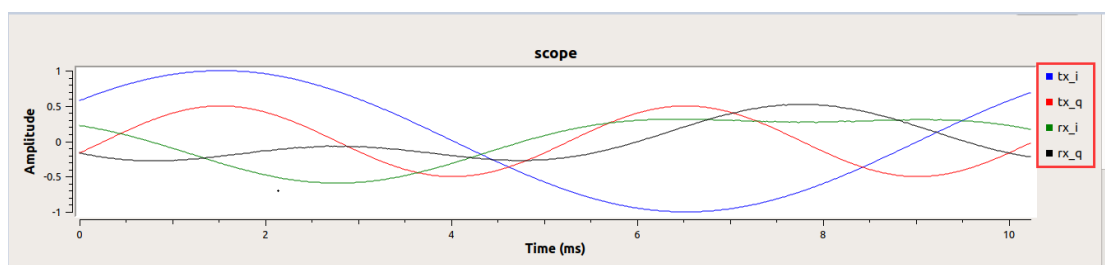


图 2-42

可以看到，这时候绿色和棕色的是 ANTSDR 接收到的信号，从图中可以看出，已经成功接收到了两个单音信号。

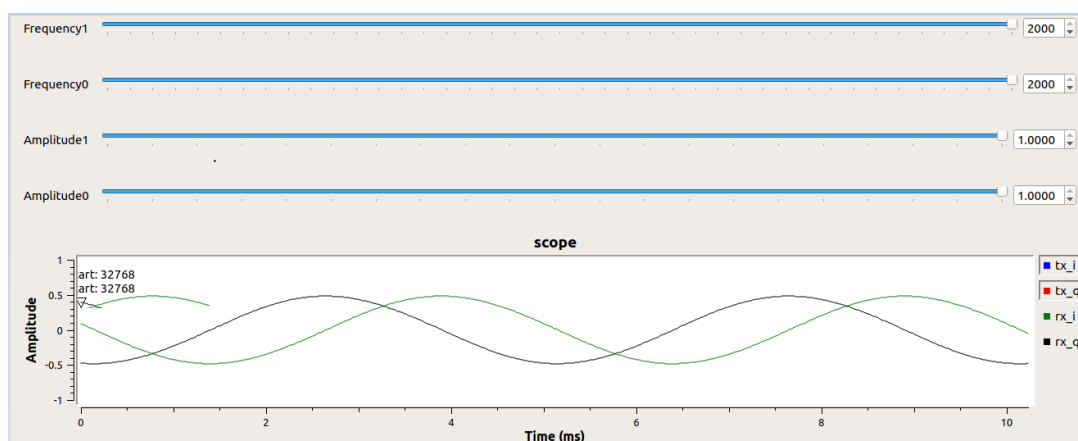


图 2-43 只显示接收到的信号

2.4 小结

在本章的实验当中，首先了解了一下 GNU Radio 的基本使用方法，添加自己的模块，生成了单音信号，并且使用示波器和频谱仪来观察生成的信号，然后介绍了在 GNU Radio 当中变量的使用，能够较为灵活地控制生成的单音信号。

最后使用 ANTSDR 连接到 Ubuntu 操作系统上之后。使用 GNU Radio 来访问 SDR 设备，最终完成了单音信号的发送和接收。

第3章 滤波器使用

在数字信号处理和通信系统当中，我们经常会使用到滤波器来对信号进行处理。常见的滤波器有 FIR 和 IIR 滤波器。FIR 滤波器，又称作有限长脉冲响应滤波器，其不带有反馈结构，在数字信号的处理当中进场会使用到，因为其结构很容易在数字域当中实现。

在本章当中，将会介绍如何在 GNU Radio 当中来实现常用的滤波器，如低通滤波器，高通滤波器器，带通滤波器，以及一些特殊的滤波器如根升余弦滤波器等。

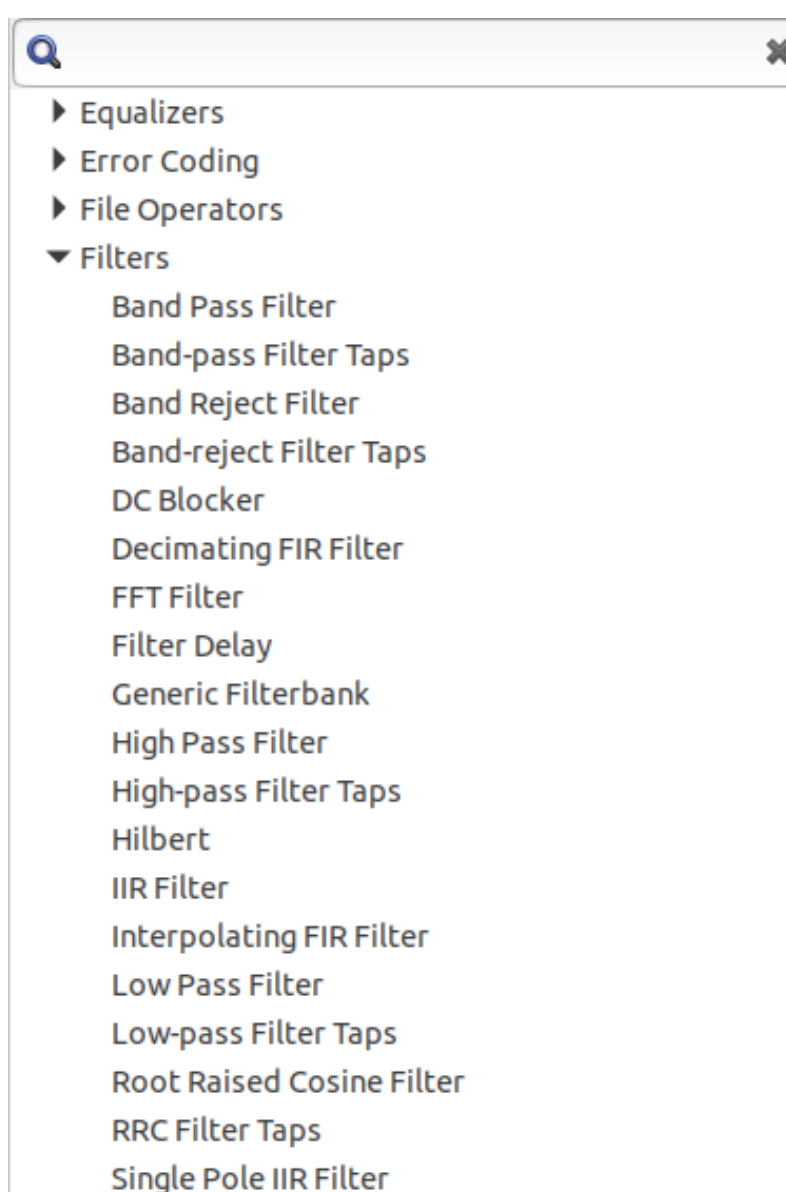


图 3-1 滤波器相关模块

滤波器相关的模块如图 3-1 所示，可以看到 GNU Radio 能够支持多种的滤波器的设计。

3.1 低通滤波器设计

3.1.1 滤波器设计基础

低通滤波器的幅频响应如下：

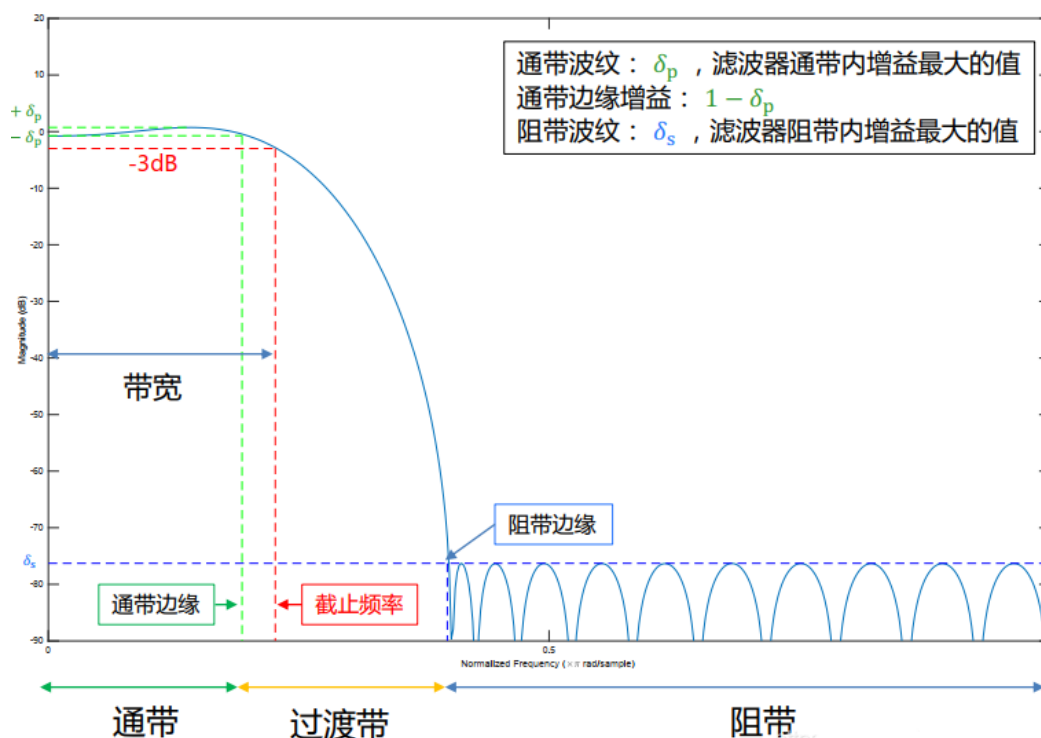


图 3-2 低通滤波器的幅频响应

在低通滤波器当中，频率位于通道的信号将会被保留，频率分量位于阻带的信号将会被滤除。通带和阻带之间是过度带。滤波器的截至频率通常位于信号幅频响应的-3dB 处，该处的信号幅度下降为原始信号幅度的一半。

低通滤波器是数字信号处理当中最常用的一个模块，能够有效地滤除高频分量，保留低频分量。

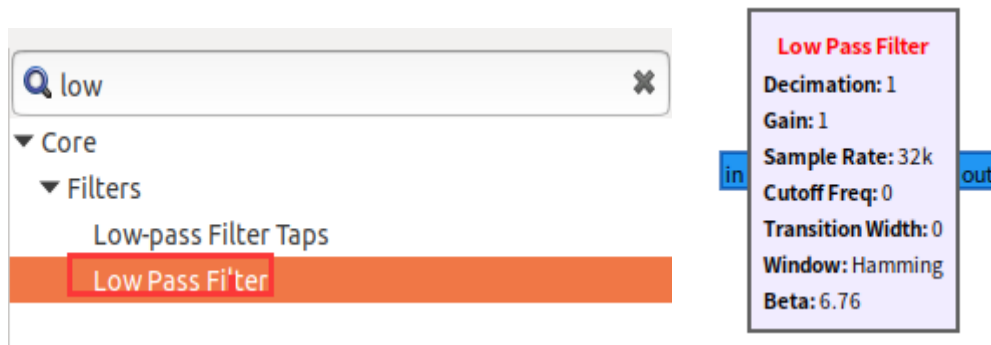


图 3-3 低通滤波器

低通滤波器的参数如下：

FIR Type: 根据输入的数据是实数还是复数确定类型。

Decimation: 滤波器的抽取系数，必须是一个整数并且在实时处理中不能改变。

Gain (R): 对输出信号的增益

Sample Rate (R): 输入信号采样率

Cutoff Freq (R): 滤波器的截至频率

Transition Width (R): 通带和阻带之间的过渡带的长度，单位是 Hz

Window (R): 窗函数

Beta (R): beta 系数只对 kaiser 窗有效

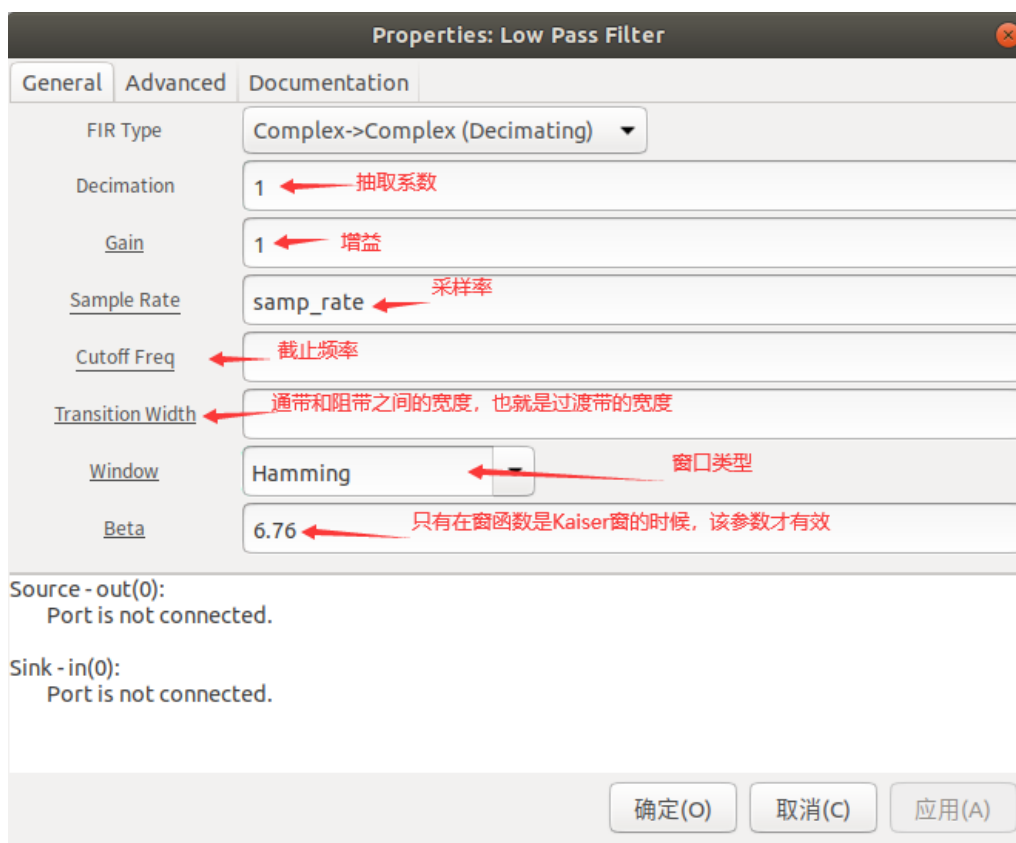


图 3-4

了解了滤波器的各种参数之后，接下来就可以来着手设计一个滤波器了。

3.1.2 低通滤波器实现

首先，产生两个单音信号，作为信号源，然后将这两个累加之后的结果经过低通滤波器处理，然后观察经过低通滤波器处理之后的信号。关于信号的产生和控制，在前面的实验当中已经介绍过了，在这里不做过多的介绍。创建流图：
lab1_lowpass_filter.grc。

在信号源生成模块如图 3-5 所示，使用两个单音信号，分别使用两个可变的变量来分别控制这两个单音信号，这样就能够生成不同频率的单音信号。然后将这两个信号进行相加，在将相加之后的结果给到低通滤波器进行滤波。

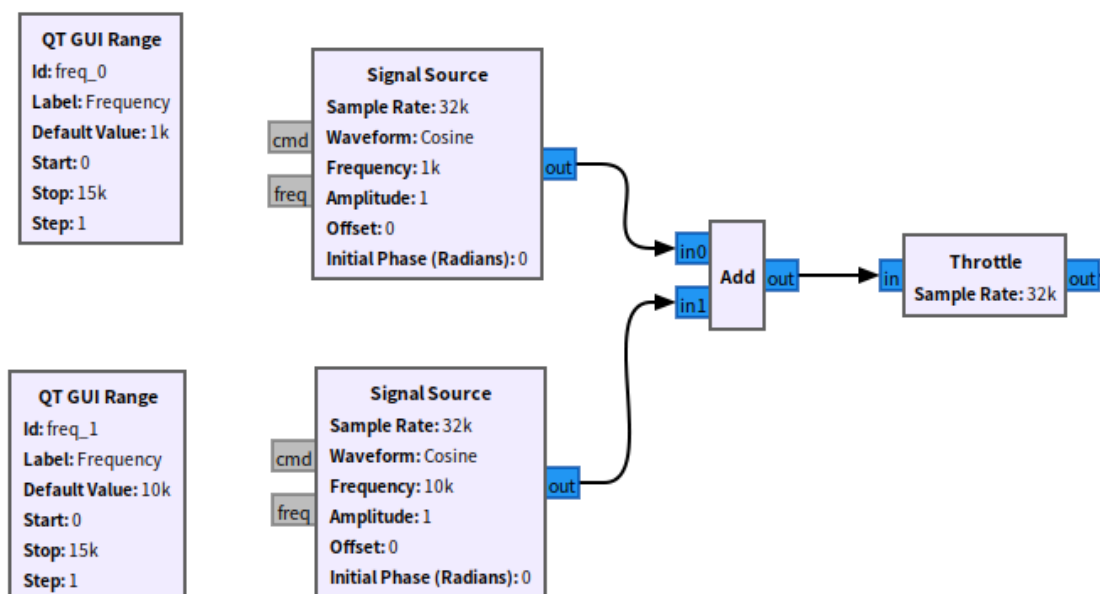


图 3-5 信号源产生模块

低通滤波器模块设计如下：其中截至频率为 5KHz，过渡带宽度为 1KHz，窗函数为 hamming 窗。

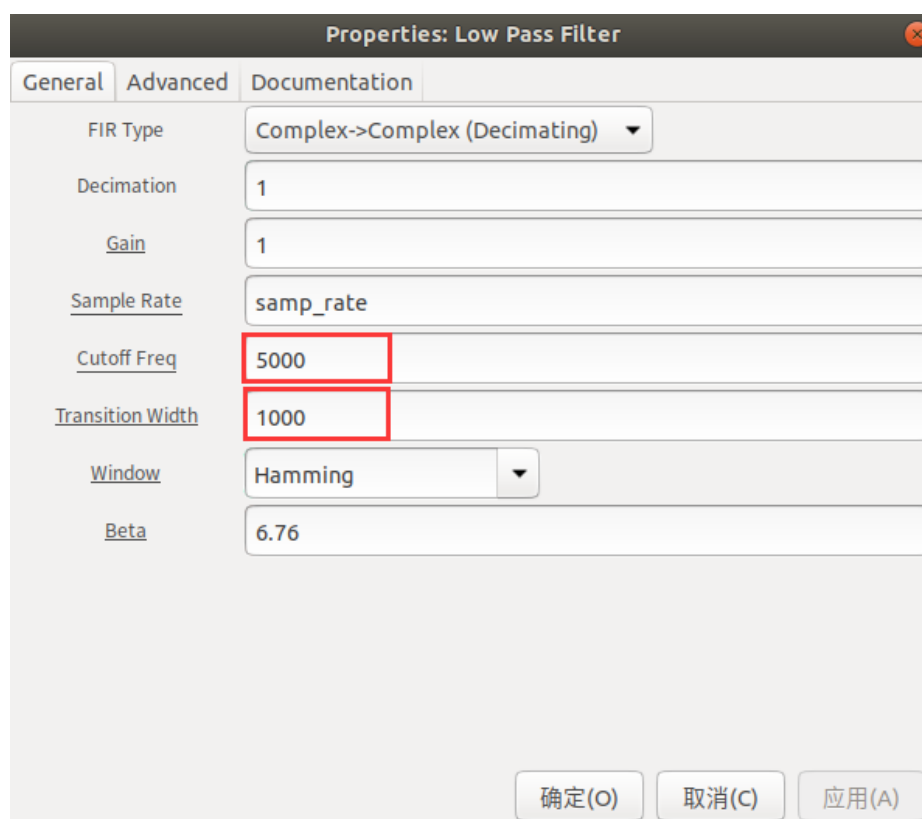


图 3-6 低通滤波器设计参数

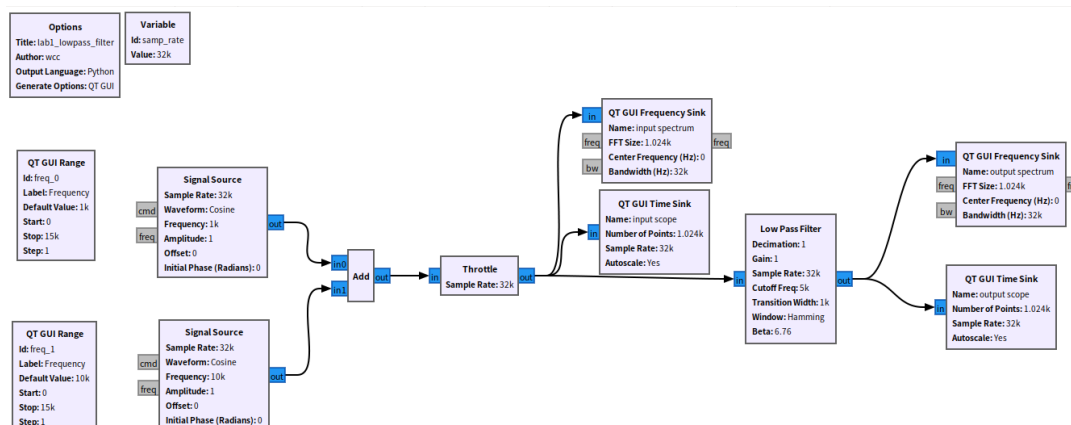


图 3-7 低通滤波器 GRC 流图

最终的 GRC 流图如图 3-7 所示，分别观察输入信号的时域波形和频域波形和输出信号的时域和频域波形。

点击开始仿真，就能够对这个 GRC 流图进行仿真了。

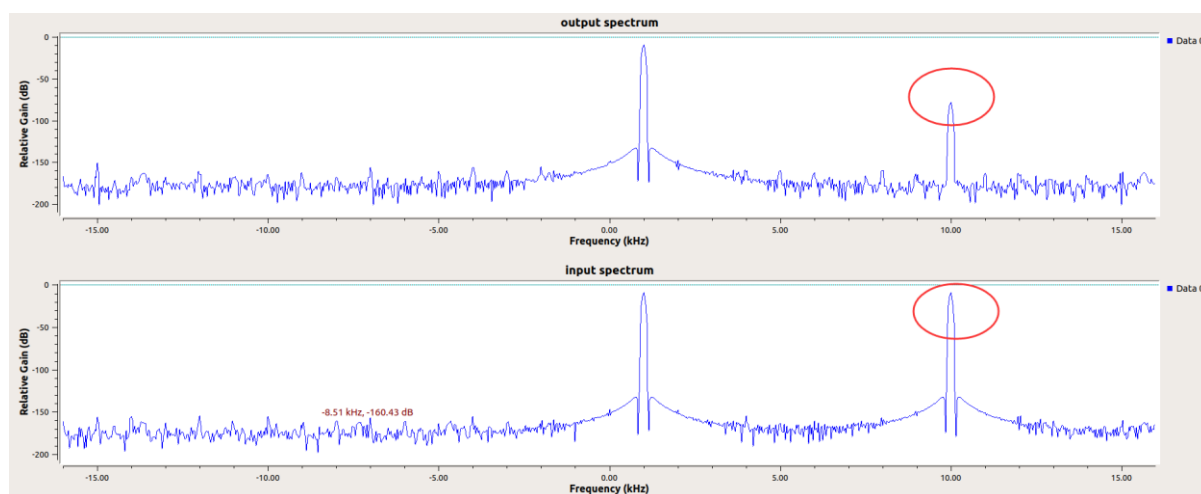


图 3-8 初始状态

默认情况下，初始信号源的频率一个为 1KHz，一个为 10KHz，可以看到 10KHz 的信号在经过低通滤波器之后，被有效的抑制了。我们通过时域波形能够直观地看到滤波器的效果，如图 3-9 所示：

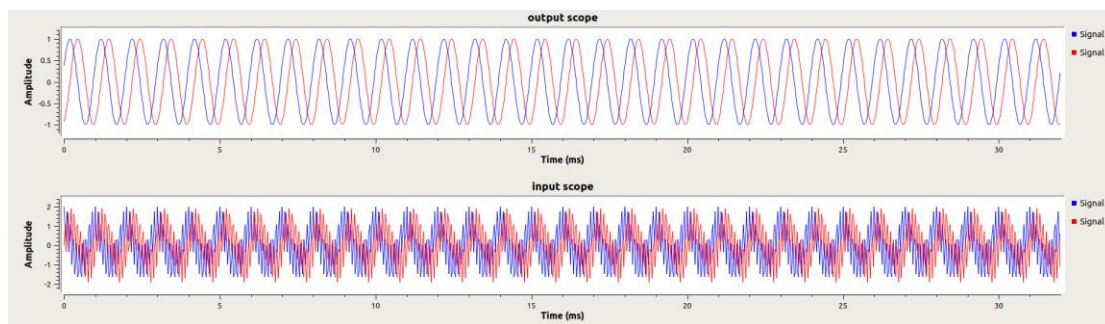


图 3-9 经过低通滤波器之后的信号

可以看到，在没有经过低通滤波的时候，输入的滤波器的信号是一个高频分量和一个低频分量的叠加，经过低通滤波器之后，成功将高频分量滤除，只保留了低频分量。

接下来可以改变输入信号的频率，使两个信号频率都在低通滤波器的通带内，然后观察现象，调整上方滑块就可以，如图 3-10、图 3-11 所示：

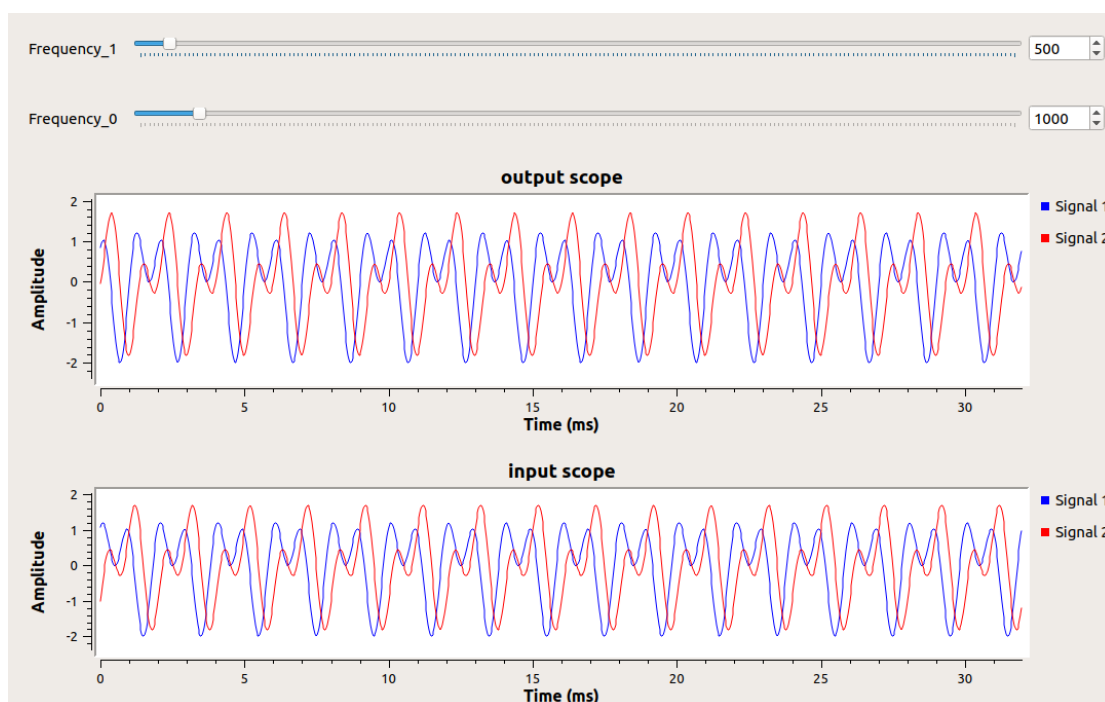


图 3-10 时域波形

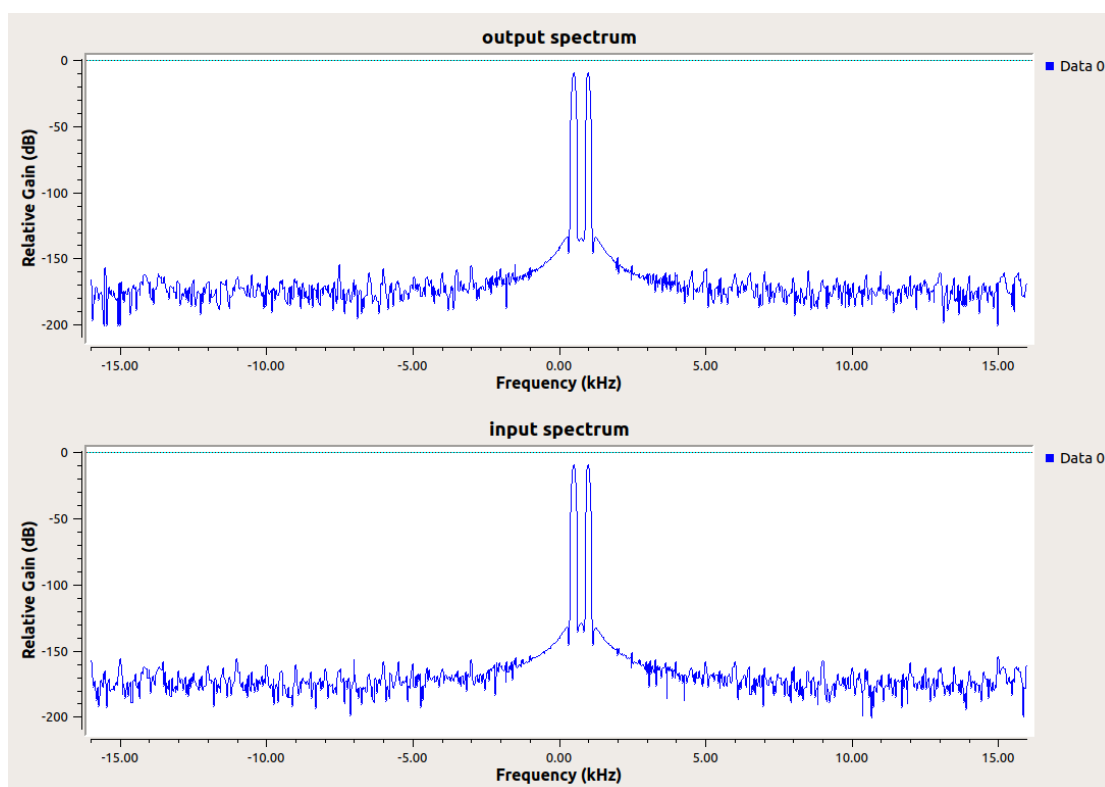


图 3-11 经过滤波之后的频谱

可以看到，当输入信号频率都位于通带内的时候，两个信号分量都能够通过低通滤波器，所以可以看到输出的信号和输入的信号基本一致。

到这里就完成了低通滤波器的设计了，可以看到，使用 GNU Radio 设计一个低通滤波器还是很简单了，只需要将低通滤波器所需要的一些参数提供给对应的模块，就能够完成一个低通滤波器的设计了。

有了低通滤波器的设计基础之后，实现其他类型的滤波器就十分简单了。

3.2 高通滤波器

在前面实现了低通滤波器之后，实现一个高通滤波器就显得十分简单，高通滤波器的幅频响应和低通滤波器的幅频响应类似，只是对低频信号有抑制效果，高频信号能够通过。

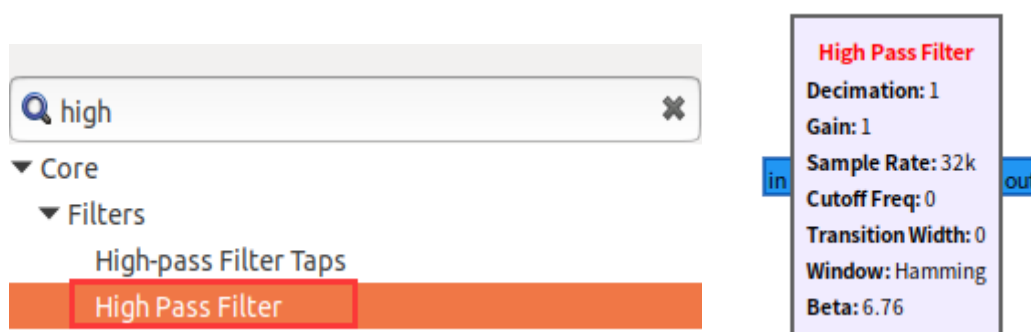


图 3-12 高通滤波器

高通滤波器的参数设置基本和低通滤波器一致，同样需要设置比较重要的截至频率和过渡带宽度等参数。在本次实验中，基于前面的实验来进行，在这里会介绍如何来抽取系数的使用，创建工程 `lab2_high_pass.grc`。

FIR 滤波器经常会和抽取系数和插值系数来一起使用，实现一个多速率的 FIR 滤波器，这在数字上变频 DUC 和数字下变频 DDC 当中经常会使用到。

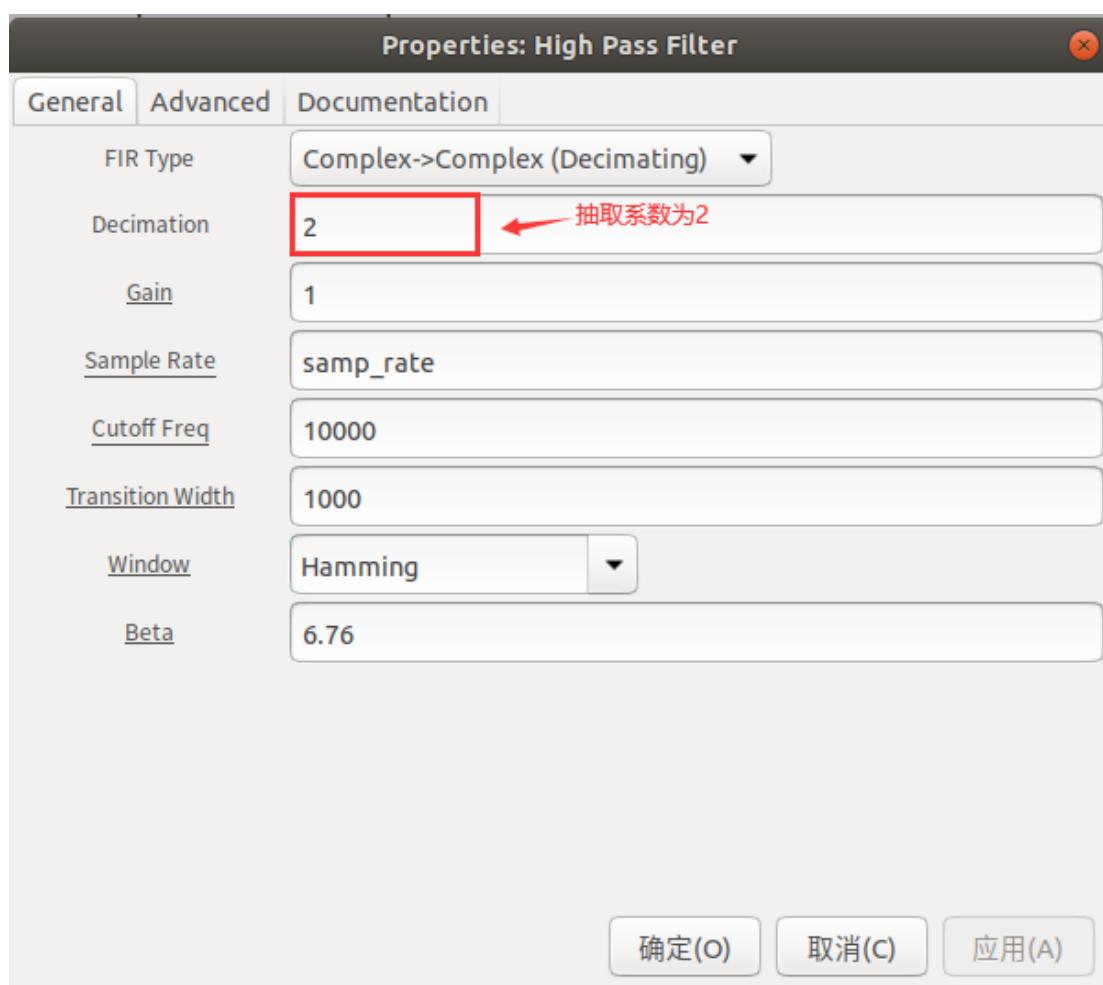


图 3-13 高通滤波器设置

高通滤波器的设置如下，其中抽取系数为 2，那么在经过这个高通滤波器之后，系统的采样率将会降为原来的一半。

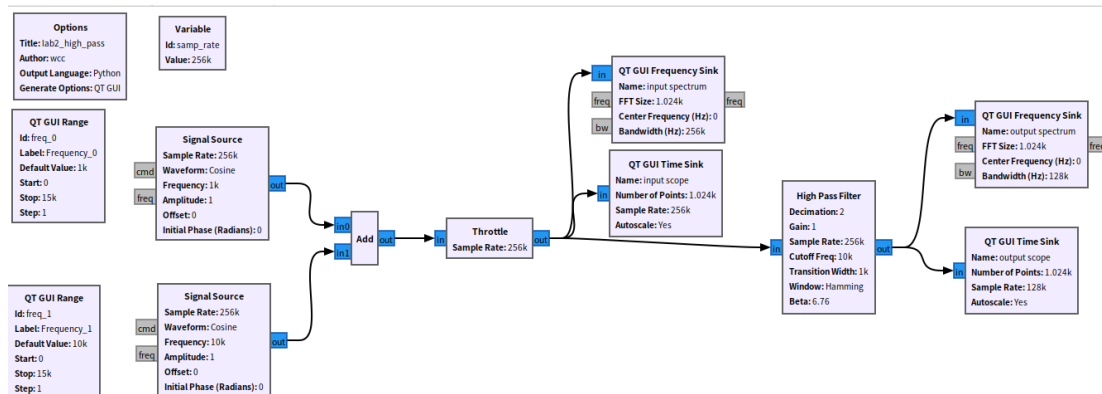


图 3-14 高通滤波仿真 GRC

对两个示波器进行设置，这样在显示的波形图上，能够够将实际的采样点用具体的标识标识出来，以其中一个示波器的设置为例，双击示波器进行设置，然后选中 config 页面。

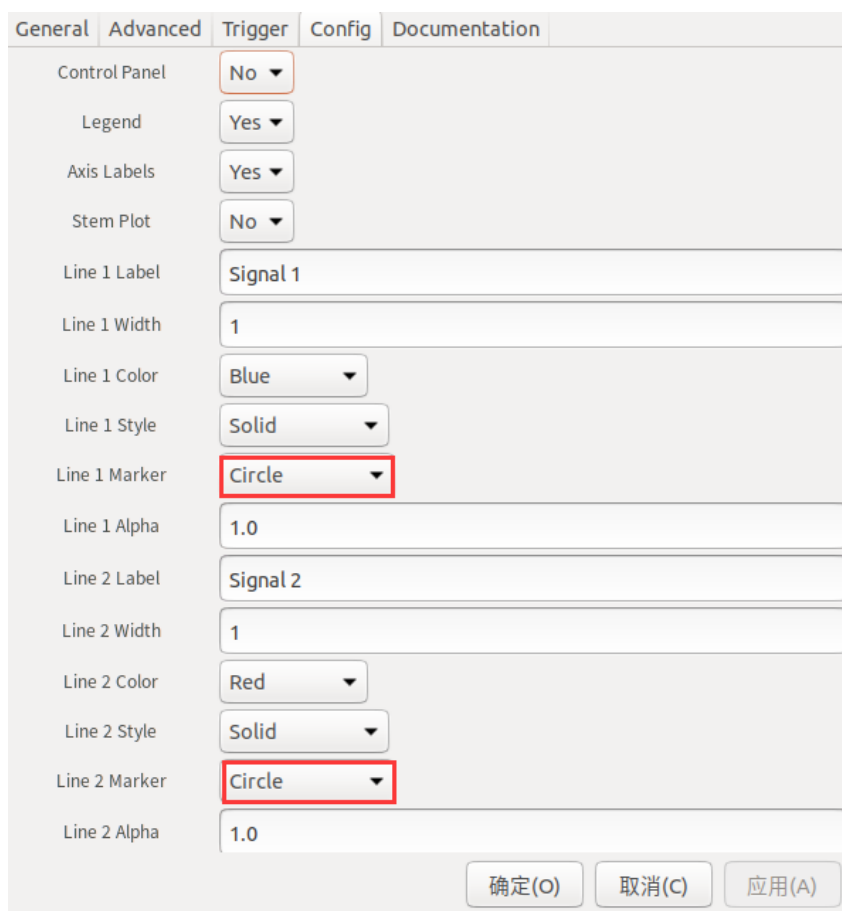


图 3-15 显示波形设置

在 config 页面中, 选择 marker, 这样就能在实际的波形当中将采样点显示出来了。

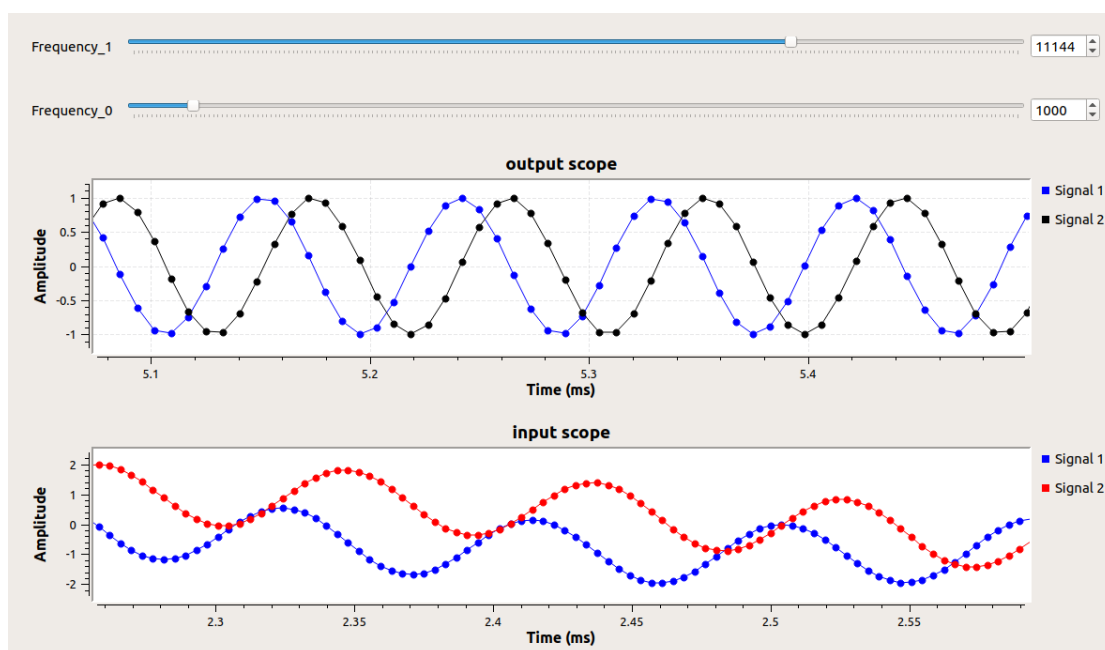


图 3-16 经过抽取之后的波形

可以看到经过抽取之后采样率, 相较于原始的采样速率下降, 采样点之间的间隔增大。

同时可以看到, 经过高通滤波器之后, 滤除了低频分量。

可以看到, 高通滤波器的设计, 和低通滤波器基本一致, 起始带通滤波器和带阻滤波器的设计也基本是同样的设计方法, 在这里就不再过多的介绍了。

3.3 根升余弦滤波器

根升余弦滤波器在数字调制中经常会使用到, 使用该滤波器能够有效减少符号间串扰。

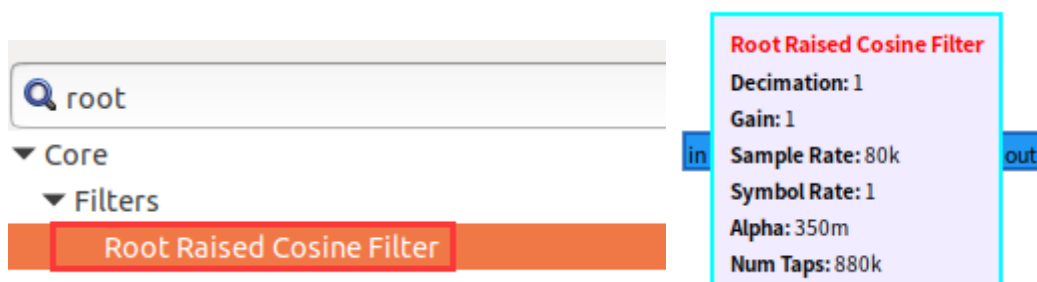


图 3-17 根升余弦滤波器

根升余弦滤波器一般在发送端和接收端中都存在，两个根升余弦滤波器滤波器组合形成一个升余弦滤波器。因此根升余弦滤波器也被称作匹配滤波器。其参数如下：

FIR Type: 根据输入的数据是实数还是复数确定类型。

Decimation: 滤波器的抽取系数，必须是一个整数并且在实时处理中不能改变。

Gain (R): 对输出信号的增益

Sample Rate (R): 输入信号采样率

Symbol Rate: 输入信号的符号速率，符号速率一般是采样率的一个因数。

Alpha: 滚降因子

Number Taps : 抽头系数



Properties: Root Raised Cosine Filter		
General	Advanced	Documentation
FIR Type	Complex->Complex (Decimating) ▼	
Decimation	1	
Gain	1	
Sample Rate	samp_rate	
Symbol Rate	1.0	
Alpha	0.35	
Num Taps	11*samp_rate	

Source - out(0):
Port is not connected.

Sink - in(0):
Port is not connected.

确定(O) 取消(C) 应用(A)

图 3-18 根升余弦滤波器默认设置

为了能够直观地演示出根升余弦滤波器滤波器的作用，可以使用一个简单的

GRC 来演示，创建流图 `lab3_rrc_filter.grc`。

首先，产生一个符号序列，然后将序列进行上采样，提升信号的采样率，然后使用两个根升余弦滤波器来对信号进行脉冲成型。最终观察得到的结果。与此同时，我们可以将原始信号也在示波器中进行显示，观察经过匹配滤波之后的结果和原始信号的关系。

首先添加一个 **vector source** 模块，用于产生符号序列：

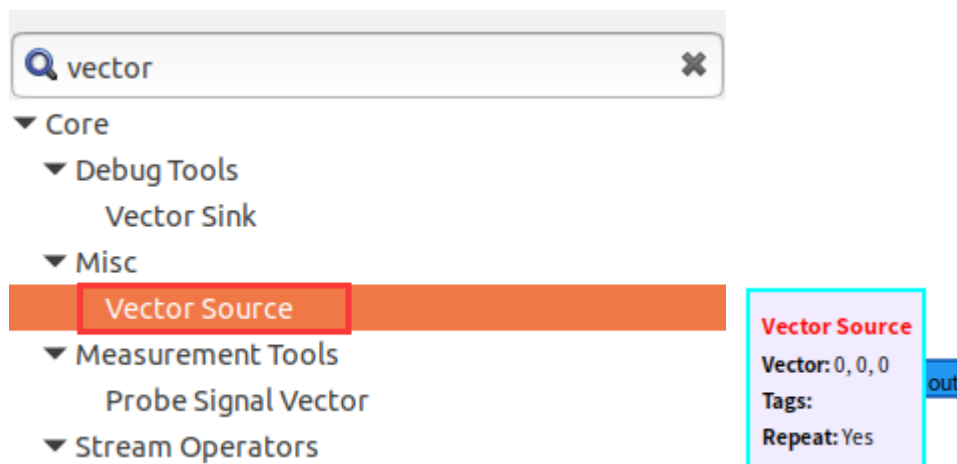


图 3-19 信号源

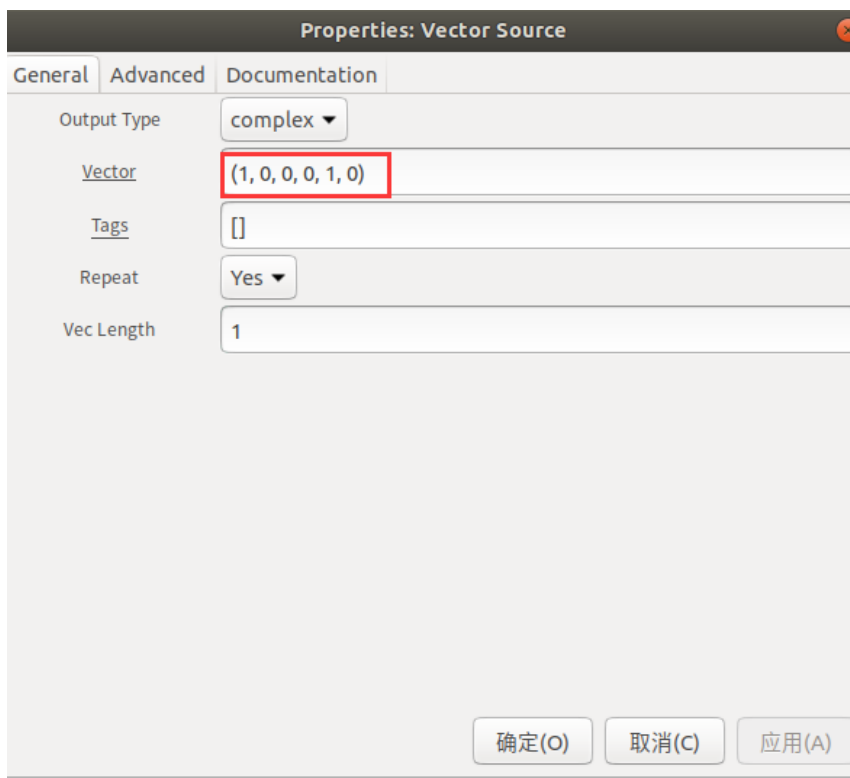


图 3-20 vector source 设置

然后进行上采样，这里使用 FIR 插值滤波器：

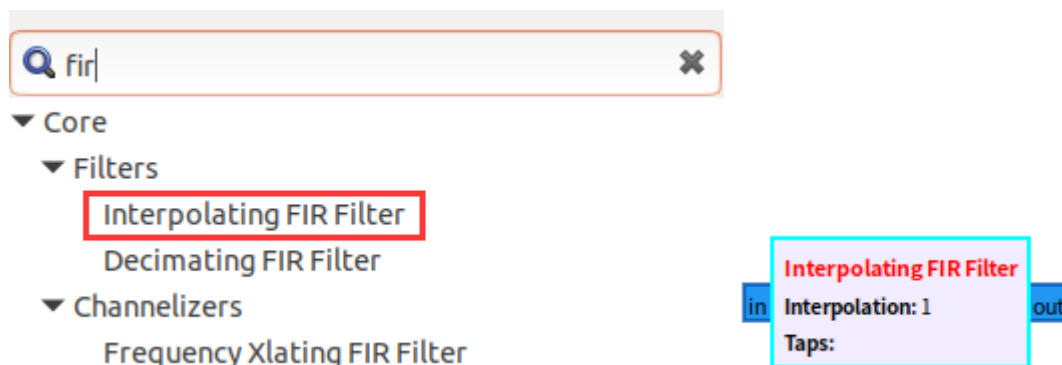


图 3-21 插值滤波器

插值滤波器进行 8 倍的插值。

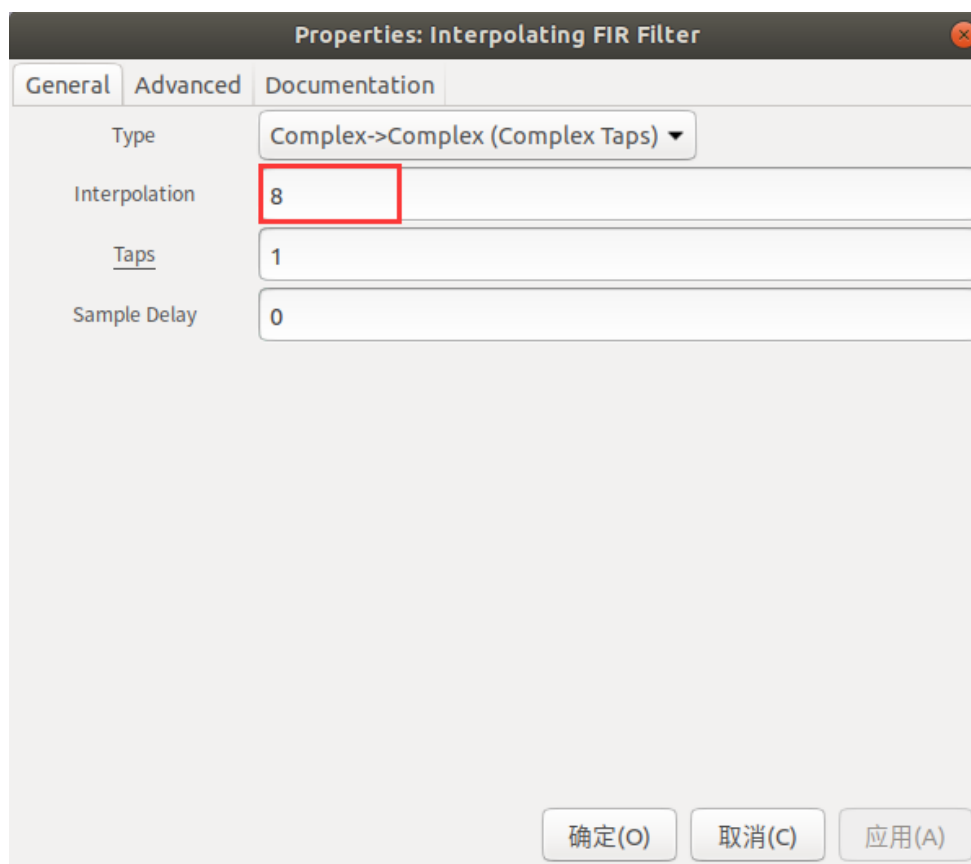


图 3-22 插值滤波器设置

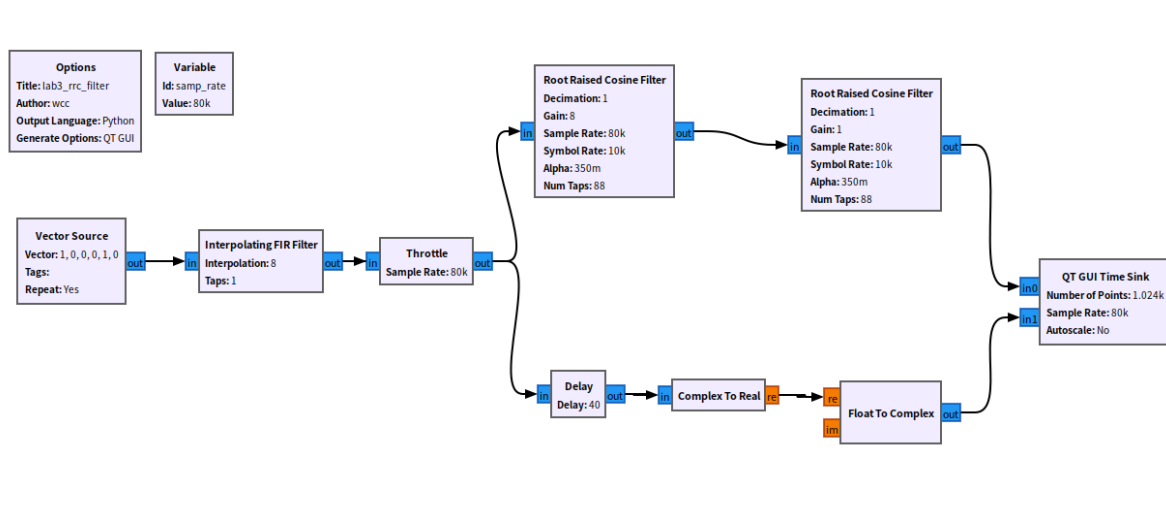


图 3-23 最终的 GRC 流程图

点击运行，可以仿真该 GRC，从仿真结果上可以看到，在接收到的信号用蓝色曲线标识，原始的符号用绿色标识出，从图中可以看出，使用了匹配滤波器之后，相邻符号间没有出现干扰，相邻符号可能会延展几个符号周期，但是在对应符号的位置上，对该符号的影响为 0。

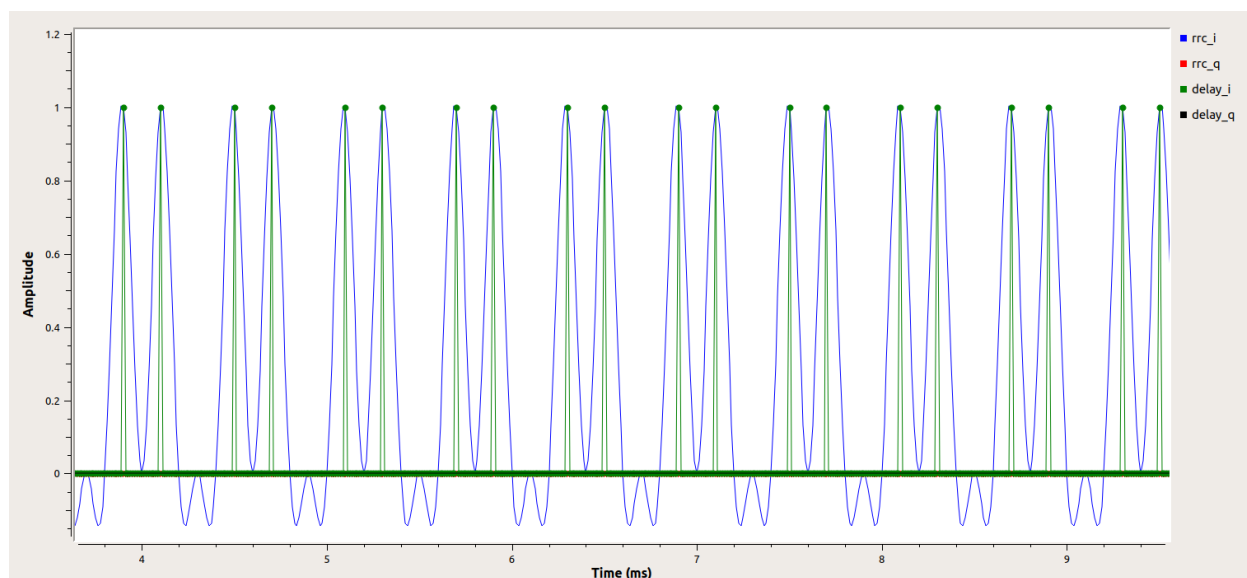


图 3-24 仿真的结果

3.4 总结

在本章当中，介绍了有关 GNU Radio 当中常用的滤波器模块的设计，具体包括低通滤波器，高通滤波器和根升余弦滤波器等。并对这几种滤波器都进行了仿真。可以看到使用 GNU Radio 能够十分简单地来设计滤波器，完成对信号的处理。

对于常见的带通、带阻滤波器其设计方法和低通滤波器都是相似的，在本章中没有具体介绍，感兴趣的用户可以自己实现带通和带阻滤波器。

第4章 GNU Radio 中的数据流标签和控制消息

GNU Radio 最初是一个流传输系统，没有其他机制能够在不同的模块之间进行数据的传递。数据流的传输模型是一种适用与采样点，比特位这种传输，但是并不是控制数据，元数据或者数据结构包传输最适合的传输系统机制。

为了解决 GNU Radio 当中的控制消息，元数据和数据流在整个系统中能够正常的传输，GNU Radio 引入了 Stream Tags 数据流标签和 Message Passing 消息传递两种机制。

4.1 数据流标签 Stream Tags

数据流标签是和主数据流并行运行的同步数据流，数据流标签可以由 GNU Radio 当中特定的模块来进行添加。从这个特定的模块开始随着特定的样本开始沿着数据流进行流动，直到这个数据流运行到 GNU Radio 当中的 sink 模块或者被另外的一个模块强制停止。

数据流标签为数据流当中的特定的数据进行标记，并且形成一个键值对 (key:value)，键标识所代表的内容，而值保留标签所标识的流数据。Key 和 value 都是多态类型(PMT, Polymorphic Types),其中 key 是 PMT 的符号而 value 可以实任意的 PMT 类型，因此可以用来承载我们想要传输的任意数据类型的数据。

标签的另外一部分是 srcid(源 ID)，用于标识创建这个标签的源模块，通常是这个模块的别名。

4.1.1 在 GNU Radio 当中给数据打上标签

在上面的介绍中，可能对流标签的了解不是太直观，简单来说就是给数据流当中的某些特殊数据进行标记。接下来我们使用 GNU Radio 来实际演示一下如何给数据打标签。

首先新建一个 GRC 流图，然后添加一个数据源模块，在这里我们使用 vector source，如图 4-1 所示

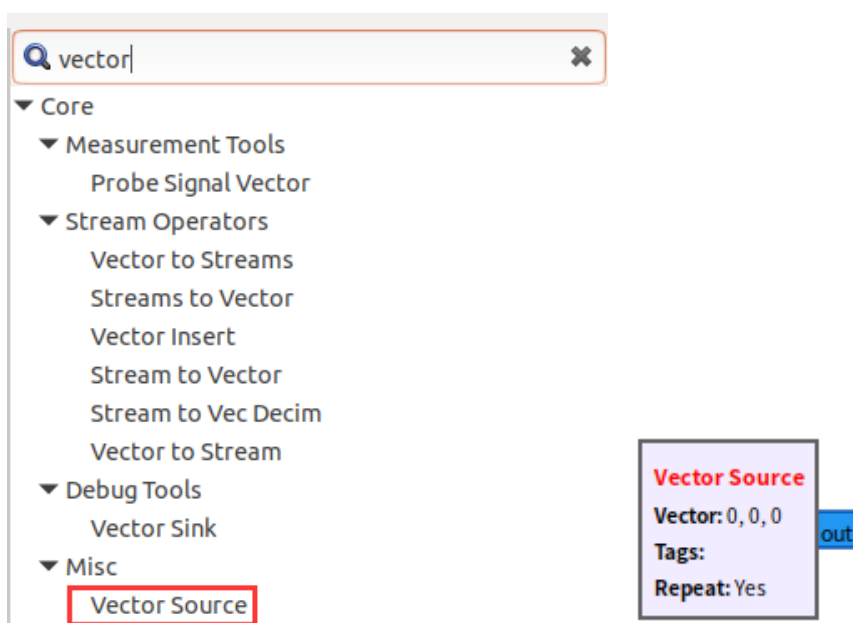


图 4-1 Vector 源模块

对该模块进行配置，使其产生 0 到 9 的递增数。如图 4-2 所示

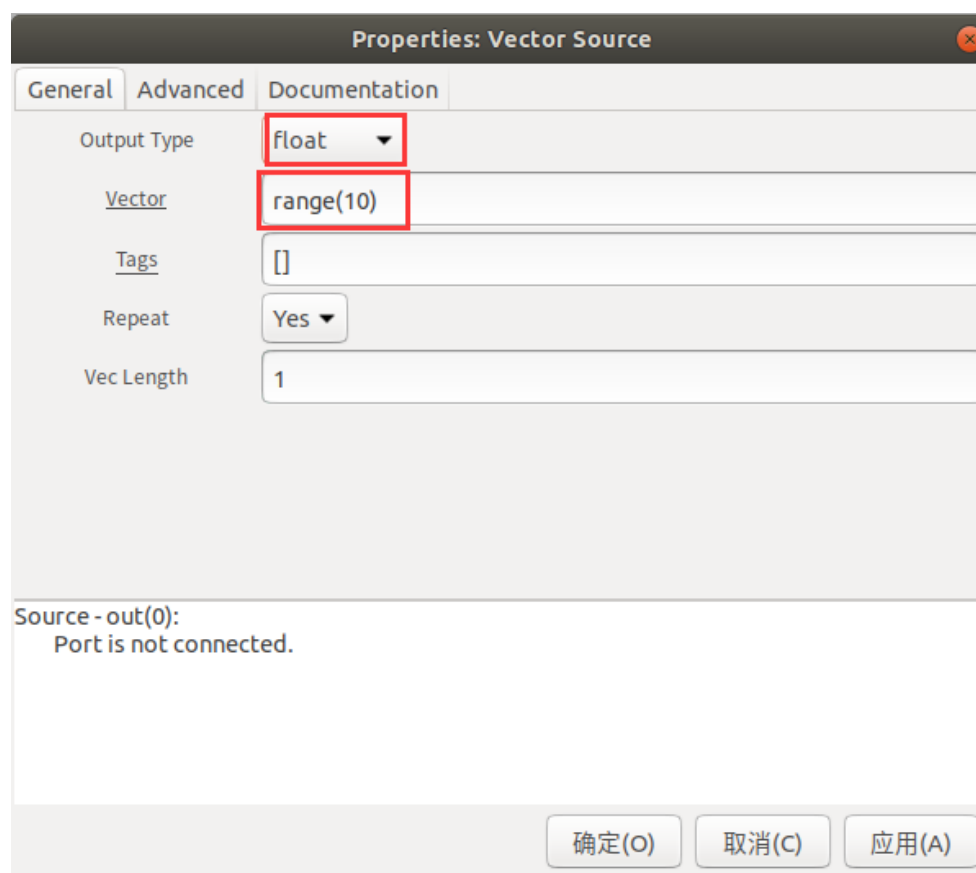


图 4-2 数据源模块设置

然后添加一个增加标签的模块用于给这个数据流打上标签，其设置如图 4-4 所示。

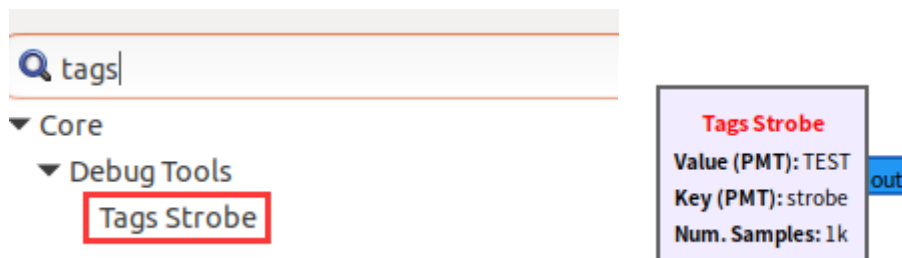


图 4-3 标签模块

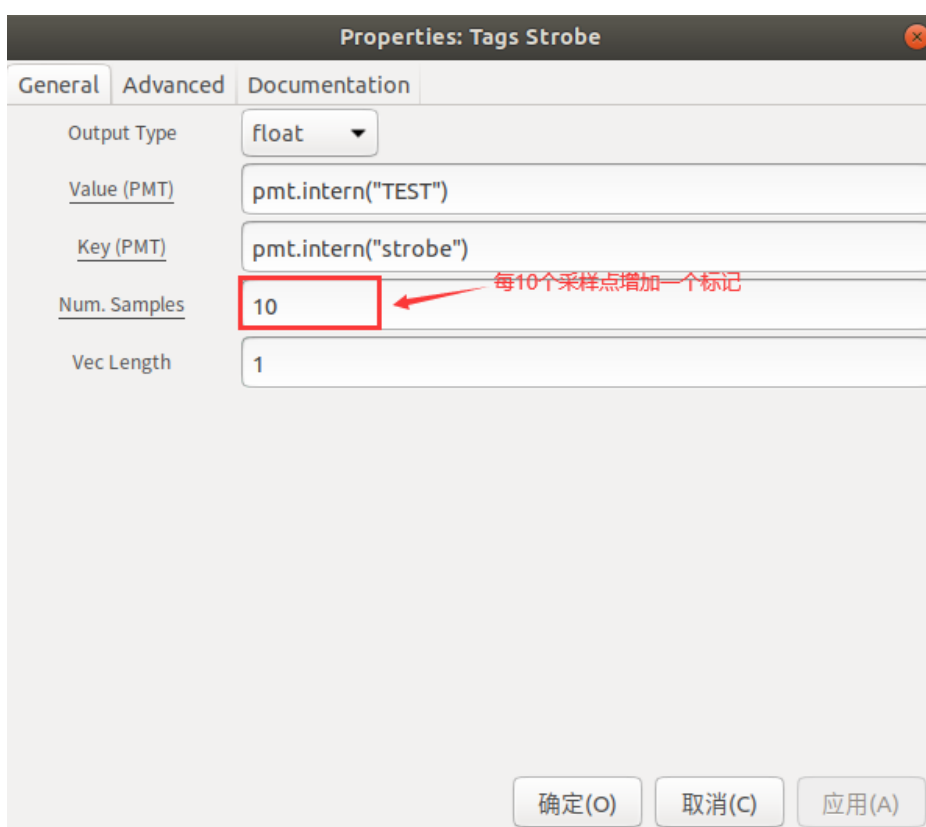


图 4-4 标签设置

然后把数据源模块和标签模块进行相加，这样就能够把标签添加到数据源模块之上了。

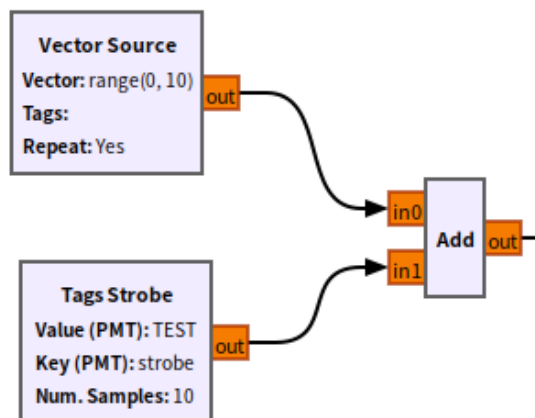


图 4-5 给数据流增加标签

为了能够更加直观地表示出加上了标签之后的数据流，使用一个 repeat 模块将原始的数据流进行扩充。

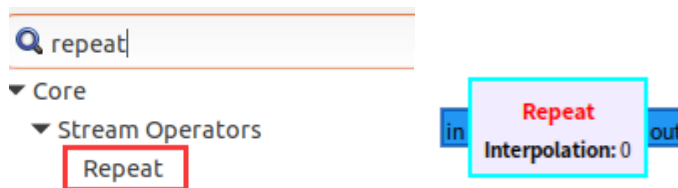


图 4-6

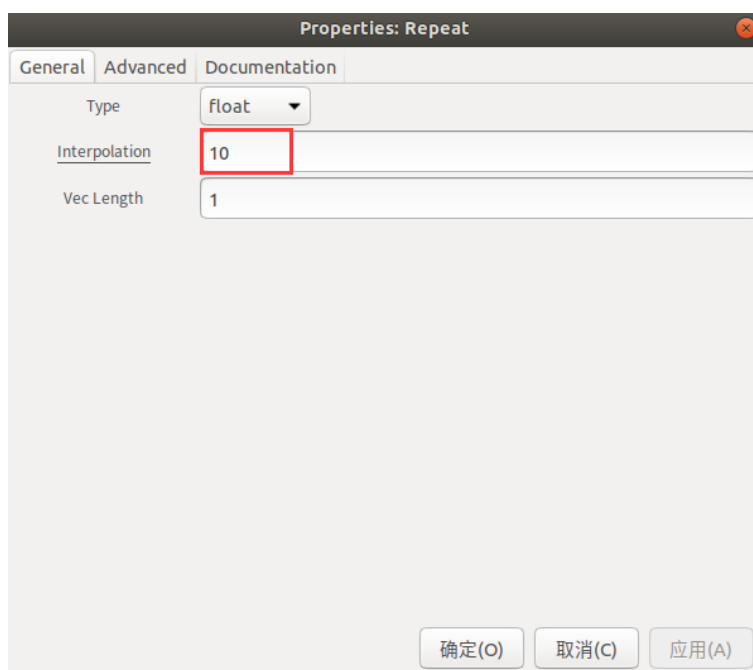


图 4-7 将原始数据重复 10 次

增加这个模块折后，会将原始数据流当中的采样点数据复制 10 份。

最终的 GRC 流图如下：

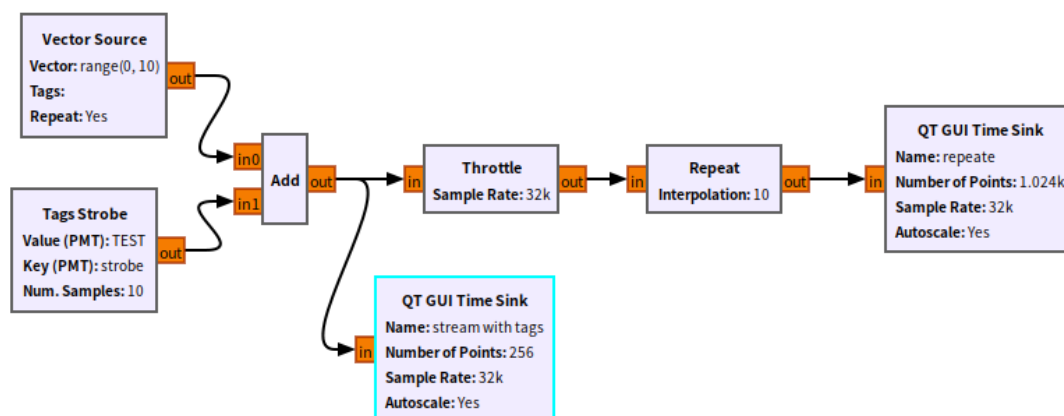


图 4-8 最终的 GRC 流图

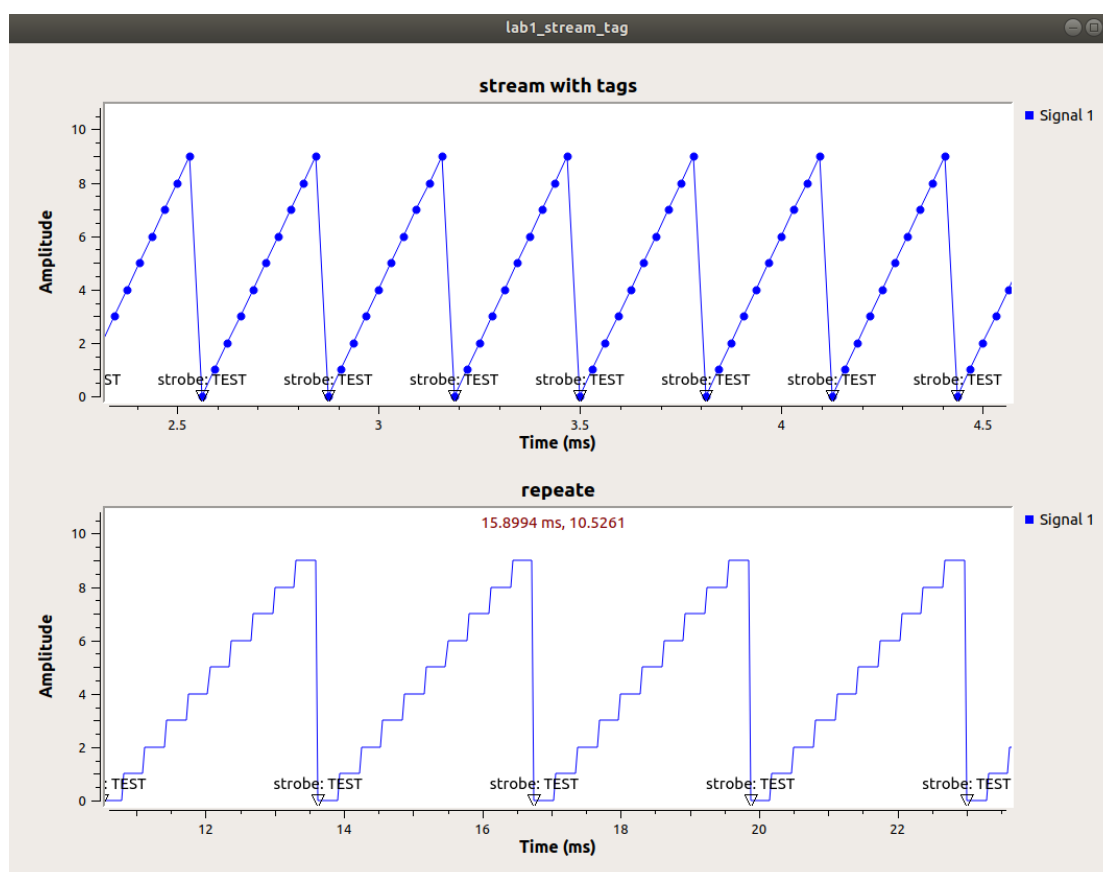


图 4-9 仿真结果

从仿真结果上看，在增加了标签之后，每隔 10 个点会增加一个标签，这个标签也就是我们在前面设计的值。

与此同时，可以看到在下面的时域波形当中是将数据进行了 `repeat` 之后的结果，可以看到对原始采样点进行复制过后，标签并没有随着复制的点进行移动，这是因为标签只会对样本当中特定的样本点进行标记。

4.2 消息传递

在前面介绍了流标签 `Stream Tags`，标签能够和数据流中特定的样本关联，并一起传递。这种模型允许其他模块来标识在特定采样点上的事件，这种模型的主要限制是标签流只能在模块内部进行访问，并且只有一个方向的流动。它的好处是与数据同步。

在某些时候，还需要一个更加通用的消息传输系统，比如下游模块想要与上游模块进行通信，又或者能够为外部的应用程序和 `GNU Radio` 来进行异步的通信。

在前面的章节当中，我们在介绍数据类型的时候，看过一个数据类型的图，如图 4-10 所示。

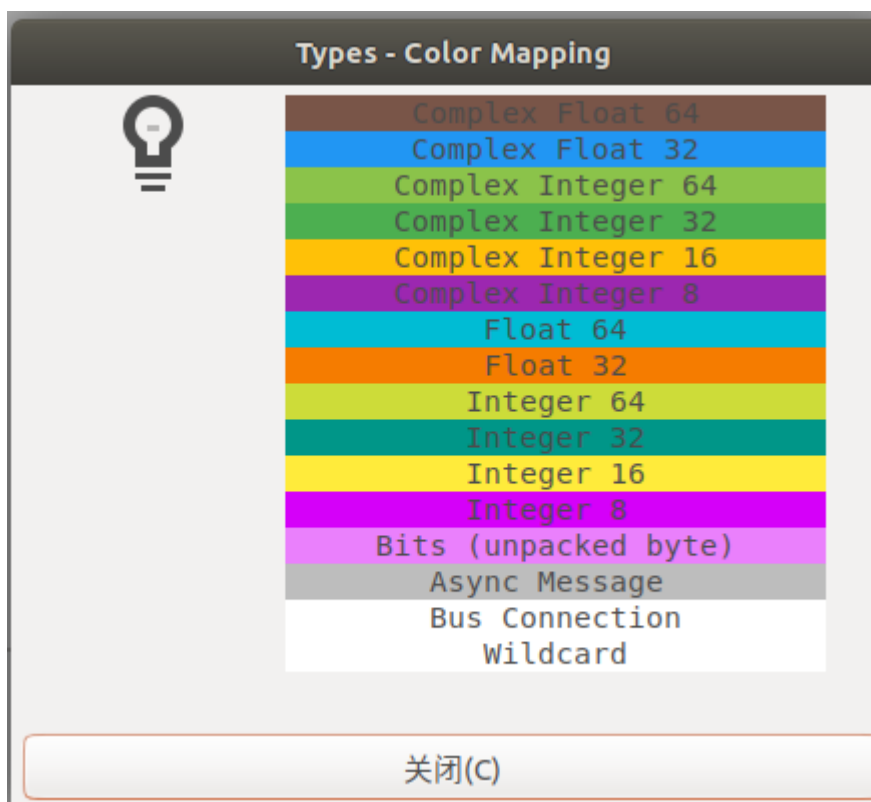


图 4-10 GNU Radio 中的数据类型

在之前我们介绍这些数据类型的时候，值介绍了上面的数学上的数据类型，其实 `GNU Radio` 中还有异步消息这种结构，在对应的模块上反应出来就是一个灰

色的接口。

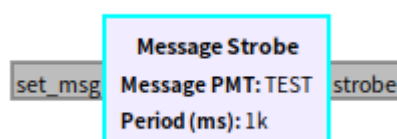


图 4-11 消息类型接口

接下来，我们就试着在 GNU Radio 当中完成一个简单的消息传递的例子。

4.2.1 GNU Radio 中消息传递

首先产生一个用于产生消息的模块

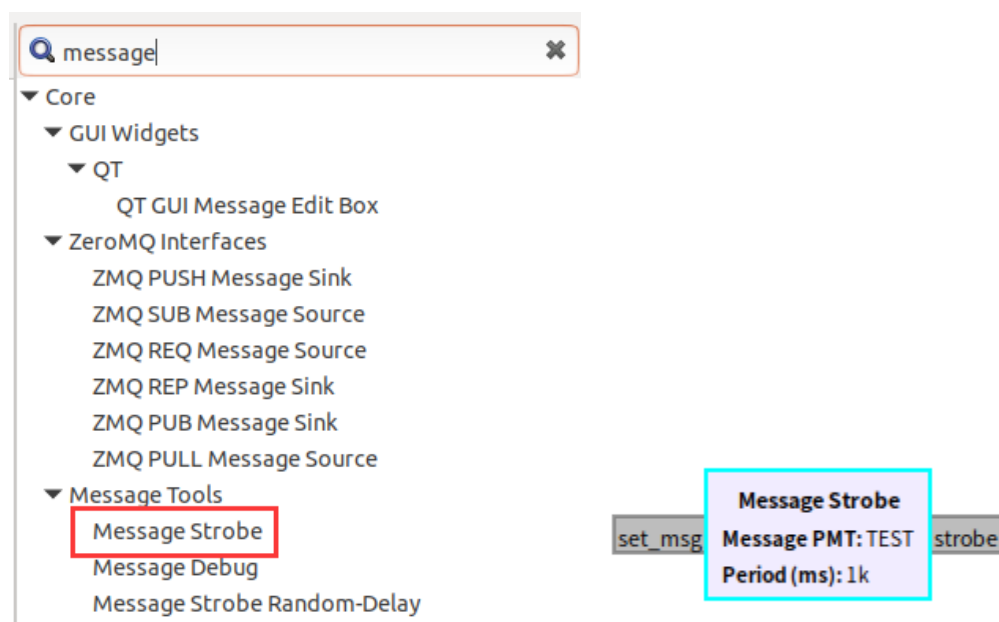


图 4-12 消息产生模块

其设置如图 4-15 所示

在这个模块当中，产生的消息信号是一个 hello world，引入了一个需要引入 numpy 某块。就和在 python 当中引入对应的工具包模块是一致的。

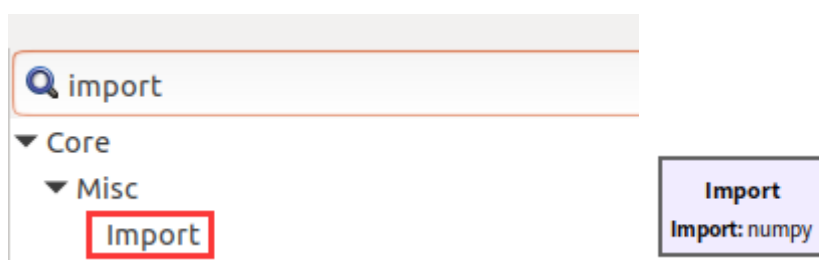


图 4-13

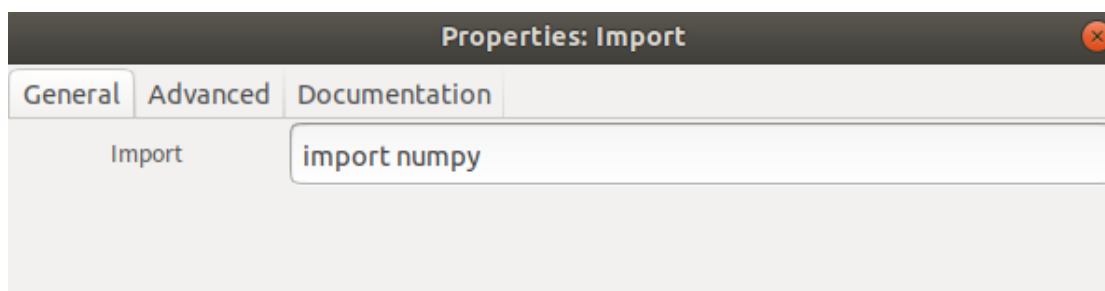


图 4-14

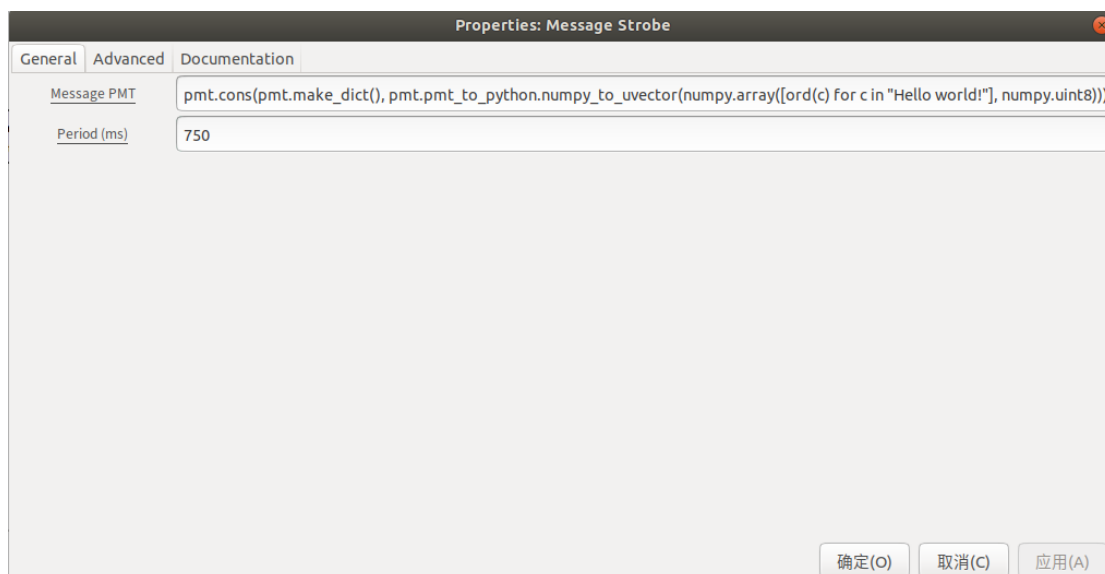


图 4-15

产生消息信号的语句如下：

```
pmt.cons(pmt.make_dict(), pmt.pmt_to_python.numpy_to_uvector(
numpy.array([ord(c) for c in "Hello world!"], numpy.uint8)))
```

这样就会每隔 750ms 产生一个消息信号，这种产生的消息信号被称位 PDU，PDU 也就是 protocol data unit，协议数据单元。这种 PDU 能够转换为带标签的数

据流，与此同时，带标签的数据流也能够转换为 PDU。接下来将 PDU 转换成带标签的数据流，并将带标签的数据流重新转换成消息

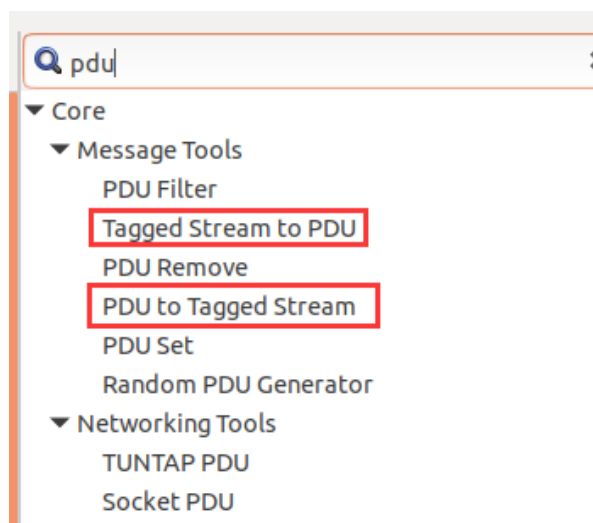


图 4-16 带标签的数据流与消息转换



图 4-17

最终的转换的流图如下图所示：

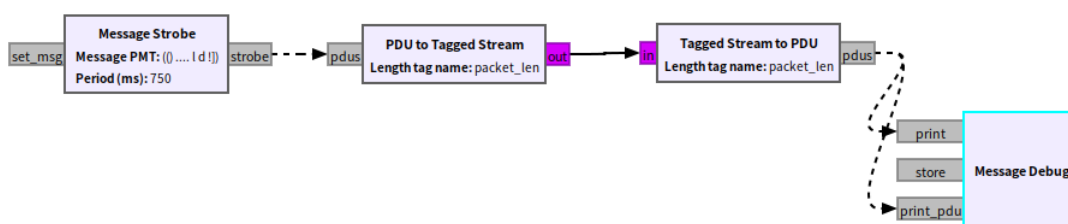


图 4-18 转换流图

可以看到消息信号的连线是用虚线来标识的，在 Message Debug 模块当中，最上面的 print 将会把消息按照字符串的形式打印出来，而下面的 print_pdu 将会把具体的数据用 16 进制的形式标识出来。

运行仿真，然后可以观察在窗口中打打印信息。

```

***** MESSAGE DEBUG PRINT *****
()[Hello world!])
*****
* MESSAGE DEBUG PRINT PDU VERBOSE *
()
pdu_length = 12
contents =
0000: 48 65 6c 6c 6f 20 77 6f 72 6c 64 21
*****
***** MESSAGE DEBUG PRINT *****
()[Hello world!])
*****
* MESSAGE DEBUG PRINT PDU VERBOSE *
()
pdu_length = 12
contents =
0000: 48 65 6c 6c 6f 20 77 6f 72 6c 64 21
*****

```

图 4-19

可以看到，每隔一段时间就能够在窗口上打印出前面生成的文字。

4.3 基于消息和流标签构建数据帧

在数字通信中，一般都需要根据协议来完成数字通信的调制与解调。那么在 GNU Radio 当中有没有什么办法能够比较简单地来构建一个自己的协议呢，比如添加帧头等工作。

在 GNU Radio 3.8 之前的版本中有两个模块可以用来实现这种工作，这两个模块分别是 Packet Encoder 和 Packet Decoder。但在 Gnu Radio 3.8 之后，这两个模块被移除了。因此想要在 GNU Radio 3.8 以及之后的版本中想要来构建自己的数据帧，就需要使用到前面介绍的数据流标签和消息传递了。

接下来，我们会搭建一个简单的数据帧，来完成一个简单的数据帧的组帧和解帧的过程。

4.3.1 使用 GNU Radio 构建数据帧

一般来说构建一个数据帧，需要使用到一个数据源，在这个例子当中只是为了演示如何来构建一个数据帧，为了简单起见，在这里使用一个消息产生模块，每隔一段时间产生一个消息来作为数据源。

和前面的实验一致，要产生一个消息信号源，需要导入 numpy 库，然后生成一个 hello world 的消息。然后将这个消息转换为带标签的数据流。这个数据流的标签的键值对的值，就是这个消息的实际长度。

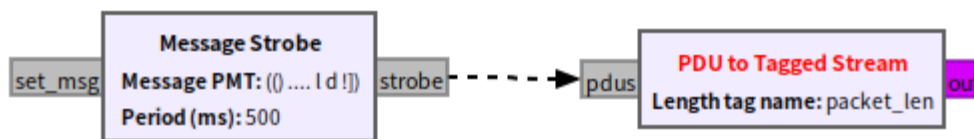


图 4-20 将会消息转换为带标签到的数据流

然后我们需要给这个转换之后的数据流添加帧头，也就是添加 header。在这里需要使用到一个用于给带有标签的数据流添加 header 的模块，Protocol Formatter 模块。

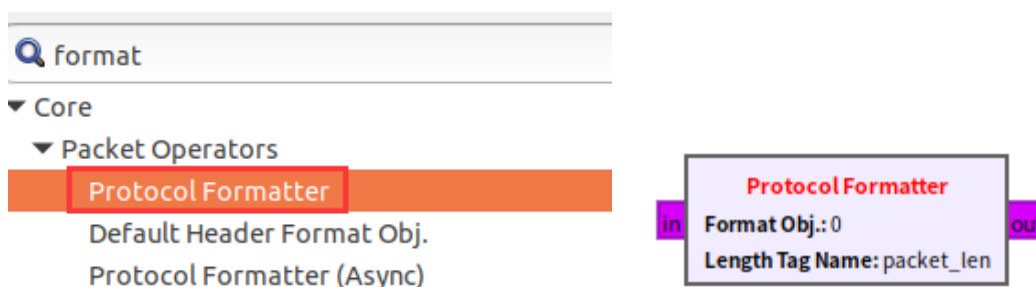


图 4-21

在这个模块当中，需要我们填入一个具体的 format 的对象，用户若对 GNU Radio 的底层信息比较数据可以使用 api 函数来完成。这里我们使用默认的 format 对象来完成，添加一个 parameter 模块，其模块的设置如所示：

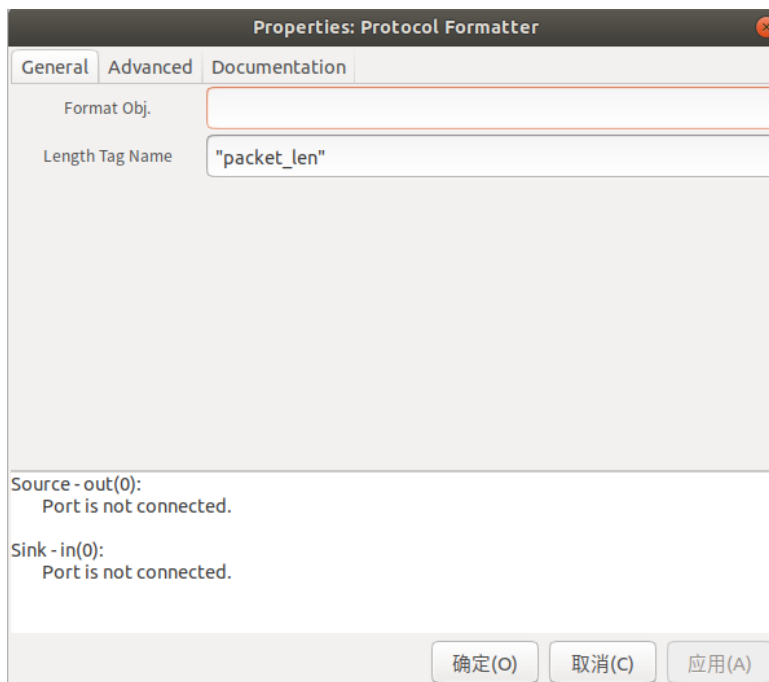


图 4-22 需要填入具体的 format 对象

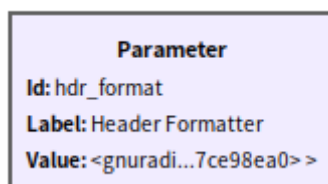


图 4-23 参数模块

```
digital.header_format_default(digital.packet_utils.default_access_code, 0)
```

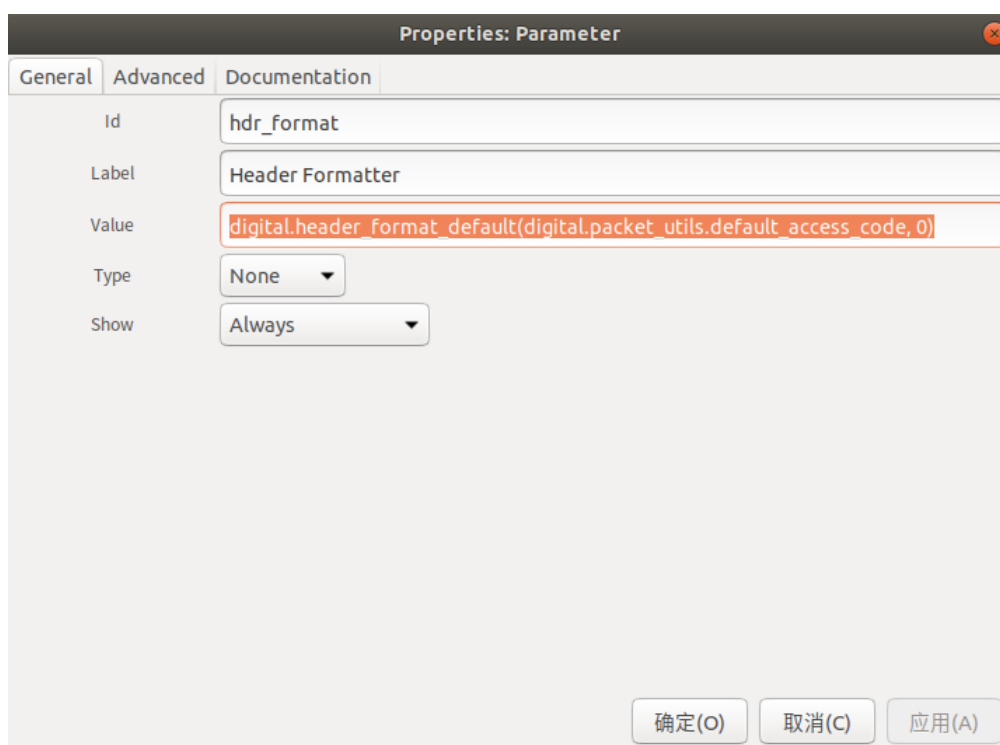


图 4-24 添加一个 parameter 模块

然后把这个 parameter 模块的模块名填写到 protocol format 模块当中。

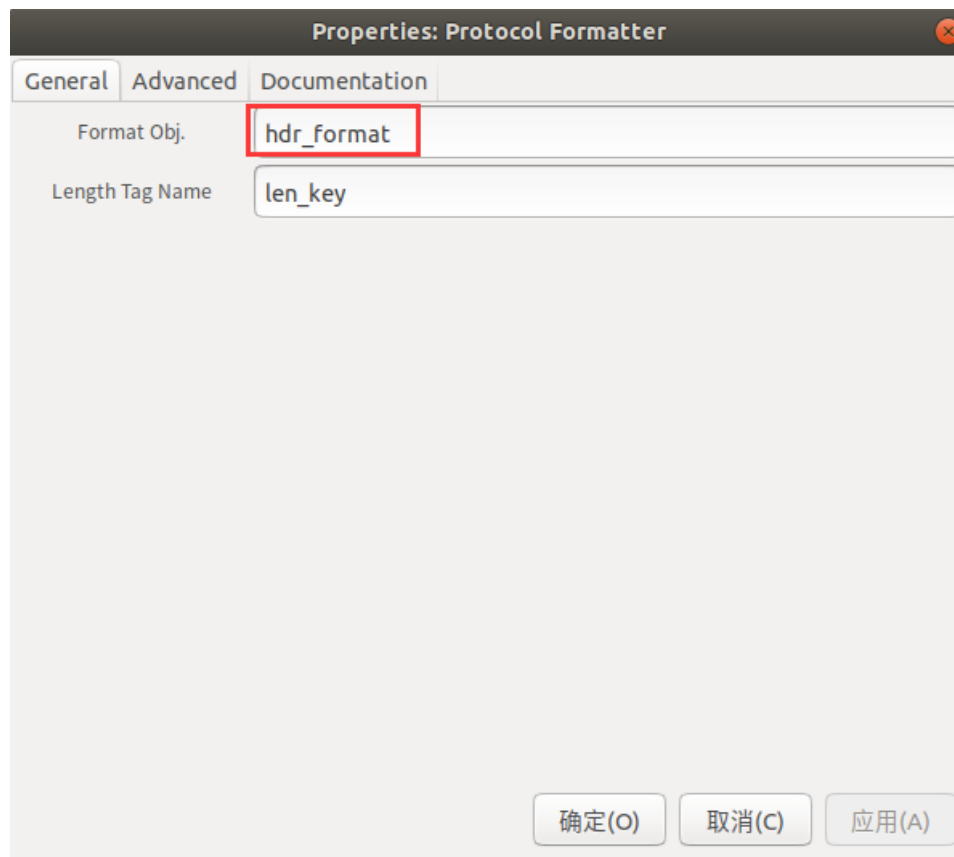


图 4-25 将 format 对象填入到 protocol formatter 中

这个 format 对象会给带标签的数据流生成一个帧头，然后我们需要将这个帧头与原始数据流进行合并，组成一个完整的数据。这里可以使用 Tagged Stream Mux 模块。

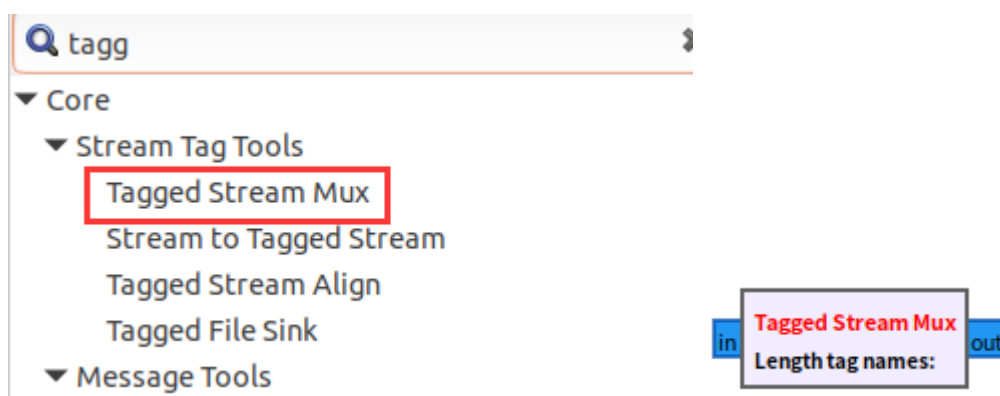


图 4-26 数据流合并模块

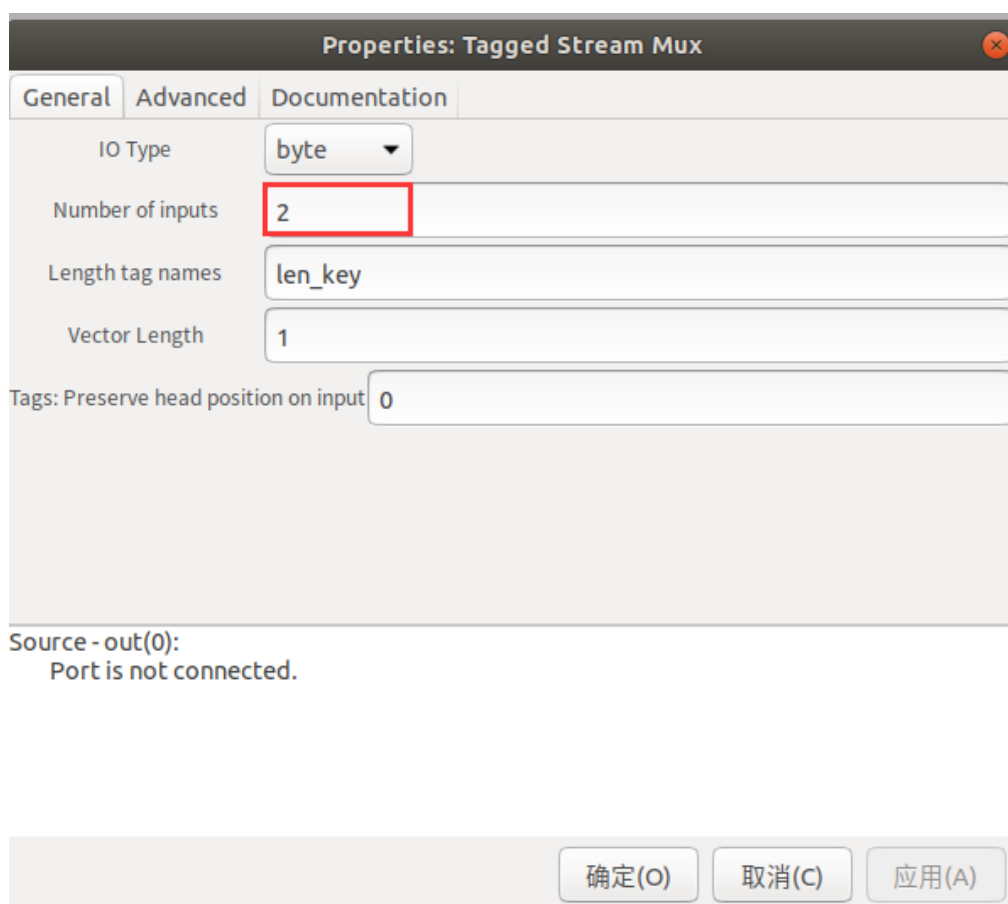


图 4-27 数据流合并模块

该模块能够将输入的带标签的数据流合并，并且将原始的标签用新生成的数据流长度的标签替换。

到这里我们就生成了一个带帧头的数据流了。

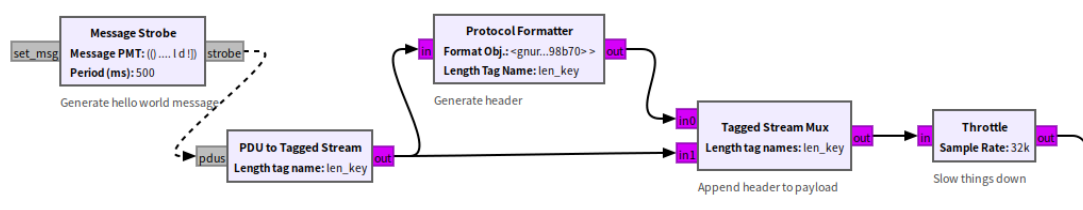


图 4-28 带帧头的数据流

可以看到当前的数据流中的数据还是以字节的形式来传递的，在实际的传输中，比如星座图的映射当中，需要将数据转换成 bit 的形式。因此我们需要使用一个模块将数据转换成 bit。

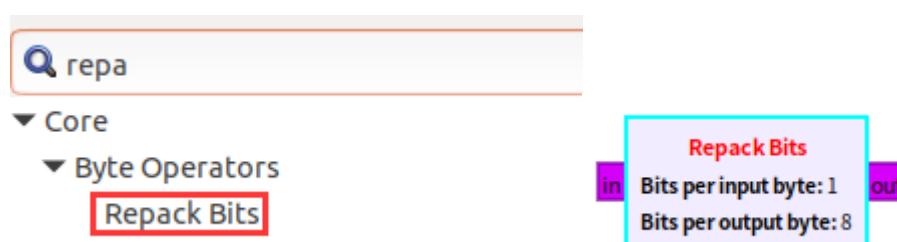


图 4-29

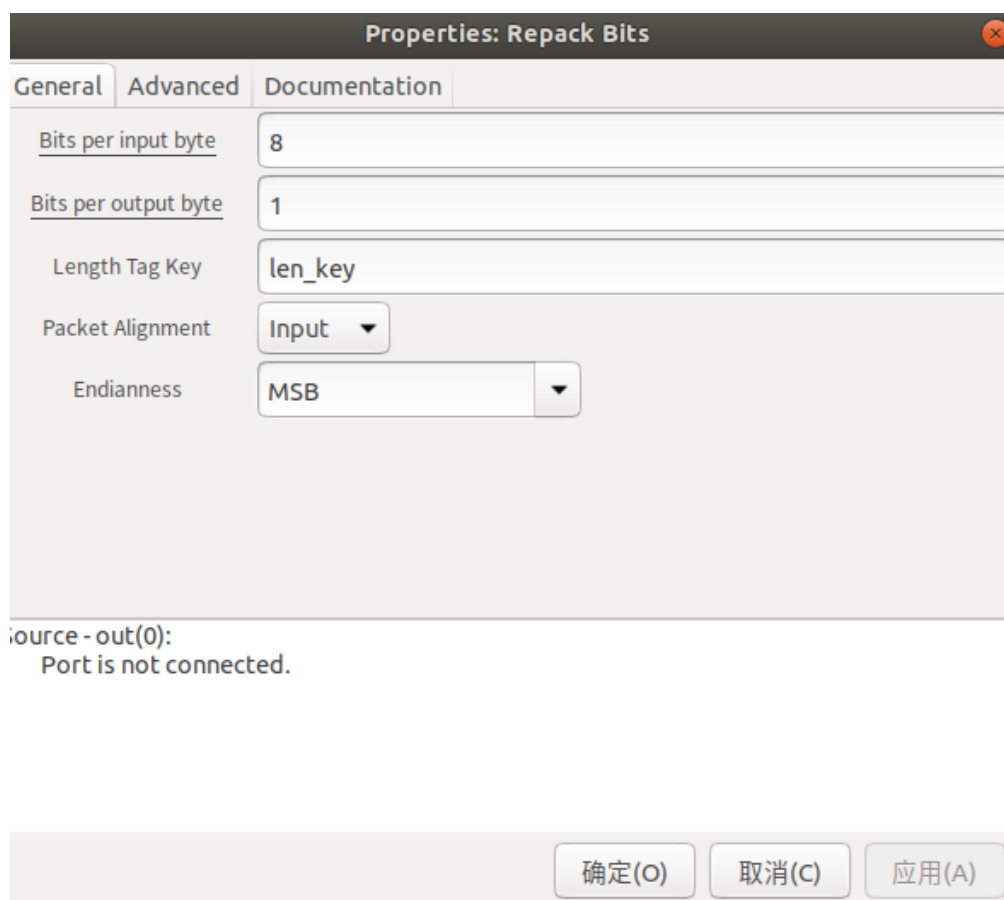


图 4-30 Repack Bits 设置

到这里我们就成功的生成了 bit 流了。接下来需要考虑的就是如何来复原原始的信号。

像前面所介绍的，我们在传输数据的时候，在原始数据流加上了帧头。在接收这个数据帧的时候，我们只需要正确地检测到了数据帧的帧头，就能说明当前位置有一个数据帧，接下来的地方就是数据所在的位置。因此需要一个模块用来检测数据帧的帧头。

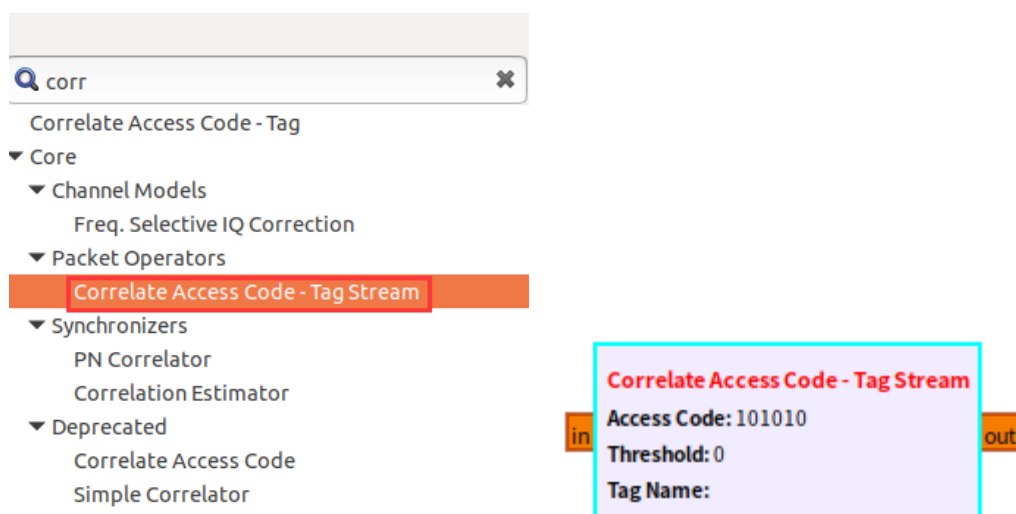


图 4-31 检测帧头模块

该模块能够检测出，在之前添加的 access code，并将帧头从数据流中剔除。其设置如下：

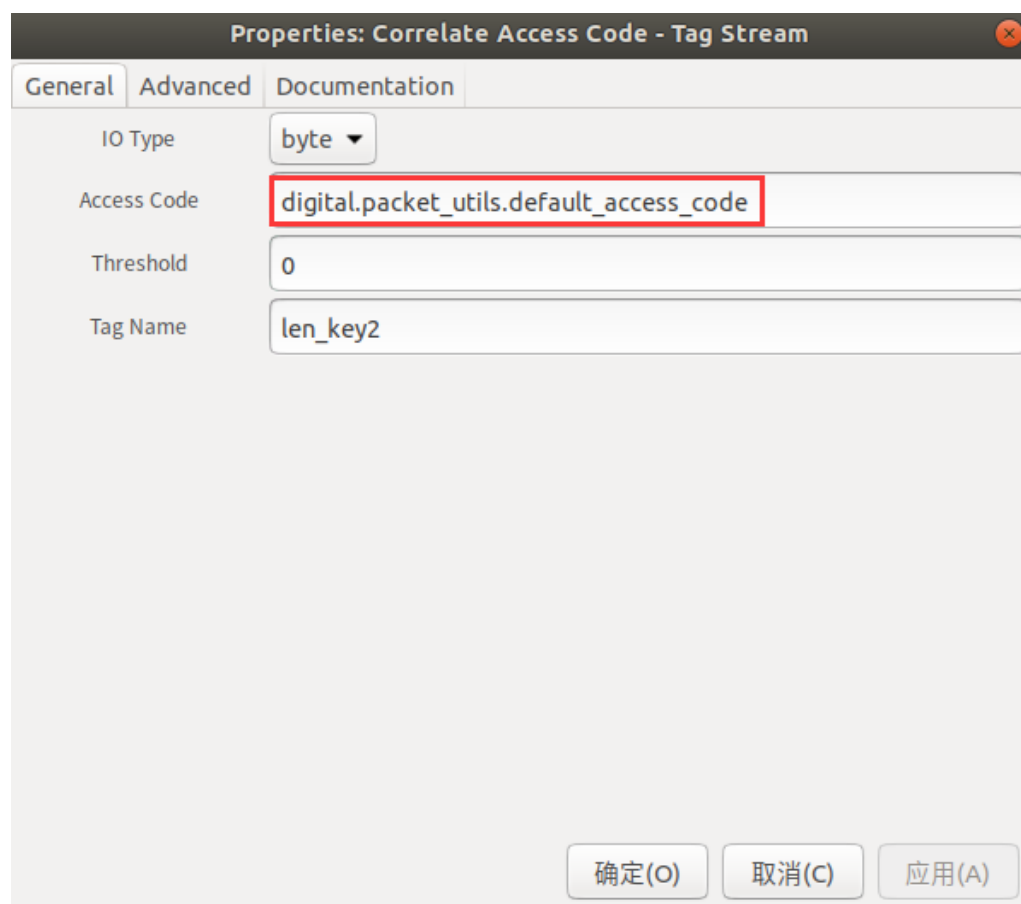


图 4-32 帧头检测模块

去掉帧头之后，我们就可以将 bit 流数据复原成原始的数据帧了。在这里需要再次使用到 Repack Bits 模块，将 bit 数据转换成数据帧。

最后就可以通过把带标签的数据流转换成消息然后通过调试工具打印到窗口当中了。

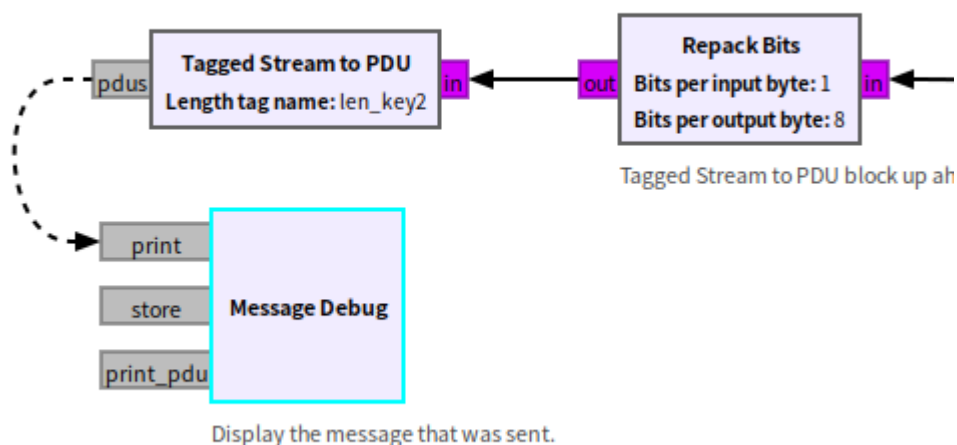


图 4-33 获取消息

为了能够观察到带有标签的数据流，我们将比特流数据进行一个映射。需要使用到 Chunks to Symbols 这个模块，这个模块在进行星座图的映射的时候，会经常使用到。

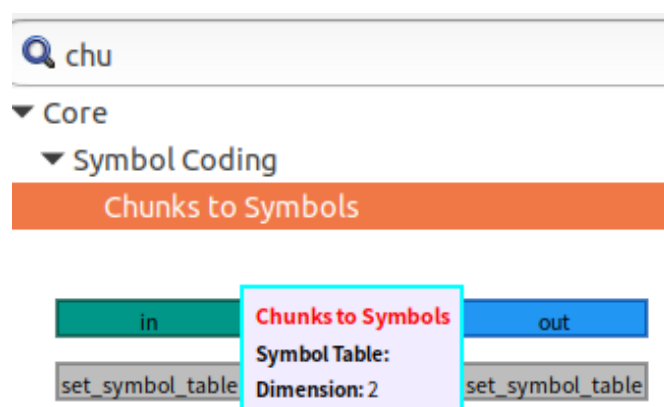


图 4-34

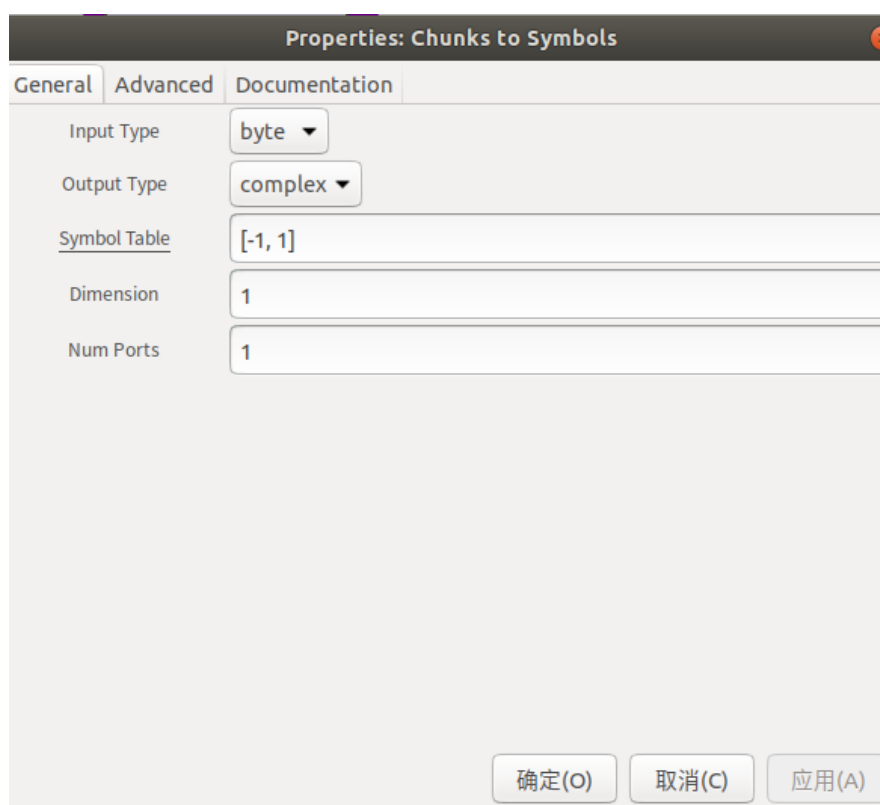


图 4-35 星座图映射模块

我们可以使用一个示波器，来观察这个映射的结果。

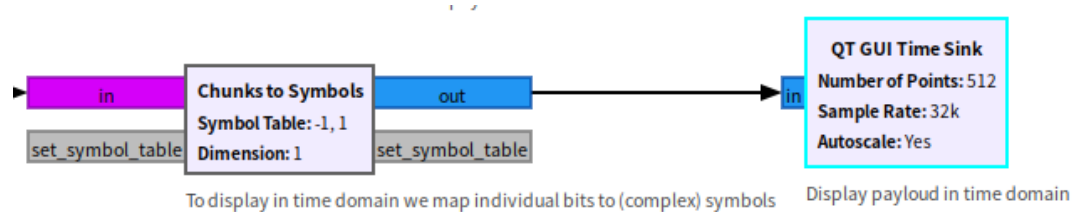


图 4-36 观察星座映射结果

最终，整个 GRC 流图如图 4-37 所示

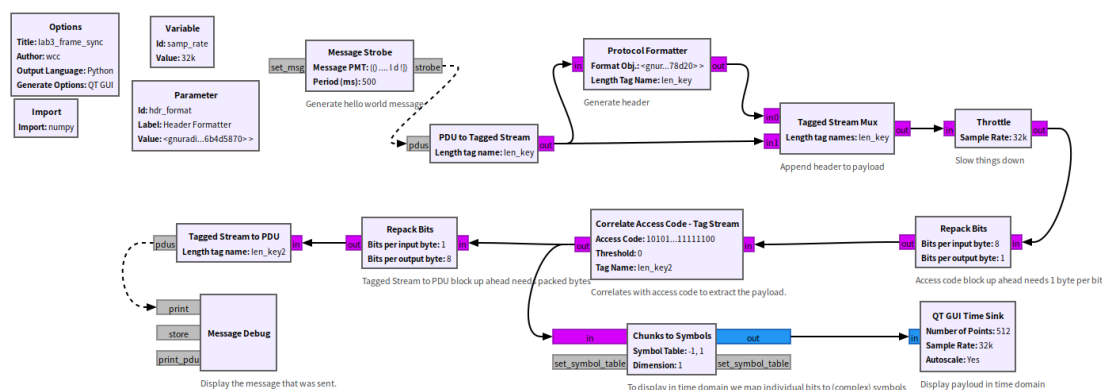


图 4-37 整个 GRC 流程图

点击运行仿真，可以查看运行的结果，用户可以分别将原始的消息信号和经过帧同步之后的消息信号分别打印出来观察结果是否一致。

```

***** MESSAGE DEBUG PRINT *****
(().#[Hello world!])
*****
***** MESSAGE DEBUG PRINT *****
(().#[Hello world!])
*****
***** MESSAGE DEBUG PRINT *****
(().#[Hello world!])
*****
***** MESSAGE DEBUG PRINT *****
(().#[Hello world!])
*****
***** MESSAGE DEBUG PRINT *****
(().#[Hello world!])
*****

```

图 4-38 经过帧同步之后的打印消息

除了观察打印消息之外，还可以观察映射后的时域波形，如图 4-39 所示

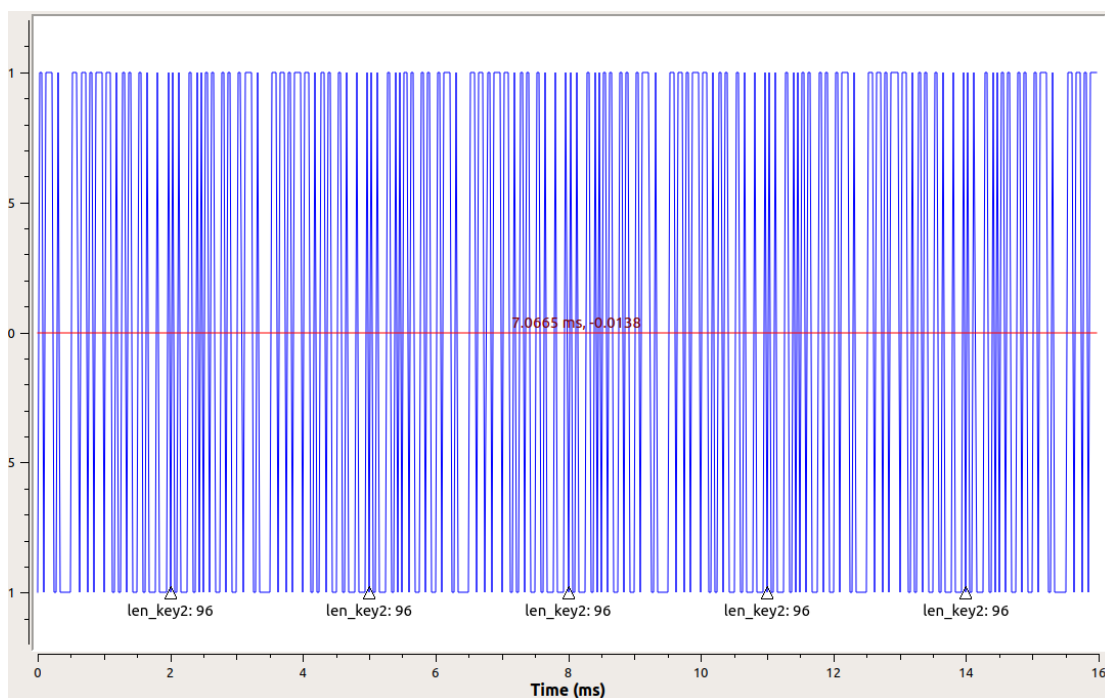


图 4-39 经过映射之后的时域波形

可以看到，时域波形上对于每一个数据帧都有一个标签，能够很好地区分每一个数据帧。

4.4 总结

在这一章当中，主要介绍了 GNU Radio 当中的两个机制，消息传递和流标签。简单的了解了一下消息传递和流标签的作用。并且使用消息和流标签完成了数据帧的组帧和解帧操作，这些在数字通信当中都是十分基础和重要的操作，构建新的协议或者实现一个已有协议，其实都是如何来设计一个帧的过程，在 GNU Radio 当中都需要使用到流标签和消息等特性。

关于流标签和消息的详细内容，可以参考 GNU Radio 官方 wiki 上的内容。

https://wiki.gnuradio.org/index.php/Message_Passing

https://wiki.gnuradio.org/index.php/Stream_Tags

https://www.gnuradio.org/doc/doxygen/page_packet_comms.html

第5章 AM-DSB-TC 的调制与解调

5.1 常规调幅原理

基带信号 $s_i(t)$ 和直流偏置信号 A_0 相加，然后与频率为 f_c 的载波进行混频得到了已调信号 $s_{am-dsb-tc}(t)$ 。AM-DSB-TC 调制器的结构如图 5-1 所示

$$s_{am-dsb-tc}(t) = (s_i(t) + A_0) \cos 2\pi f_c t \quad 5-1$$

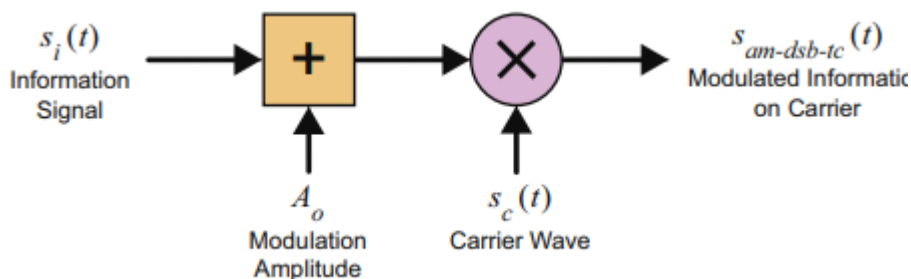


图 5-1 AM 调制器结构

知道了 AM-DSB-TC 调制的基本原理之后，就可以来举个例子看看具体的调制过程是怎么样的，在这里用 AM-DSB-TC 的调制方法，来调制一个单音信号为例。

5.1.1 单音基带信号调制

单音信号的幅度为 A_i 频率为 f_i

$$s_i(t) = A_i \cos(2\pi f_i t) = A_i \cos(\omega_i t) \quad 5-2$$

载波信号的幅度为 A_c 频率为 f_c

$$s_c(t) = A_c \cos(2\pi f_c t) = A_c \cos(\omega_c t) \quad 5-3$$

已调信号为：

$$s_{am-dsb-tc}(t) = \left[A_o + A_i \cos(\omega_i t) \right] A_c \cos(\omega_c t) \quad 5-4$$

在这里可以使用积化和差公式：

$$\begin{aligned}
 s_{am-dsb-tc}(t) &= A_o A_c \cos(\omega_c)t + \frac{A_i A_c}{2} \cos(\omega_c - \omega_i)t + \frac{A_i A_c}{2} \cos(\omega_c + \omega_i)t \\
 &= A_o A_c \cos(\omega_c)t + \frac{A_i A_c}{2} \left(\cos(\omega_c - \omega_i)t + \cos(\omega_c + \omega_i)t \right) .
 \end{aligned}
 \tag{5-5}$$

调制的过程如图 5-2 所示，可以看到经过 AM-DSB-TC 调制过后，在已调信号的频谱中产生了三个频率分量，分别是处于下边带的 $f_c - f_i$ 处于上边带的 $f_c + f_i$ 和载波频率分量 f_c 。在已调信号中，把包络绘制出来，这里包络的最大幅度与上方基带信号的最大幅度刚好对应起来。

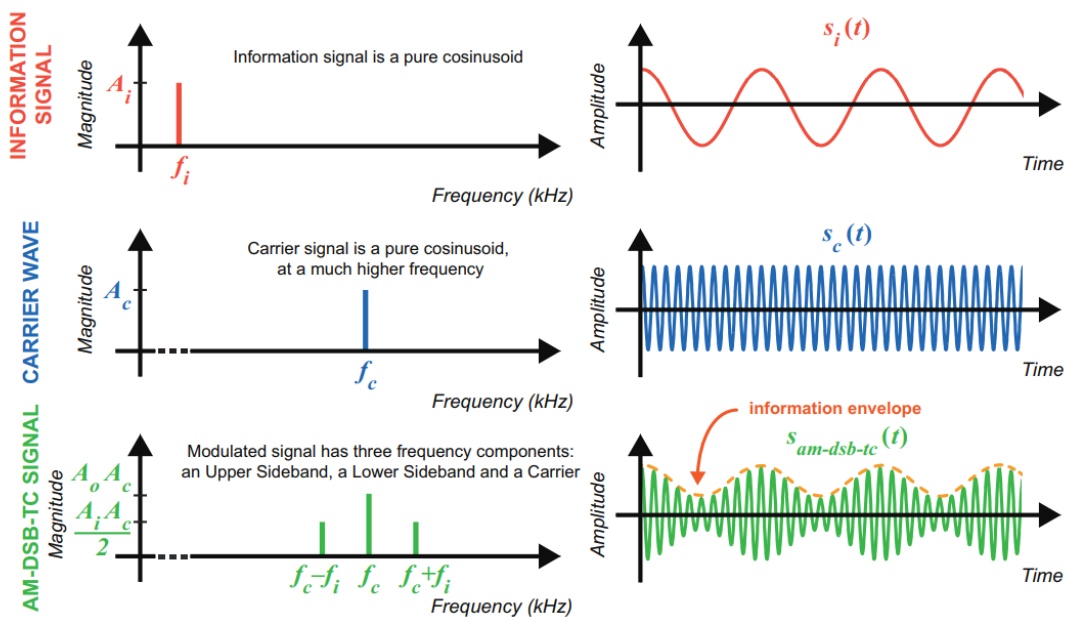


图 5-2 AM 调制过程

接下来需要考虑的是，AM-DSB-TC 在什么时候会出现异常，还是考虑上面的已调信号的表达式，可以对其进行简单的变形。

$$\begin{aligned}
 s_{am-dsb-tc}(t) &= A_o \left[1 + m \cos(\omega_i t) \right] A_c \cos(\omega_c t) \\
 &= A_o A_c \cos(\omega_c t) + \frac{A_o A_c m}{2} \left(\cos(\omega_c - \omega_i)t + \cos(\omega_c + \omega_i)t \right) \\
 \text{where: } m &= \frac{A_i}{A_o} .
 \end{aligned}$$

5-6

在这里使用一个调制系数 m 来表示 AM-DSB-TC 的调制程度，这个调制系数就是基带信号的振幅和载波频率的振幅的比值。下图表示的是不同调制系数下已调信号的时域波形，可以看到随着调制系数的增大，已调信号的波形将会失调。所以，当调制系数为 1 的时候称作满调幅。

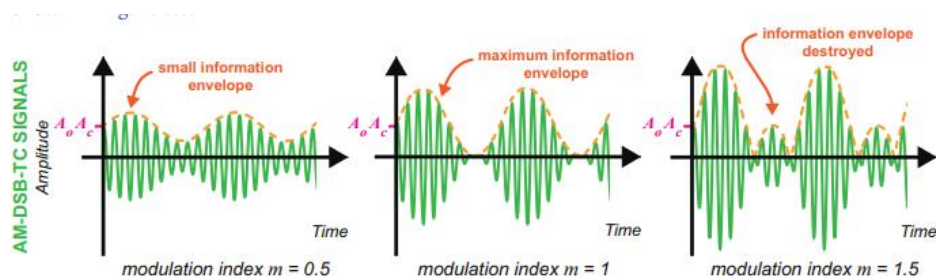


图 5-3 不同调制系数下的时域波形

5.1.2 基带信号的调制

在前面介绍了一个单音信号的常规调幅，在实际中，基带信号一般不会是一个单音信号这么简单，基带信号一般都有一定的带宽。若对基带信号使用常规调幅，其过程和上面的过程基本一致。如下图所示：

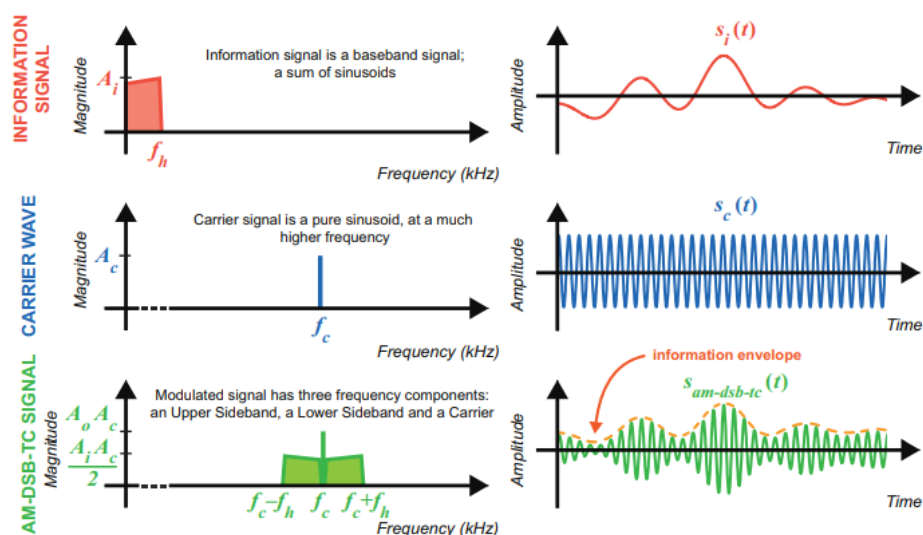


图 5-4 基带信号的常规调幅

由信号与系统的知识可以知道，任何信号都可以用正余弦信号来表示，因此基带信号可以看作是有不同频率的单音信号组成的，这样一想就能够简单地来弄清楚基带信号调制过程中的频域变化。

为了更直观地描述这一过程，我们从基带信号中，任意找出三个频率的单音信号，观察其在调制前和调制后的变化，如图 5-5、图 5-6 所示：

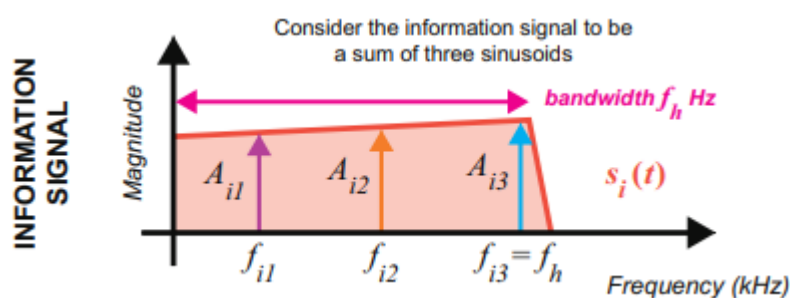


图 5-5 调制前的基带信号

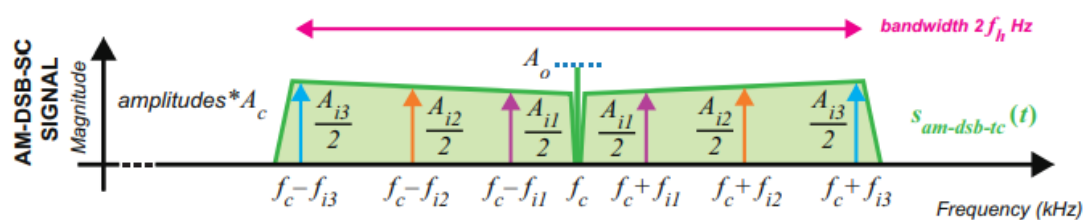


图 5-6 调制过后的基带信号

可以看到使用常规调幅过后，在频域上回生成对称的两个边带。

5.2 使用 GNU Radio 仿真常规调幅

在前面简单介绍了常规调幅的原理之后，接下来就可以使用 GNU Radio 来对常规调幅进行仿真了。创建流图 `lab1_am_dsb_tc.grc`。

首先进行常规调幅，其调制部分如图 5-7 所示，基带信号加上一个直流偏置之后与载波信号相乘，完成调制。

其整个 GRC 流图如图 5-8 所示。

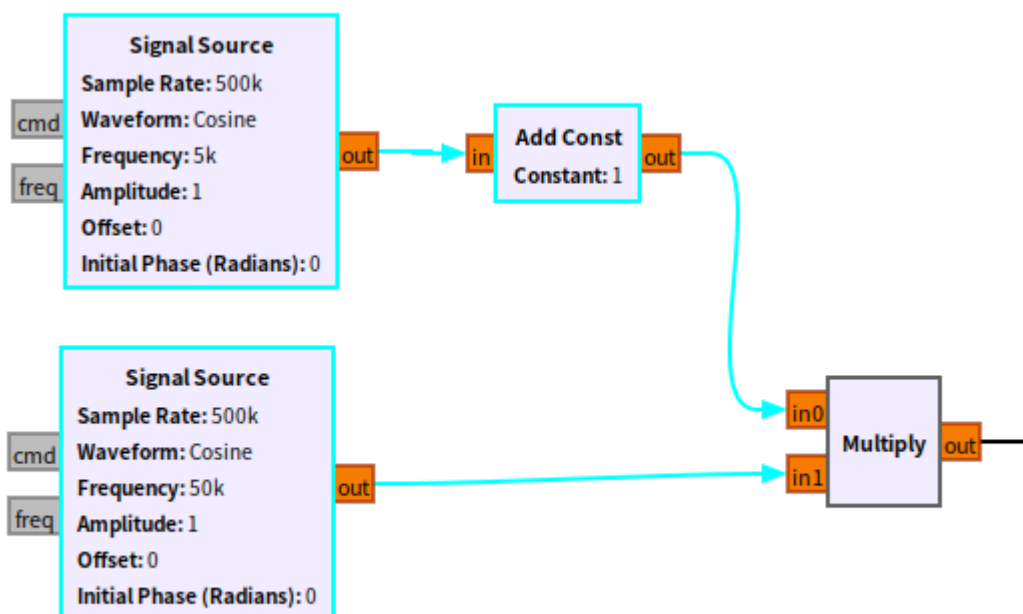


图 5-7 常规调幅调制部分

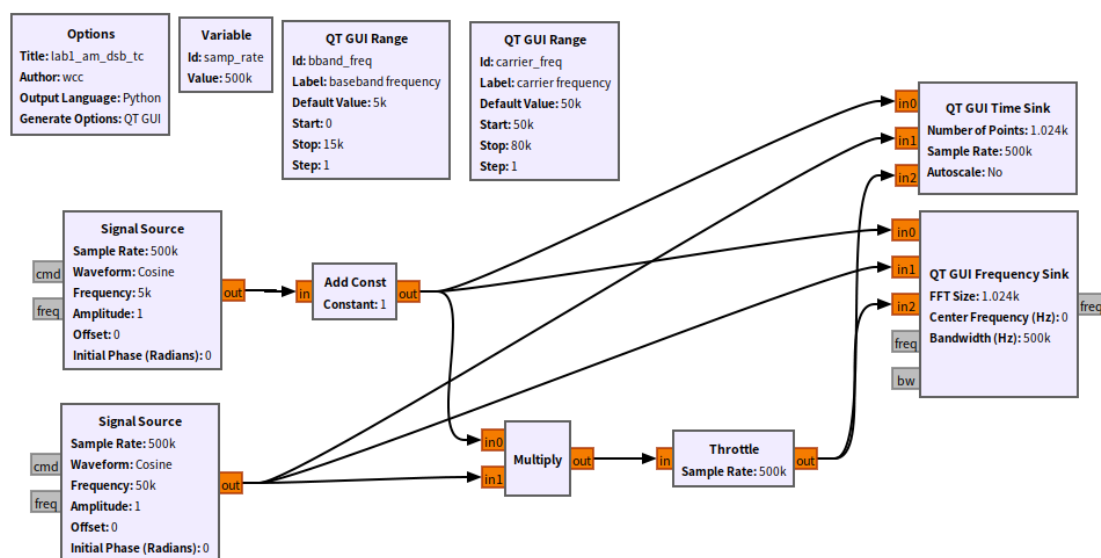


图 5-8 常规调幅 GRC 流图

点击运行，就可以开始运行这个流图了。

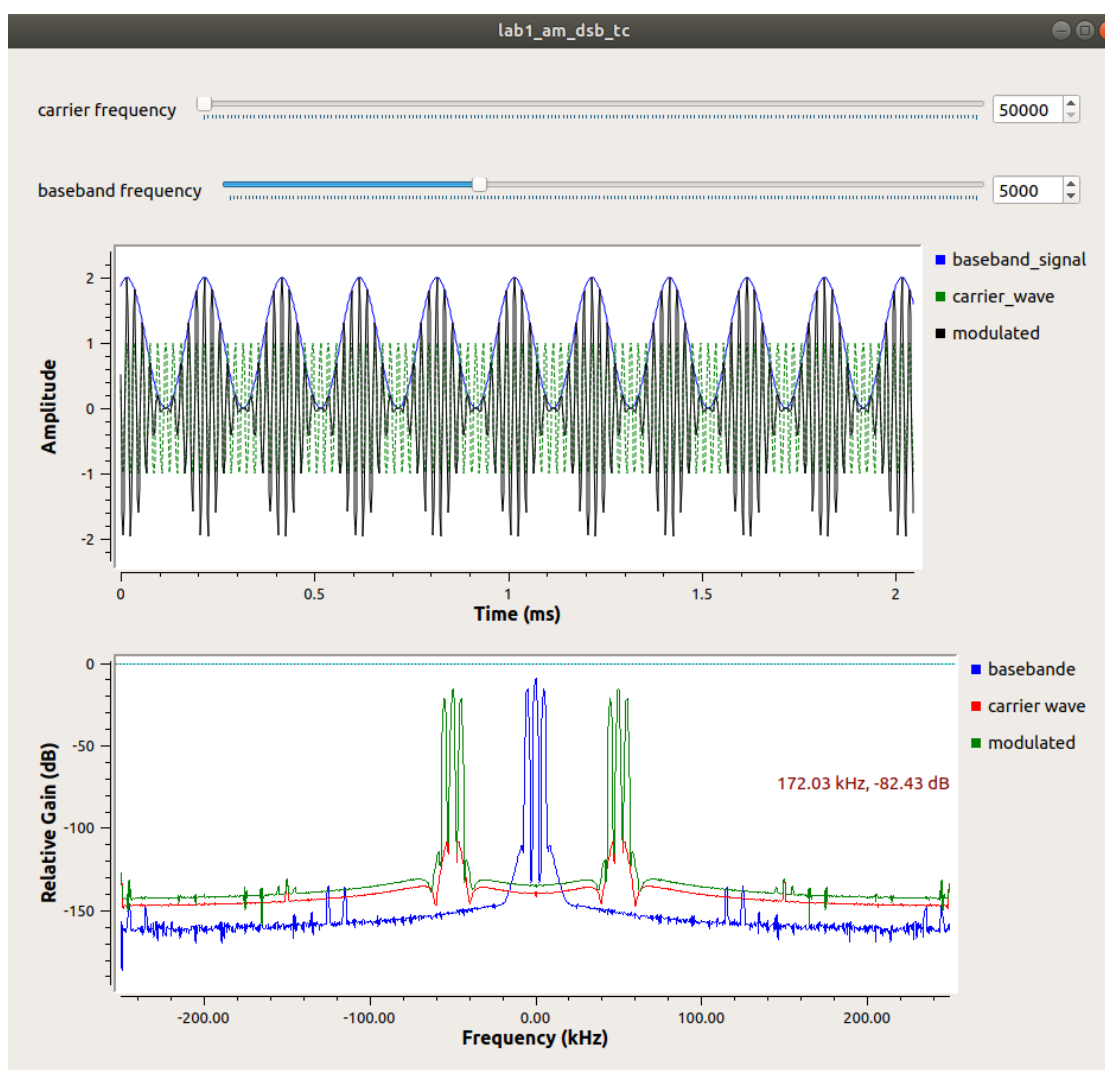


图 5-9 常规调幅结果

可以看到，使用常规调幅调制一个单音信号，在已调信号当中，信号的包络就是基带的消息信号。这与前面介绍的常规调幅的基本原理一致。

用户可以移动上面的滑块，来更改基带信号的频率和载波信号的频率，观察频谱图和时域波形，分析常规调幅的作用。

5.3 使用常规调幅发送音频信号

接下来，我们可以使用 ANTSDR 基于常规调幅的方式，来发送音频信号，并在其他的 SDR 设备上接收。**需要注意的是，在使用到具体的硬件的时候，最好使用一台实体机来完成 GNU Radio 的开发。使用虚拟机经常会出现数据卡顿的状况。**

在进行使用 ANTSDR 发送音频数据之前，我们需要知道 ANTSDR 采用的是

AD936x 系列芯片，其调制载波的生成是在 `ad936x` 当中生成的，也就是在进行调制到高频载波的时候，不需要在基带数据中完成。又因为 ANTSDR 与上位机之间进行数据传递是采用的复数基带信号的形式，因此在进行数据传递的时候，用户只需要提供好复数基带信号就可以了。

新建流图 `lab2_am_dsb_tc_antsdr.grc`，本次实验将会发送音频文件，因此我们准备了结果音频文件放在 `source_audio` 文件夹下。

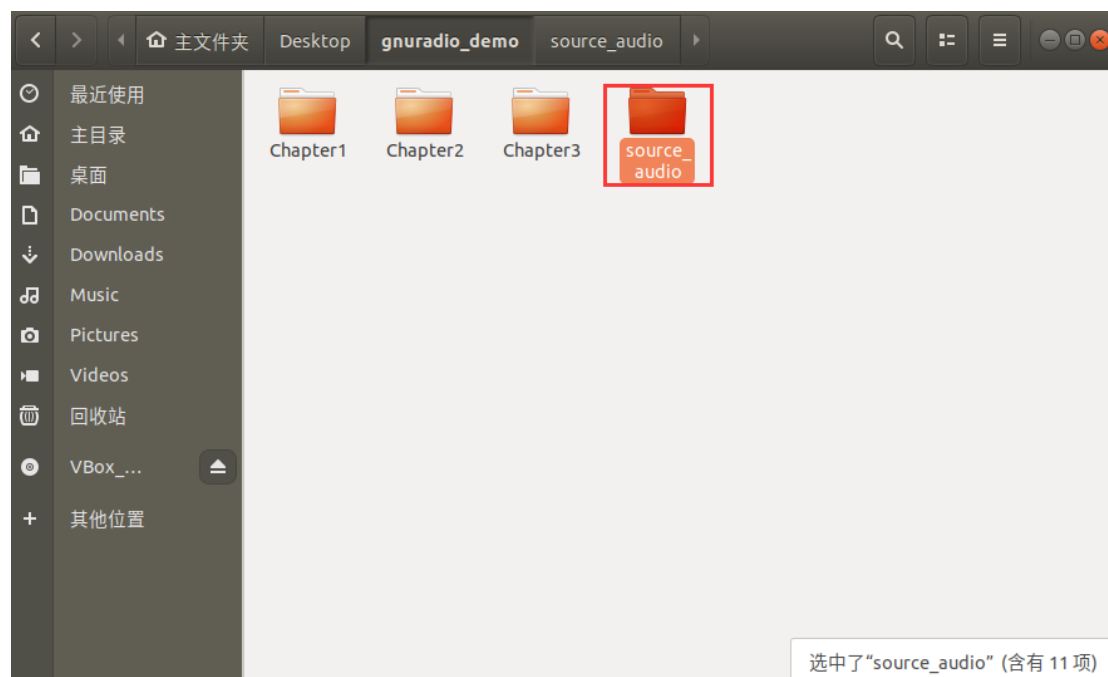


图 5-10 音频源文件文件夹

在 GNU Radio 当中，需要使用到音频文件的时候，需要添加一个模块，来获取音频文件数据，这个模块是 `Wav File Source`。

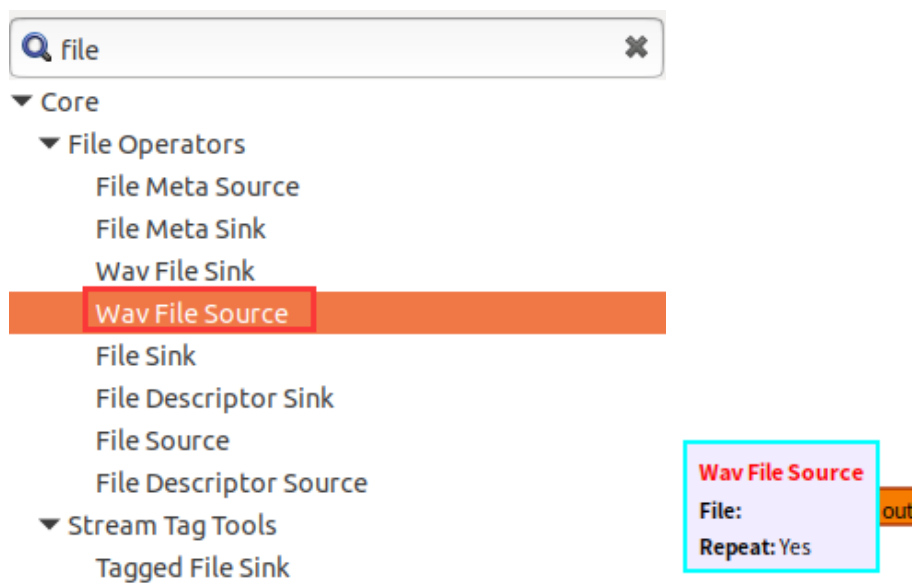


图 5-11 音频源模块

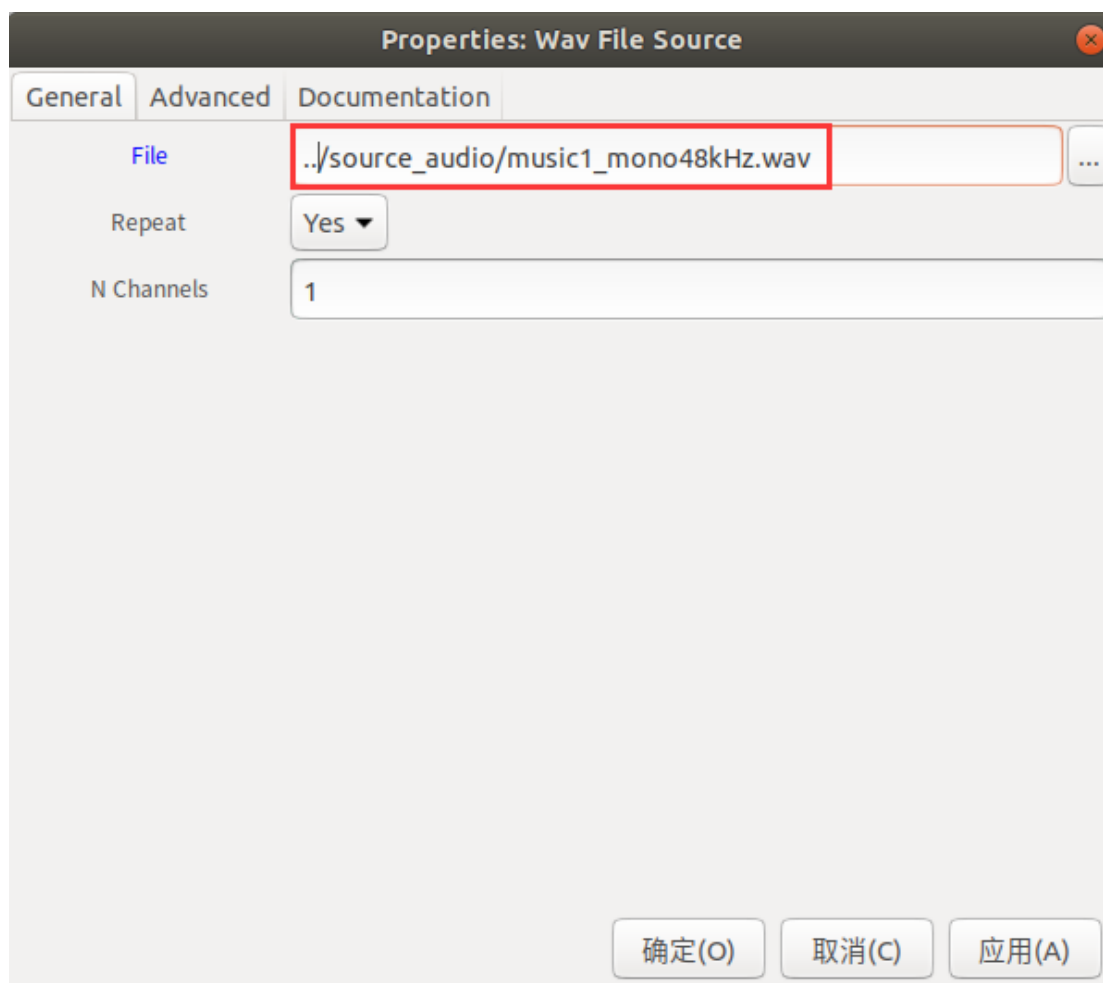


图 5-12 该模块需要指定音频文件所在的路径即可

由于基带信号的采样率比较低，音频信号的采样频率只有 48KHz，因此需要对基带信号进行一个上采样之后，才能把数据给到复数基带，因此在这里需要使用到一个上变频模块。

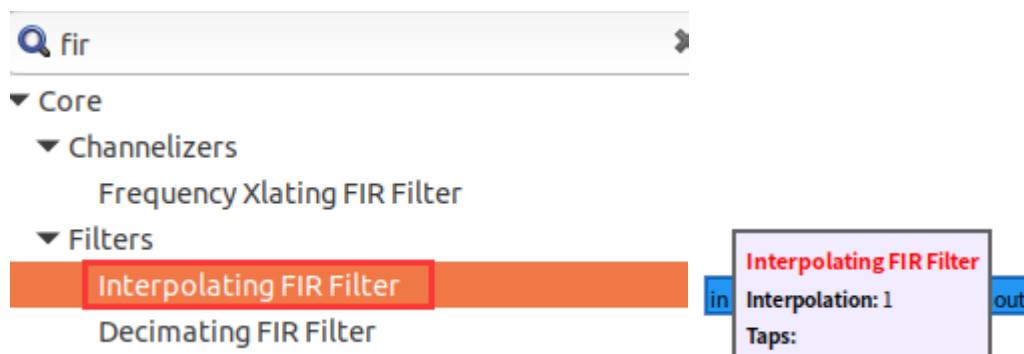


图 5-13 上变频模块

FIR 插值器的参数如图 5-14 所示，可以看到，主要有两个参数，一个是插值倍数，该值确定在进行插值过程中，采样频率变化的倍数。再一个重要的参数是 FIR 滤波器的抽头系数，为了获取到这个抽头系数，我们需要再添加一个用于获取 FIR 抽头系数的模块。

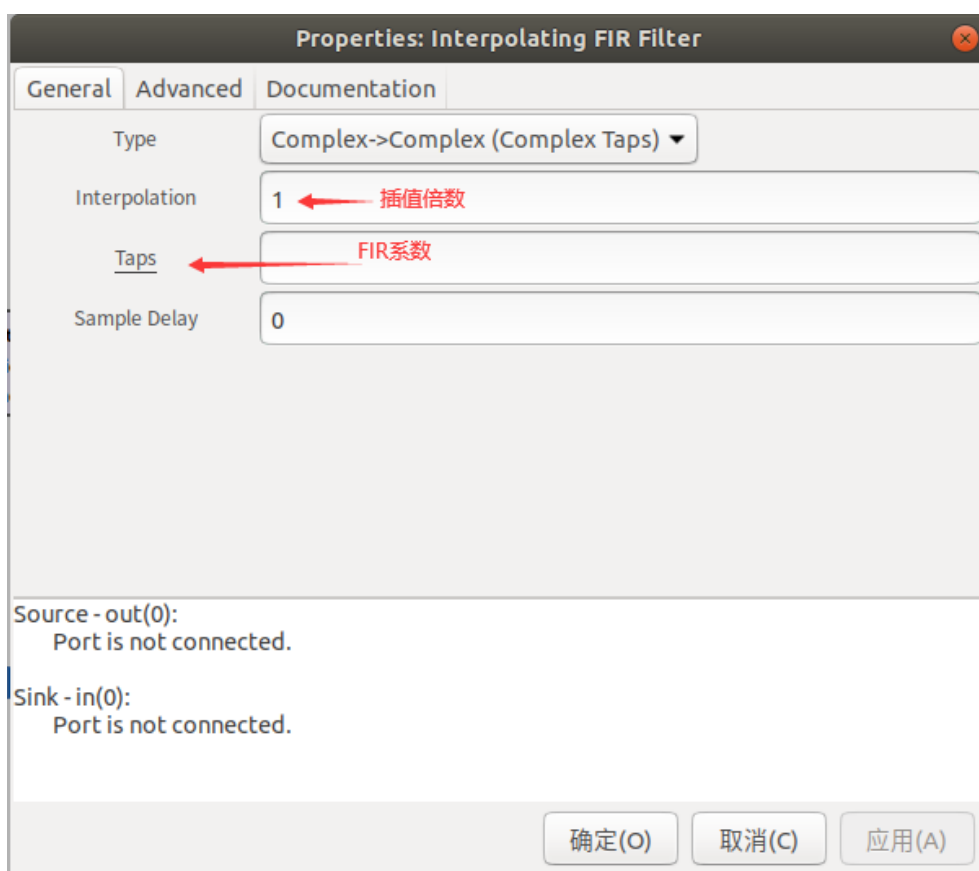


图 5-14 FIR 插值器参数

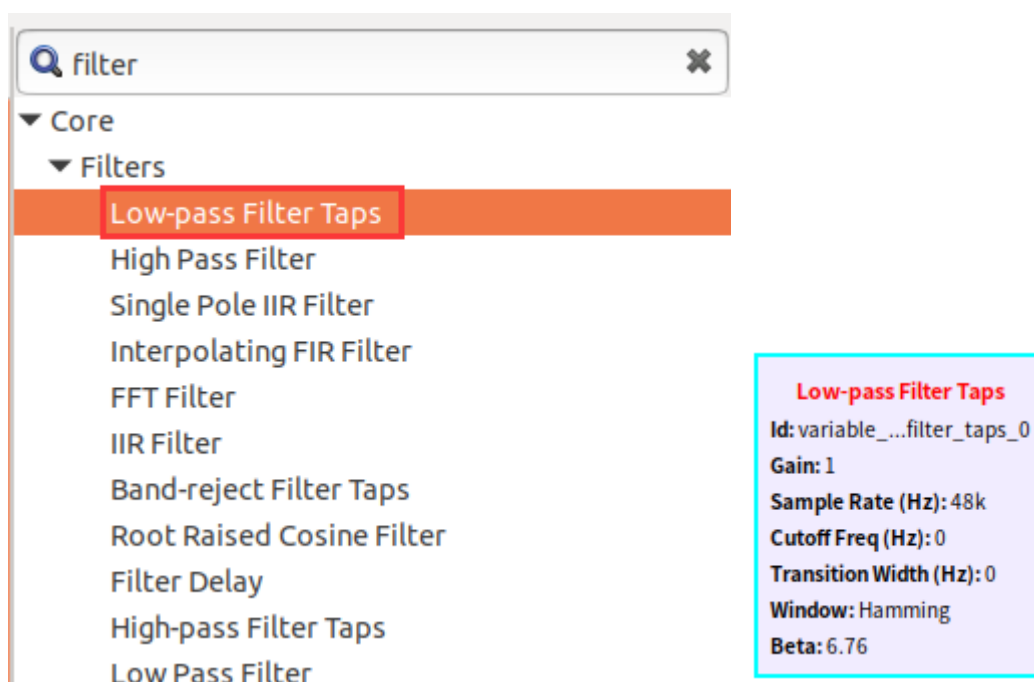


图 5-15 FIR 抽头系数模块

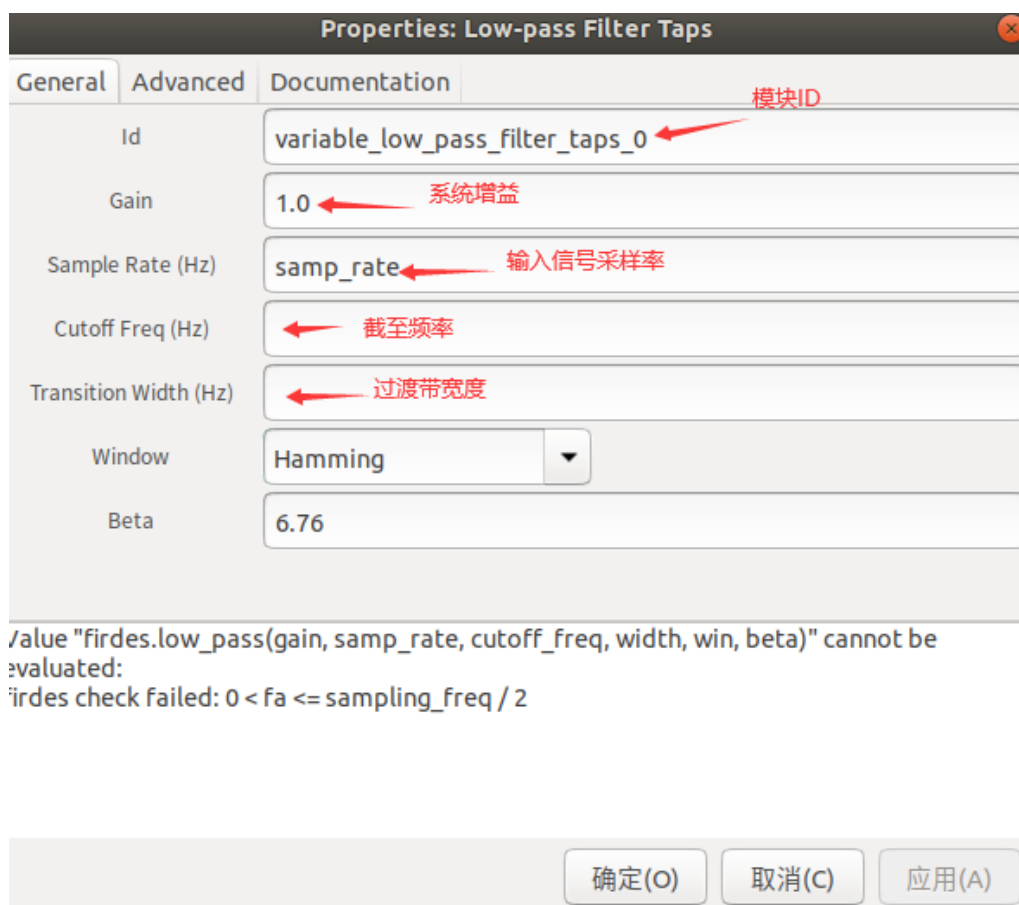


图 5-16 FIR 抽头系数模块参数

该 FIR 低通滤波器抽头系数模块的设置与前面设置低通滤波器的时候类似，按照前面设置低通滤波器的方法进行设置就可以。其设置如图 5-17 所示：

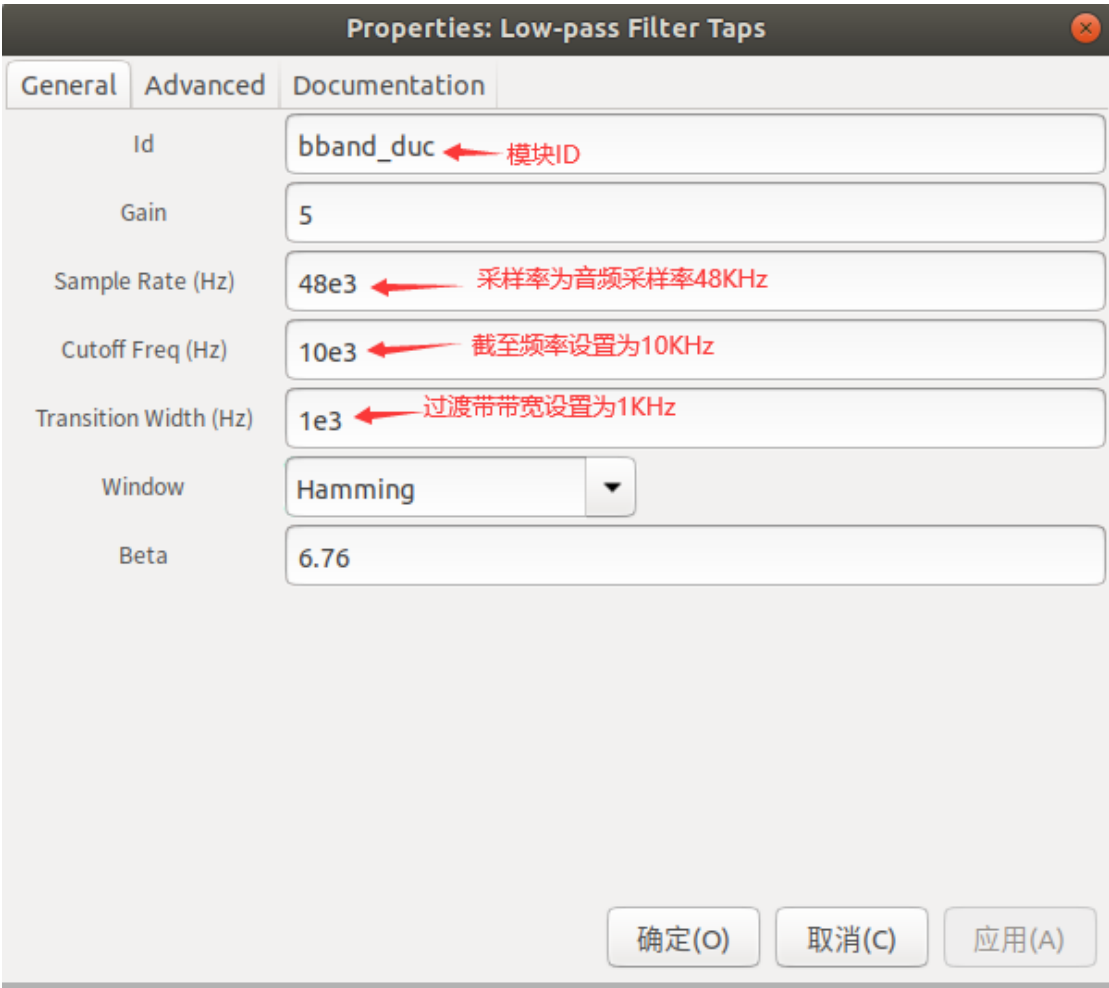


图 5-17 抽头系数模块设置

设置好抽头系数模块之后，这个模就会为我们提供 FIR 低通滤波器的系数了，然后就可以将这个系数用在 FIR 插值器当中了，FIR 插值器的设置如图 5-18 所示：

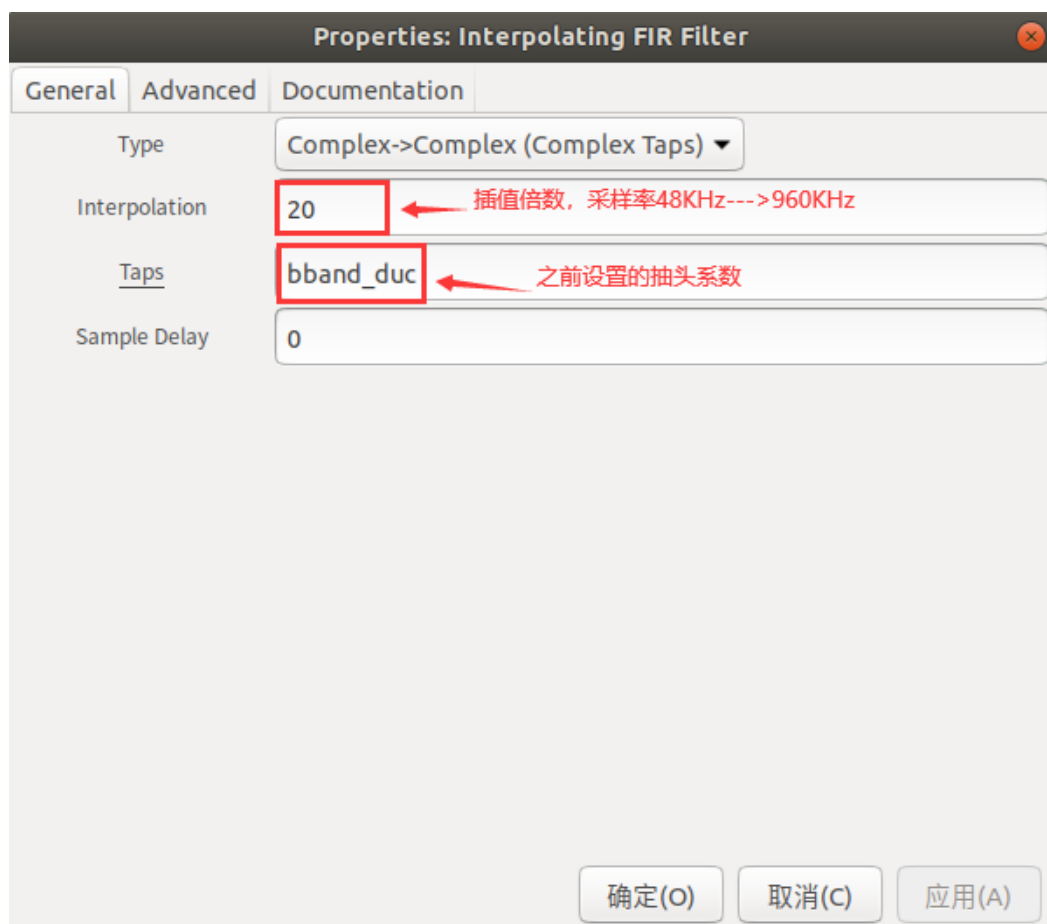


图 5-18 FIR 插值器设置

再进行数字上变频 DUC 的时候，一般再经过了上采样之后，需要将上采样之后的信号经过一个低通滤波器，这样能够有效地减轻旁瓣对信号的干扰，因此我们可以再这之后再增加一个低通滤波器，其设置如图 5-19 所示。可以看到，此时低通滤波器的输入采样率应该设置为上采样之后的信号采样率。

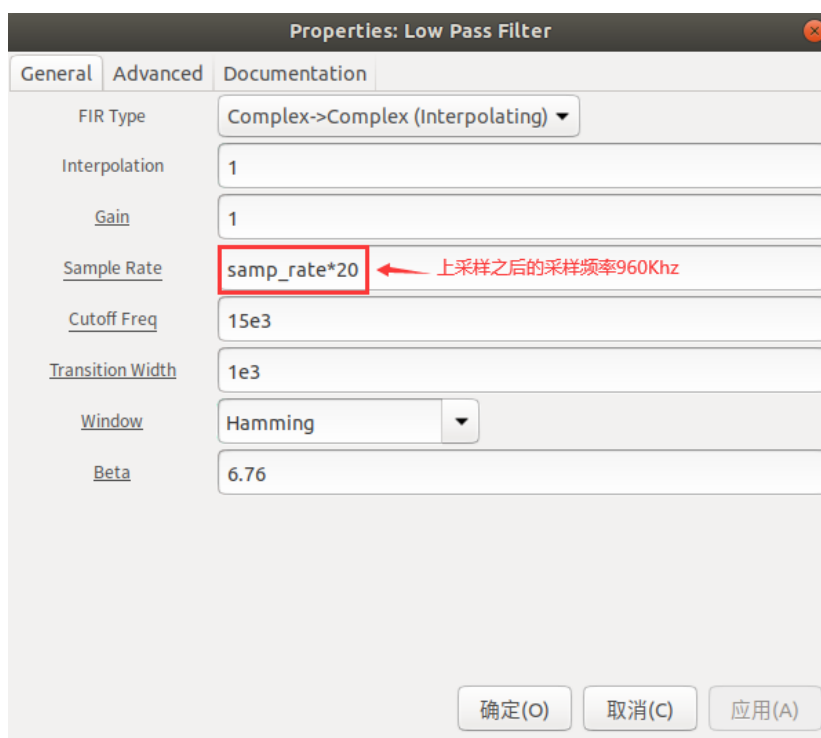


图 5-19 低通滤波器设置

最后，就可以将这个信号给到 pluto 发送模块，从而将这个信号发射到信道之中了。Pluto sink 模块的设置如图 5-19 所示。

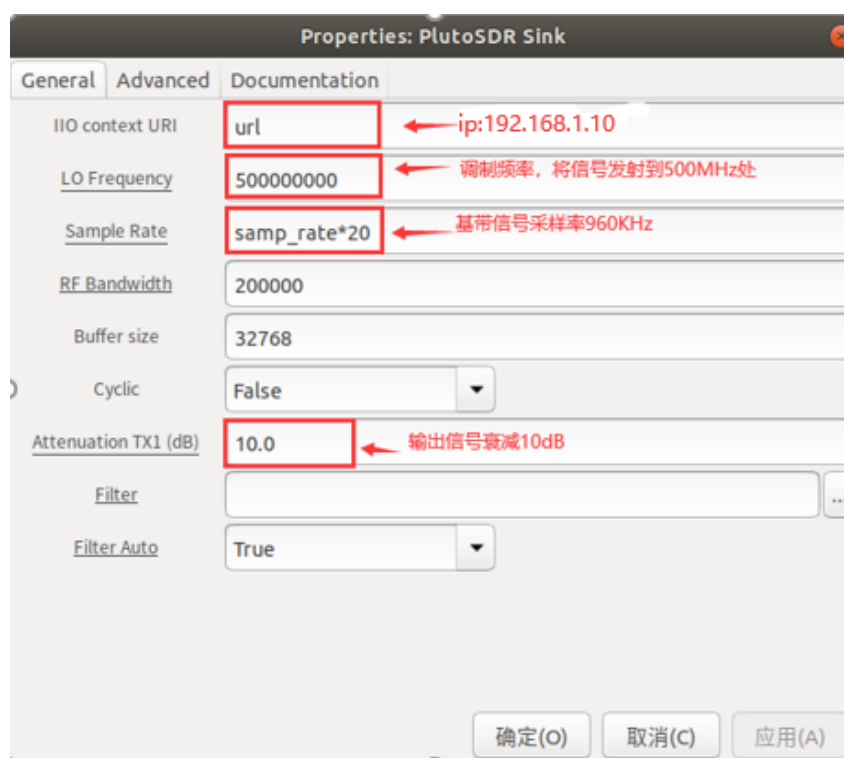


图 5-20 Pluto 输出模块设置

最终，整个 GRC 流程图如所示：

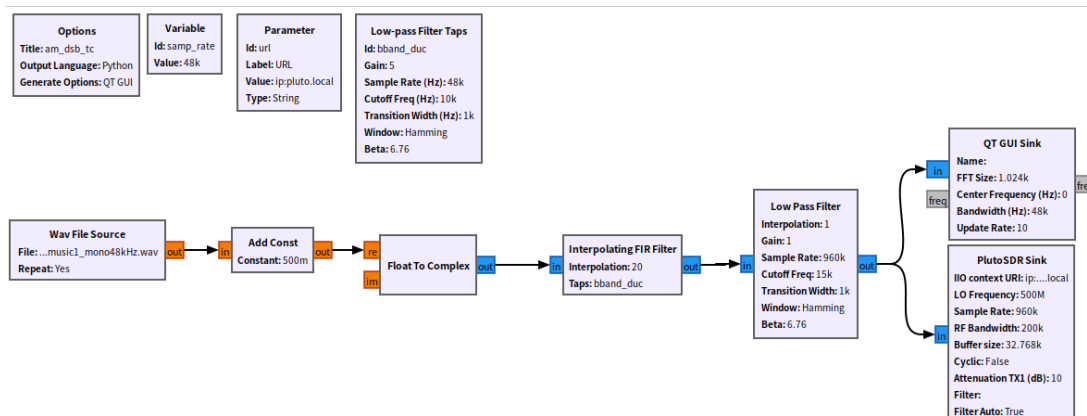


图 5-21 常规调幅 GRC 流程图

点击运行仿真，仿真的时候，可以观察在基带上的信号。

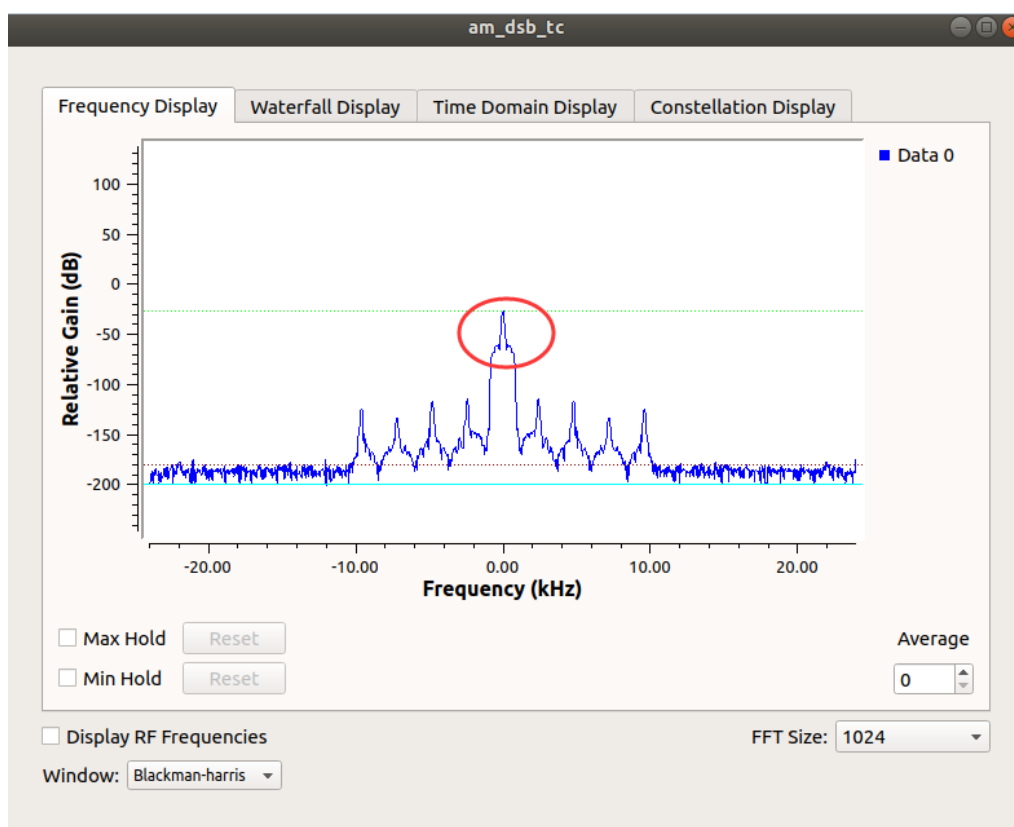


图 5-22 频域信号

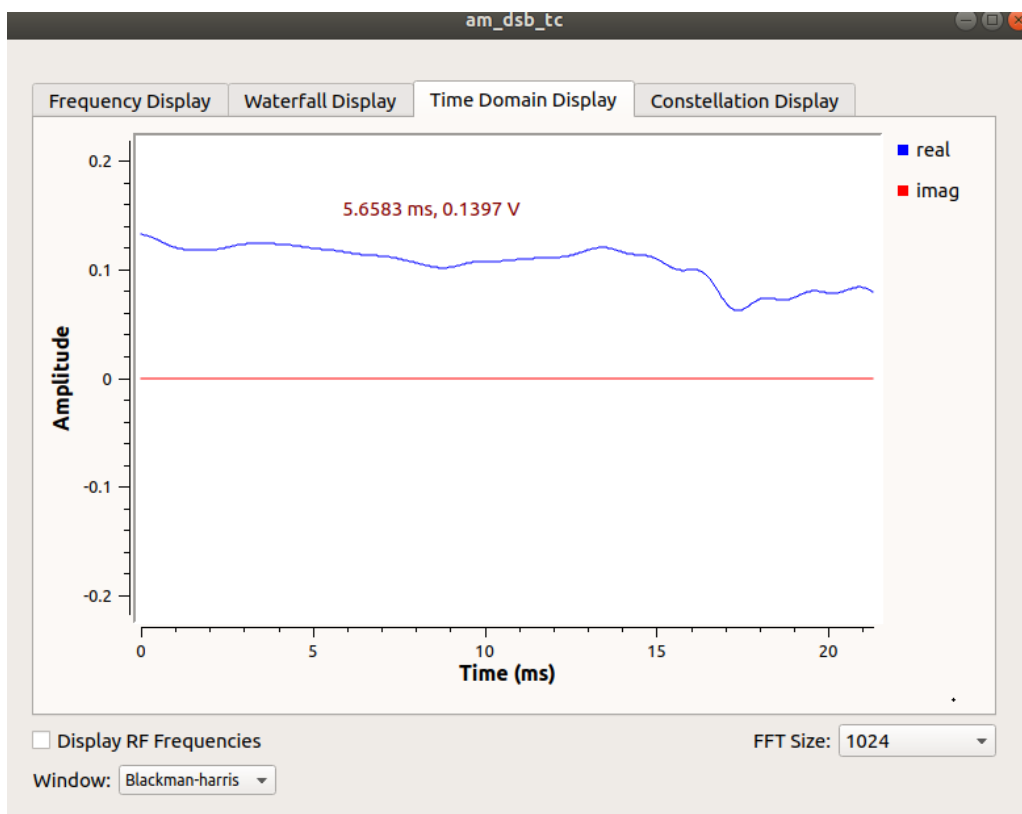


图 5-23 时域波形

若此时用户手上有两外一个 SDR 设备，比如 ANTSDR 或者其他 SDR 设备比如 RTL_SDR，USRP 等，用户可以使用这些 SDR 设备使用 SDR 软件来接收这些信号。

在这里笔者是用另外一块 ANTSDR，并在 windows 系统上使用 SDR# 来进行音频信号的接收。将频率调制 500MHz，选择解调方式为 AM 解调，打开 SDR 设备的时候，就能够听到前面发送出来的音频信号了。

使用 SDR#接收到的频谱如图 5-24 所示，可以看到在 500Hz 附近有一个明显的信号起伏，从瀑布图上也能够看出该处信号强度较强。此时音频播放器也能够听到播放出来的音乐。

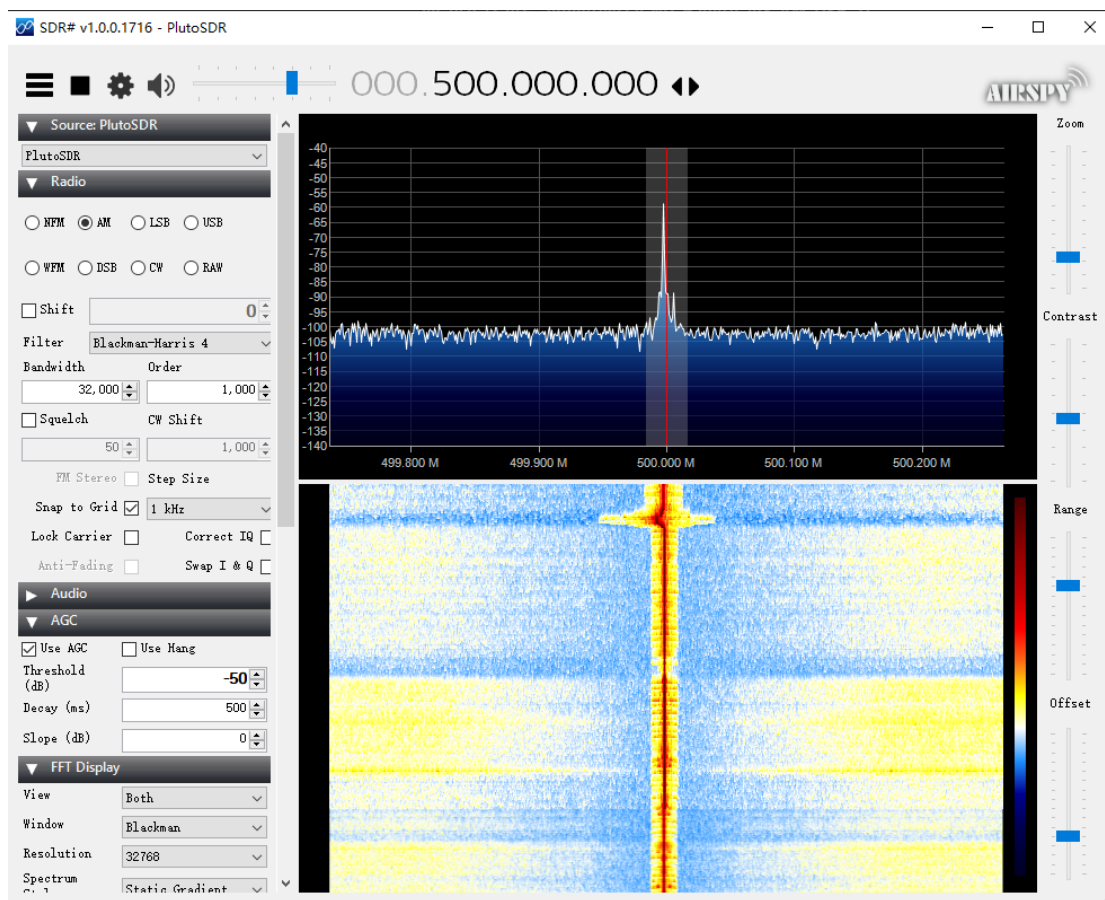


图 5-24 使用 SDR#接收到信号

5.4 常规调幅的非相干解调原理

在前面介绍了常规调幅的调制原理，并且通过仿真实现了对常规调幅更加直观的认识，信号调制完成之后，在接收机里面需要对信号进行正确的解调，从而恢复出数据。

常规调幅的一个有点就是，消息信号能够在已调信号的包络中提取出来，从而能够使用非相干的方式进行解调。所谓非相干解调，就是在接收机当中，不需要生成一个和调制载波同频同相的信号就能将基带信号从已调信号中解调出来。

5.4.1 一般情况下的包络检波

在 Matlab 和 ANTSDR 共同作用来实现包络检波的时候，不能够使用模拟的方法，因为 Matlab 接收到的信号是已经经过采样之后的信号，是数字信号。因此在数字信号处理中可以饱和操作来实现包络检波的功能。所谓饱和操作就是使信号的幅度值并使其平滑。一般的结构如下图所示

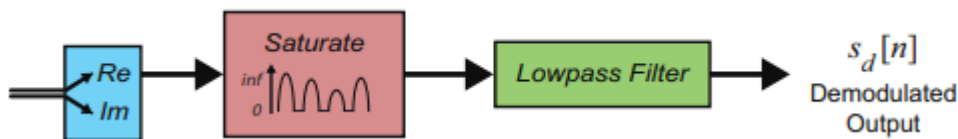


图 5-25 离散时间包络检波

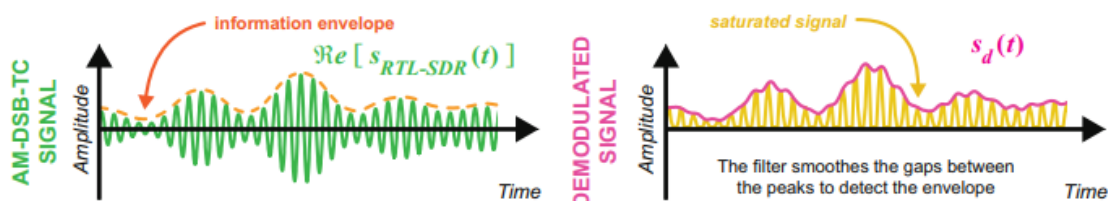


图 5-26 包络检波示意图

为了使低通滤波器能够成功的把波峰平滑化，并检测出信息包络线，AM-DSB-TC 信号的载波频率要相对信息信号的频率尽可能地高。如果载波频率不够高，那么滤波器就很难把它消除，这样输出的信号就会含有噪声。

5.4.2 实数解调的优化

上面的包络检波是在实数域下进行的，在前面也提到过，想要使解调出来的包络平滑，载波的频率需要比基带信号的频率要尽可能的高。我们可以看到在前面的解调过程中，复数部分的包络和正数部分的包络是对称的，但是在解调的使用，只使用了正数部分的包络。若我们现在对信号取绝对值，然后再来求取信号的包络，就相当于增大了载波的频率。这样，更加能够使包络平滑。

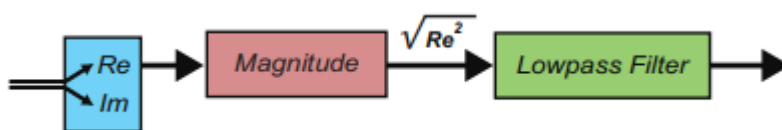


图 5-27 改进包络检波

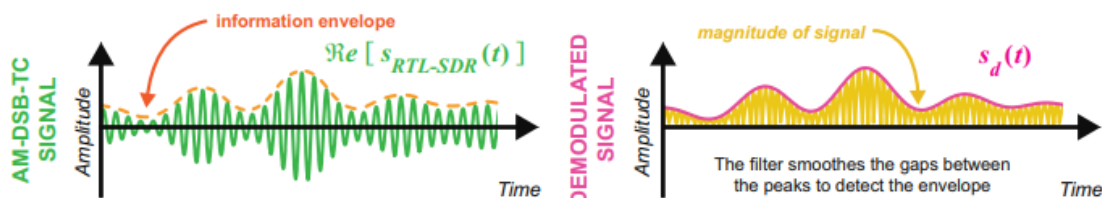


图 5-28

5.4.3 复包络检波

在前面的章节中，介绍了使用复数来表示信号的方法，其实在进行包络检波的时候，使用复包络会使整个过程显得更加直接。

我们使用的 ANTSDR 是在和上位机进行数据传输的时候，使用的是 IQ 两路数据，可以将其看作是复数信号的形式，如下图所示：

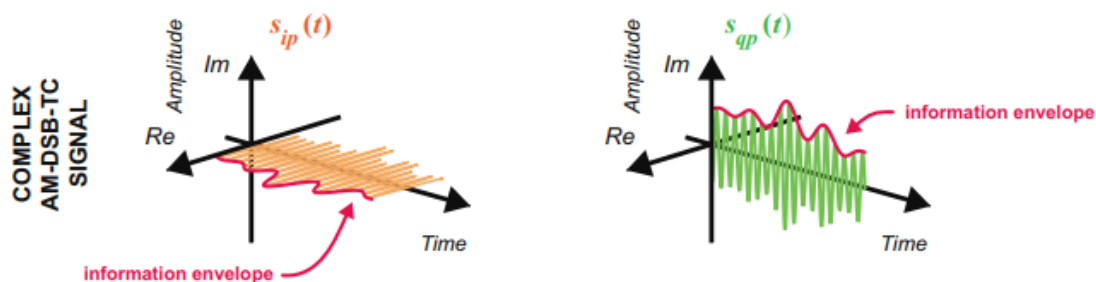


图 5-29 IQ 路包络

在复数空间内，其实可以将这虚部和实部的信号放在同一坐标轴下随着时间的变化，那么信号就可以使看作围绕着时间轴在不停旋转的一个点。

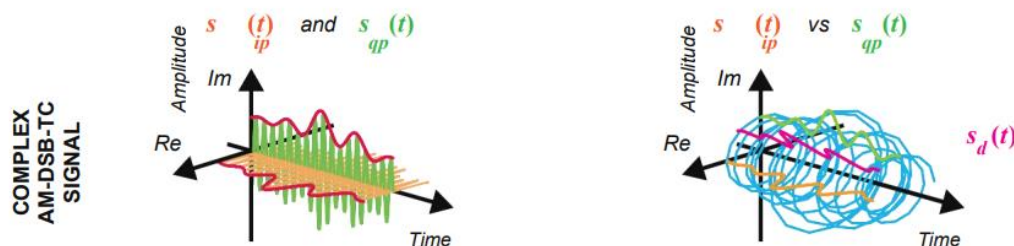


图 5-30

因此，在求取信号的包络的时候，我们只需要求得信号得复包络，便能够得到消息信号。所以包络检波得结构可以进一步简化：

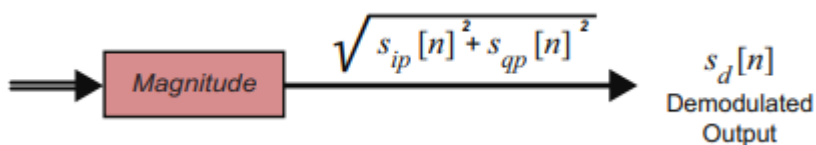


图 5-31 复包络检波

可以看到，求一个复数的包络，只需要知道这个信号的模就可以了，这个也就是相当于求这个复数的模。

5.5 常规调幅的非相干解调仿真

在前面介绍了常规调幅的包络检波方法之后，可以使用 GNU Radio 来进行简单的仿真，验证一下非相干解调的方法。

对于常规调幅解调的仿真，只需要在前面常规调幅的基础上进行就可以了，可以看大，对于常规调幅的包络检波只需要将信号进行在较高的频率下进行取绝对值，并进行低通滤波就能够得到原始的信号。

因此在进行非相干解调的时候需要添加一个求信号绝对值的模块和一个低通滤波器。

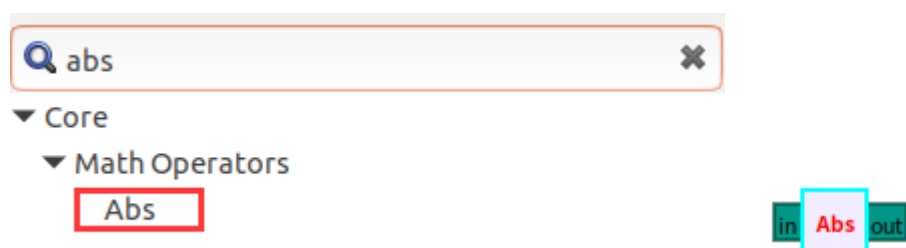


图 5-32 ABS 模块

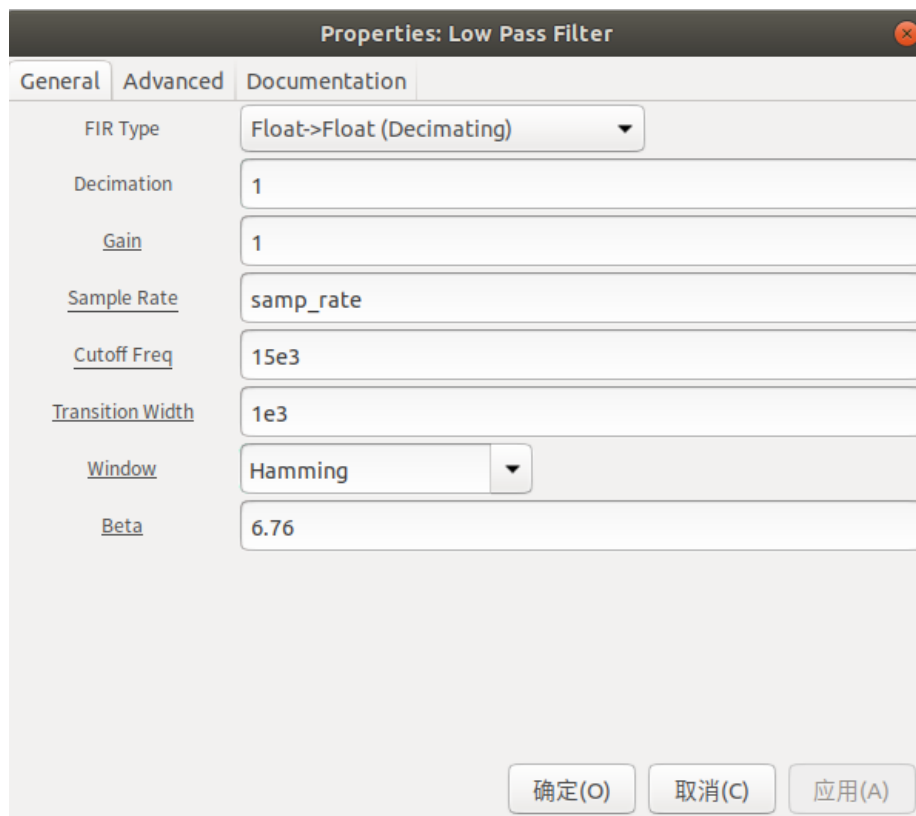


图 5-33 低通滤波器设置

低通滤波器的设置如图 5-33 所示。需要将进行取绝对值的信号滤除其高频分量，就能够得到信号的包络了。

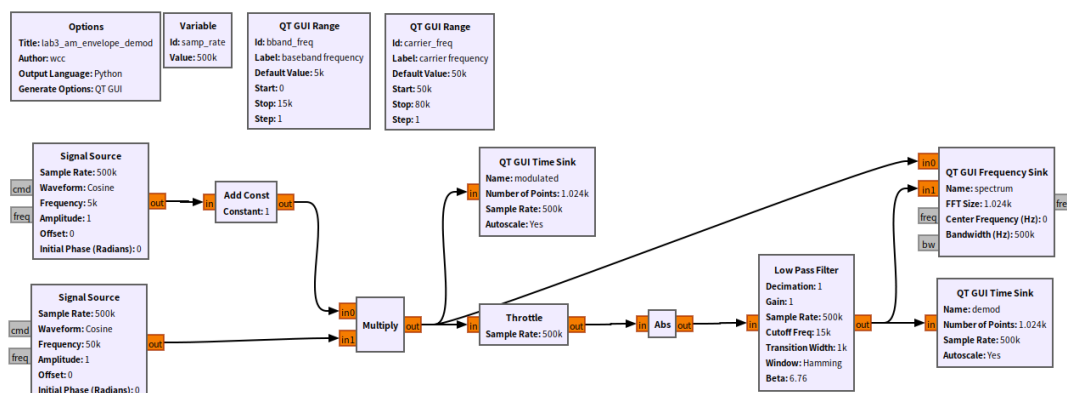


图 5-34 GRC 流图

整个 GRC 流图如图 5-34 所示可以看到，这个 GRC 流图十分简单就是在之前的基础上增加了两个模块就能够完成信号的解调了。

点击仿真开始运行这个流图。

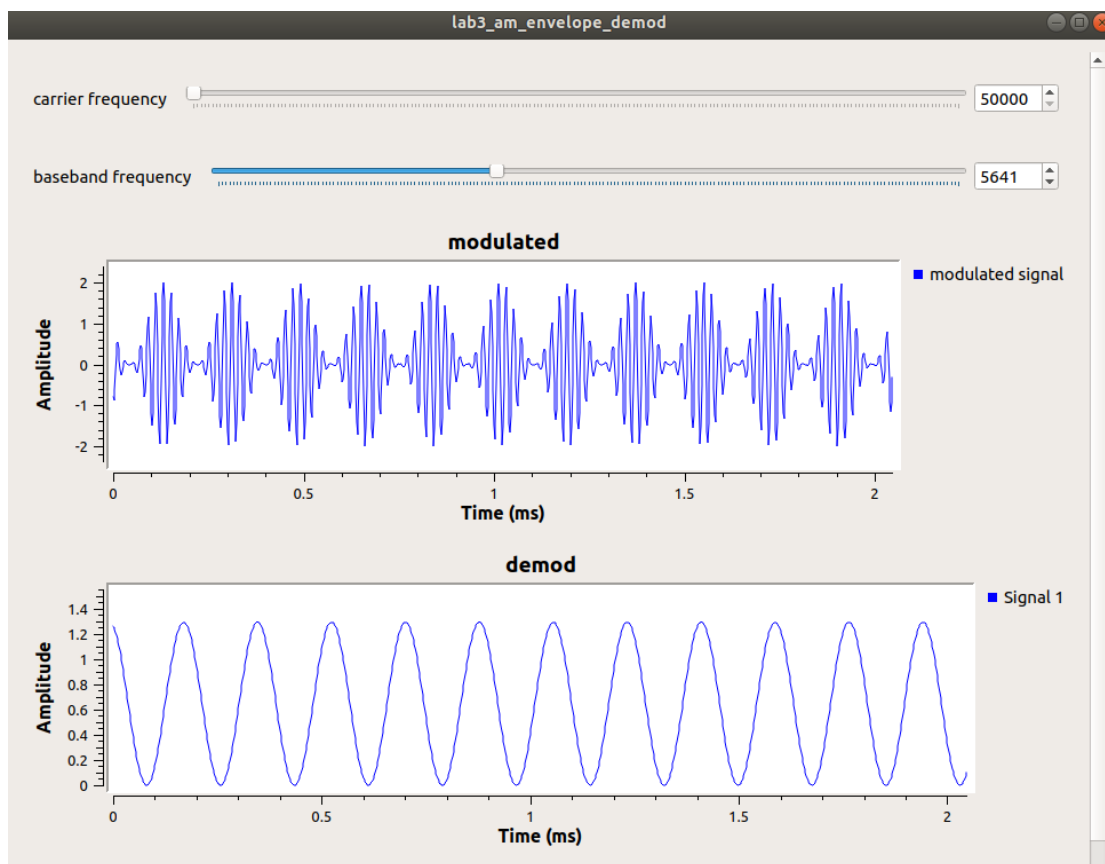


图 5-35 仿真得到的结果

可以看到，在经过包络检波之后，能够从原始的信号当中提取出信号的包络并且，用户可以更改基带信号的频率，观察解调出来的包络的变化。

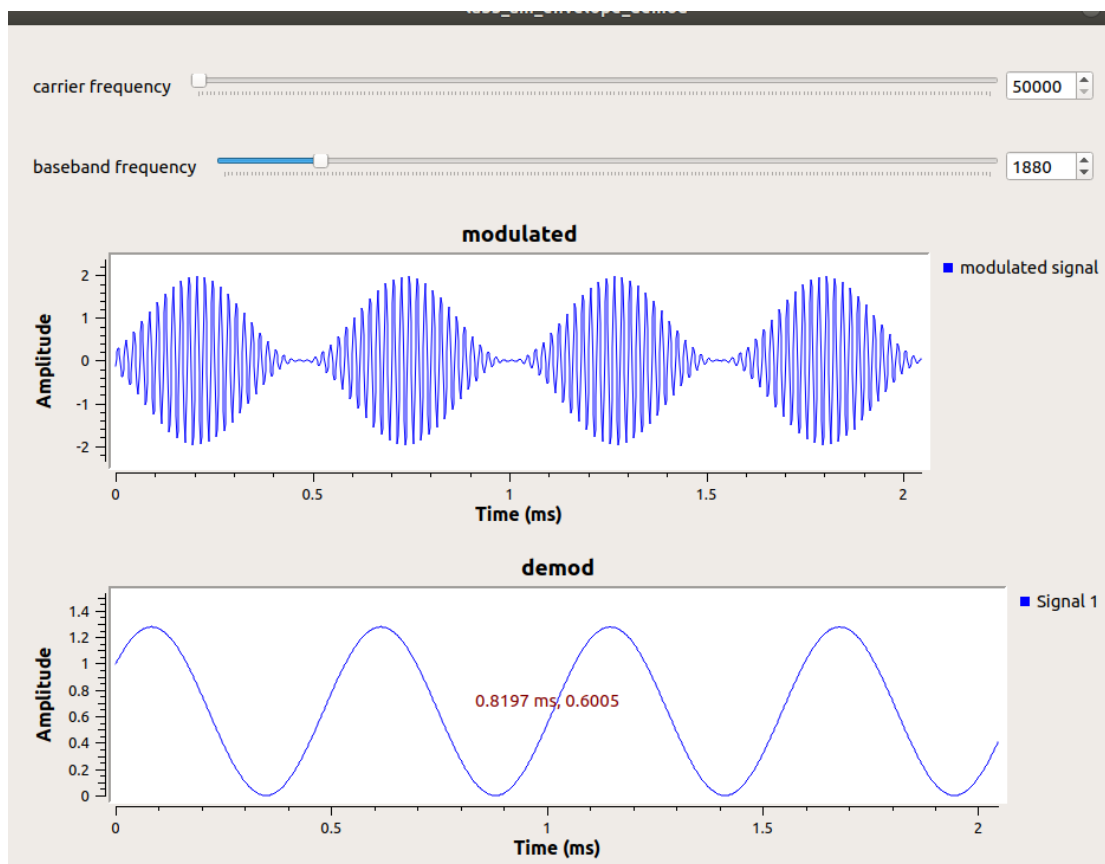


图 5-36 减小基带信号频率

5.6 解调常规调幅音频信号

接下来就可以使用包络检波来进行 AM-DSB-TC 的非相干解调了。有了前面仿真常规调幅的包络检波方法，实现 AM 信号的包络检波应该没有太大问题。

整个 AM 调制与解调的 GRC 流程图如图 5-37 所示，其中上板部分是常规调幅的调制部分，下半部分是常规调幅的解调部分。

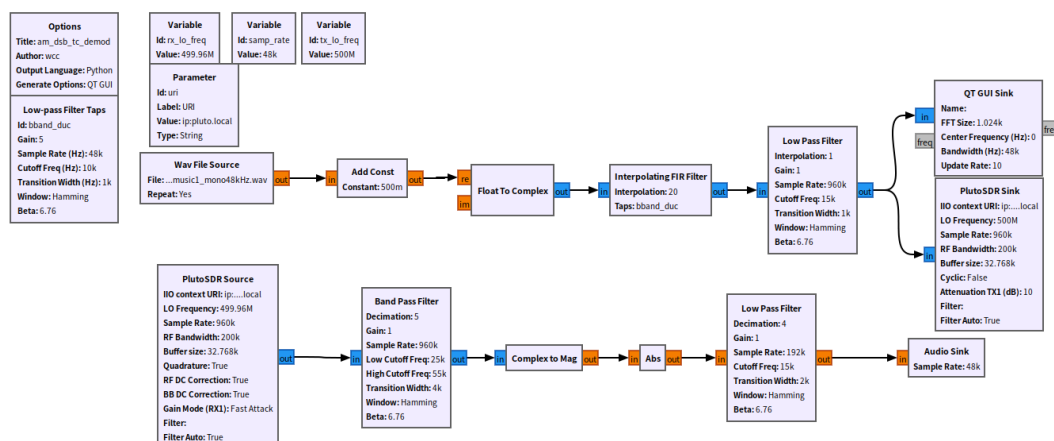


图 5-37 常规调幅调制解调 GRC

在解调部分当中，首先设置了 Pluto SDR 的接收模块，该模块的设置如下：

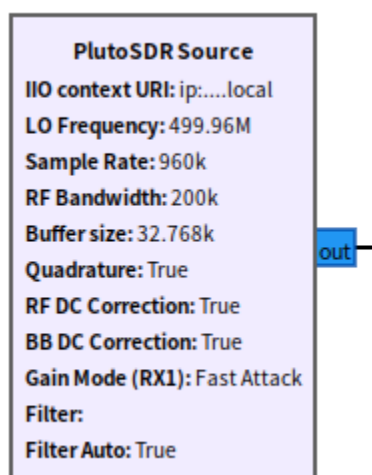


图 5-38 PlutoSDR 接收模块

在接收模块当中，设置了下变频的本振为 499.96MHz，这么做的目的是能够将信号接收到的信号下变频到基带附近，这样在传递到上位机的复数基带信号当中，还是一个带有较高频率载波的信号。

从信号源得到这个信号之后，需要使用一个带通滤波器，保留 40KHz 附近的信号，这样就能方便后续获得基带信号。

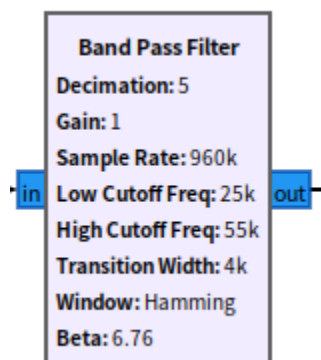


图 5-39 带通滤波器设置

带通滤波器的使用方法和前面介绍的低通滤波器和高通滤波器相似，其具体设置如下图所示：

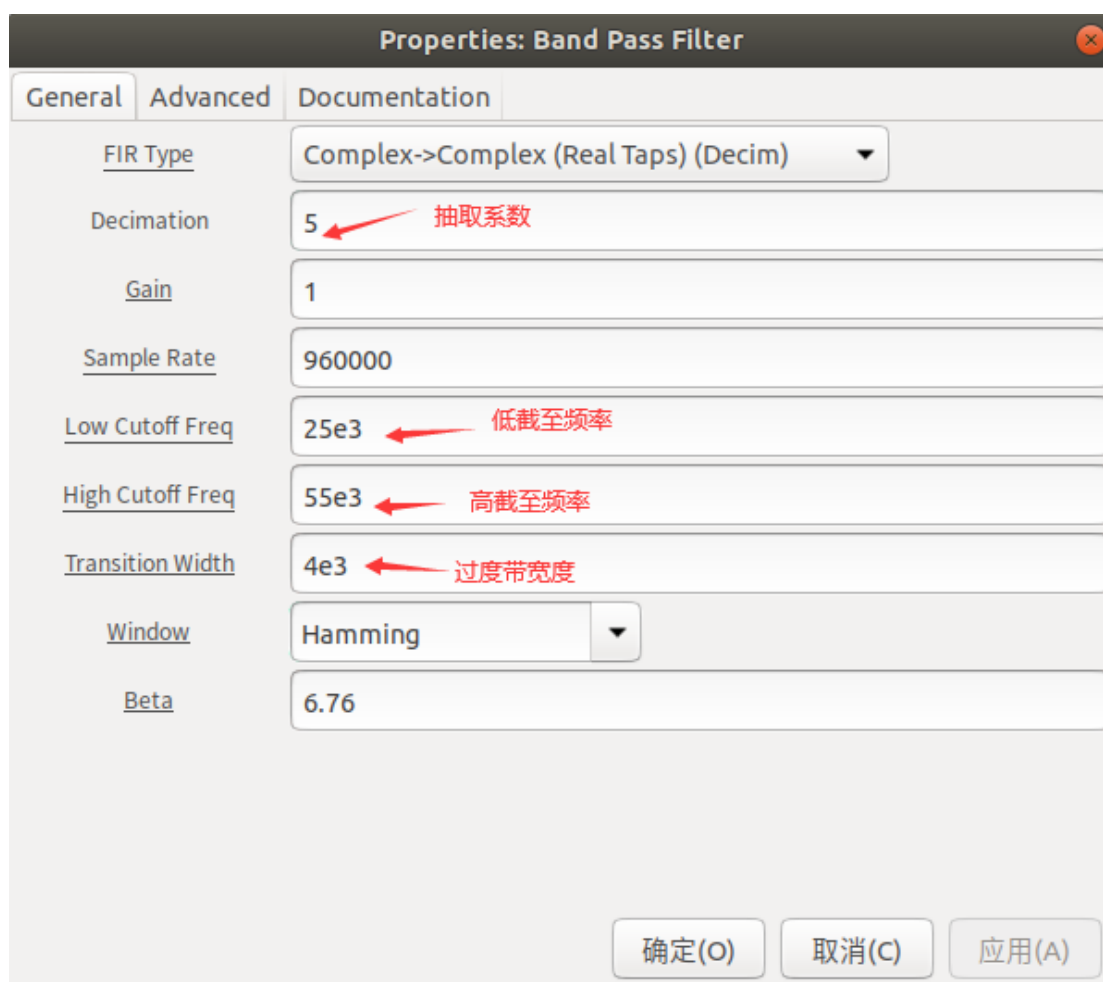


图 5-40

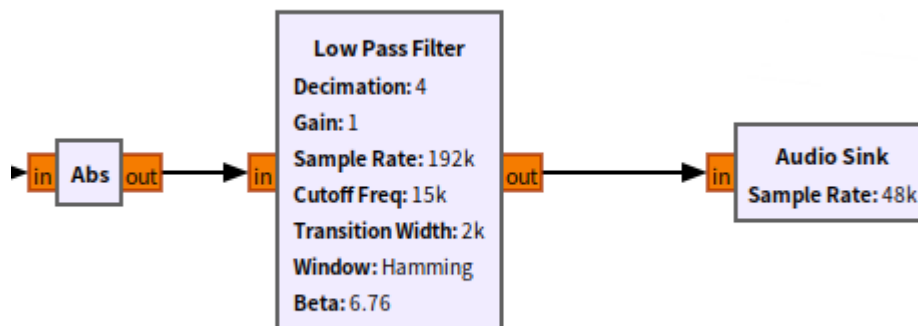


图 5-41 包络检波

最后使用包络检波就能够获取到基带信号了，这里使用音频播放器将解调出来的信号进行播放，这样就能够获取到基带信号了。

5.7 总结

在本章当中，主要介绍了常规调幅的原理和其调制解调方法，实现了使用 ANTSDR 来发送音频信号和进行音频信号的接收的工作。

第6章 FM 的调制与解调

在前面的实验当中，我们实现了使用常规调幅的方式来调制音频，由于硬件的限制，我们无法获取到商业 AM 电台的信号，所以我们自己使用 ANTSDR 发送了 AM 信号并用 ANTSDR 成功地将信号进行了解调。在本章的实验当中，我们会使用 ANTSDR 来解调商用的 FM 信号，能够收听当地电台广播。除此之外我们还会自己实现了一个 FM 发射机，并且使用 ANTSDR 来进行解调。

6.1 FM 调制的数学原理

6.1.1 FM 信号产生的过程

FM 就是频率调制，就是使用载波的频率来承载消息信号，当消息信号的幅值较大的时候，产生的频率比较大，当消息信号较小时，产生的频率比较小。在我们之前介绍过的 VCO 这个结构，就是一个最简单的 FM 的调制器。

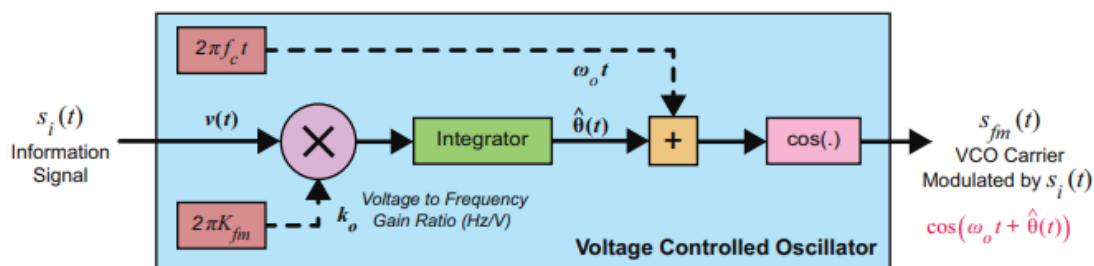


图 6-1 VCO 结构

控制信号进入了 VCO 之后，首先需要乘上一个增益，这个增益表示电压（消息信号的幅值）到频率（FM 调制的结果）的增益值， $v(t)$ 与 k_o 的乘积会经过一个积分器。积分之后的信号为：

$$\hat{\theta}(t) = k_o \int_{-\infty}^t v(t) dt$$

由 VCO 产生的正弦波会有一个初始的振幅 A_o 和频率 f_o ，正弦波的相位取决于前面这个积分信号的瞬时值。将这个值带入到产生的正弦波当中。

$$\begin{aligned} c(t) &= A_o \cos\left(2\pi f_o t + \hat{\theta}(t)\right) \\ &= A_o \cos\left(\omega_o t + k_o \int_{-\infty}^t v(t) dt\right) \end{aligned}$$

其中这个 k_o 可以表示为 $k_o = 2\pi K_{fm}$ ，因此 VCO 得到的输出信号为：

$$s_{fm}(t) = A_c \cos\left(\omega_c t + \theta_{fm}(t)\right) = A_c \cos\left(\omega_c t + 2\pi K_{fm} \times \int_{-\infty}^t s_i(t) dt\right)$$

VCO 输出信号的频率就是对相位的微分，可以表示为：

$$f_{fm\ inst}(t) = f_c + K_{fm} s_i(t) \text{ Hz}$$

可以看到使用 FM 调频之后，已调信号将会包含很多的频率分量。

6.1.2 FM 调制一个单音信号

和前面 AM 调制类似，我们首先来看使用 FM 调制一个单音信号，并且观察其时域和频域波形，从而对 FM 调制有一个更加直观地认识。

首先定义一个消息信号振幅为 A_i 频率为 f_i ，则这个消息信号的表示为：

$$s_i(t) = A_i \cos(2\pi f_i t) = A_i \cos(\omega_i t)$$

由此我们可以算出调制后的信号的瞬时相位，就是使用这个信号与增益系数相乘然后进行积分：

$$\begin{aligned} \theta_{fm}(t) &= 2\pi K_{fm} A_i \times \int_{-\infty}^t \cos(\omega_i t) dt \\ &= 2\pi K_{fm} A_i \times \frac{\sin(\omega_i t)}{\omega_i} \\ &= \frac{K_{fm} A_i}{f_i} \sin(\omega_i t) \\ &= \frac{\Delta f}{f_i} \sin(\omega_i t) \\ &= \beta_{fm} \sin(\omega_i t) \end{aligned}$$

在上面的这个式子中 Δf 叫做频率变化量， β_{fm} 叫做 FM 的调制系数。频率变化量增益系数和当前信号的振幅的乘积，在上面这个消息信号是一个单音信号例子中， Δf 也就是频率分辨率，调制系数就是频率变化量和这个消息信号频率的比值。

因此一个单音信号经过调制之后就可以使用如下方式来进行表示：

$$s_{fm}(t) = A_c \cos\left(\omega_c t + \beta_{fm} \sin(\omega_i t)\right)$$

$$\text{where: } \omega_c = 2\pi f_c \text{ and } \omega_i = 2\pi f_i$$

6.1.3 FM 调制一个基带信号

基带信号一般都是由很多的频率分量所组成的，相较于单音信号，要考虑的东西更多。在本章中，我们还是采用和前面章节当中类似的例子，用三个单音信号来表示一个基带信号，然后采用 FM 的方式来调制这一个基带信号。

假设有三个频率不同的单音信号组成的基带信号，其表示如下：

$$\begin{aligned}
 s_i(t) &= A_{i1} \cos(2\pi f_{i1} t) + A_{i2} \cos(2\pi f_{i2} t) + A_{i3} \cos(2\pi f_{i3} t) \\
 &= A_{i1} \cos(\omega_{i1} t) + A_{i2} \cos(\omega_{i2} t) + A_{i3} \cos(\omega_{i3} t), \\
 \text{where: } \omega_i &= 2\pi f_i.
 \end{aligned}$$

采用 FM 进行调制，有了前面对单音信号的调制过程的介绍，我们很容易得到这个基带信号进入 VCO 产生的瞬时角度如下：

$$\begin{aligned}
 \theta_{fm}(t) &= 2\pi K_{fm} \times \int_{-\infty}^t \left[A_{i1} \cos(\omega_{i1} t) + A_{i2} \cos(\omega_{i2} t) + A_{i3} \cos(\omega_{i3} t) \right] dt \\
 &= 2\pi K_{fm} \left[A_{i1} \int_{-\infty}^t \cos(\omega_{i1} t) dt + A_{i2} \int_{-\infty}^t \cos(\omega_{i2} t) dt + A_{i3} \int_{-\infty}^t \cos(\omega_{i3} t) dt \right] \\
 &= \frac{K_{fm} A_{i1}}{f_{i1}} \sin(\omega_{i1} t) + \frac{K_{fm} A_{i2}}{f_{i2}} \sin(\omega_{i2} t) + \frac{K_{fm} A_{i3}}{f_{i3}} \sin(\omega_{i3} t).
 \end{aligned}$$

最终产生的一条信号的表达式如下：

$$s_{fm}(t) = A_c \cos \left(\omega_c t + \left[\frac{K_{fm} A_{i1}}{f_{i1}} \sin(\omega_{i1} t) + \frac{K_{fm} A_{i2}}{f_{i2}} \sin(\omega_{i2} t) + \frac{K_{fm} A_{i3}}{f_{i3}} \sin(\omega_{i3} t) \right] \right)$$

可以看到这个信号包含了很多的频率分量，但是现在还不能很直观地表示出来，在之后的仿真模型中，我们会直观地来认识这些频率分量，并且能够计算出已调信号的带宽。

6.1.4 宽带调频

在前面的 FM 调制单音信号和 FM 调制基带信号的结果中，不同于 AM 调制，FM 调制很难看出已调信号具有哪些频率分量，占用多少带宽。其实 FM 的带宽也是可以确定的，其中有窄带调频和宽带调频的区别。窄带调频一般用于业余无线电，宽带调频一般用于商用广播。

FM 究竟是窄带调频还是宽带调频取决于调制系数 β_{fm} 。绝大多数时候，所说的调频广播，都是指的宽带调频 WFM。WFM 是大多数商用电台所准寻的标准，其中最大频偏 Δf 是 75KHz，商用电台的带宽也会有限制，比如下图中，是使用

SDR#这个软件接收到的电台广播，可以看到每个电台广播都占有特定的频率。

在下图中所示的大约 2MHz 的频率范围内有 3 个电台，从瀑布图上可以看到每个电台大约占据的频谱的带宽。

在国内商用广播电台的频率一般是 88MHz~108MHz 之间。

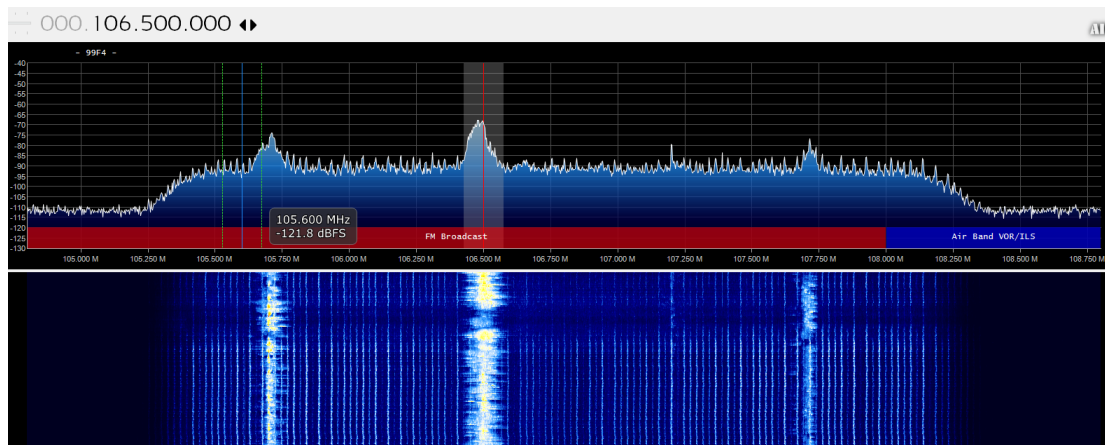


图 6-2 SDR# 接收到广播信号

如果信号的带宽不加限制，那么它在经过 WFM 调制之后将会在最终的载波频率附近产生无限多的频率分量，随着频率分量原理中心载波，其对应频率分量的信号的能量也会降低。

在前面我们介绍 NFM 的时候，使用和差化积公式，将已调信号进行了展开：

$$\begin{aligned}
 s_{fm}(t) &= A_c \cos(\omega_c t + \beta_{fm} \sin(\omega_i t)) \\
 &= A_c \cos(\omega_c t) \cos(\beta_{fm} \sin(\omega_i t)) - A_c \sin(\omega_c t) \sin(\beta_{fm} \sin(\omega_i t))
 \end{aligned}$$

对于含有信息信号的这两个分量，我们可以使用贝塞尔函数或者傅里叶级数将其展开：

$$\begin{aligned}
 \cos(\beta_{fm} \sin(\omega_i t)) &= J_0(\beta_{fm}) + 2 \sum_{n=1}^{\infty} J_{2n}(\beta_{fm}) \cos(2n\omega_i t) \\
 &= J_0(\beta_{fm}) + 2J_2(\beta_{fm}) \cos(2\omega_i t) + 2J_4(\beta_{fm}) \cos(4\omega_i t) + \dots \\
 &\equiv J_0 + 2J_2 \cos(2\omega_i t) + 2J_4 \cos(4\omega_i t) + \dots \\
 \sin(\beta_{fm} \sin(\omega_i t)) &= 2 \sum_{n=1}^{\infty} J_{2n-1}(\beta_{fm}) \sin((2n-1)\omega_i t) \\
 &= 2J_1(\beta_{fm}) \sin(\omega_i t) + 2J_3(\beta_{fm}) \sin(3\omega_i t) + 2J_5(\beta_{fm}) \sin(5\omega_i t) + \dots \\
 &\equiv 2J_1 \sin(\omega_i t) + 2J_3 \sin(3\omega_i t) + 2J_5 \sin(5\omega_i t) + \dots
 \end{aligned}$$

在上面的式子当中，我们 $J_n(\beta_{fm})$ 做了简化，使用 J_n 来代替，实际上这些数据都比较难以求得，我们在这里观察他们的规律。由傅里叶级数可以知道，频率越高的分量其前面的系数一般也越小。所以我们可以看到，随着频率的增加，其频率分量的大小也逐渐减小。

将上面的式子，带入到 FM 调制的表达式当中，就能够得到已调信号的表达式：

$$\begin{aligned}
 s_{fm-wfm}(t) &= A_c \cos(\omega_c t) [J_0 + 2J_2 \cos(2\omega_i t) + 2J_4 \cos(4\omega_i t) + \dots] \\
 &\quad - A_c \sin(\omega_c t) [2J_1 \sin(\omega_i t) + 2J_3 \sin(3\omega_i t) + 2J_5 \sin(5\omega_i t) + \dots] \\
 s_{fm-wfm}(t) &= A_c J_0 \cos(\omega_c t) \\
 &\quad - A_c J_1 [\cos(\omega_c - \omega_i)t - \cos(\omega_c + \omega_i)t] \\
 &\quad + A_c J_2 [\cos(\omega_c - 2\omega_i)t + \cos(\omega_c + 2\omega_i)t] \\
 &\quad - A_c J_3 [\cos(\omega_c - 3\omega_i)t - \cos(\omega_c + 3\omega_i)t] \\
 &\quad + A_c J_4 [\cos(\omega_c - 4\omega_i)t + \cos(\omega_c + 4\omega_i)t] + \dots
 \end{aligned}$$

可以看到，最终一条信号的结果是在围绕着载波中心频率的地方，每隔基带信号的频率依次递减。

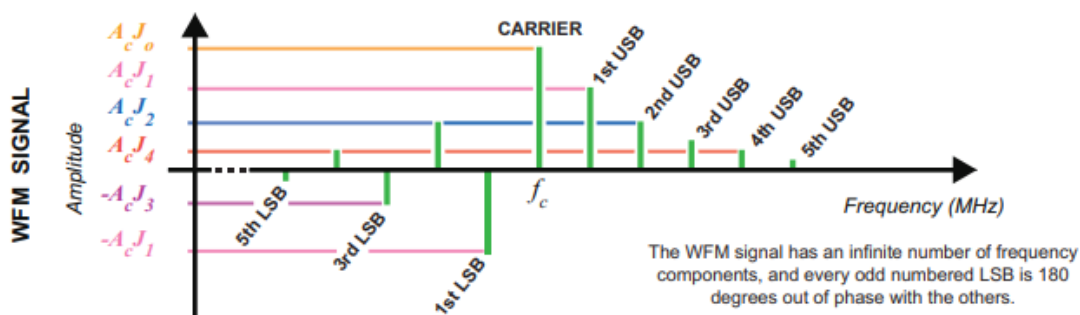


图 6-3 WFM 调制单音信号

对于 WFM 已调信号的带宽，可以根据其中能量比较显著的几个频率分量来确定，绝大部分的信息都承载在能量比较明显的这一区域。通常来说 WFM 已调信号的带宽可以通过卡森公式进行计算。

$$B = 2(\beta_{fm} + 1)f_i$$

当 WFM 调制的不是单音信号而是一个基带信号的时候，这个时候的频率可以用基带信号当中最大的频率分量 f_h 来表示。

6.2 使用 GNU Radio 仿真 WFM

在前面介绍而来 WFM 实现的原理之后，接下来就可以在 GNU Raio 当中来实现这个仿真仿真模型了，直观地感受一下使用 WFM 调频的时候，时域和频域的信号的变化情况。

像前面在原理当中介绍的那样，使用频率调制的时候，基带信号使用调制载波的频率来承载，需要对基带信号进行积分，并将积分的结果作为调制载波的相位。因此在 GNU Radio 中，我们也需要使用一个用于积分的模块。

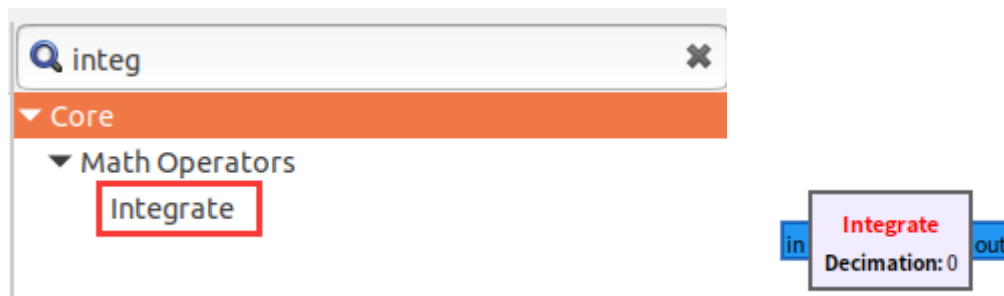


图 6-4 积分模块

得到了积分之后的信号，需要将这个信号作为调制载波的相位，那么如何来生成这么一个信号呢。回忆复数的结构，一个复信号可以跟为幅度和相位两个部分，因此，只需要将这个积分的结果作为这个复数信号的角度即可，因此需要使

用到一个数据类型转换的模块，将幅度和角度转换为复数。

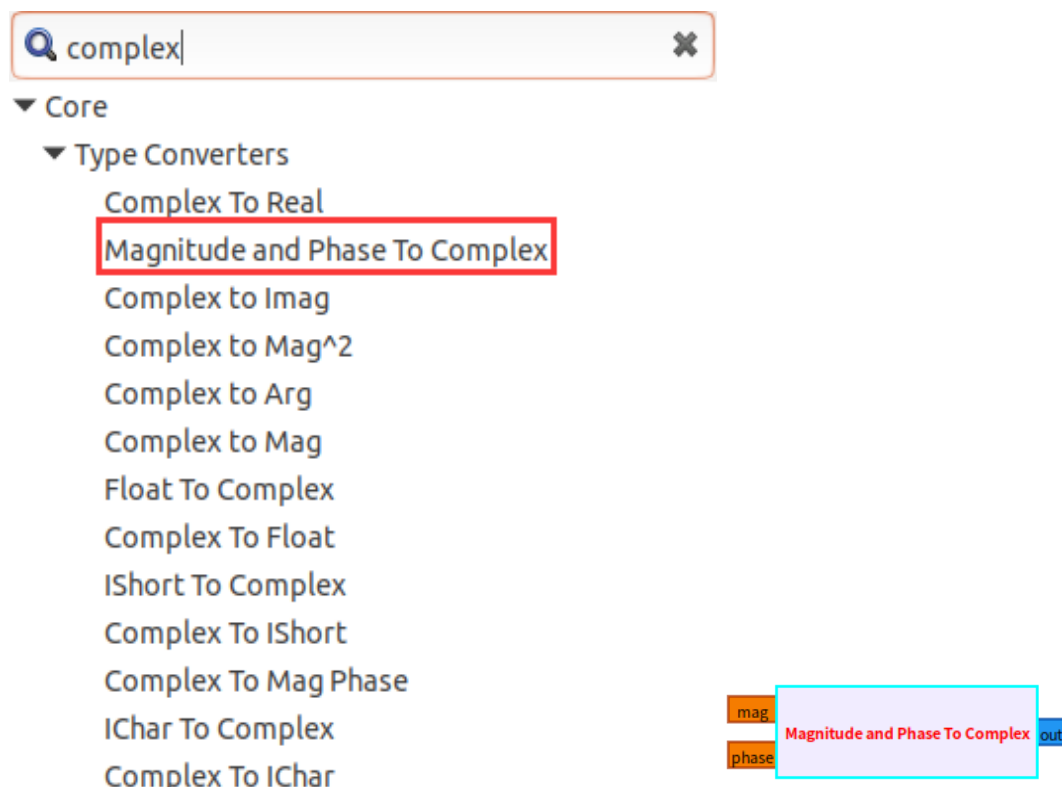


图 6-5 数据类型转换模块

该模块能够将幅度和相位转换成一个复数信号。

有了上面的这些模块之后，我们就可以产生几个单音信号作为基带信号，然后对这几个基带信号进行 FM 调制了。

总体的 GRC 流程图如图 6-6 所示：

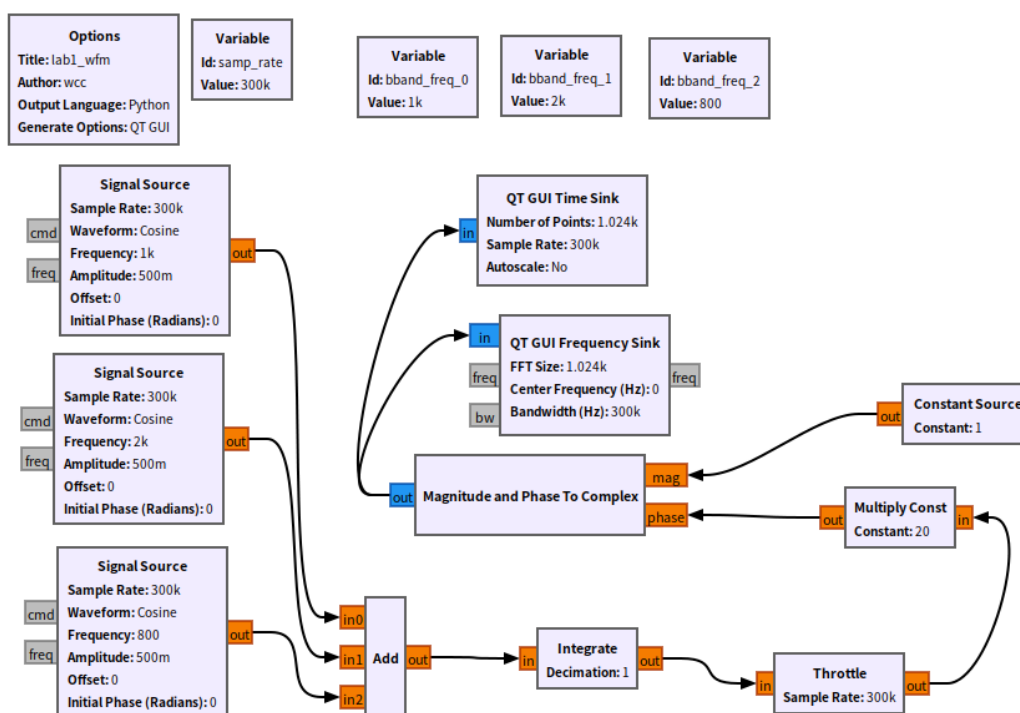


图 6-6 整体 GRC 流程图

点击仿真就可以开始运行这个流程图了。

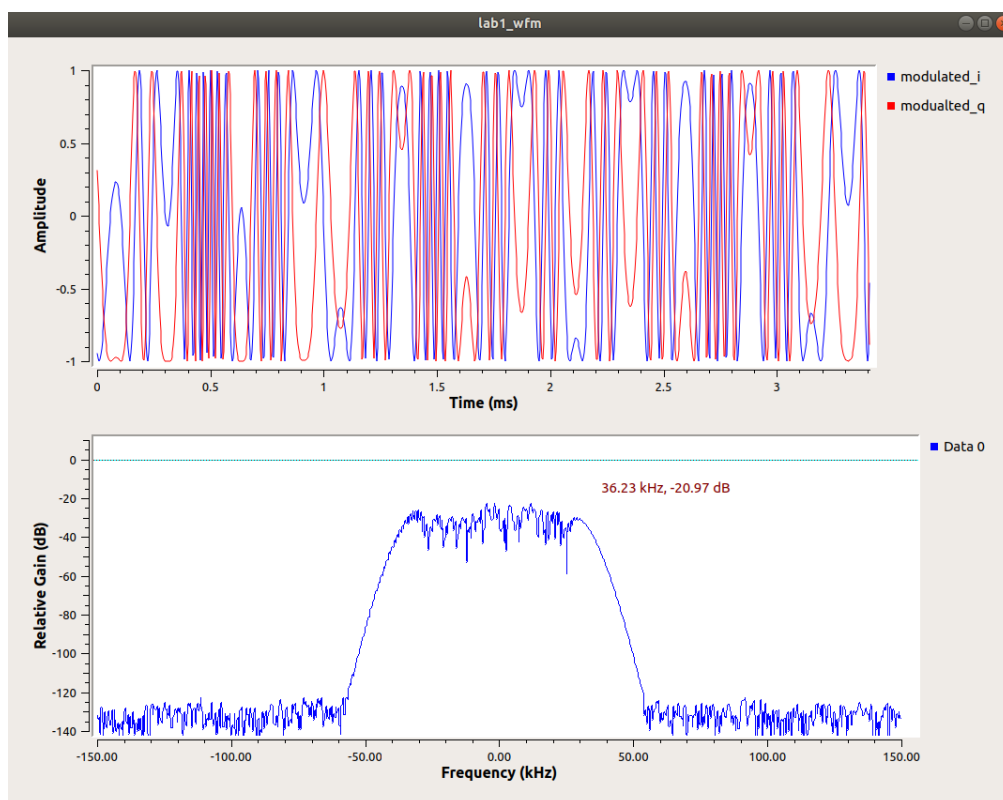


图 6-7 WFM 仿真

可以看到,基带信号是几个频率比较低的单音信号,但是经过 FM 调制之后,占用的频带宽度进行了很大的扩展。

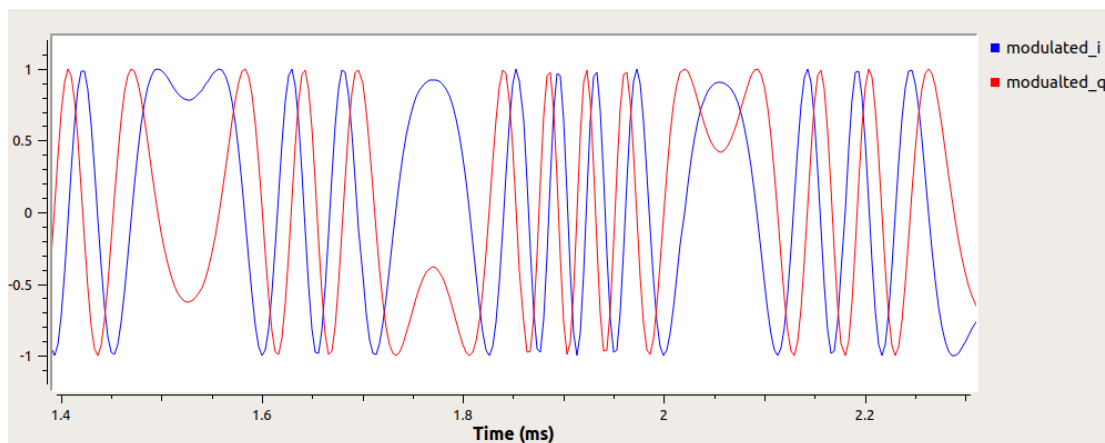


图 6-8 时域波形

从时域波形上不难看出,基带信号幅度越大的地方,其产生的信号的频率也就越快。

6.3 使用 ANTSDR 发送 FM 信号

完成了 WFM 的仿真之后,就可以使用 ANTSDR 来发送一个 FM 信号,通过本次实验,我们可以发送一个 FM 到广播频段,用户若有其他 SDR 设备,或者收音机的话就能够接收到广播并播放了。

和前面 AM 调制的实验类似,首先需要使用一个读取音频文件的模块来获取原始的音频数据。

由于原始音频数据是 48KHz 的采样率,而 ANTSDR 在基带上的采样率会比这个采样率高很多,因此需要对信号进行上采样,所以会使用到两个插值滤波器,完成对信号的上采样工作。

然后根据前面的 FM 的仿真实验,需要对基带信号进行积分之后,作为信号的一个相位,因此还需要使用积分模块来对这个基带信号积分。

具体使用到的模块在前面的实验当中都已经介绍到,在这里不做过多解释。最终整体的 GRC 流程图如图 6-9 所示。

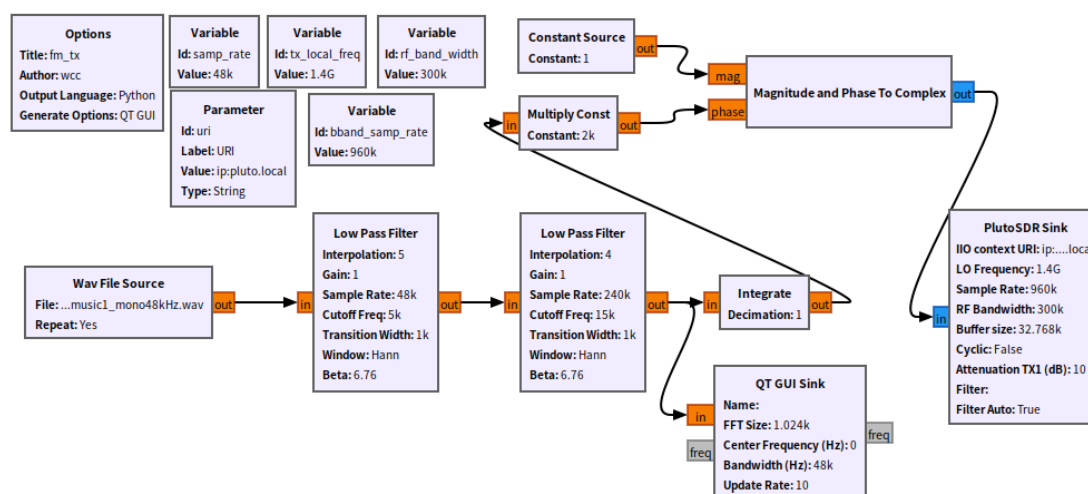


图 6-9 整体流程图

可以看到在上面的 GRC 流图当中，对基带信号进行上采样使用了两级的低通滤波器，可以注意其插值的倍数分别为 5 和 4，因此整体插值为原来的 20 倍。因此经过两级滤波器之后的信号的采样率为 960KHz。

此时，我们将 Pluto Sink 模块的发射频率设置为 1.4GHz，那么最终信号将会被调制到 1.4GHz 上，当然，用户也可以将该信号调制到广播频段，比如 88~108MHz 中的某一处，这样，就可以使用收音机来接收这个广播信号了。

在这里我们调制到 1.4GHz 然后使用 SDR 设备来对这个信号进行接收，在这里笔者使用另外一块 ANTS DR 使用 SDR#来对这个信号进行接收，并且选择解调方式为 WFM，这样，我们就能够使用在播放器当中收听到音频信号了。

接收到的信号如图 6-10 所示：

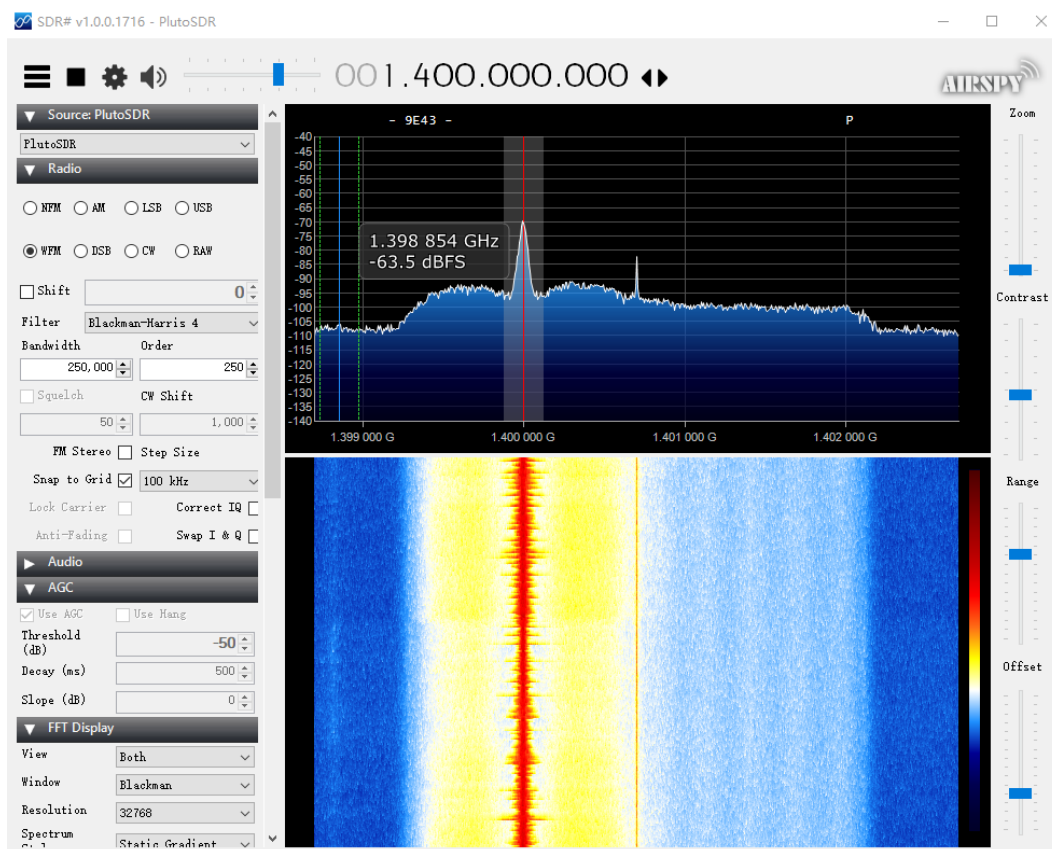


图 6-10 接收到的音频信号

6.4 复数延迟解调

为了更加直观地来介绍如何使用复数延迟鉴别器来解调这个信号，我们还是以解调一个单音信号的 FM 已调信号为例。

FM 信号经过 ANTSDR 进行下变频之后在 GNU Radio 中的复基带信号如下：

$$s_{iq}(t) = \frac{A}{2} e^{j(\theta_{fm}(t))}$$

$$s_{ip}(t) = \frac{A}{2} \cos(\theta_{fm}(t))$$

$$s_{qp}(t) = \frac{A}{2} \sin(\theta_{fm}(t))$$

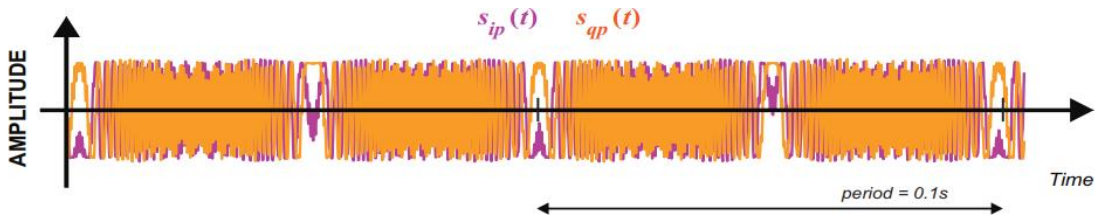


图 6-11

接下来，我们对这个复基带信号进行处理，首先使用一路信号来对这个复基带信号进行取共轭的操作，取共轭后的信号如下：

$$s_{conj}(t) = \frac{A}{2} e^{-j(\theta_{fm}(t))}$$

然后对没有取共轭之前的原始信号进行延时操作，延时过后的信号如下：

$$s_{delay}(t) = \frac{A}{2} e^{j(\theta_{fm}(t-\tau))}$$

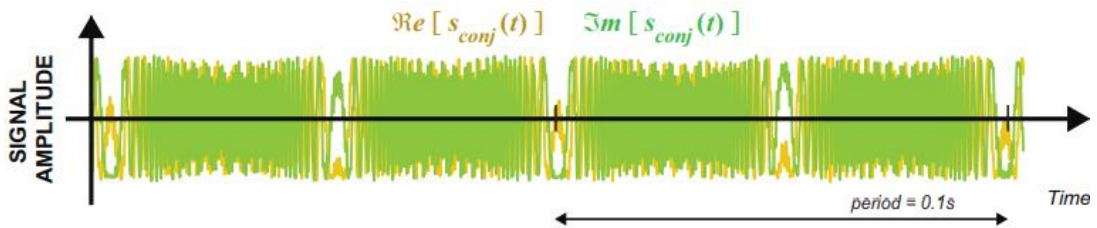


图 6-12 取共轭的信号

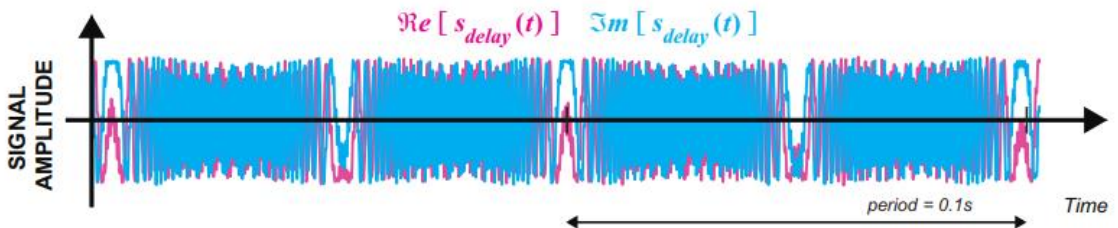


图 6-13 延时之后的信号

然后将这个延时之后的信号和共轭的信号进行相乘：

$$\begin{aligned} s_{pd}(t) &= s_{conj}(t) \times s_{delay}(t) \\ &= \frac{A^2}{4} e^{-j(\theta_{fm}(t) - \theta_{fm}(t-\tau))} \end{aligned}$$

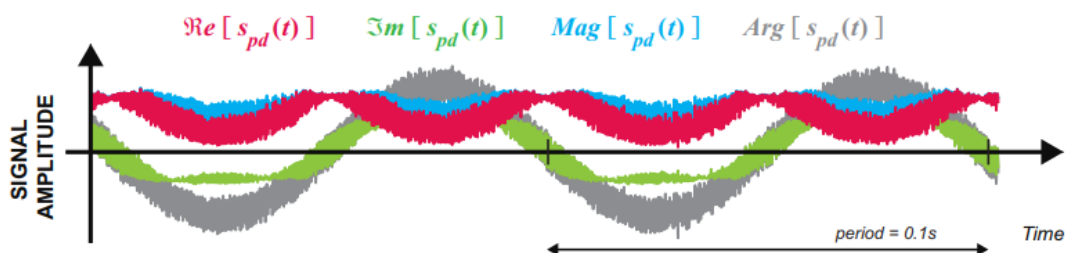


图 6-14 相乘之后的信号

可以看到，经过鉴相后的信号，其角度中包含有消息信号的有效信息，此时我们可以求取信号的角度。

$$s_d(t) = \angle s_{pd}(t) = -(\theta_{fm}(t) - \theta_{fm}(t - \tau))$$

当这个延时很小的时候，我们可以将这个角度看作近似微分的关系：

$$s_d(t) = \frac{d}{dt} \theta_{fm}(t) = \theta_{fm}'(t)$$

现在这个结果和前面使用复数微分鉴相器得到结果是一致的，所以我们通过这种方法就成功地将消息信号从复基带信号中提取了出来。

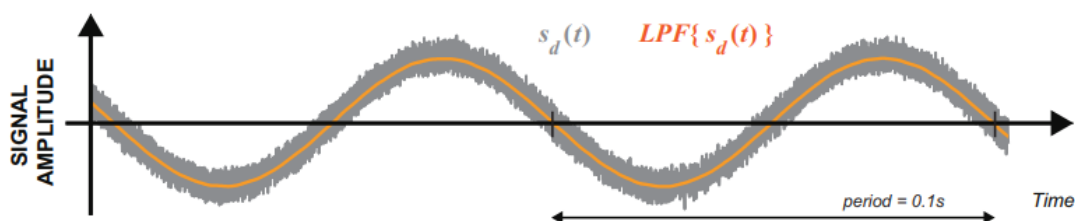


图 6-15

复数延迟鉴别器也被称作 FM 转 AM 操作，它将 FM 的频率变化转换到了幅度的变化。

6.5 复数延迟解调仿真

从上面的原理中可以看出，对于 FM 的复数延迟解调器，其主要部分就是实验对接收到的已调信号进行延时和取共轭操作，在 GNU Radio 当中，也为我们提供了这些模块。

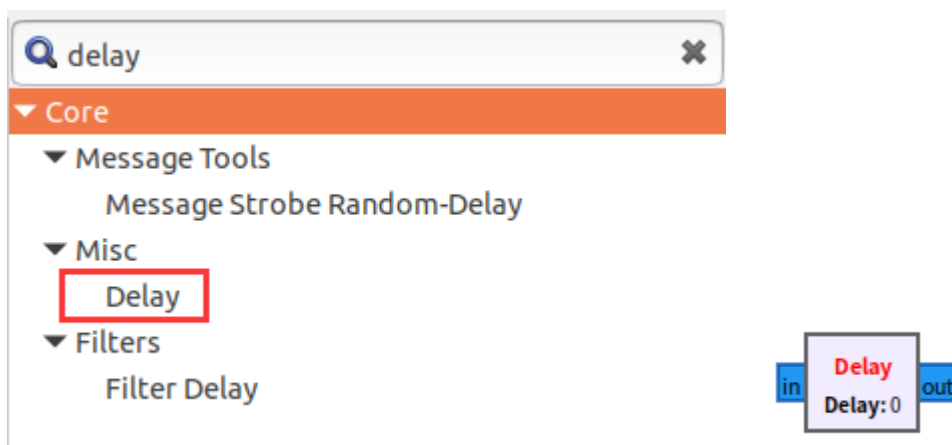


图 6-16 延迟模块

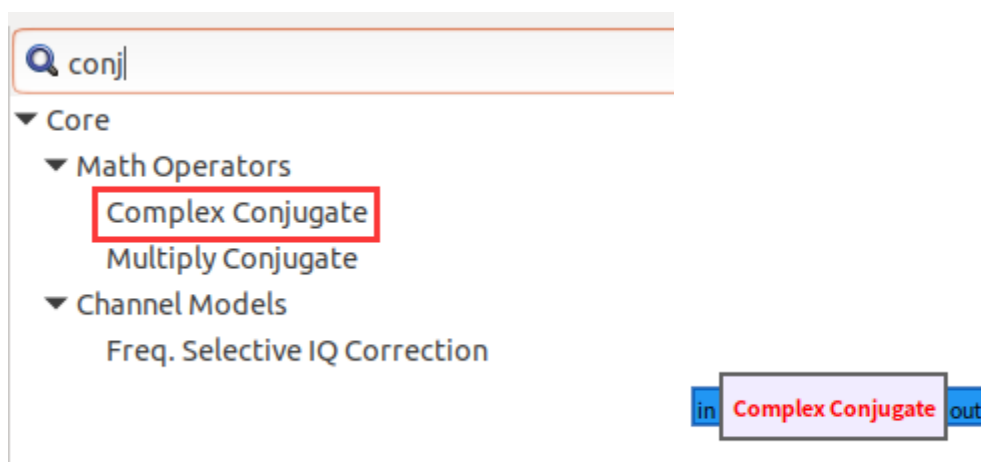


图 6-17 取共轭模块

在将信号进行延时和取得共轭之后，需要将延时信号和共轭信号进行相乘，这样便于我们在之后提取出基带信号。相乘之后得信号，只需要将其中的高频分量滤除，就能够得到低频信号了。

最终整体得 GRC 流图如图 6-18 所示。

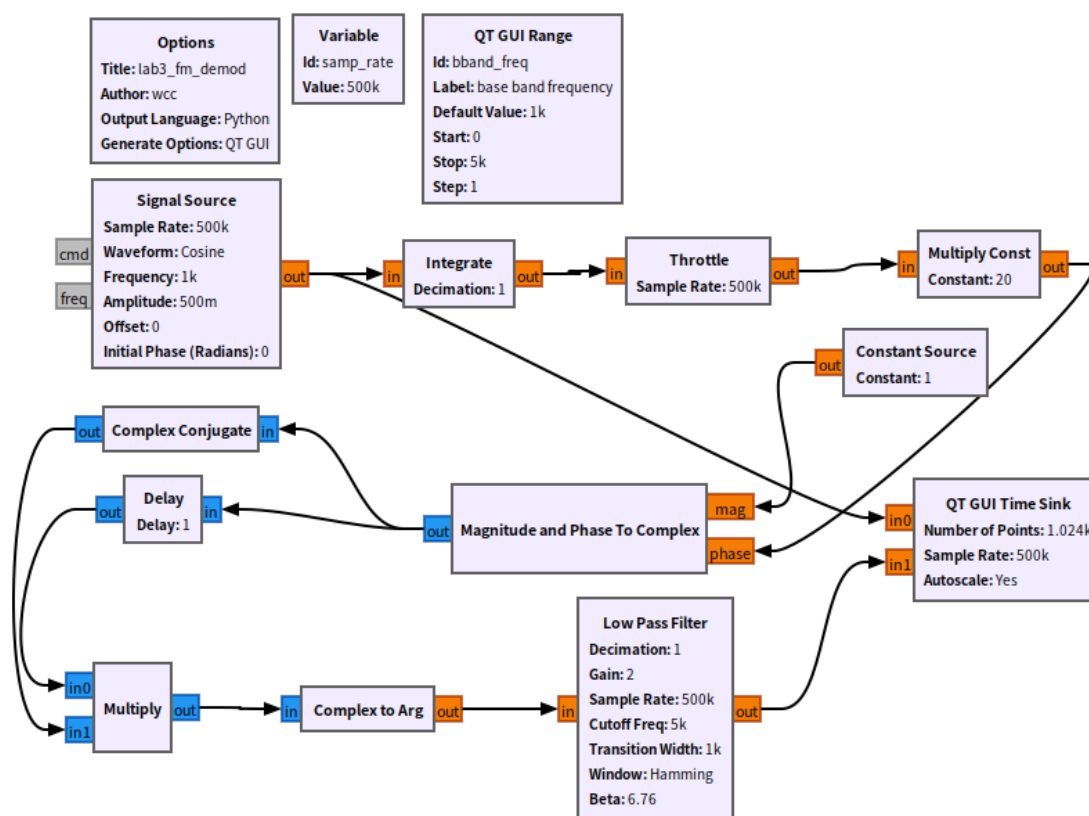


图 6-18 FM 解调整体流程图

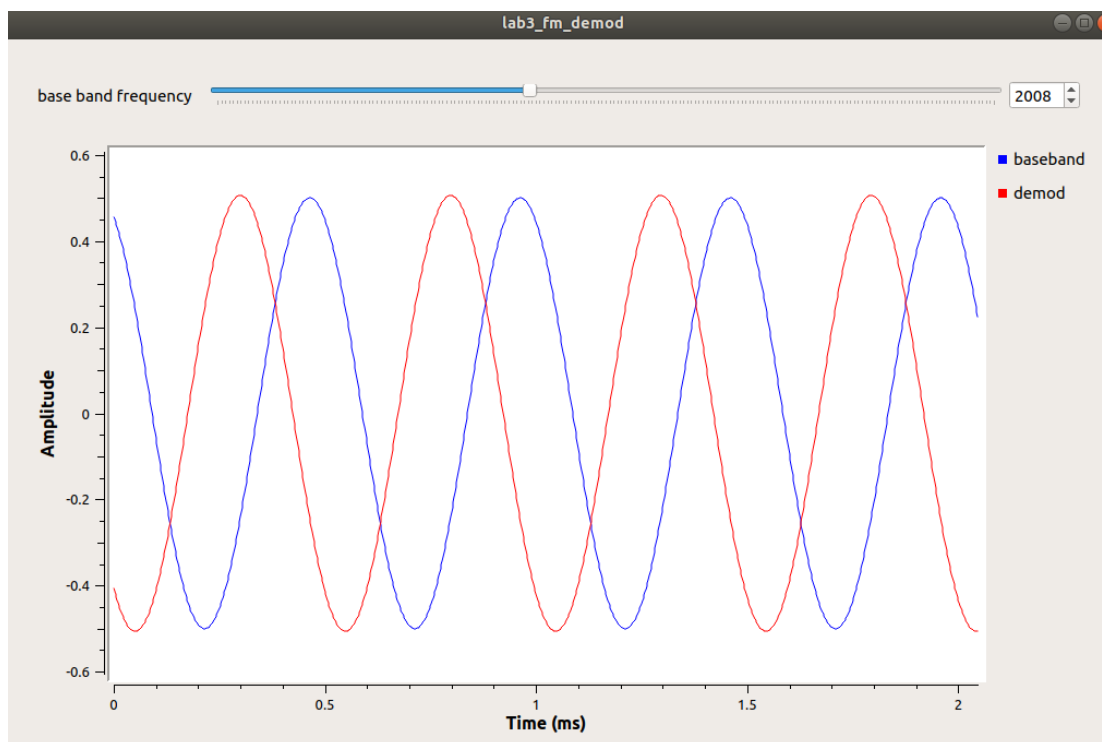


图 6-19

可以看到，经过解调之后，得到得基带信号的频率和原始信号一致，在不同

频率下得到的信号的原始信号相比，得到的信号的幅度可能会有一个比例缩放的关系。

6.6 使用 ANTSDR 解调 FM 信号

弄清楚了 FM 解调的原理之后，就可以来实现一个 FM 解调器了。FM 解调器，在前面进行 FM 解调仿真的时候，其实已经实现了，在使用 ANTSDR 进行解调商用广播电台的时候，只需要将 pluto source 模块添加进来就可以了。

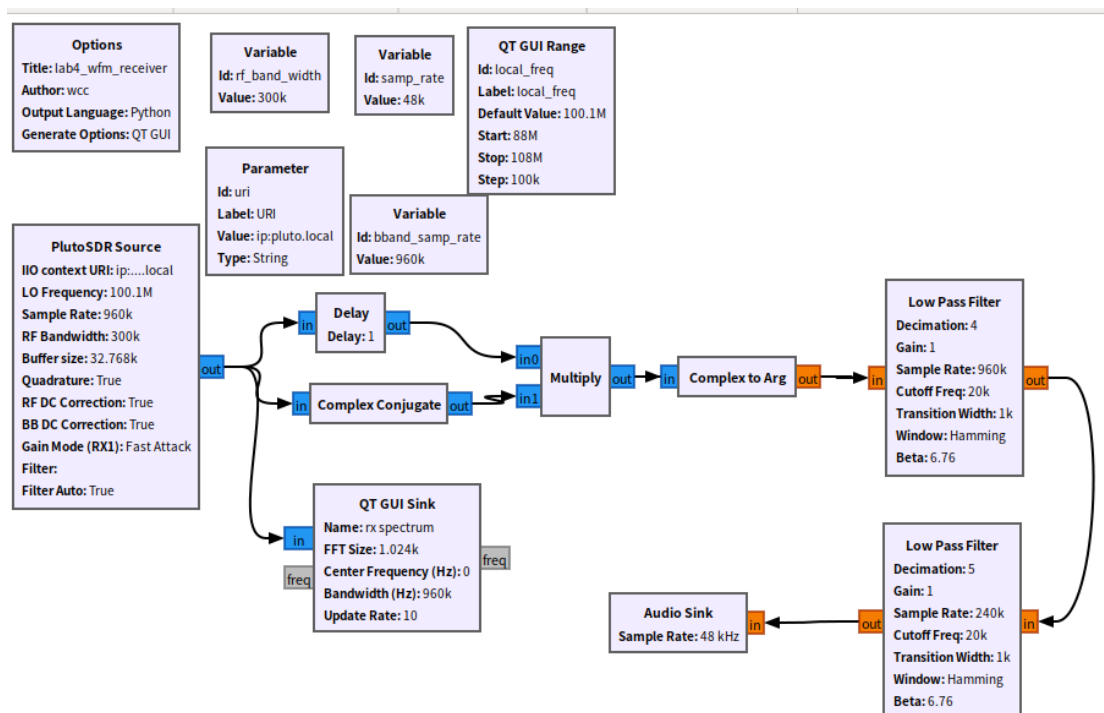


图 6-20 FM 解调 GRC 流图

在上面的流图中，首先将中心频率设置为 100.1MHz，在笔者所在位置，该处频率为一个音乐广播电台，在该位置能够收听到商用广播电台信号。

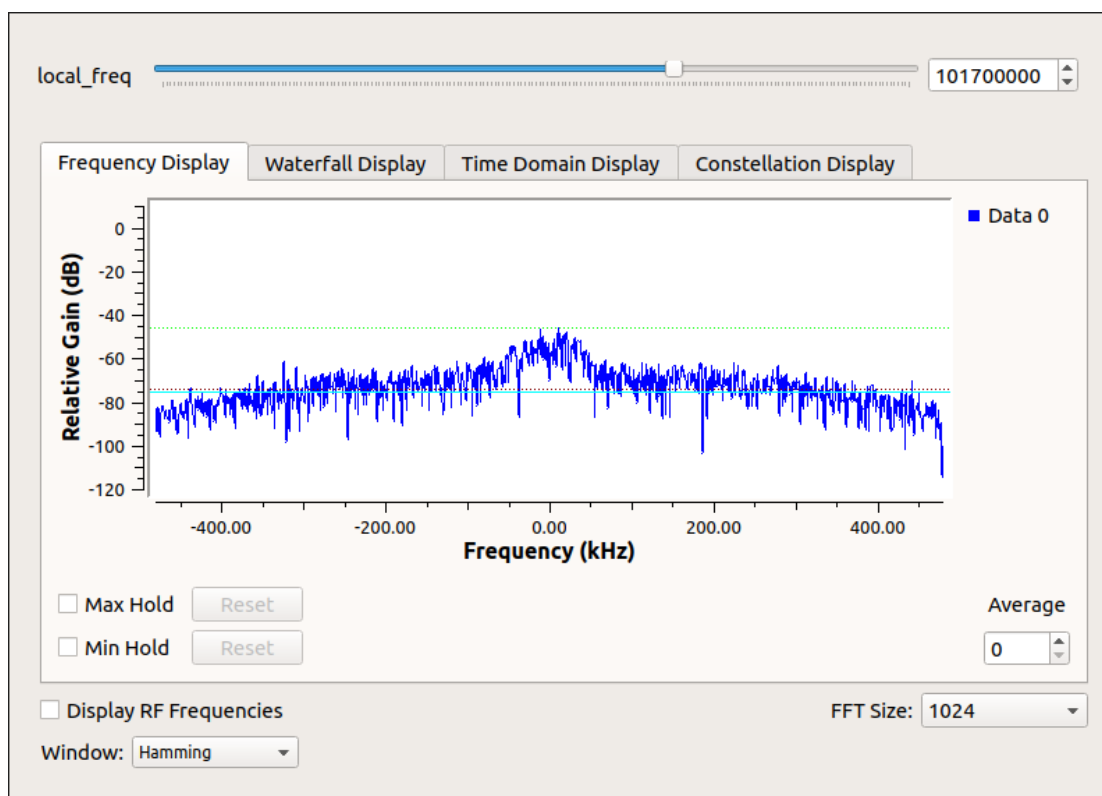


图 6-21 频谱，101.7MHz 处有一电台

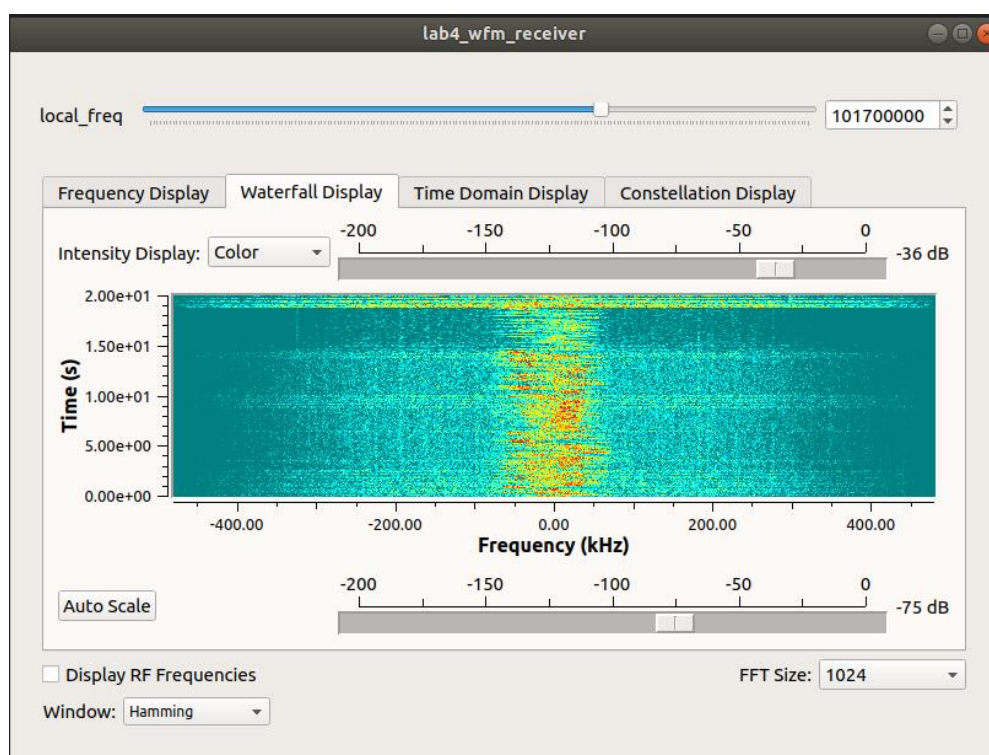


图 6-22 根据瀑布谱分析信号电台所处位置

6.7 总结

在本章中介绍了 FM 调制解调的原理，并且给出了 FM 调制解调的具体实现方法。能够发送 FM 信号到信道中，在其他设备上接收，除此之外，还接收了商用电台的广播信号。

在本章当中使用到的调制解调方法，都是使用自己的方式来实现的。其实在 GNU Radio 当中，也已经提供了一些用于 FM 调制解调的模块，在实际的使用中，其实可以直接使用这些模块来进行调制解调，用户可以自己实现一下，实现的方法十分简单。

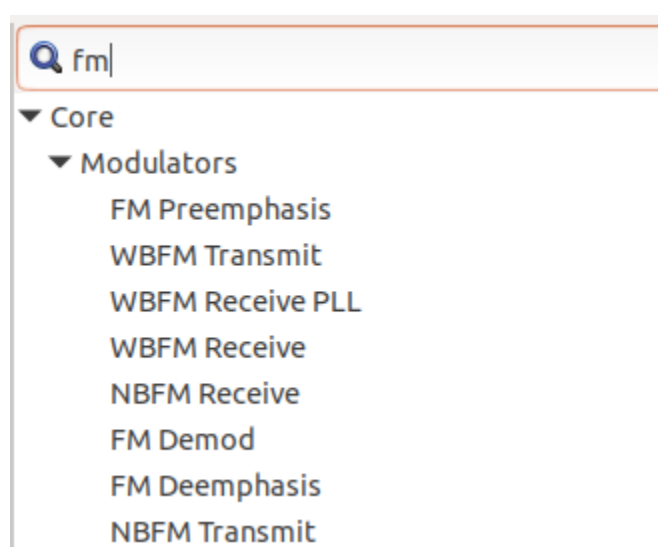


图 6-23 FM 解调模块

第7章 ASK 的调制与解调

7.1 ASK 调制

有两种类型的 ASK 调制，一种是 On Off Keying(OOK)，一种是 Amplitude Shift Keying(ASK)。

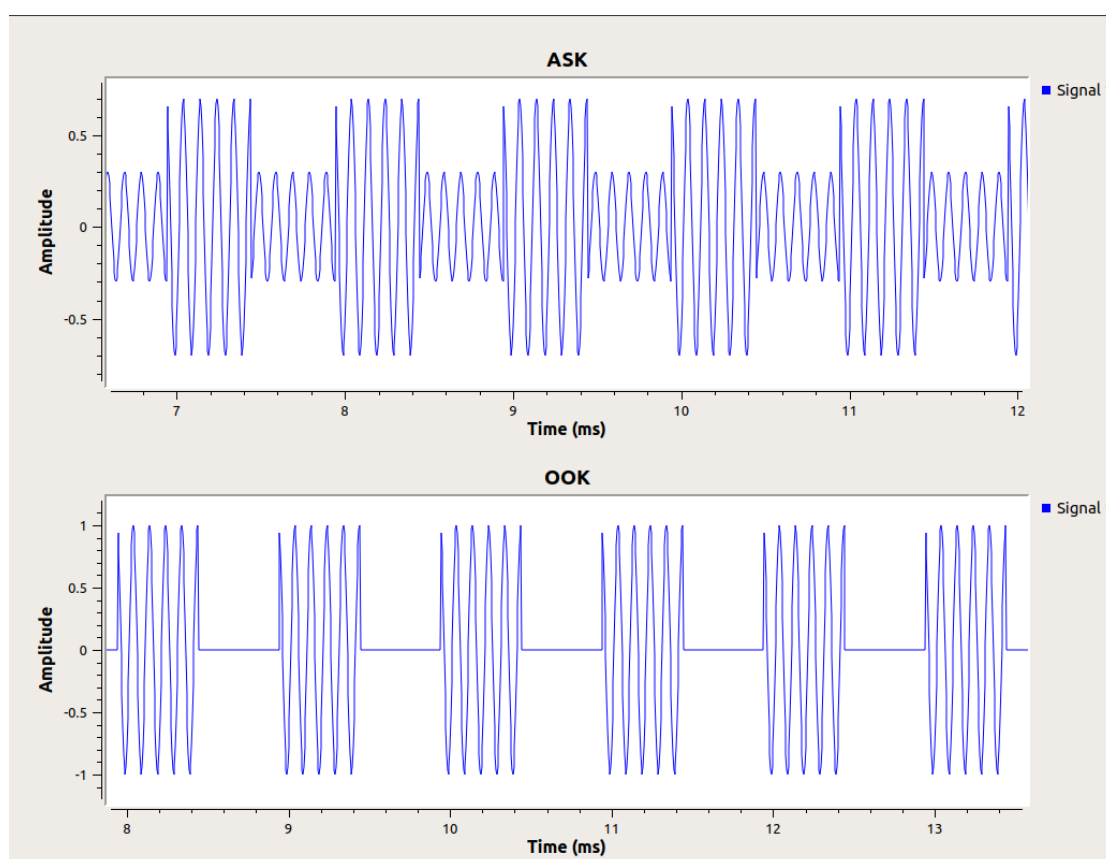


图 7-1 两种不同的 ASK

在图 7-1 的上方是 ASK 信号，在图 7-1 的下方是 OOK 信号，这两种调制信号的区别就是当基带信号是 0 的时候是否还需要发送波形。对于 OOK 信号，在基带信号为 0 的时候，传输信号幅度为 0，对于 ASK 信号，在基带信号为 0 的时候，传输的信号幅度不为 0。

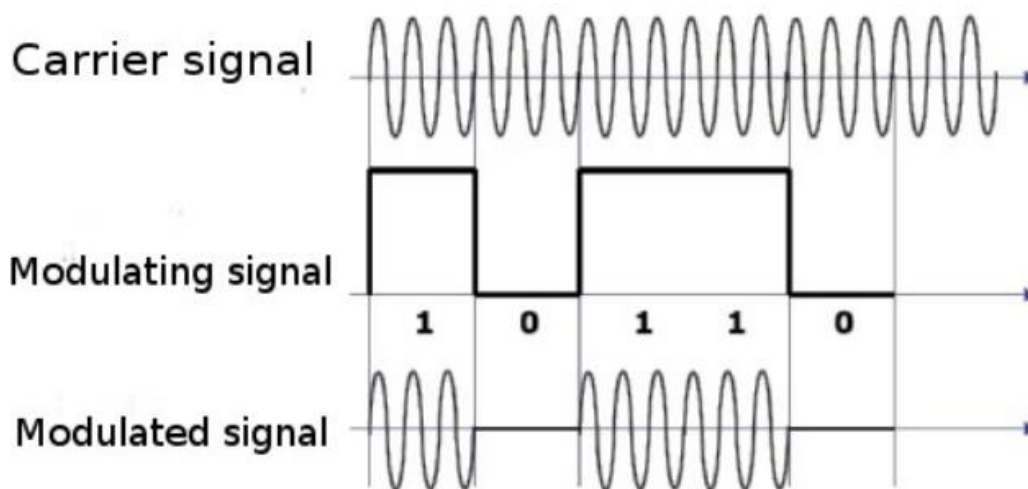


图 7-2 OOK 调制的形式

7.2 在 GNU Radio 当中生成 OOK 信号

新建 GRC 流图 `lab1_ask_mod.grc`。

在 GNU Radio 当中生成一个 OOK 信号十分简单，首先需要产生一个高频载波信号，在图 7-3 的流图当中高频载波的频率为 10KHz。然后产生需要发送的基带信号，这里基带信号使用一个方波信号发生器来产生，其信号产生频率为 1KHz，如图 7-4 所示

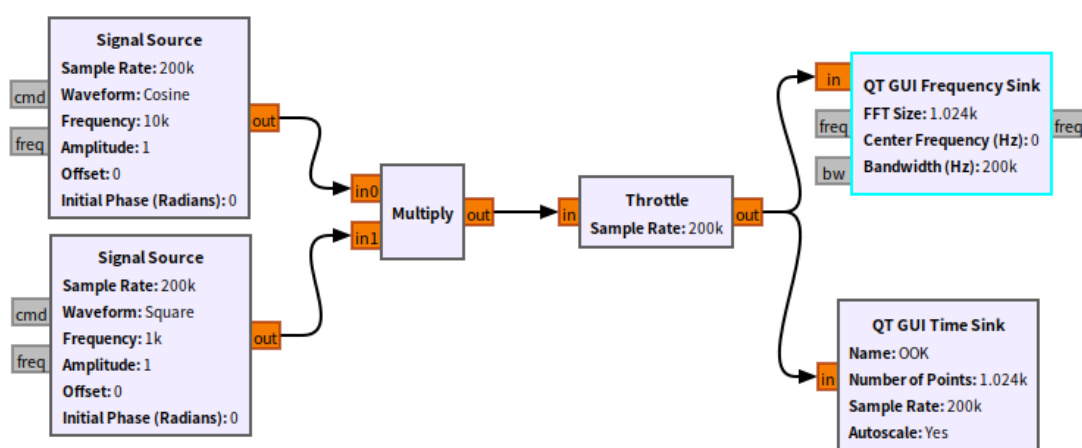


图 7-3 OOK 产生流图

因为已调信号中包含有直流分量，所以可以看到再经过调制后的信号的频率分

量会扩展到整个频谱上。

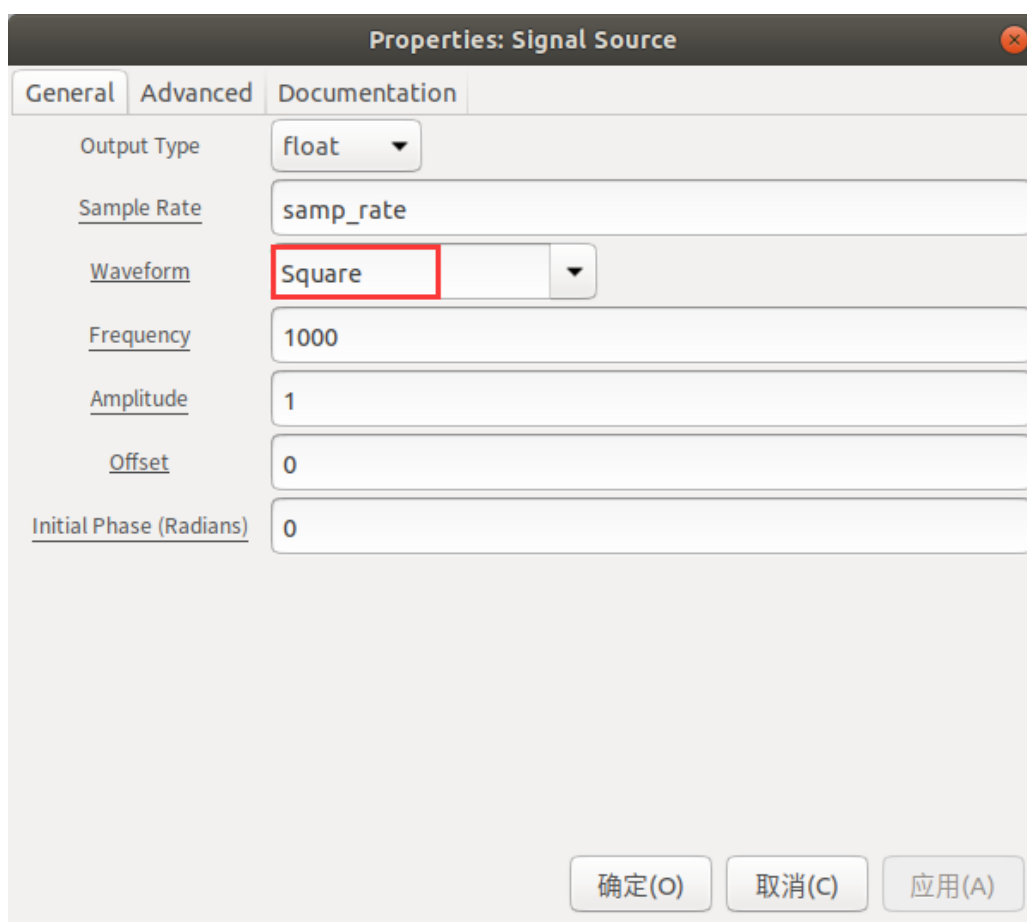


图 7-4 方波信号

运行仿真，就能够得到一个 OOK 的已调信号。

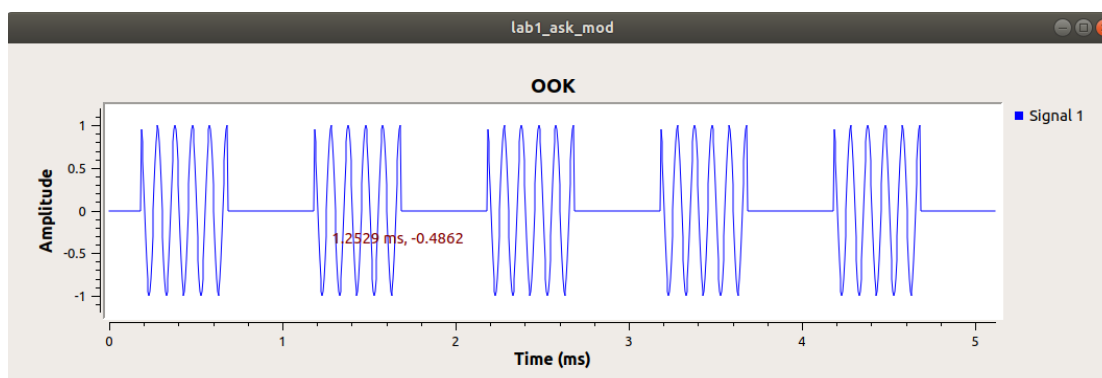


图 7-5 OOK 已调信号

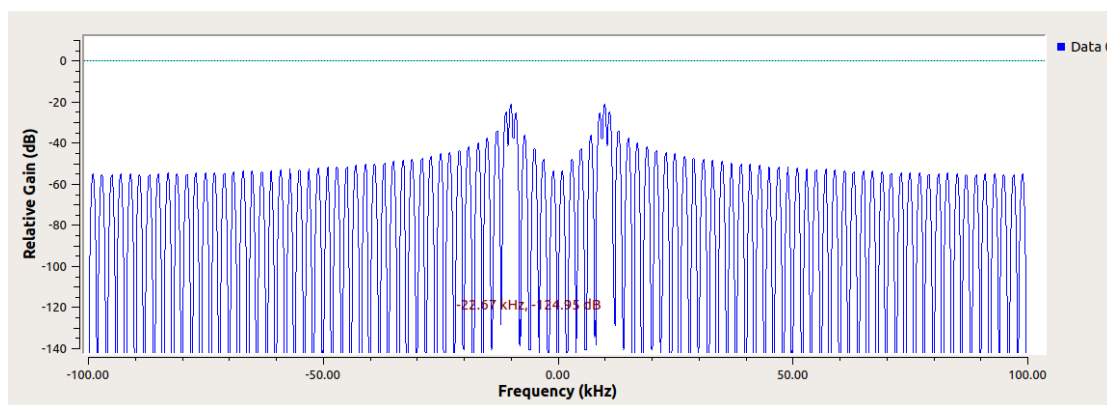


图 7-6 OOK 已调信号频谱

可以看到 OOK 信号的产生十分简单，这种调制方法下，将基带信号的幅度使用承载与载波之上。当然这是最简单的 2ASK 信号。这在基带信号是 0, 1, 0, 1 这种 bit 序列的时候，能够使用 2ASK 这种方式很容易地进行调制。

7.3 ASK 信号的解调

从 ASK 信号的形式上来看，和之前模拟调制时使用到的 AM 调制是很类似的，只是在模拟调制中，信号的幅度有多种，而在数字调制中，调制的是 0, 1 的序列，因此在进行 ASK 信号的解调的时候，可以参考 AM 信号的解调，使用包络检波的方法来对 ASK 信号进行解调。

具体的方法就是对已调信号进行取绝对值或者求得模的平方，然后就再使用低通滤波器将高频信号过滤，只保留低频信号。这样就能够信号进行解调了。新建流图 `lab2_ask_demod.grc`。

整个 ASK 调制解调的 GRC 流图如图 7-7 所示

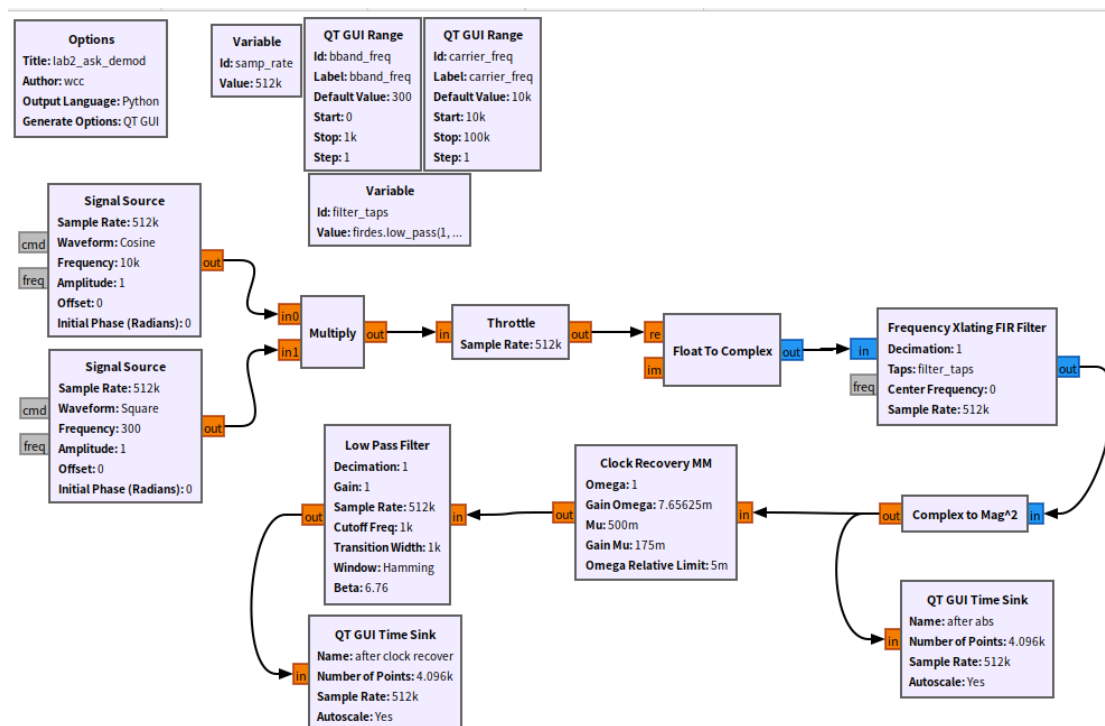


图 7-7 ASK 解调 GRC

在上面的图中，首先使用了一个 FIR 滤波器，该滤波器的作用是保留带内信号，去除噪声信号，虽然在上面的流图中，并未引入噪声信号，但是添加这个模块，还是能够有效地保留带内信号。

然后使用复数转模方的模块，该模块用于求复数的模。

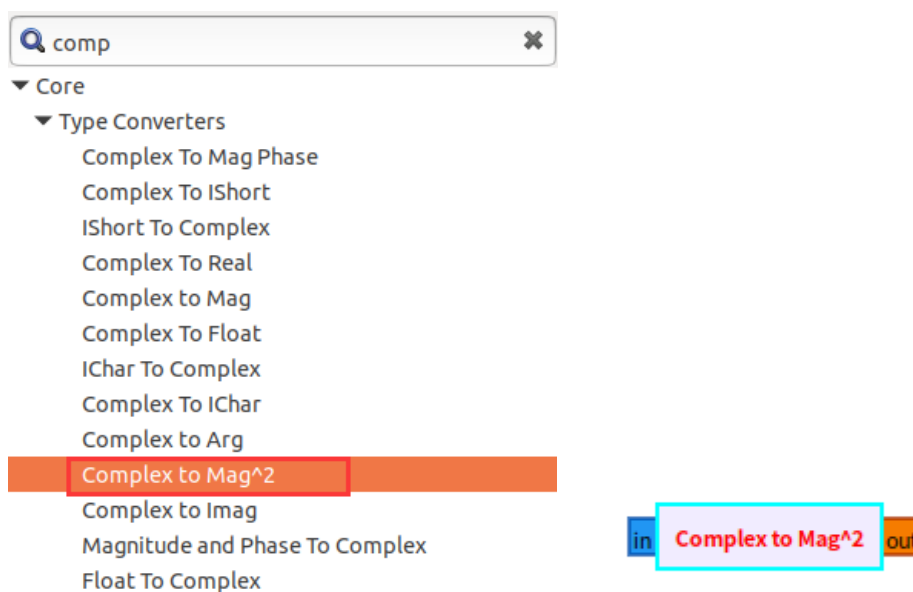


图 7-8

然后使用一个时钟恢复模块，该模块能够完成时钟恢复和同步的作用，以便能够用于提取符号。

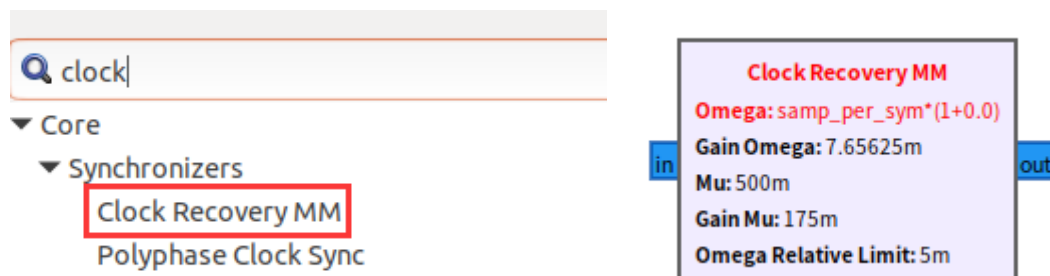


图 7-9

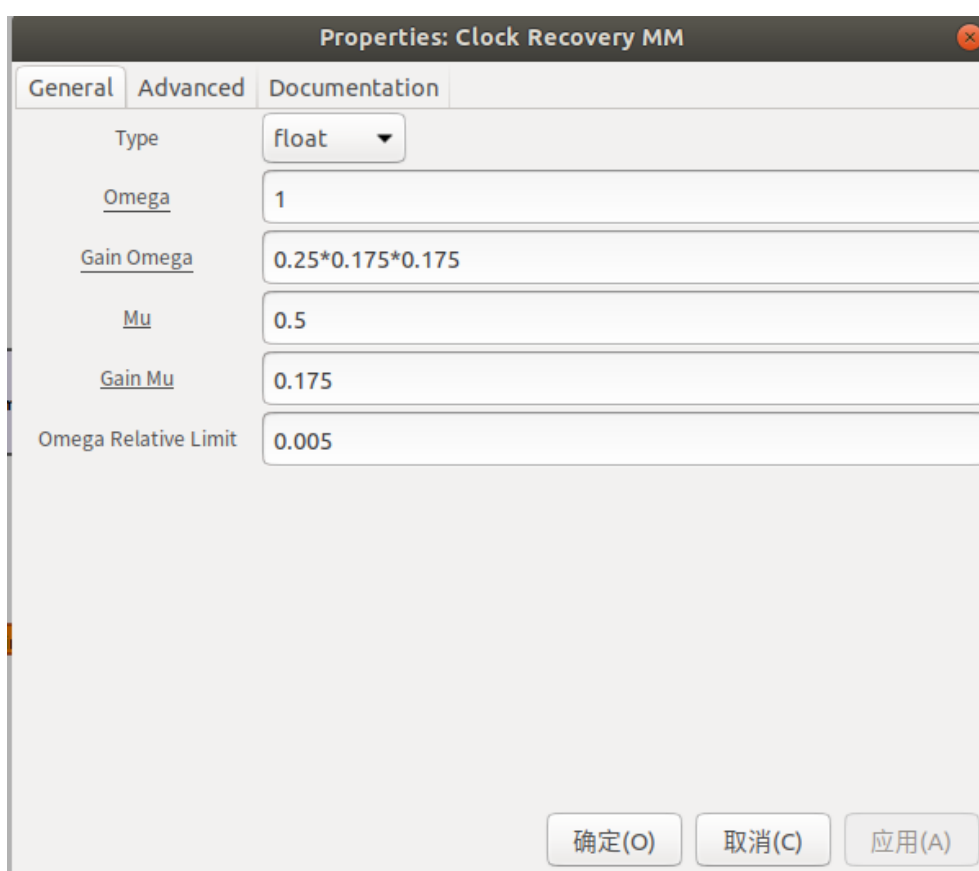


图 7-10 时钟恢复模块设置

在最后使用一个低通滤波器来滤除高频分量保留低频分量，这样就能够复原原始信号了。

然后就可以运行上面的 GRC，来对 ASK 的调制解调进行仿真了，其运行的结果如图 7-11 所示。

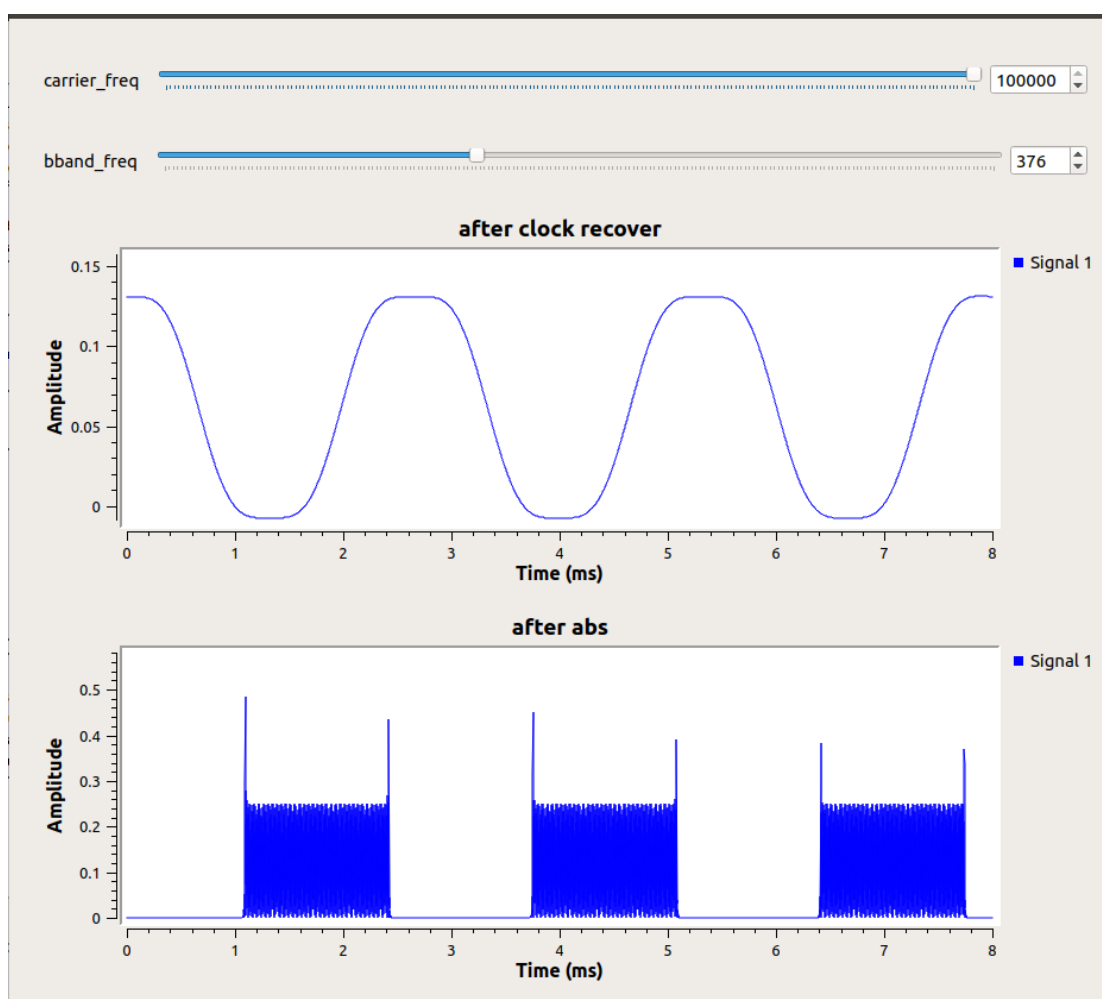


图 7-11 运行结果

可以看到在进行了求取复数模方之后，信号变得十分密集，其中含有高频分量。在最终经过时钟恢复和低通滤波之后，成功地将这些高频分量进行了滤除，得到了解调之后的信号。

在这之后，只需要按照定时脉冲将数据正确地恢复出来即可。

7.4 使用 ASK 发送文本数据

在前面介绍了 ASK 的基本原理并且进行仿真之后，我们了解了 ASK 调制解调的基本的方法，当时前面在进行 ASK 调制与解调的时候，使用都是固定的数据来进行收发。接下来可以使用 GNU Radio 来使用 ASK 信号收发文本数据。

在进行文本数据的收发的时候，就需要使用到前面第 4 章当中介绍的流标签以及帧格式等基础知识，来组成一个用户自己的帧，用户可以参考第 4 章中的内容来完成接下来的实验。新建 GRC 流图 `lab3_ask_file.grc`。

在进行传输文本数据之前，先新建两个文本文件，其中一个文本文件用作数据源，另外一个文本文件用作接收解调之后的数据。其中 `text.txt` 是数据源文件，`text.out` 是接收数据的文本文件。

```
wcc@wcc-dev:~/Desktop/gnuradio_demo/Chapter6$ ll
总用量 92
drwxrwxr-x 3 wcc wcc 4096 4月 19 15:52 ./
drwxrwxr-x 9 wcc wcc 4096 4月 16 09:40 ../
drwxrwxr-x 3 wcc wcc 4096 4月 19 15:01 gnuradio_ask_decoder/
-rw-rw-r-- 1 wcc wcc 6180 4月 15 10:25 lab1_ask_mod.grc
-rwxrwxr-- 1 wcc wcc 8623 4月 15 10:01 lab1_ask_mod.py*
-rw-rw-r-- 1 wcc wcc 9901 4月 15 10:36 lab2_ask_demod.grc
-rwxrwxr-- 1 wcc wcc 11971 4月 15 10:36 lab2_ask_demod.py*
-rw-rw-r-- 1 wcc wcc 12658 4月 19 15:45 lab3_ask_file.grc
-rwxrwxr-- 1 wcc wcc 12756 4月 19 15:23 lab3_ask_file.py*
-rw-rw-r-- 1 wcc wcc 0 4月 19 15:10 text_out.txt
-rw-rw-r-- 1 wcc wcc 1900 4月 19 14:41 text.txt
wcc@wcc-dev:~/Desktop/gnuradio_demo/Chapter6$
```

图 7-12 新建两个文本文件

在数据源模块当中，其文件内容如下，一些 `hello world` 和递增数组组成的文件。

```
wcc@wcc-dev:~/Desktop/gnuradio_demo/Chapter6$ cat text.txt
hello world 00 !!!
hello world 01 !!!
hello world 02 !!!
hello world 03 !!!
hello world 04 !!!
hello world 05 !!!
hello world 06 !!!
hello world 07 !!!
hello world 08 !!!
hello world 09 !!!
hello world 10 !!!
hello world 11 !!!
hello world 12 !!!
hello world 13 !!!
hello world 14 !!!
hello world 15 !!!
hello world 16 !!!
hello world 17 !!!
hello world 18 !!!
hello world 19 !!!
```

图 7-13 text.txt 的文件内容

准备好这两个文件之后就可以开始在 GNU Radio 当中来实现基于 ASK 的文本信息发送了。

首先，使用一个文件源模块，将文本文件读取进来，需要使用到 file source 模块。读取文件之后，对数据流进行添加标签。使用到 stream to tagged stream 模块。

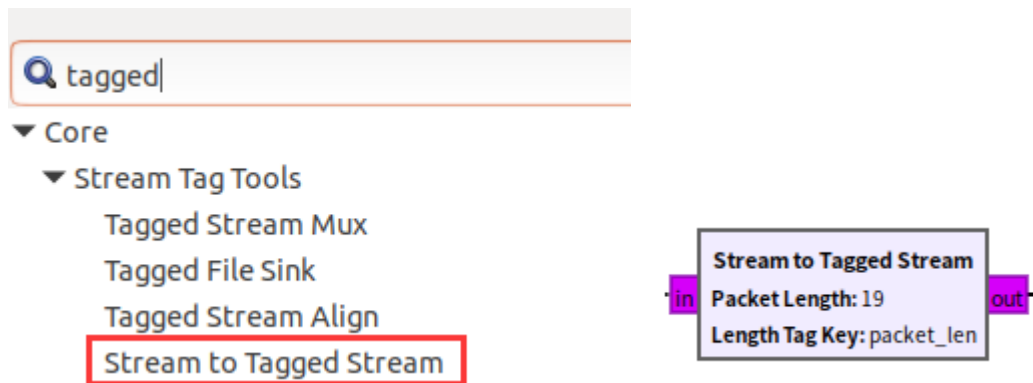


图 7-14 给流打上标签

然后就可以生成协议头，并将协议头添加到数据中，如图 7-15 所示

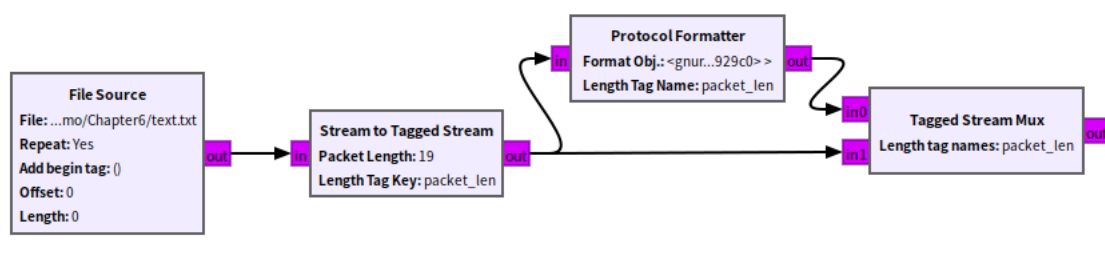


图 7-15 组成数据帧

在之后，就可以将数据帧转换成 bit 流了，并且将每一个点都 repeat 100 次，这样做的目的是能够生成 OOK 的基带信号。由于在系统中限制了最大采样率是 100KHz，而基带信号的速率远低于这个速率，因此需要将原始采样点复制 100 次，这样就能够得到频率较低的基带信号，如图 7-17 所示

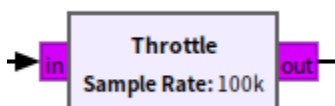


图 7-16 限制最大采样率

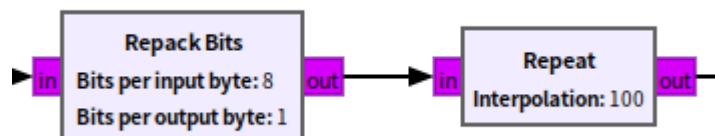


图 7-17 生成 OOK 基带信号

然后我们就可以进行 OOK 的调制和非相干解调了。在调制的时候，使用一个频率为 10KHz 的载波进行调制，让后对这个调制之后的信号进行非相干解调。在前面的 ASK 解调和 AM 解调当中，我们都介绍过非相干解调的实现的方法。这里直接取绝对值，然后对信号进行低通滤波就能够将基带信号复原了。

需要注意的是，在进行信号的复原的时候，对解调出来的信号进行了 100 倍的抽取，因为在产生 OOK 基带信号的时候，对 1 个点重复了 100 次，在这里需要对信号进行复原，需要进行 100 倍抽取。如图 7-18 所示。

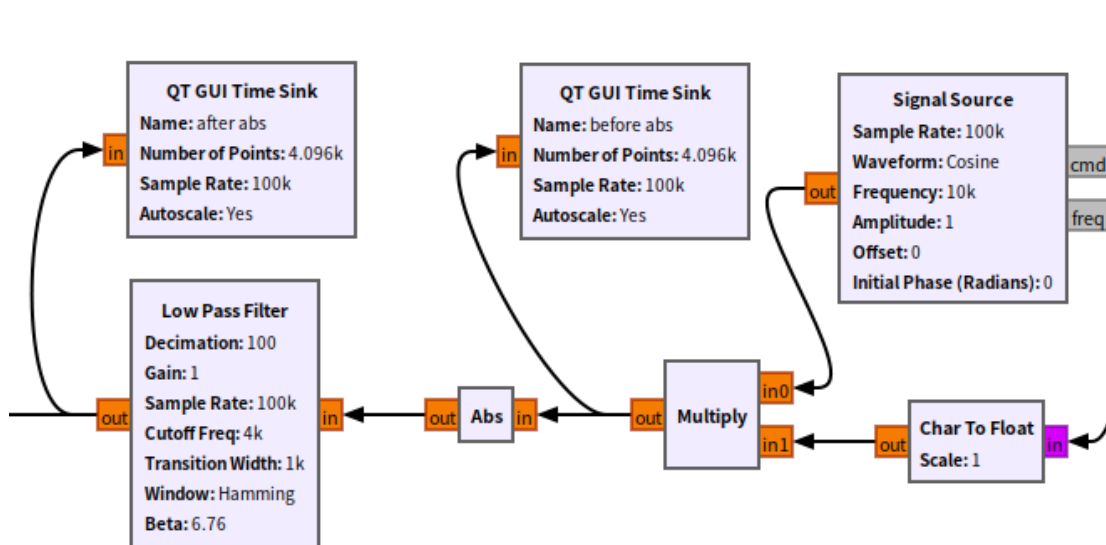


图 7-18 进行 OOK 调制和非相干解调

最后需要进行帧的同步，在这里就是来检测帧头，当检测到帧头之后，就可以将后续的数据恢复成原始的数据。如图 7-19 所示。

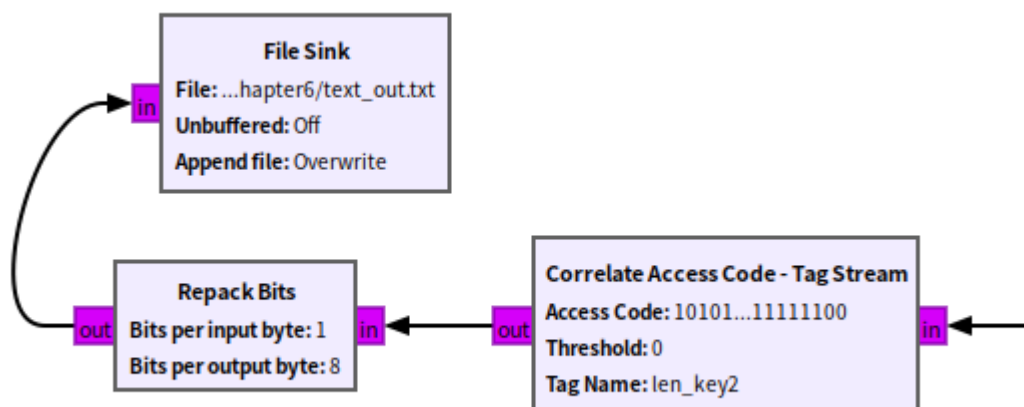


图 7-19 检测帧头，并解调出数据

最终，整个 GRC 流程图如图 7-20 所示

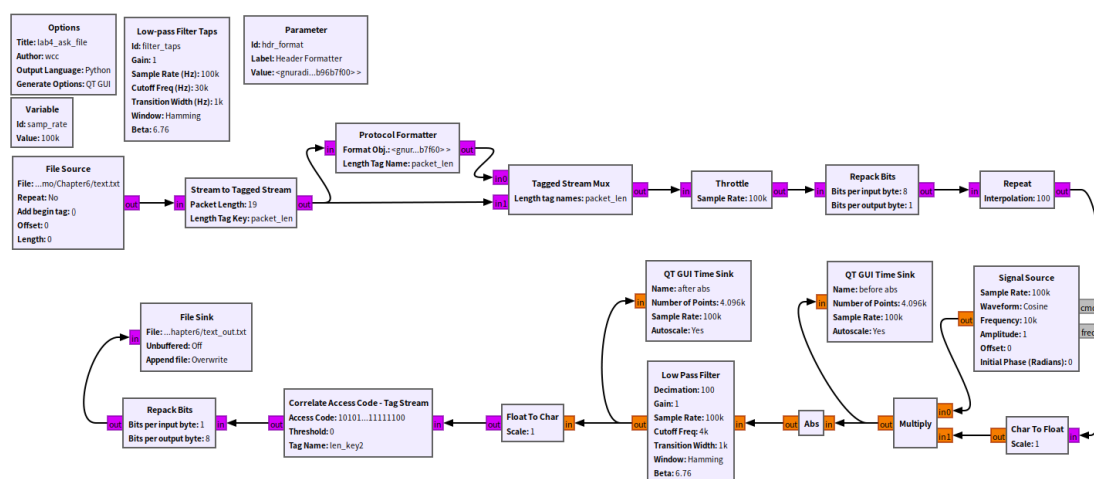


图 7-20 ASK 文件传输流程图

点击运行仿真，就可以开始运行这个流程图了，运行一段时间之后，可以停止运行仿真，然后看此时的 text_out.txt 文件。

我们可以查看该文本当中的内容，与发送的文本的内容一致。并且是在不停的重复，这是因为在 GRC 当中，一直在从 file source 模块中读取数据。

```
wcc@wcc-dev:~/Desktop/gnuradio_demo/Chapter6$ ls -lh
总用量 152K
drwxrwxr-x 3 wcc wcc 4.0K 4月 19 15:01 gnuradio_ask_decoder
-rw-rw-r-- 1 wcc wcc 6.1K 4月 15 10:25 lab1_ask_mod.grc
-rwxrwxr-- 1 wcc wcc 8.5K 4月 15 10:01 lab1_ask_mod.py
-rw-rw-r-- 1 wcc wcc 9.7K 4月 15 10:36 lab2_ask_demod.grc
-rwxrwxr-- 1 wcc wcc 12K 4月 15 10:36 lab2_ask_demod.py
-rw-rw-r-- 1 wcc wcc 13K 4月 19 15:55 lab3_ask_file.grc
-rwxrwxr-- 1 wcc wcc 13K 4月 19 15:23 lab3_ask_file.py
-rwxrwxr-- 1 wcc wcc 13K 4月 19 15:55 lab4_ask_file.py
-rw-rw-r-- 1 wcc wcc 49K 4月 19 15:55 text_out.txt
-rw-rw-r-- 1 wcc wcc 1.9K 4月 19 14:41 text.txt
wcc@wcc-dev:~/Desktop/gnuradio_demo/Chapter6$
```

图 7-21 解调出来的文件

```
wcc@wcc-dev:~/Desktop/gnuradio_demo/Chapter6$ cat text_out.txt
hello world 00 !!!
hello world 01 !!! File Rate (Hz): 100k
hello world 02 !!! Bit Freq (Hz): 30k
hello world 03 !!! Bit Rate (Hz): 1k
hello world 04 !!! Low: Hamming
hello world 05 !!! 6.76
hello world 06 !!!
hello world 07 !!!
hello world 08 !!!
hello world 09 !!!
```

图 7-22 解调出来的文件内容

7.5 使用 ANTSDR 基于 ASK 发送文字和图片

在实现了前面在 GNU Radio 上的调制解调的一些列的仿真实验之后，我们可以结合 SDR 设备来尝试着发送一些数据，并对这些数据进行接收了。

在本次实验中，使用 ANTSDR 和 GNU Radio 来发送文本数据，并且使用 ANTSDR 自身来接收这个发射出去的文本数据，并最终能够将文本数据保存到文本当中。新建 GRC 流图 `lab4_ask_antsdr.grc`。

首先完成 ASK 的调制部分的工作，其调制和发射部分的工作和前面在进行 ASK 仿真的时候基本一致。只是需要将原始的采样率较低的基带信号，上采样到 ANTSDR 支持的基带采样率上，这样就能够将数据发送到信道当中了，还需要注意的是，在调制端，发送信号的调制载波的频率是 2.4GHz。

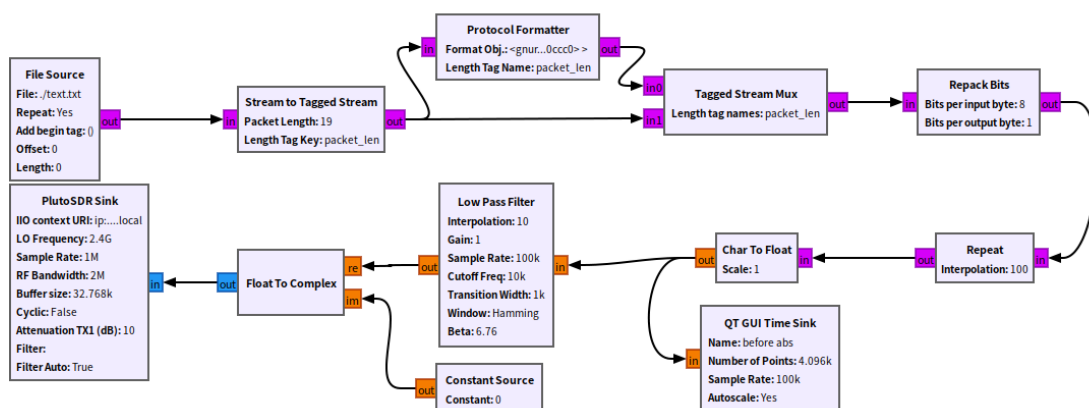


图 7-23 ASK 调制部分

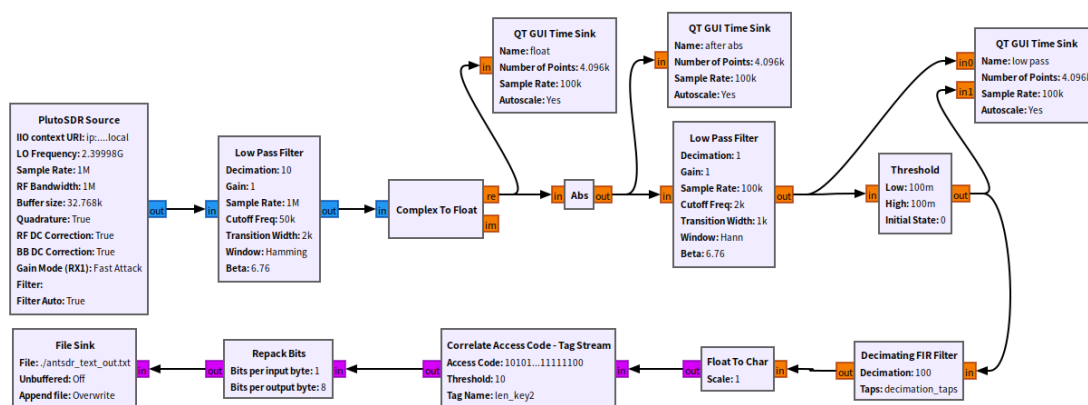


图 7-24 ASK 解调部分

在一开始，我们进行的工作只是进行一个简单的文本的发送，文本的内容和前面在进行仿真的时候的文本的内容一致。仿真大概进行到 30s 的时候，就能够在存放接收数据的文件当中看到接收到的信号。

```

text_out.txt x
home > wcc > GNU_Radio > Lab > Lab7_ASK > text_out.txt
1  hello world 00 !!!
2  hello world 01 !!!
3  hello world 02 !!!
4  hello world 03 !!!
5  hello world 04 !!!
6  hello world 05 !!!
7  hello world 06 !!!
8  hello world 07 !!!
9  hello world 08 !!!
10 hello world 09 !!!
11 hello world 10 !!!

```

图 7-25 接收到的文本数据

在上面的文本接收实验当中，我们使用的数据源是一个 txt 格式的文本数据，其实 GNU Radio 也能够直接使用图像格式的数据作为数据源，我们只需要在 file source 模块和 file sink 模块当中指定具体的图像的路径就可以。

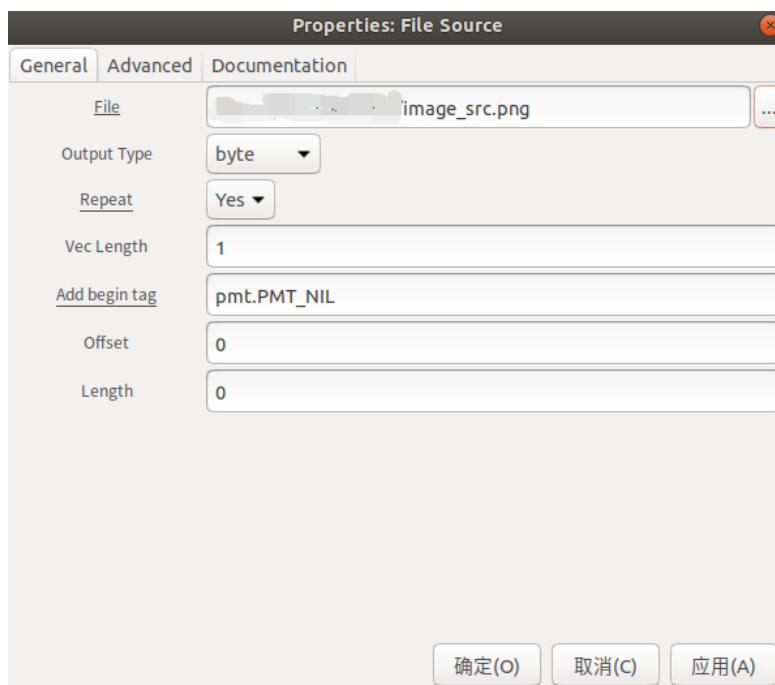


图 7-26 源文件为图像数据文件

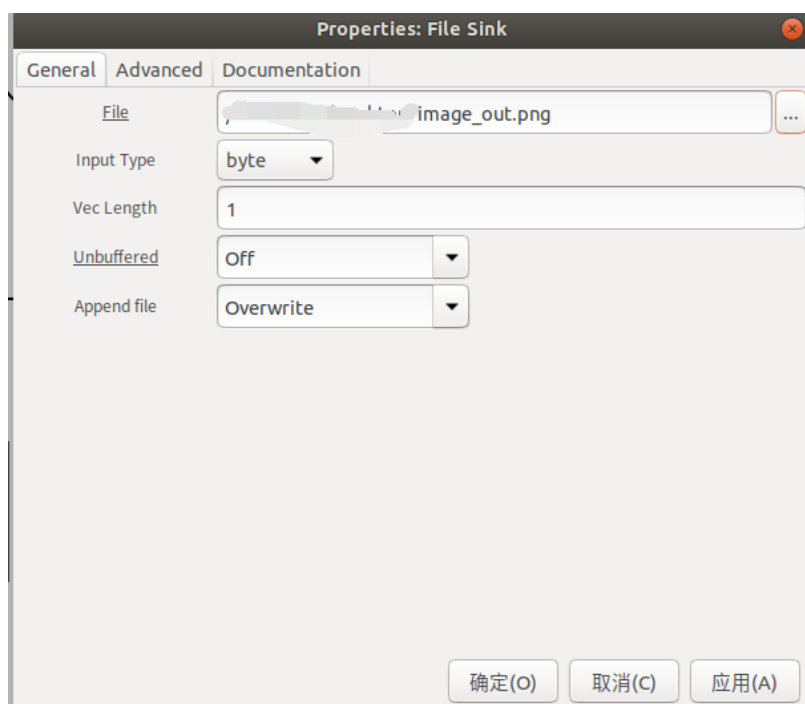


图 7-27 接收图像保存文件

在本次实验中，使用到的图像数据源名称为 `image_src.png`，接收图像数据存放在 `image_out.png` 当中。下图是正在进行 ASK 图像的传输过程中，使用图像查看器分别查看原始图像和接收图像的结果。

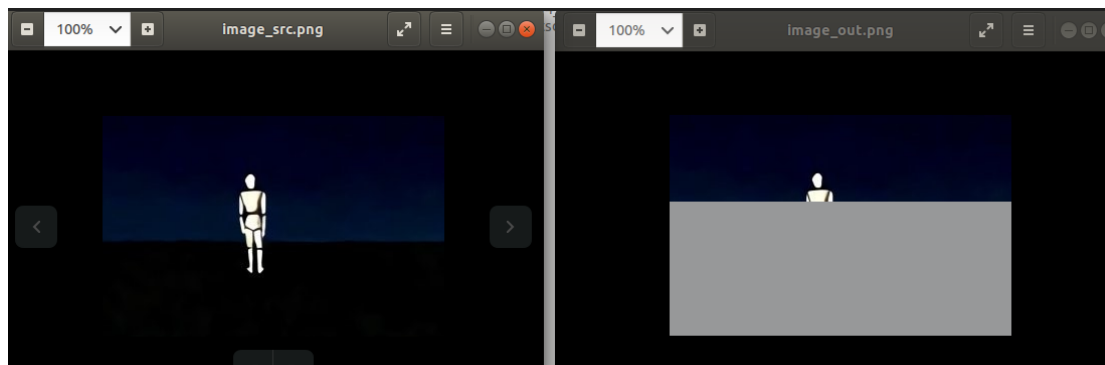


图 7-28 ASK 进行图像传输的过程

7.6 总结

在本章实验当中，我们先简单介绍了一下 ASK 调制和解调的原理，并通过在 GNU Radio 当中通过搭建 GRC 流图的方式来对 ASK 调制与解调进行了仿真，进一步加深了对 ASK 调制的时域和频域的认识。

然后我们基于 ASK 调制，结合前面第三章中的流标签和消息等完成了 ASK 传输的具体的例子，用具体的例子，演示了 ASK 如何进行数据的传输。

最后我们结合具体的 ANTSDR 硬件，来完成了真实的 ASK 信号的发送和接收。并且使用 ANTSDR 来传输了文本数据和图像数据。

ASK 是数字调制当中比较简单和基础的调制方法，在本章实验当中的 GRC 流图，可能性能不是最好的，用户可以在这个基础上进行进一步的优化。

第8章 BPSK 的调制与解调

8.1 相移键控

PSK 是一种数字调制方法，通过更改(调制)恒定参考频率信号(载波)的相位来传输数据。通过在精确的时间来改变正弦信号和余弦信号输入来完成调制，广泛地应用与无线局域网，RFID 和蓝牙中。

任何数字调制方案都会使用有限数量地信号来标识一个数字数据。PSK 调制使用有限的相位来标识，每一个相位标识一个二进制的数字 **bit**。

标识 PSK 调制的一种简单的方法是使用星座图，星座图显示的是在复平面上的点。实轴和虚轴分别对应着同相分量(In-Phase)和正交分量(Quadrature)。

在 PSK 的星座图当中，所选择的星座点通常具有均匀的角间距，这样可以在相邻的点之间实现最大的相位分里，从而提高抗干扰能力。PSK 的星座点通常在位于一个圆上，因此各个星座点都具有相同的能量，与此同时这些星座点所标识的复数的模是相同的，用于承载信号的正选信号和余弦信号的振幅也是相同的。

两个比较常见的相移键控是 BPSK 和 QPSK。因为传输的数据通常是二进制数据，因此 PSK 的方案通常以星座点的点数是 2 的幂进行设计的。

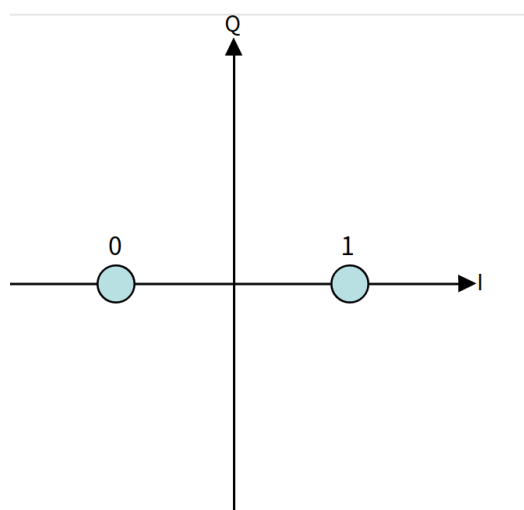


图 8-1 BPSK 星座图

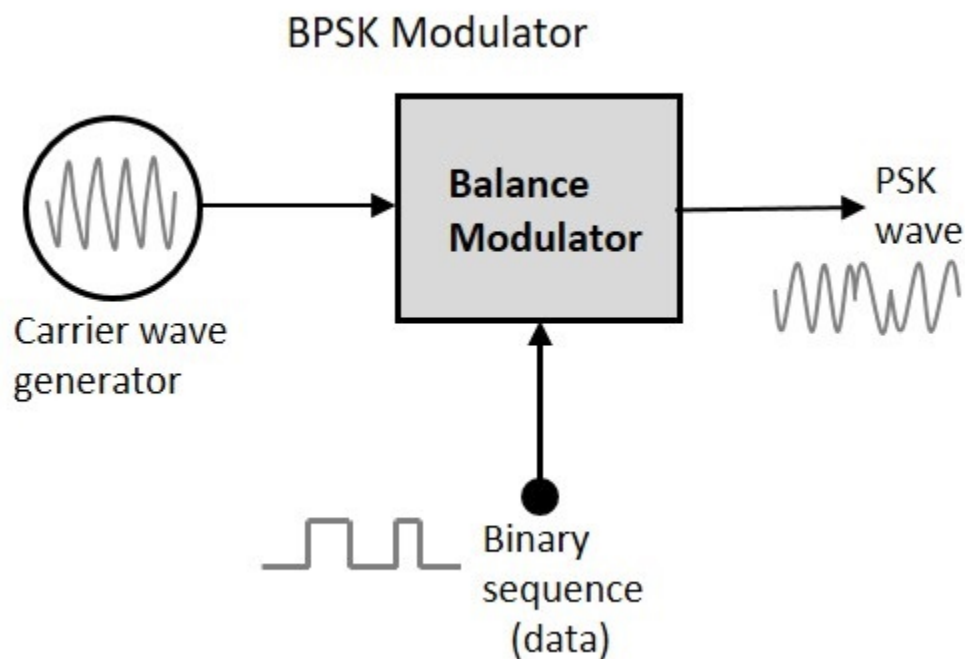


图 8-2 BPSK 调制器

一般的 BPSK 的调制器的示意图如图 8-2 所示，BPSK 的调制器将载波信号和 2 进制序列作为输入。当输入的二进制 bit 序列为高的时候，输出信号的相位为 180° ，当输入二进制 bit 序列为低的时候输出的相位为 0° 。

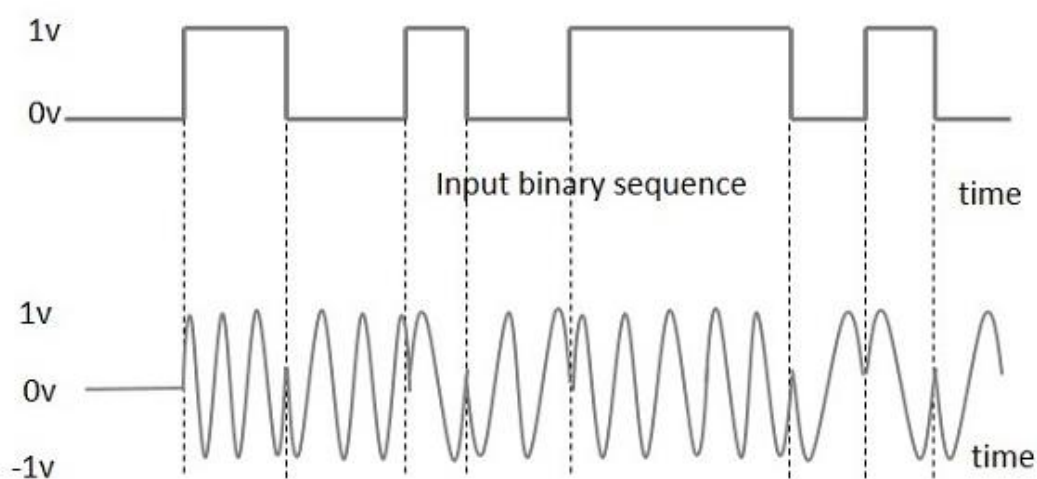


图 8-3 BPSK 调制示意图

8.2 在 GNU Rado 中生成 BPSK 信号

在前面的小节当中，我们介绍了 BPSK 调制的基本的原理，简单来说就是使用不同的相位来承载信号。又因为 BPSK 信号的相位一般情况下只有两种相位也就是 0° 和 180° 。

在 GNU Radio 当中，实现这种映射模块有几种，比较常用的一种映射模块是星座图调制模块。

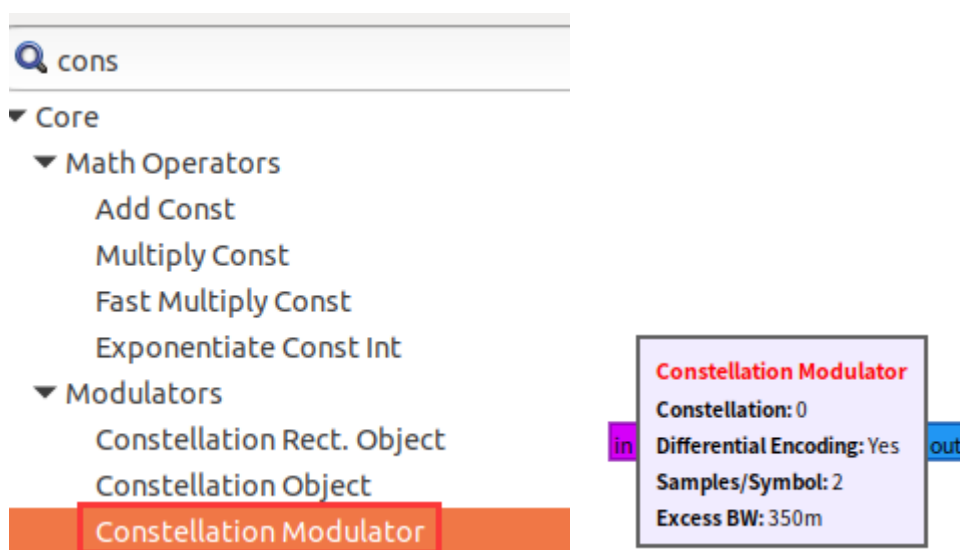


图 8-4 星座图映射模块

使用星座图调制模块的好处是，在模块内部已经使用了 RRC 成型滤波器对输出信号的波形进行了整型。除此之外，该调制器还能够用来调制 QAM 信号，因为该调制器是基于星座点的映射来实现的，给定输入的符号与对应的输出的星座点之间的一一对应来实现调制。在本次实验当中使用到的是 PSK 的星座映射。

Constellation Modulator 模块的参数如下：

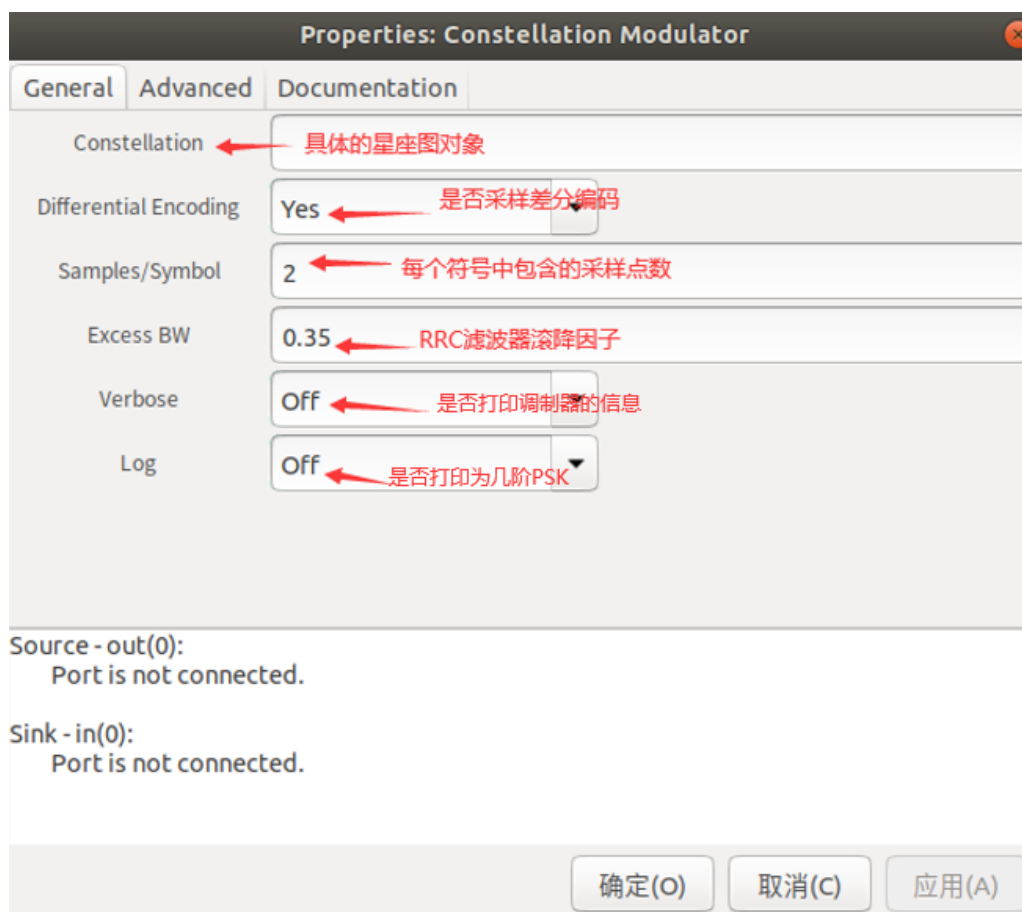


图 8-5 星座图调制模块参数

从星座图调制模块的参数中可以看到，首先需要有一个具体的星座图对象。这个对象就是包含了符号与星座点之间的对应关系。在 GNU Radio 当中，可以使用 Constellation Object 模块来完成符号与星座图的映射。

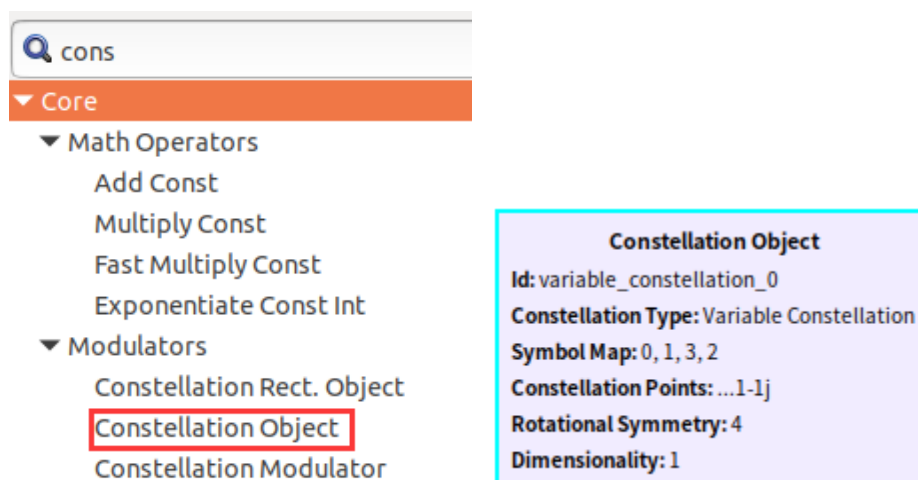


图 8-6 星座图对象



图 8-7 星座图对象参数

星座图对象主要参数设置如图 8-8 所示：

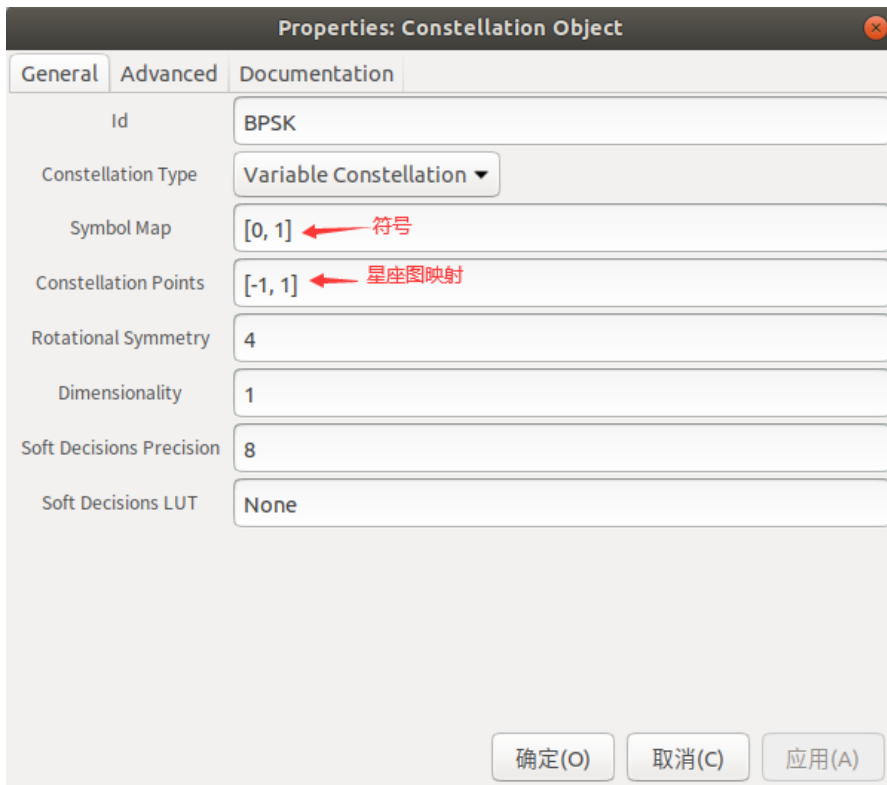


图 8-8 星座图参数设置

在星座图对象当中，符号表有[0,1]这是一个 BPSK 所支持的二进制符号。对应的星座图上的星座点是[-1, 1],更准确来说应该是[-1 + j0, 1 + j0]，这样就能够将星座图从符号映射到对应的星座上。

准备好了星座图映射的对象之后，就可以来填写星座图调制模块的参数，其设置如图 8-9 所示：

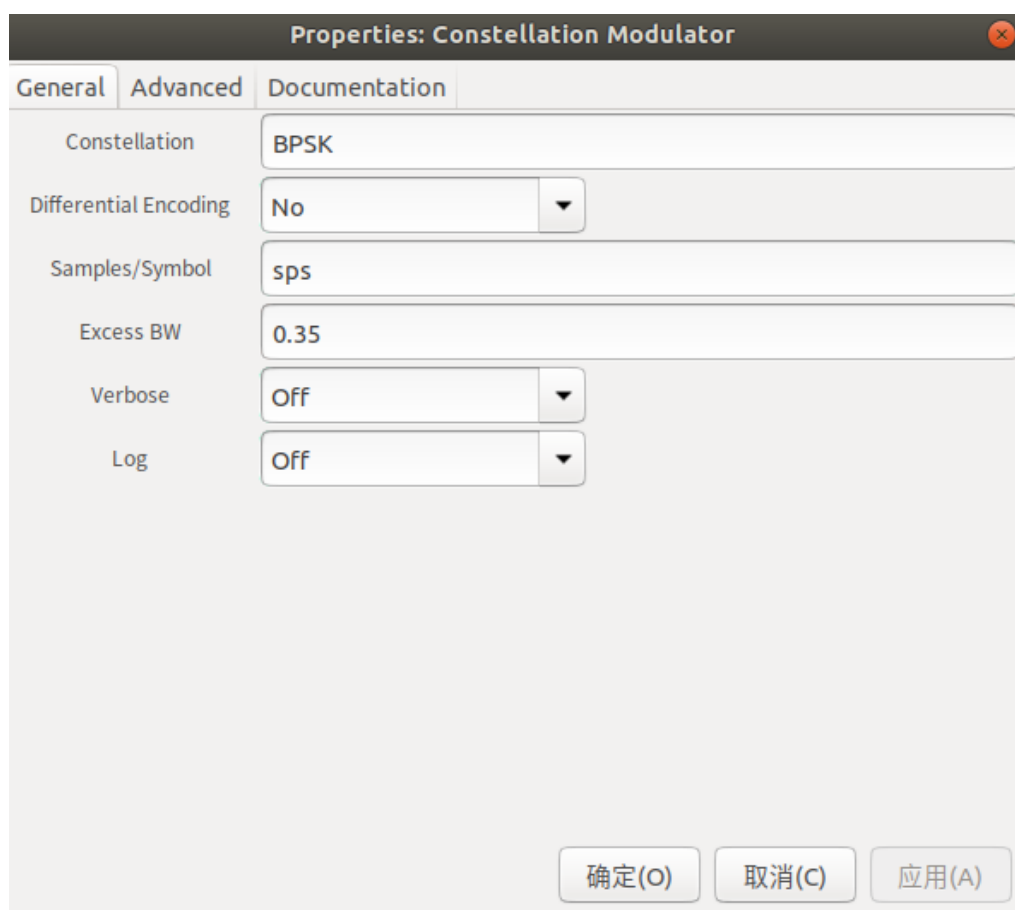


图 8-9 星座图调制模块参数设置

其中 sps 标识每个符号当中的采样点数，使用一个变量模块来设置这个参数就可与。

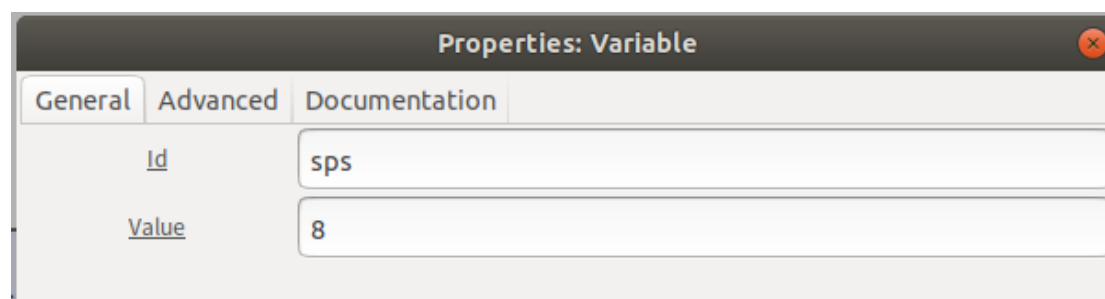


图 8-10 sps 设置

最终 BPSK 的调制 GRC 流程图如图 8-11 所示。

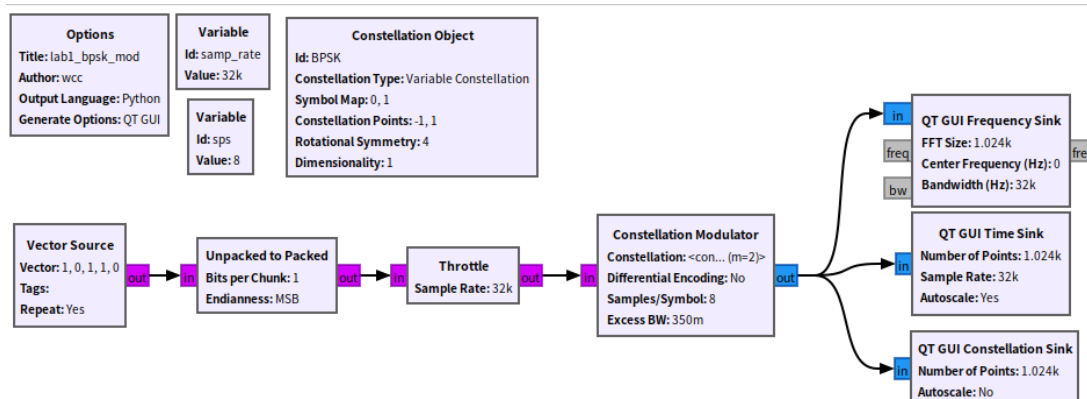


图 8-11 BPSK 调制 GRC 流程图

点击仿真开始运行这个 GRC 流程图，可以看到映射出来的时域和频域信号。

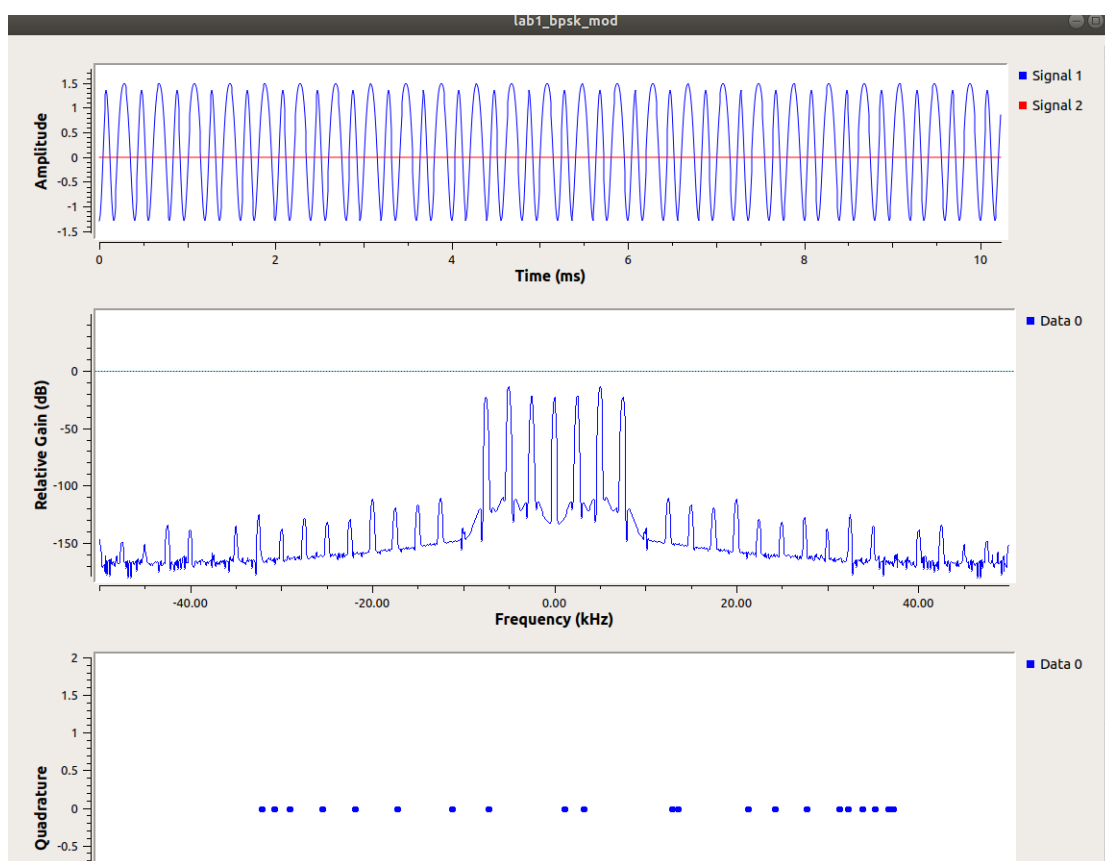


图 8-12 BPSK 调制信号

因为星座图映射模块当中使用到了根升余弦滤波器，所以经过调制的波形，没能够根明显分辨出各个符号所标识的波形。

8.3 BPSK 信号解调

为了能够模拟 BPSK 信号在真实的信道当中传输的情况，在这里我们首先引入一个信道模块。

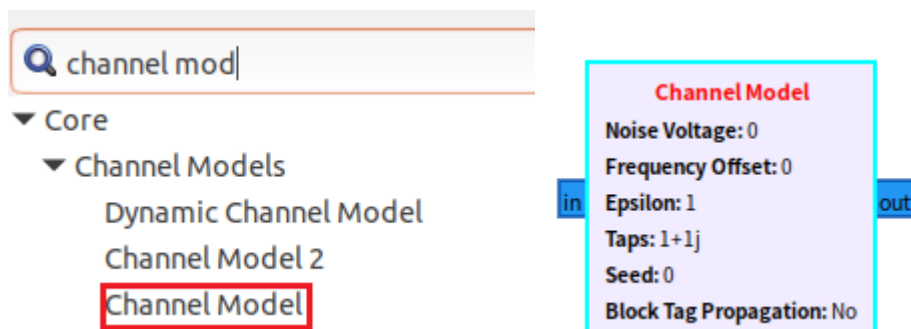


图 8-13 信道模块

使用这个模块，允许用户设置 AWGN 信道的噪声源，归一化的频率偏移和定时采样偏移，并且可以设置 AWGN 信道的随机种子值以实现随机的 AWGN 信道。在这个模块当中，还可以采用带有参数的多径延时曲线的 FIR 滤波器来近似模拟多径效应。其有关参数如下：

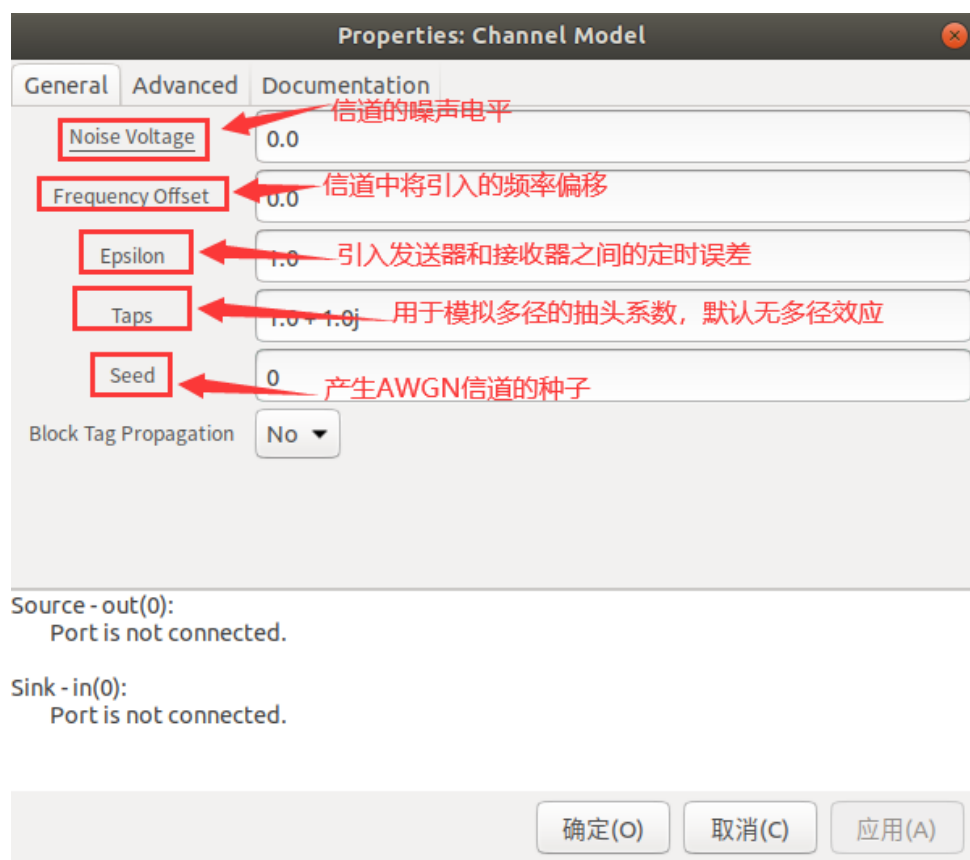
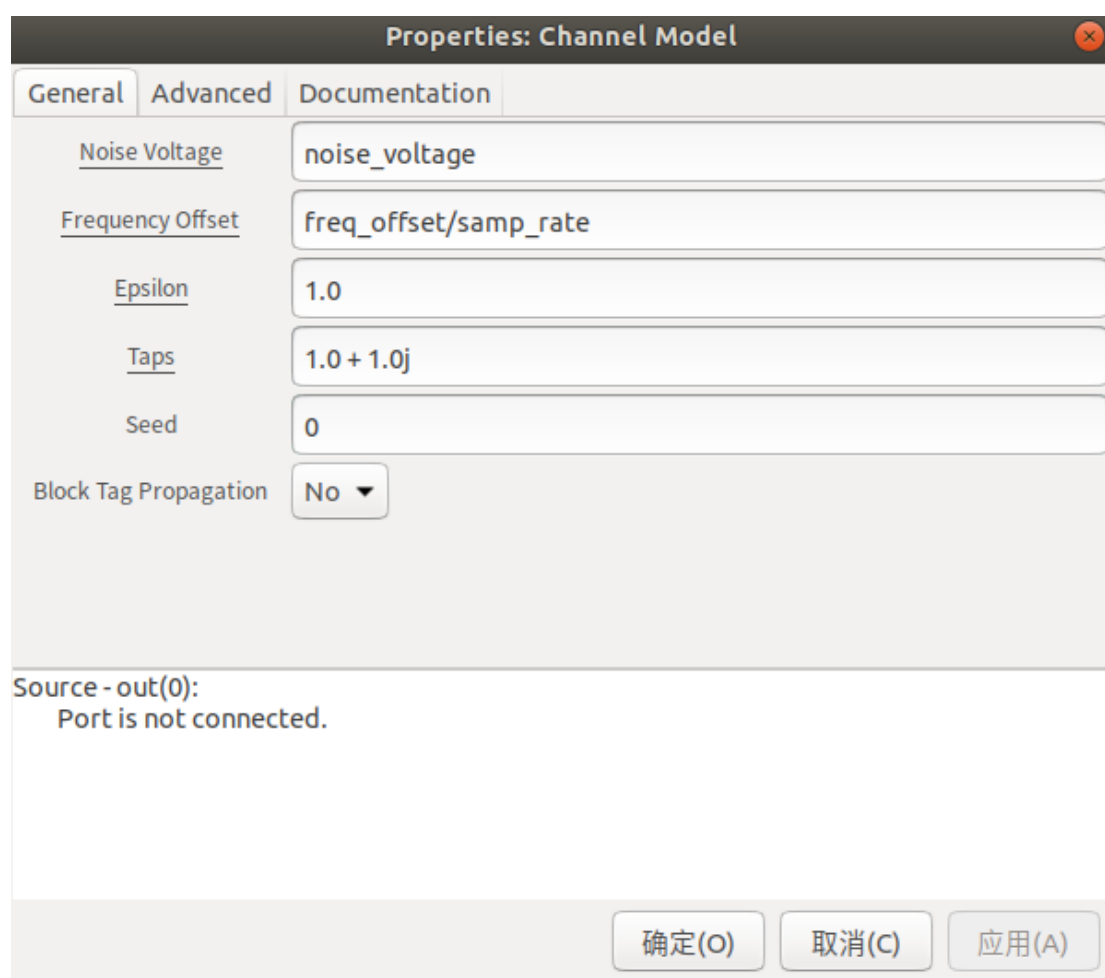


图 8-14 AWGN 信道参数

为了能够动态地更改信道地参数，在这里我们引入两个变量来控制信达的噪声电平和心如将会引入的频率偏移。

QT GUI Range	QT GUI Range
Id: noise_voltage	Id: freq_offset
Default Value: 63m	Label: freq_offset
Start: 0	Default Value: 0
Stop: 1	Start: -1k
Step: 1m	Stop: 1k
	Step: 1

图 8-15 信道参数



The image shows a Qt-style dialog box titled "Properties: Channel Model". It has three tabs: "General", "Advanced", and "Documentation". The "General" tab is selected. It contains several input fields and a dropdown menu. The "Noise Voltage" field is set to "noise_voltage". The "Frequency Offset" field is set to "freq_offset/samp_rate". The "Epsilon" field is set to "1.0". The "Taps" field is set to "1.0 + 1.0j". The "Seed" field is set to "0". The "Block Tag Propagation" dropdown menu is set to "No". At the bottom, there is a status bar that says "Source - out(0): Port is not connected." and three buttons: "确定(O)", "取消(C)", and "应用(A)".

Property	Value
Noise Voltage	noise_voltage
Frequency Offset	freq_offset/samp_rate
Epsilon	1.0
Taps	1.0 + 1.0j
Seed	0
Block Tag Propagation	No

Source - out(0):
Port is not connected.

确定(O) 取消(C) 应用(A)

图 8-16 信道参数设置

接下来就需要来进行 BPSK 信号的解调了，在 GNU Radio 当中，可以使用系统中已有的信号来进行 PSK 或者 PAM 信号信号的同步操作。

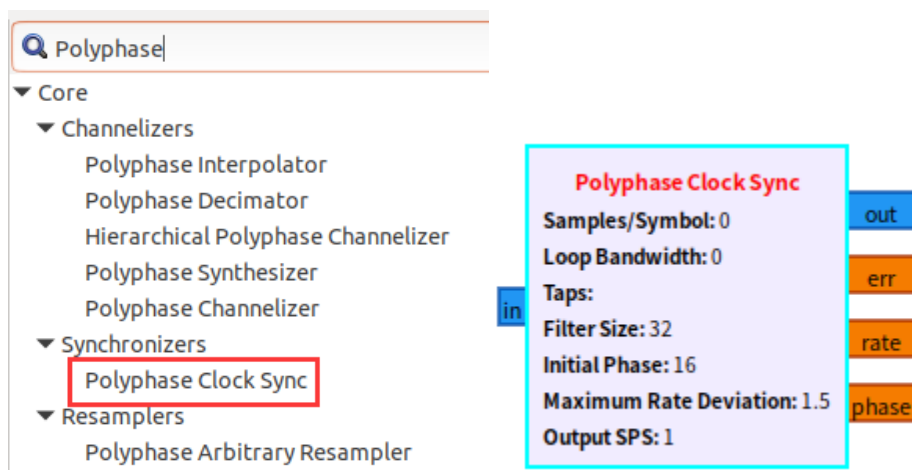


图 8-17 多相时钟同步模块

这个模块通过滤波之后的信号的导数最小来使得信号完成同步，与此同时会使信号具有最大的 SNR 并且使 ISI 达到最小。这个模块主要使用建立两个滤波器组来完成这些工作，一个滤波器组包含信号的脉冲成型的匹配滤波器比如根升余弦滤波器。第二组滤波器包含第一组滤波器组中滤波器的导数。

该模块的参数如下：

Samples/Symbol: 每个符号所包含的采样点数，因为该模块默认输出的采样点就代表了当前的符号。

Loop Bandwidth: 环路带宽用于设置内部控制环路增益，这个值应该设置地得比较小。

Taps: 这个模块最重要的一个参数就是抽头系数，对于典型的数字调制，这里需要使用根升余弦滤波器的系数。

Filter Size: 滤波器的数量，该值能够使用户获得足够好的分辨率，以生成足够小的 ISI。

Initial Phase: 初始相位指的是该模块最开始查看的滤波器的路径。

Maximum Rate Deviation: 限制速率。

Output SPS: 输出的每个符号中的采样点数。

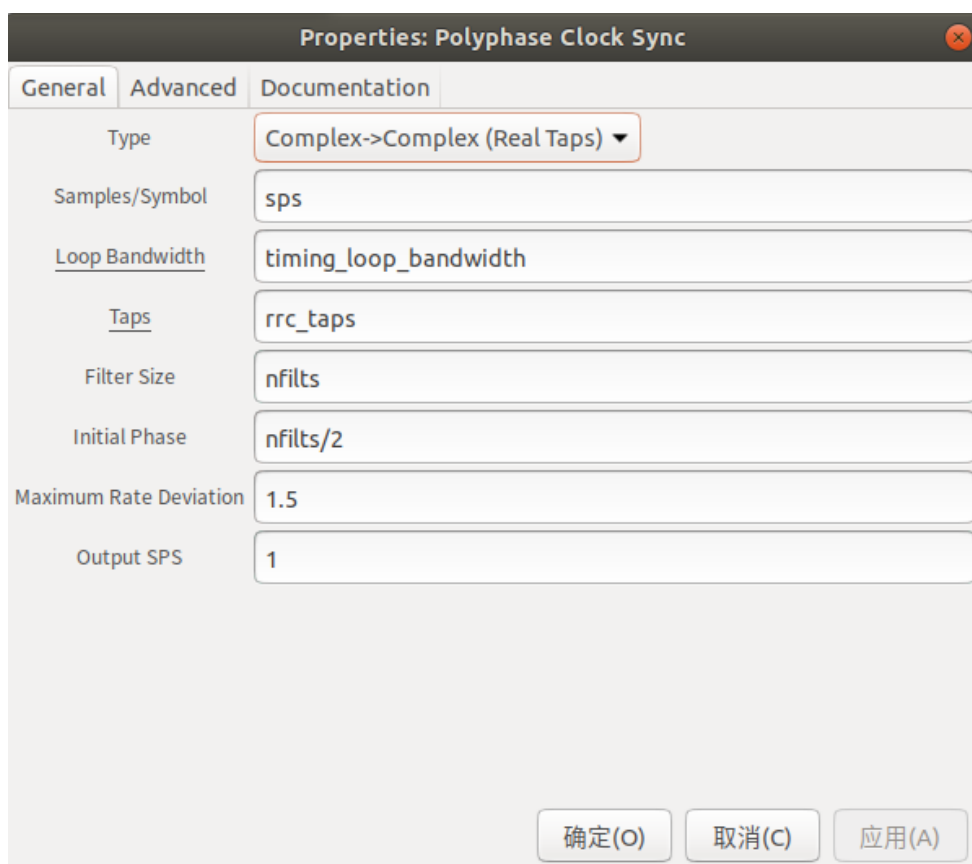


图 8-18

为了能够正确地来设置这个多相时钟同步模块，我们可以使用一些参数来对这个模块进行设置。

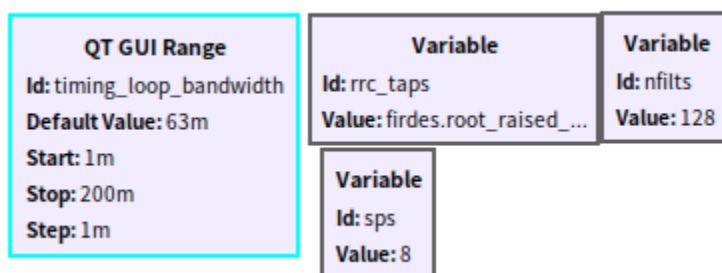


图 8-19 参数设置

其中对于 RRC 滤波器地抽头系数，可以使用如下方式来进行设置。

```
firdes.root_raised_cosine(nfilts, nfilts, 1.0/float(sps), 0.35, 45*nfilts)
```

在进行了同步之后，我们就可以来观察一下经过同步之后的星座图了，到目前位置所设计的 GRC 流程图如所示。

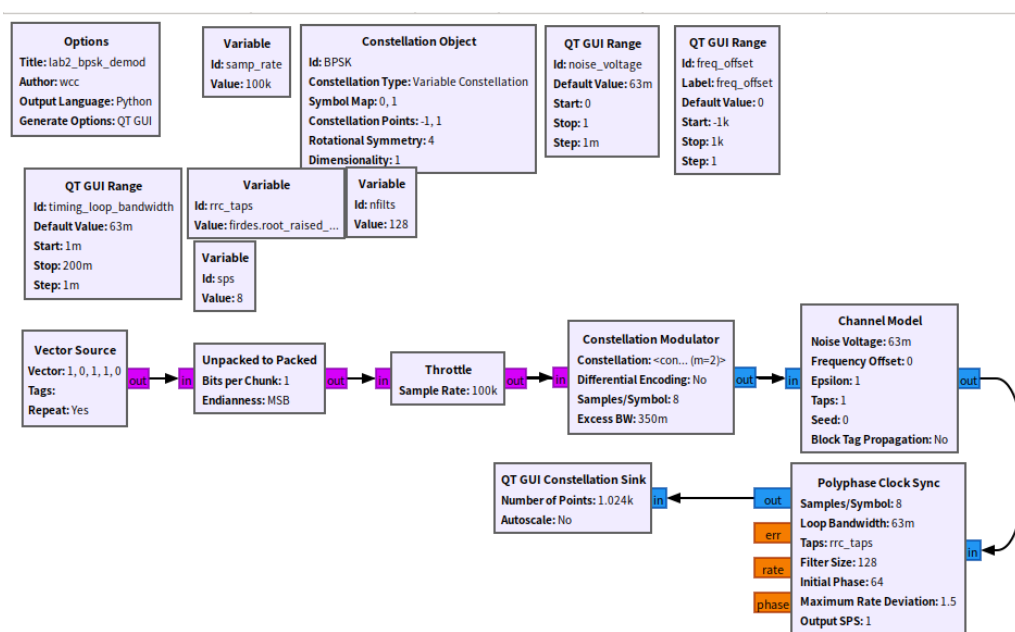


图 8-20 完成同步的 GRC 流图

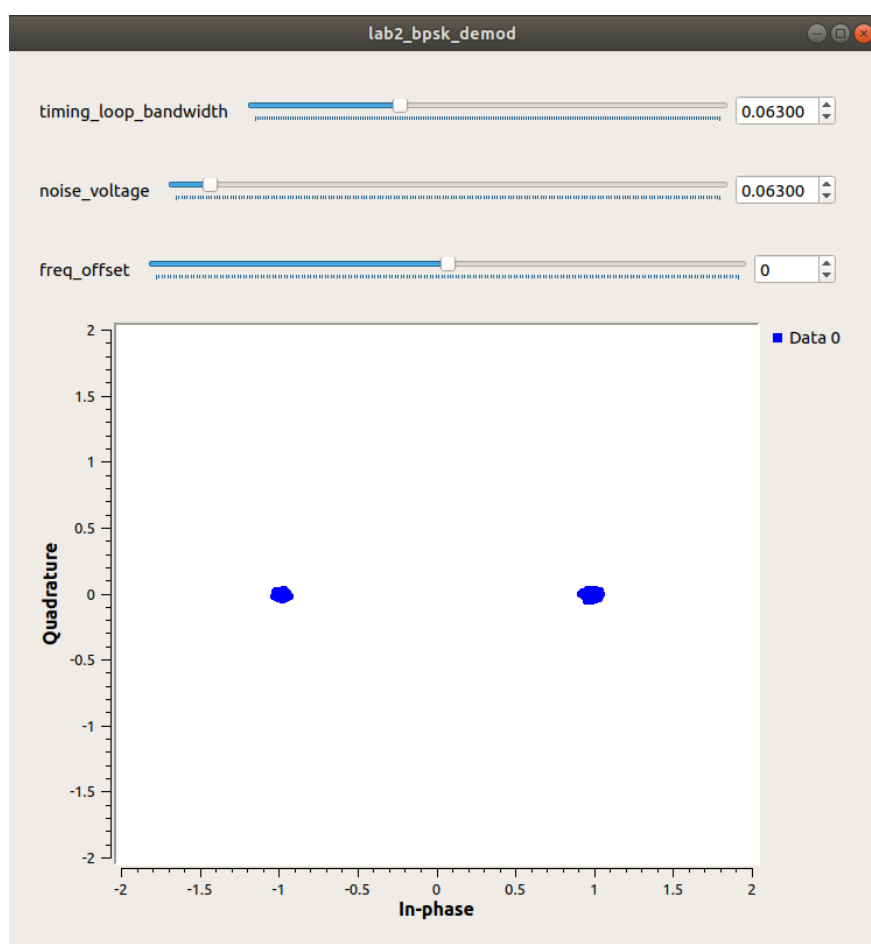


图 8-21

可以看到经过了同步之后的 BPSK 信号的星座图能够正确地分布在标准的星座点的附近。着说明当前已经能够正确地获取到接收到数据了。

在接下来需要进行的操作就是从接收到的数据当中恢复出原始的数据了。

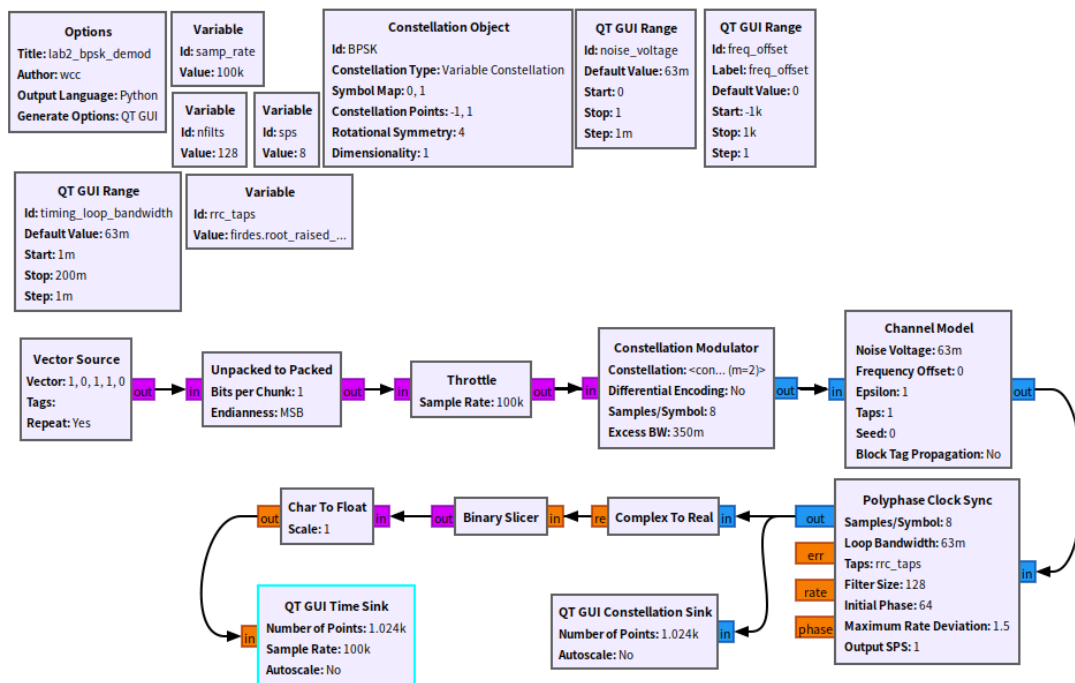


图 8-22 解调出原始数据

点击运行仿真，可以查看解调出来的符号序列，如图 8-23 所示，可以看到，解调出来的符号序列 10110，与前面发送的符号序列保持一致。

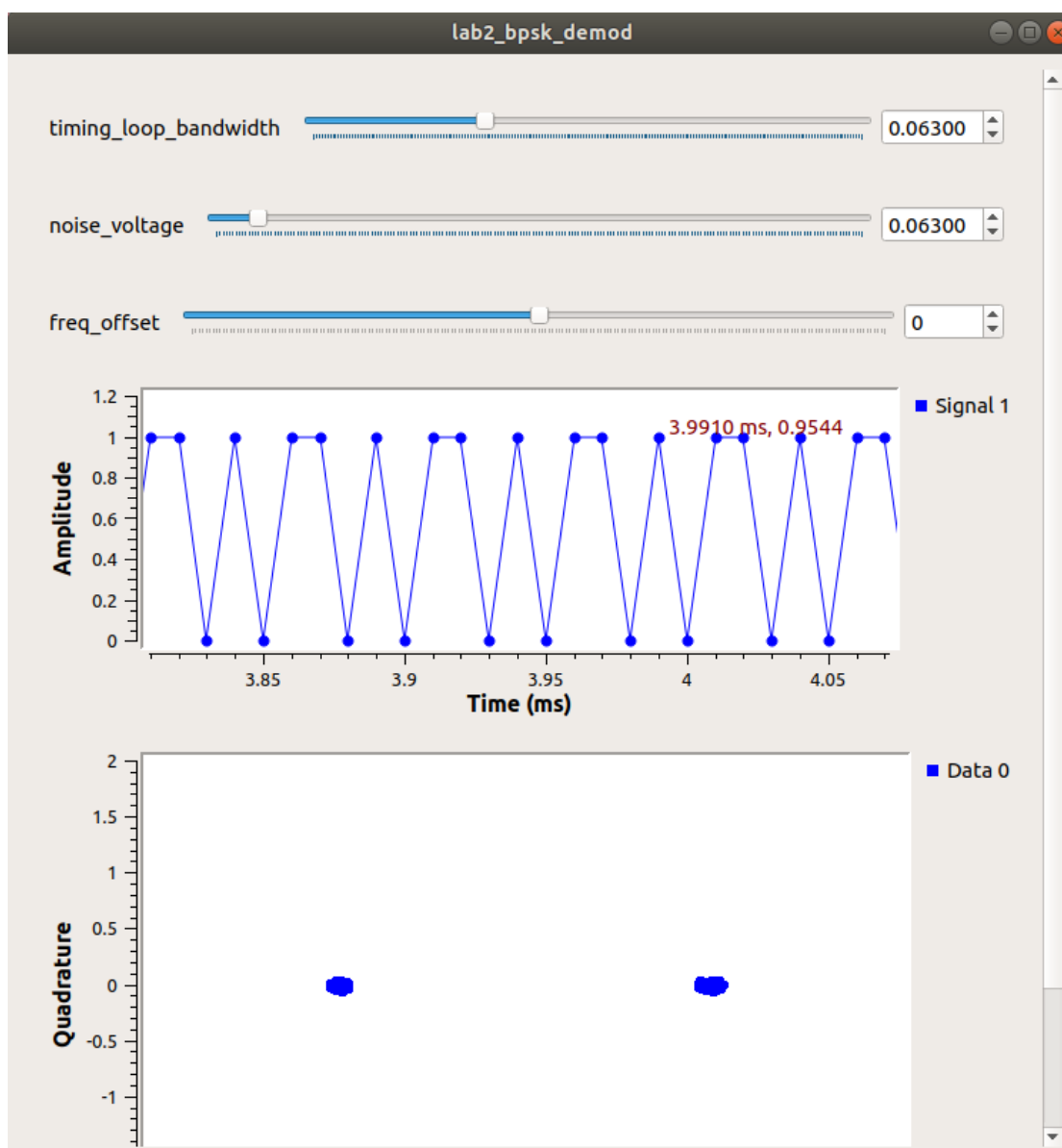


图 8-23 解调出的符号序列

到这里，我们可以再回顾一下上面的 GRC 流图在运行过程中，从源到终点所经历的过程。

- 源模块提供重复的 1、0、1、1、0 的数字数据流。这个数据流是 bit 流(每个字节当中有一个相关的 bit)。
- Unpacked to packed 模块从原始的数据流中，提取低位，并将输出的字节数据 bit 流依次填充。
- 样本数据流通过流控模块，使得平均速率不超过 100KHz。
- 然后将这些 packed 字节流送到星座调制模块，在星座调制模块当中将会完成以下工作：

- 将二进制 0, 1 编码位符号, 其中 0, 1 分别对应 -1, 1。因此原始的重复序列 1、0、1、1、0... 转换为 +1、-1、+1、+1、-1...
- 信号然后经过由系数 `sps` 确定的比例进行上采样。上采样是在原始的样本当中插入 0 值的过程。因此映射之后的 +1、-1、+1、+1、-1... 转换为 +1、0、0、0、0、0、0、-1、0、0、0、0、0、0、0、+1、0、0、0、0、0、0、0、+1、0、0、0、0、0、0、0...
- 上采样的信号会通过跟升余弦滤波器 `SRRC`, 这样, 消息当中的每个符号都会产生一个 `SRRC` 脉冲, 该脉冲将会经过符号的值(+1、-1)进行缩放。相邻的脉冲的峰值之间的间距为上采样系数 `sps` 决定的采样点间隔。波特率 `baud`(每秒钟发送的符号更改次数)等于采样率除以上采样系数 $\text{baud} = \text{samp_rate} / \text{sps}$ 。在 `BPSK` 当中, 波特率等于比特率, 因此如果采样率为 32KHz 并且上采样系数为 `sps=8`, 那么实际的数据速率是 4Kb/s。
- 基带信号通过 `Channel` 模块, 这个模块模拟了我们的传输的路径。在这个模块当中我们可以添加噪声和频率偏移。
- 接下来信号到达多相时钟同步模块。在这个模块当中, 模块首先通过匹配滤波器, 然后该模块采用迭代的方式来估计信号的采样时刻(基于上采样系数 `sps`, 环路带宽, 滤波器大小, 初始相位和最大频率偏移等)。如果该模块工作正常的话, 它将输出一系列的数字信号, 这些信号与原始的信号非常接近。比如原始的序列是 +1、-1、+1、+1、-1, 理想情况下该模块的输出应该和原始的序列一致, 但实际中由于噪声和模块自身的影响, 输出的信号将不完全相同的, 会有一定的偏差, 比如 +1.01+j0.001、-0.99+j0.012、+1.13+j0.01、+1.03-j0.12... 在该模块当中还会对输入的序列进行一个降采样的操作。
- 接下来会提取经过同步之后的数据的实部, 然后将数据传递到 `binary slicer` 模块, 如果输入到 `slicer` 模块的数据小于 0 那么二进制输出为 0 如果输入到 `slicer` 模块的数据大于 0, 那么二进制输出为 1。

8.4 使用 BPSK 发送文本数据

在前面使用 `BPSK` 完成了基本的数据调制与解调之后, 我们可以考虑使用 `BPSK` 来做一些更加有实际意义的事情了, 我们可以使用 `BPSK` 来进行一些文本数据的收发工作。

在进行收发数据之前, 我们需要认识到, 前面我们在进行解调数据的时候, 仅仅使用了一个多相时钟同步模块来对接收到的信号进行同步, 在实际的信道中传输的时候, 可能还会有其他的一些误差的引入, 这时候需要我们对信道进行均衡等操作。

在 `GNU Radio` 当中, 已经提供了一些用于均衡信道和用于载波同步的模块, 我们可以直接使用这两个模块即可。新建 GRC 流图 [lab3_bpsk_text.grc](#)。

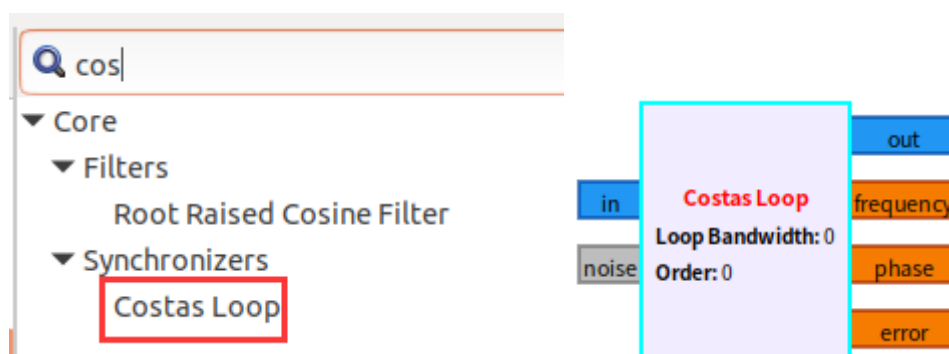


图 8-24 Costas 环

科斯塔斯环和 PLL 的功能类似，用于完成载波同步的操作，在 GNU Radio 当中我们需要使用到这个模块来进行载波的同步，其参数如下：

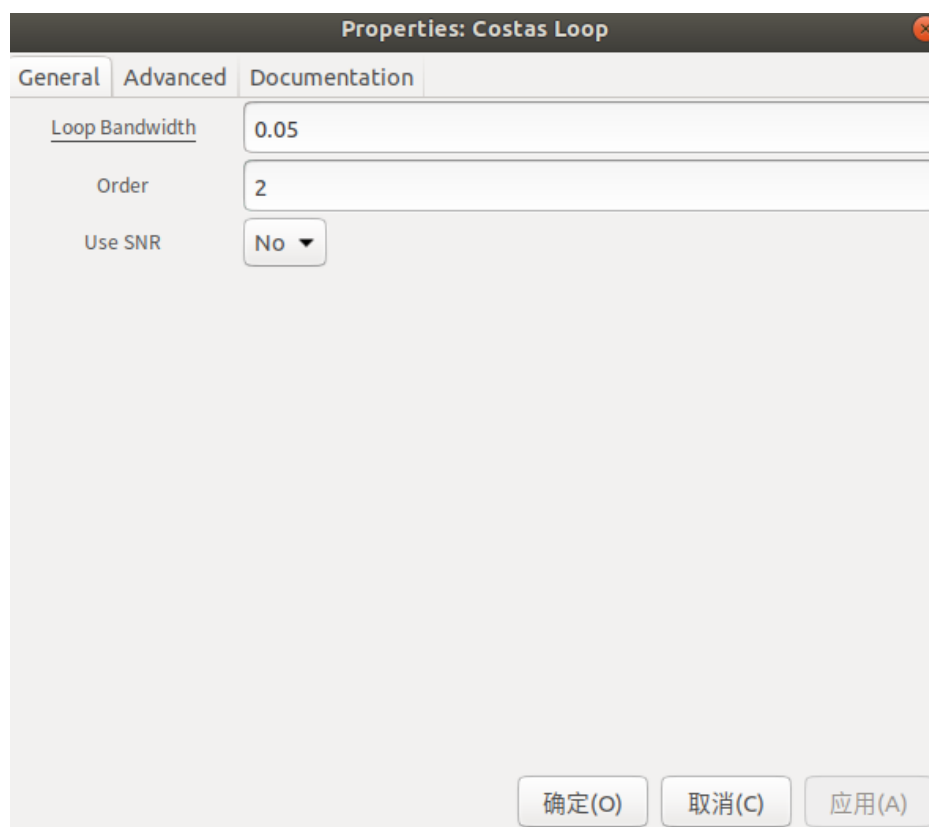


图 8-25

科斯塔斯环常用于解调 PSK 信号，用于跟中载波信号，该模块的阶数对应着不同阶数的 PSK 信号。比如 2、4、8 阶 Costas loop 分别用于解调 BPSK，QPSK，8PSK 信号。

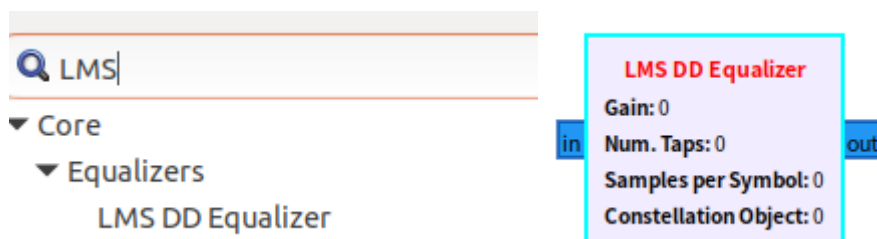


图 8-26 最小均方算法(LMS)均衡

在实际的信号处理当中，还需要对信道进行均衡，常用的一个均衡方法是最 LMS 均衡。

该模块的参数如下：

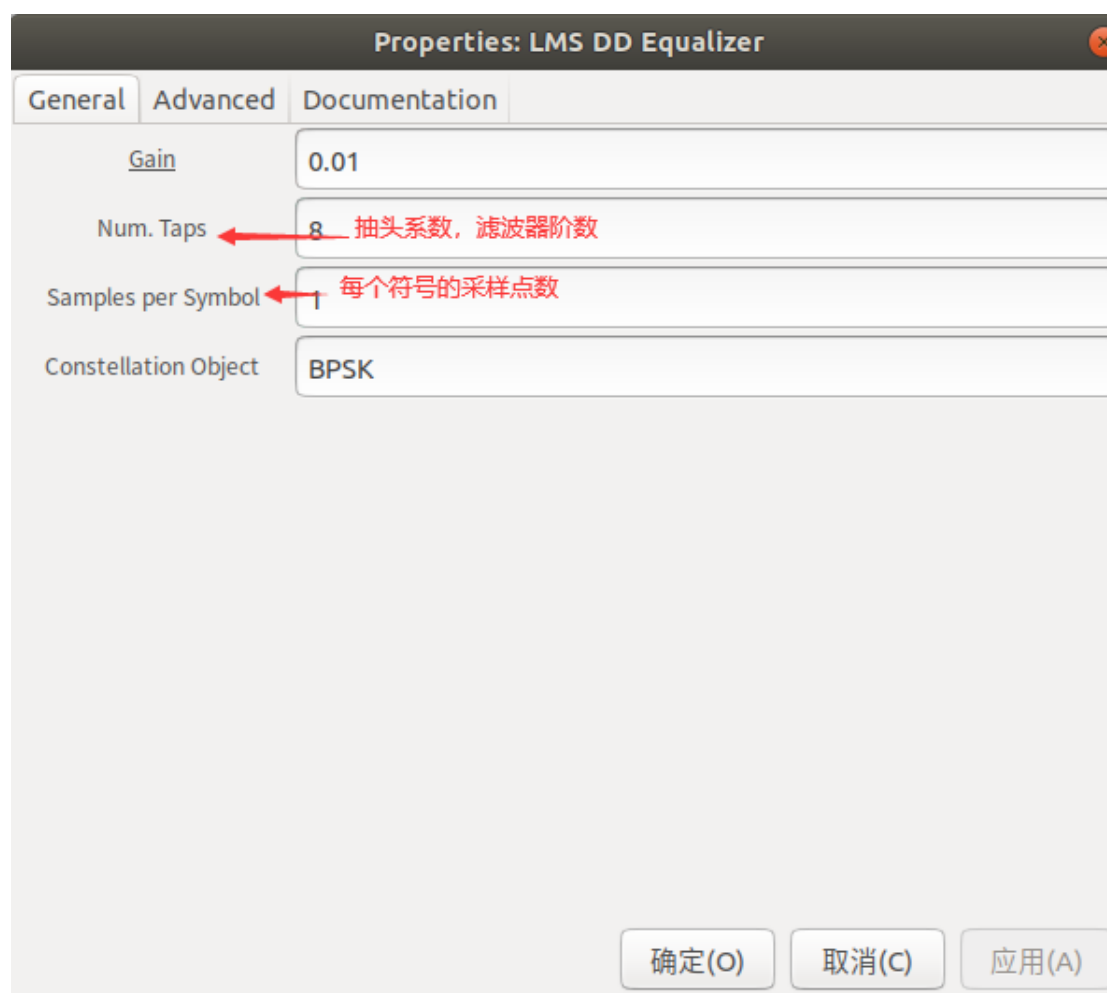


图 8-27 LMS 均衡模块参数

最终用于传输文件的整个 GRC 流程图如图 8-28 所示

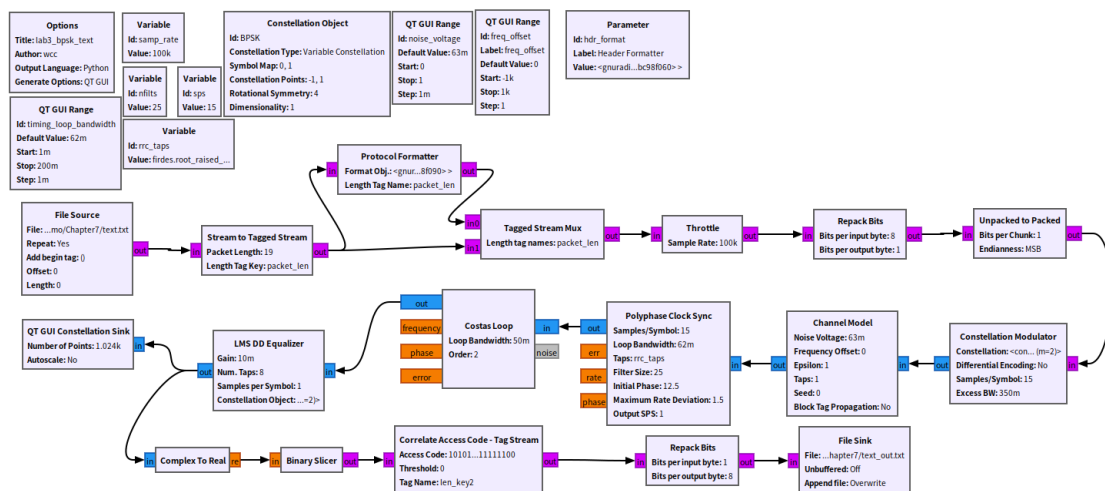


图 8-28 BPSK 传输文件流程图

点击运行仿真，可以看到最终接收到的数据和经过载波同步和均衡之后的星座图。

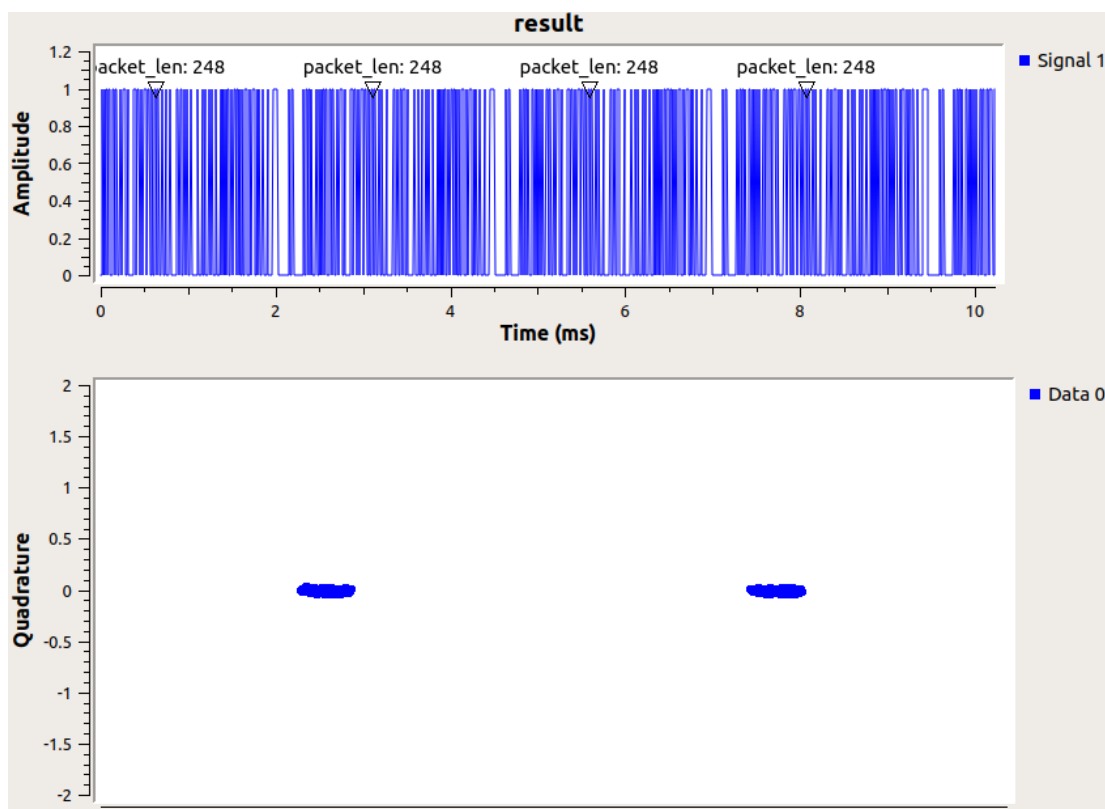


图 8-29 接收到数据和星座图

最终，我们能够在用于保存解调文件的文本当中成功地解调出文本信号。

```

home > wcc > Desktop > gnuradio_demo > Chapter7 > text_out.txt
216 hello world 15 !!!
217 hello world 16 !!!
218 hello world 17 !!!
219 hello world 18 !!!
220 hello world 19 !!!
221 hello world 20 !!!
222 hello world 21 !!!
223 hello world 22 !!!
224 hello world 23 !!!
225 hello world 24 !!!
226 hello world 25 !!!
227 hello world 26 !!!

```

图 8-30 解调出的数据

8.5 使用 ANTSDR 基于 BPSK 发送文字

在前面的实验当中，我们基于 GNU Radio 完成了 BPSK 的基本的调制和解调，并且使用 BPSK 来仿真了发送简单的字符的实验。在接下来的实验当中，我们就可以考虑来使用 ANTSDR 将 BPSK 信号发送到真实的信道当中了。

有了之前章节当中的经验和前面的仿真实验，完成本次实验就很简单了。新建 GRC 流图 [lab4_bpsk_antcdr.grc](#)。

首先，观察 BPSK 调制以及发送模块。

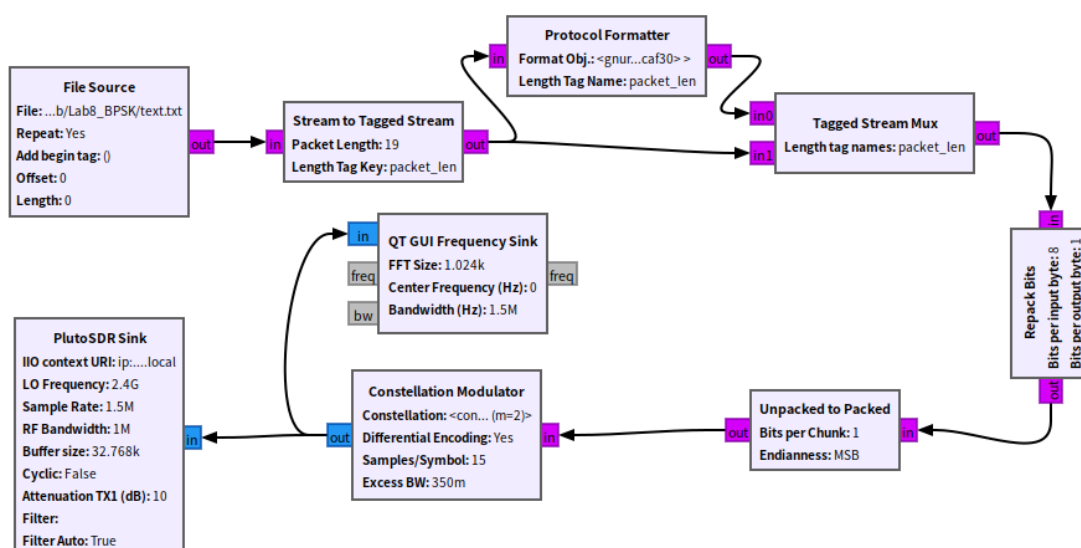


图 8-31 BPSK 调制和发送模块

该部分流图流图所实现的功能如下：

File Source 模块从 txt 文本当中获取原始的数据流, 然后经由 Stream to Tagged Stream 给原始的数据流打上标签, 打上标签的数据流需要通过 Protocol Formatter 生成头, 并和打上标签的数据流组成新的数据流, 然后将数据流转换成 bit 流, 并给到 BPSK 调制模块进行星座图的映射, 最终讲过 Pluto SDR Sink 模块将数据发送到信道当中。

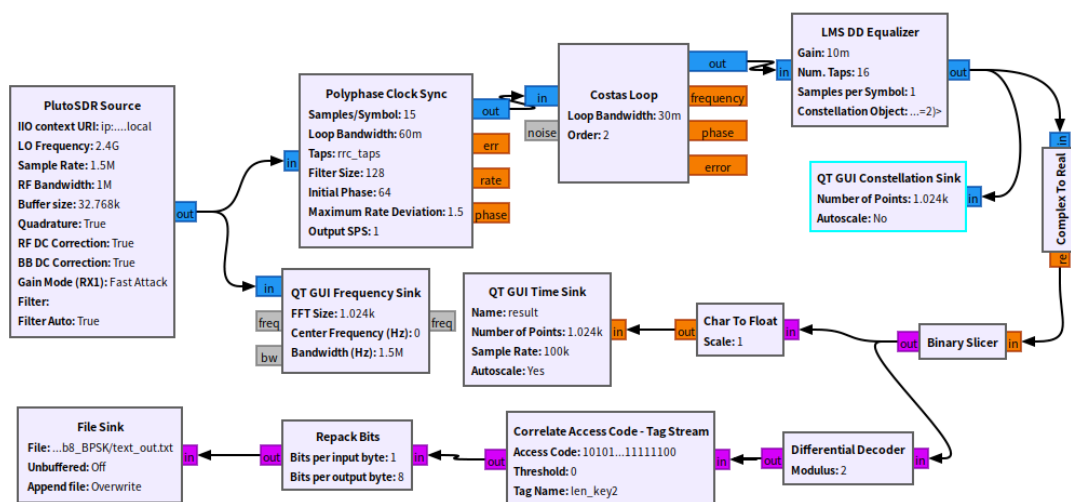


图 8-32 BPSK 解调模块

BPSK 的解调模块如图 8-32 所示, 从信道当中接收到信号通过 PlutoSDR Source 模块提供给 GNU Radio, 接收到数据流然后依次经过多相时钟同步模块进行同步, 科斯塔斯环进行载波同步和 LMS 模块进行信道均衡, 这样能够得到比较好的星座图, 由于在本次实验当中进行 BPSK 映射的时候采用的差分编码的方式, 因此在解调的时候, 需要由差分编码转换会原始的数据编码, 然后经过头检测模块检测数据帧, 进行数据帧的同步, 最终能够得解调出的文本数据。

运行仿真, 可以看到当前用于保存解调数据的文本中出现了发送的文本的数据。观察 GRC 流图运行时的信道的发送信号和接收信号的频谱图分别如图 8-33、图 8-34 所示。可以看到, 经过信道之后, 接收到的信号相较于原始的信号有了较大的误差。

最后我们可以观察经过处理之后的星座图的情况, 如图 8-35 所示。

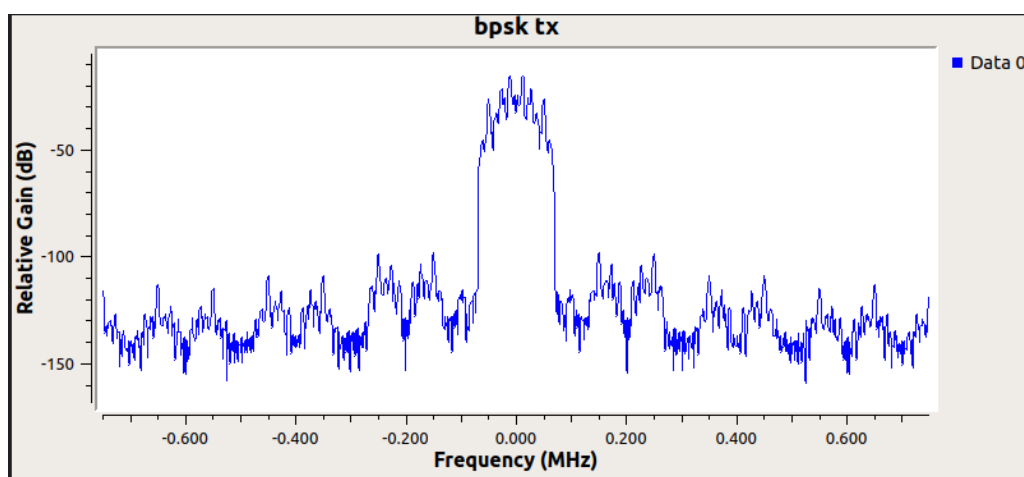


图 8-33 BPSK 发送端频谱图

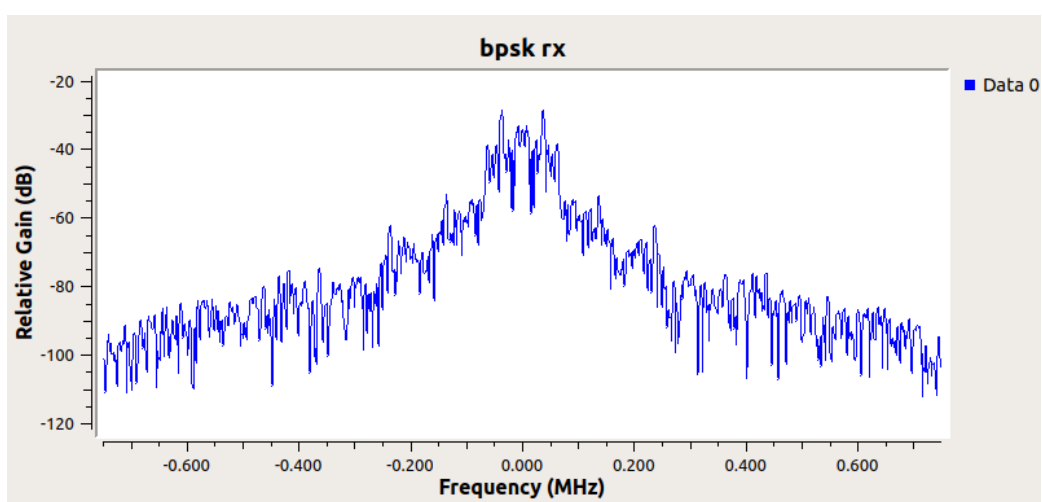


图 8-34 BPSK 接收端频谱

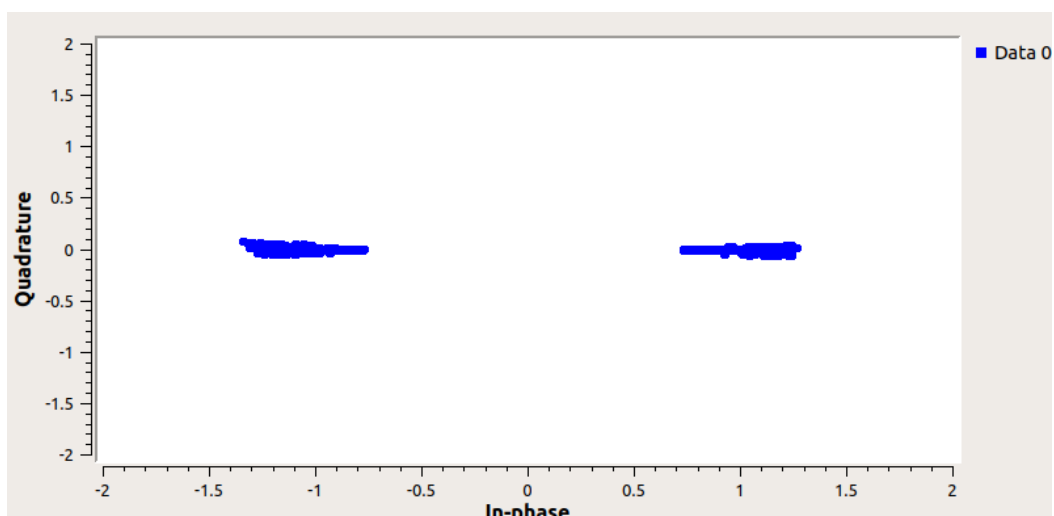


图 8-35 经过处理之后的星座图

8.6 总结

在本章当中，我们首先了解了以下相移键控的基本概念，主要介绍了 BPSK 的调制和解调的方法，并且在 GNU Radio 就 BPSK 调制和解调给出了具体的实现方法。

基于 BPSK 的基本的调制和解调方法，我们设计了 GRC 流图，能够使用 BPSK 来进行一些简单的文本数据的传输，并且最终能够使用 PlutoSDR 来完成 BPSK 信号的发送和接收，并最终实现了将文本信号发送到大气，并成功地从大气当中接收到文本信号的实验。

第9章 QPSK 的调制与解调

在前面章节的实验当中，我们实现了 BPSK 的调制与解调 BPSK 是一种比较简单相移键控调制方法，在这一章的实验当中，我们会介绍另外一种常见的相移键控，也就是 QPSK 调制。

9.1 QPSK 调制的星座图

QPSK 也是一种常见的相移键控调制方法。该调制方法的星座图如图 9-1 所示。QPSK 调制其实有两种星座图，一种是如图 9-1 所示的星座图，另外一种星座图上，四个星座点分布在 4 个坐标轴轴上。在图 9-1，所示的星座图当中，其星座点的分布和 4QAM 的分布是一摸一样的，但是二者的是不一样的。QPSK 是采样 4 个相位不同的载波来进行信号的承载。

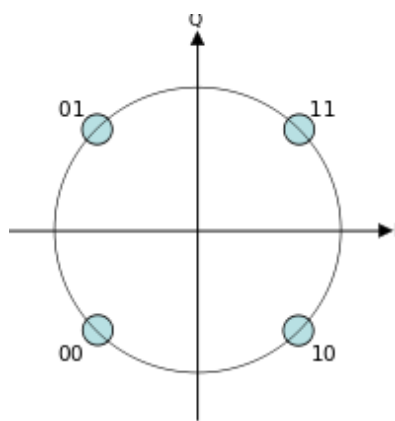


图 9-1 QPSK 星座图

QPSK 调制的符号与载波的对应关系如图 9-2 所示，可以看到，在 QPSK 调制当中，每个符号可以承载两个 bit 的信息，相比与 BPSK，在两侧 bit 速率相同的情况下，QPSK 的符号速率只有 BPSK 的一半，这样能够节省出时间间隙给其他模块进行使用。

QPSK 的符号对应着不同的载波，在 QPSK 当中，有 4 个相位不同的载波对应分别对应不同的符号。

可以看到整个 GRC 流图的调制解调流图和前面的 QPSK 的调制解调的过程基本一致，区别在于星座图的映射和使用到的星座图对象不同。QPSK 星座图的符号和星座点的映射关系如图 9-4、图 9-5 所示

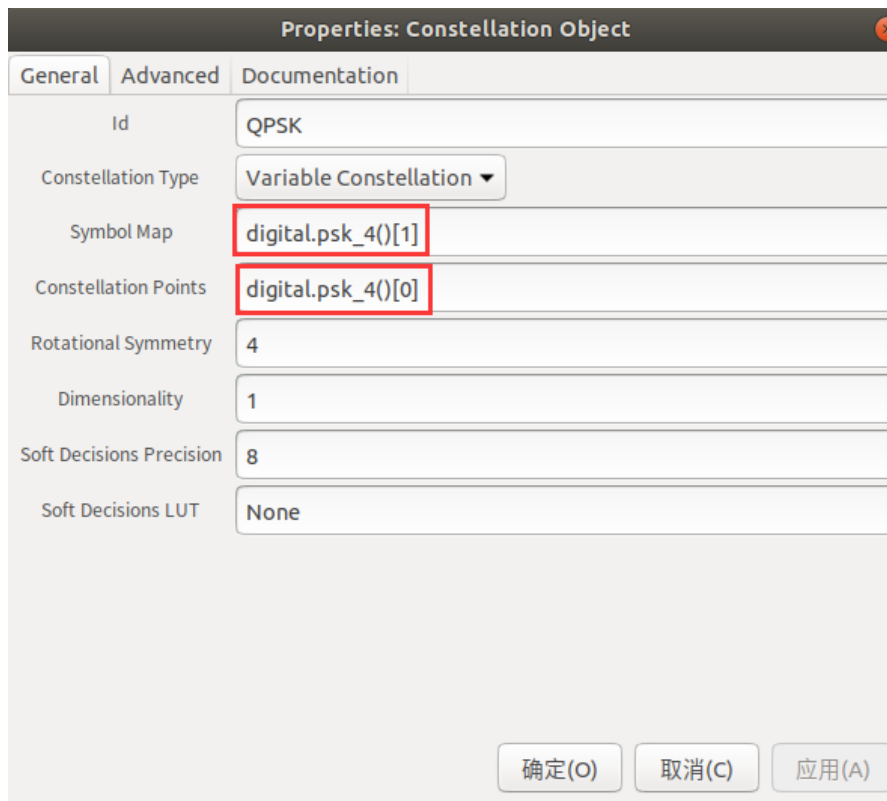


图 9-4

- digital.sd_psk_2_*: Returns (constellation, symbol_map) lists for different rotations for BPSK.
- digital.sd_psk_4_*: Returns (constellation, symbol_map) lists for different rotations for QPSK.
- digital.sd_qam_16_*: Returns (constellation, symbol_map) lists for different rotations for 16QAM.
- digital.soft_dec_table_generator: Takes in a generator function (like the digital.sd_XXX above) and creates a LUT to a specific precision.
- digital.soft_dec_table: Takes in a constellation/symbol map and uses digital.calc_soft_dec to generate a LUT to a specific precision.
- digital.calc_soft_dec: Takes a complex sample and calculates the soft decisions for a given constellation/symbol mapping.
- digital.calc_soft_dec_from_table: Given a sample and a LUT, returns the soft decisions of the LUT for the nearest point to the sample.

图 9-5

在进行 QPSK 的解调的时候，我们使用到了星座图解调模块，通过使用这个模块就能够将接收到的数据用于反映射到原始的符号上，如图 9-6 所示。

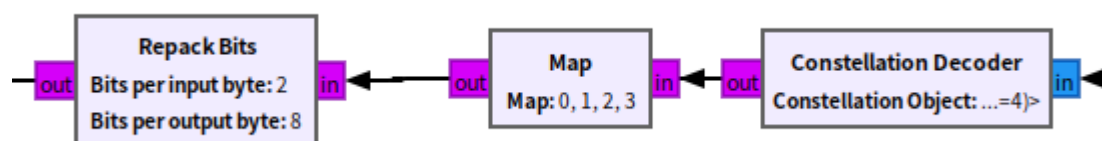


图 9-6

点击运行仿真，可以看到经过简单的同步之后的星座图和时序图，其中解调出来的数据和原始数据的时序图如图 9-7 所示。可以看到，经过解调之后的数据和原始数据一致。

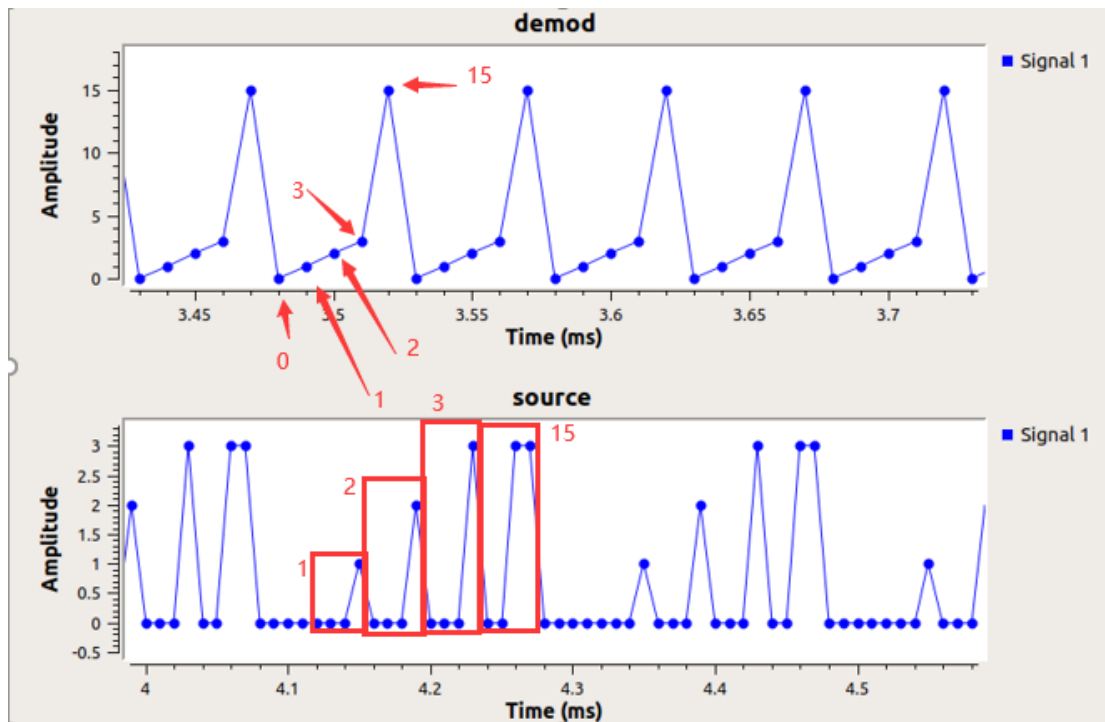


图 9-7 解调出的数据和原始数据

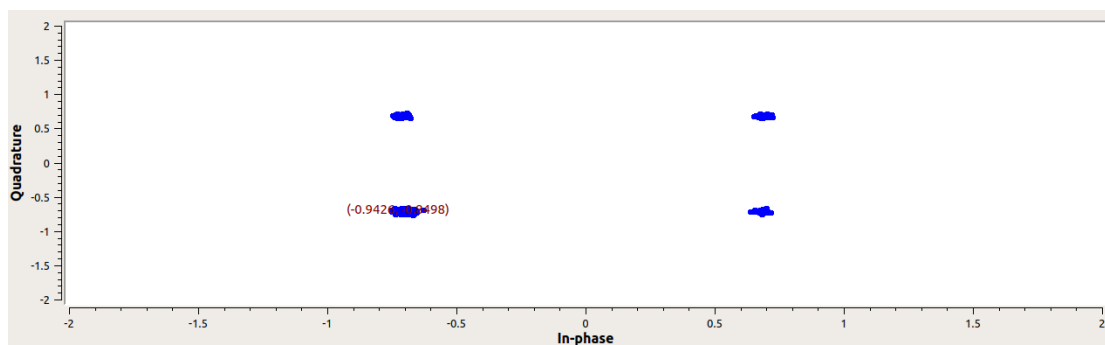


图 9-8 经过简单同步的星座图

然后我们可以观察经过简单的同步之后的星座图。可以看到 QPSK 的星座图有 4 个星座点。

9.3 使用 QPSK 发送文本数据

在前面使用 QPSK 完成了简单的调制解调的仿真实验之后，我们可以尝试使用 QPSK 来发送一些简单的文本信息，这样能够使 QPSK 在实际使用中更有意义。

在本章的实验当中，使用到的模块基本和上一章当中的实验一致，具体的设计过程，可以参考上一章当中的 BPSK 发送文本的实验。新建工程 `lab2_qpsk_text.grc`。

和前面的实验当中的 BPSK 的解调的同步相比，在本次实验当中，使用到了另外一种均衡的方法，本次实验中用于解调同步的模块如图 9-9 所示。在本次实验当中使用到 CMA 均衡模块。

在各种盲均衡算法中，恒模算法（CMA）是一种重要的盲信道均衡方法，广泛地应用在了数字通信系统中，这种算法隐含地利用了接收信号的高阶统计量。

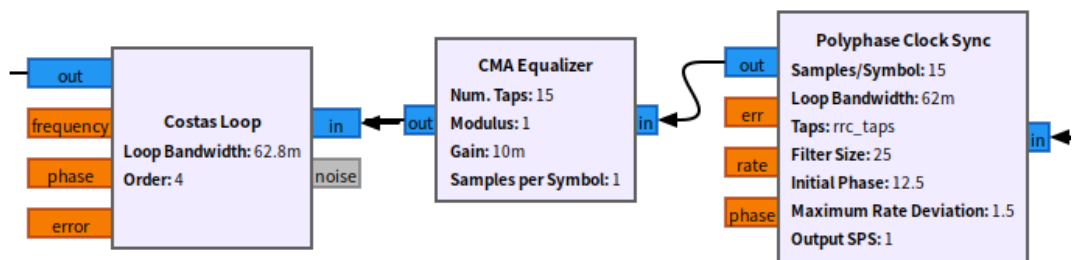


图 9-9 用于同步的模块

对于 CMA 模块的设置如下：

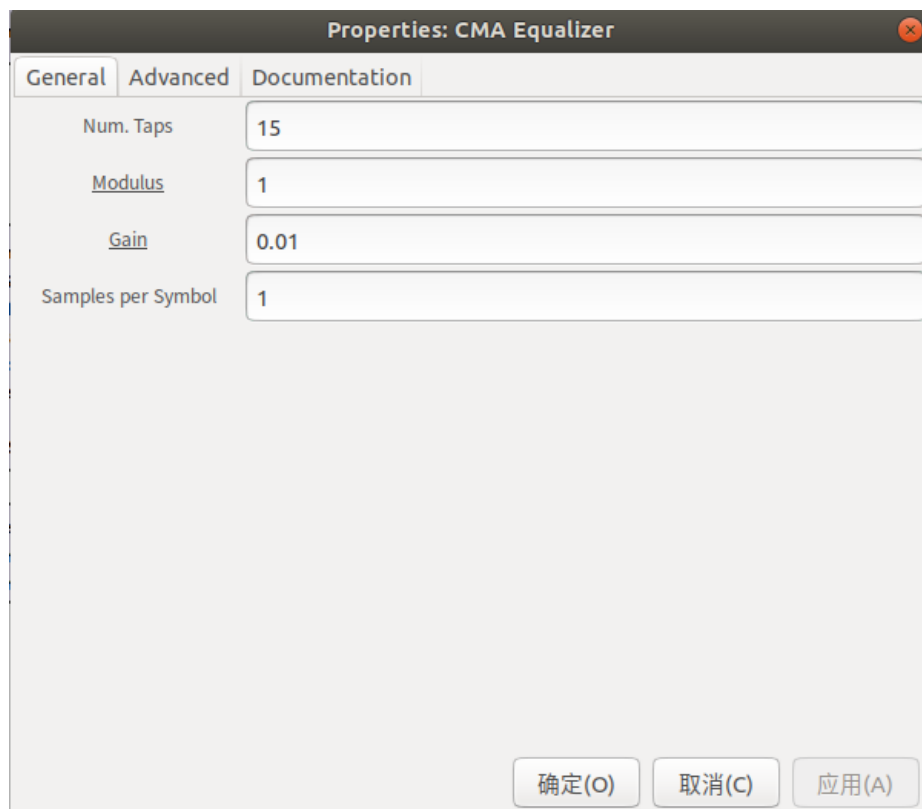


图 9-10 CMA 设置

本次实验的 GRC 流图如下，具体的 GRC 流图可以参开所提供的 demo 中的源文件。

其中 QPSK 调制部分的 GRC 流图如图 9-11 所示

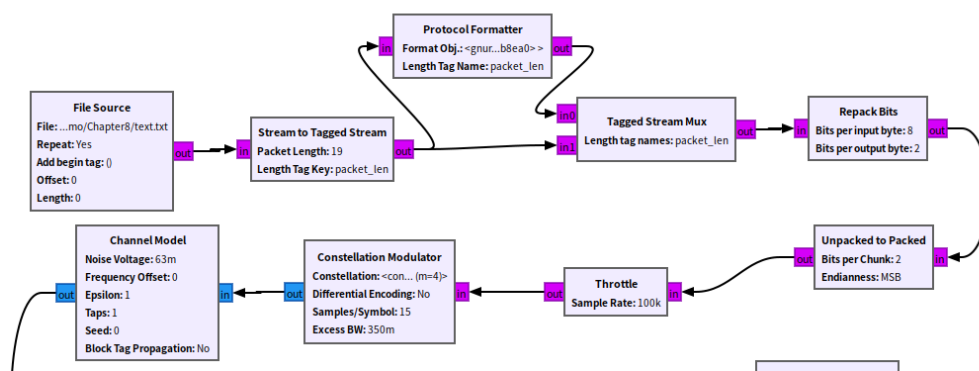


图 9-11 QPSK 调制部分 GRC

QPSK 解调部分的 GRC 流图如图 9-12 所示：

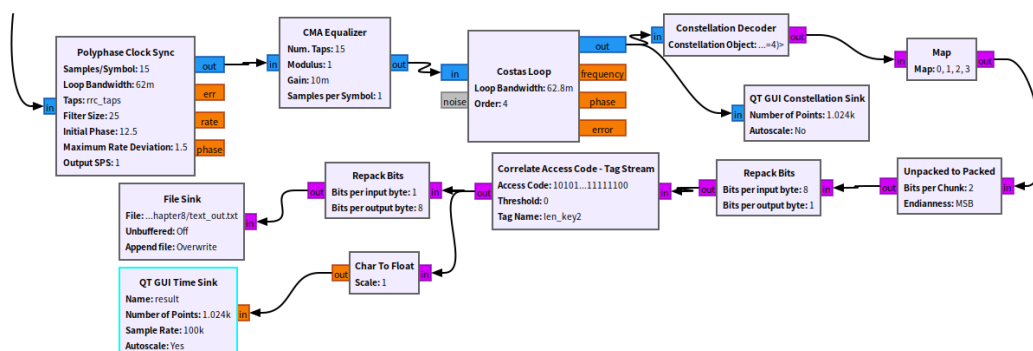


图 9-12 QPSK 解调 GRC 流图

点击运行仿真，然后可以观察用于接收文本数据的文件当中是否出现了经过解调之后的数据。

9.4 使用 ANTSDR 收发数据

在完成了上面的实验之后，我们可以着手来使用 ANTSDR 来实现真实的无线信号的收发。本次实验和上一章当中的 BPSK 发送文本实验很类似。新建 GRC 流图 **lab3_qpsk_antsdr.grc**。

本次实验的具体的 GRC 流图可以参考所提供的源文件。

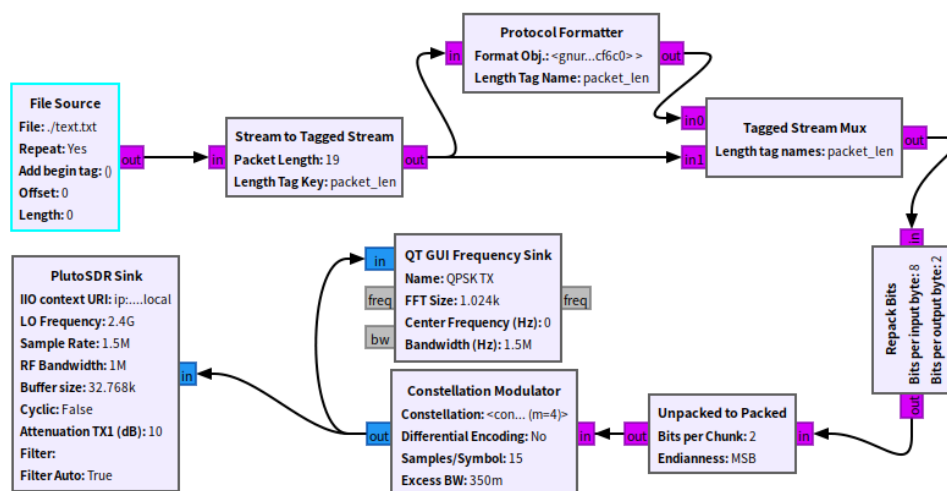


图 9-13 QPSK 调制及发送模块

QPSK 的调制模块如图 9-13 所示，该部分实验和前面在 BPSK 章节所完成的实验基本一致，区别在于星座图调制模块当中使用到的星座图映射是 QPSK 映射。

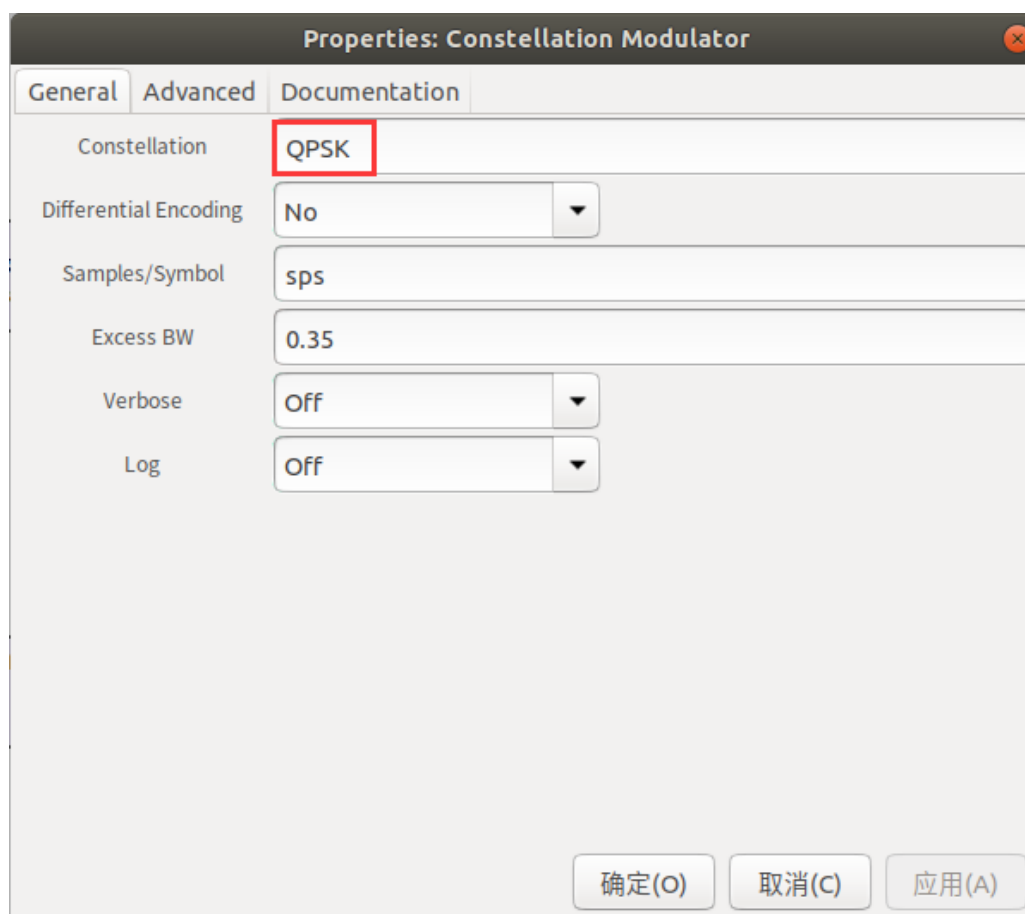


图 9-14

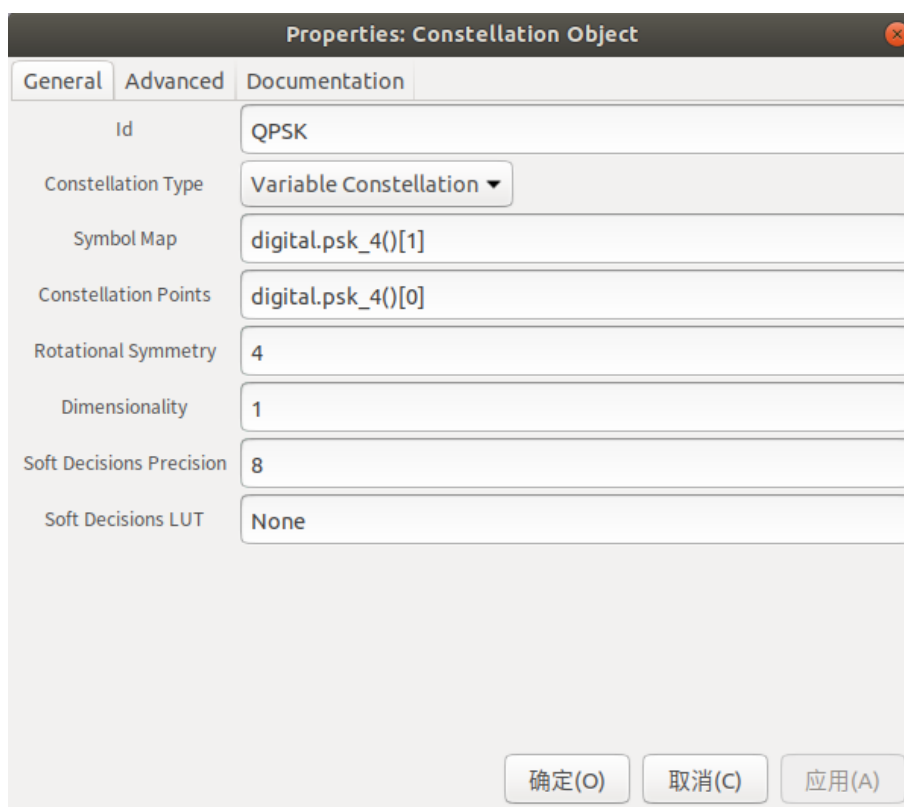


图 9-15

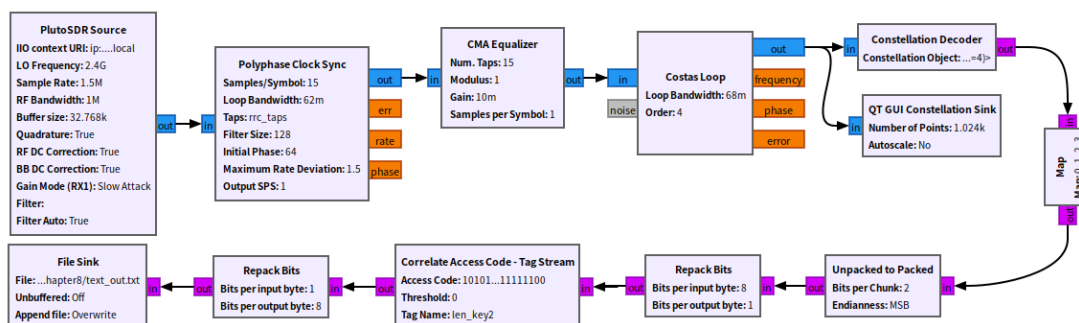


图 9-16 QPSK 解调模块

在 QPSK 解调模块中，首先进行了同步的操作，然后使用星座图解调模块，将星座点反映射到具体的符号上。最终能够将信号解调出来。

9.5 总结

在本章当中，主要完成了对 QPSK 的调制解调的仿真，然后使用 ANTSDR 来

基于 QPSK 发送了真实的数据，并且能够成功地进行接收。QPSK 是一种常用的数字调制方法，在本章当中只是使用 GNU Radio 简单地进行了仿真，在之后可能还会有更多的时候会使用到这种调制方法，这里只是给用户做个简单了解。

第10章 FSK 调制与解调

10.1 频移键控

FSK 是在低复杂度的接收机当中常用的一种调制技术，在 FSK 当中，通过更改载波的频率来发送数字信息。

Binary FSK 是最简单的 FSK 调制技术，bit 0 和 1 分别在空气中传播的不同的频率 f_0 和 f_1 ，比特和符号之间的映射关系如下：

$$\begin{array}{lcl} 0 & \rightarrow & -1 \\ 1 & \rightarrow & +1 \end{array}$$

将 bit 0 映射到 -1，将 bit 1 映射到 +1，这样我们可以得到两个不同的频率。应 i 来标识 $\{-1, 1\}$ 其中的一个元素，我们可以得到的信号的频率如下：

$$F_i = F_c + (-1)^{i+1} \cdot \Delta F = F_c \pm \Delta F$$

其中 F_i 是 FSK 当中用于承载信息所对应的信号的频率， F_c 是载波频率， ΔF 是一个与该载波频率的峰值频率偏差。

于是我们可以简单地得到一个 BFSK 的已调信号。

$$s(t) = A \cos(2\pi F_i t + \phi) = A \cos[2\pi(F_c \pm \Delta F)t + \phi]$$

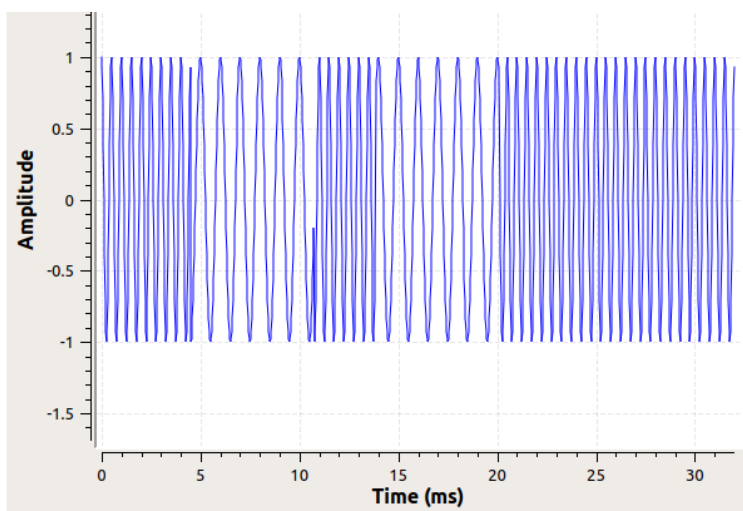


图 10-1 BFSK 时域波形

从 BFSK 的已调信号的时域波形当中，我们能够很直观地通过信号频率来分辨当前所承载的符号信息。

10.2 GNU Radio 当中生成 FSK 信号

在 GNU Radio 当中生成 FSK 信号十分简单，只需要将 0, 1 符号正确地与对应的频率对应起来。新建 GRC 流图 `lab1_fsk_mod.grc` 产生 FSK 信号的 GRC 流图如所示：

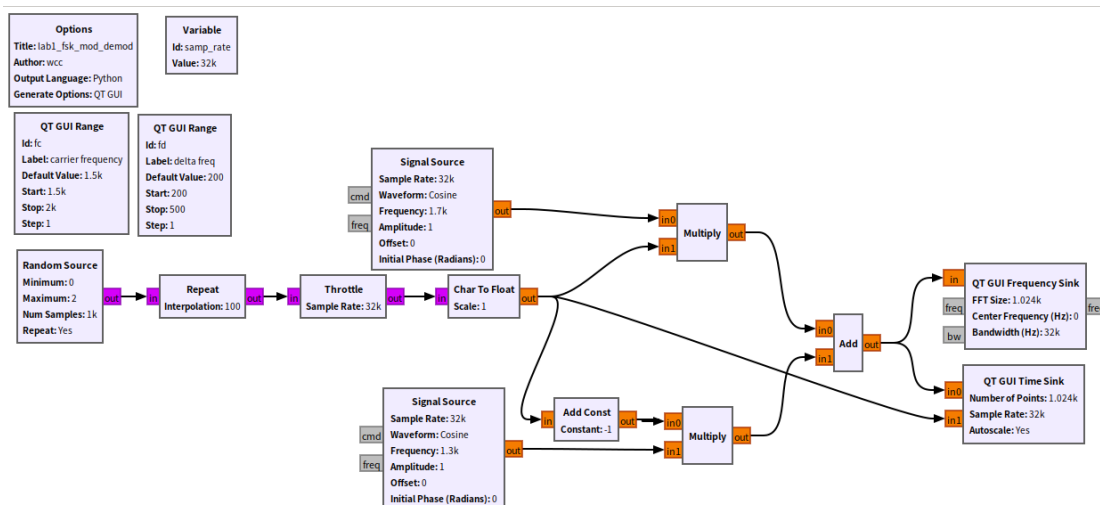


图 10-2 产生 2FSK 信号地 GRC 流图

使用一个随机数产生模块，产生从 0, 1 序列，作为信号源。

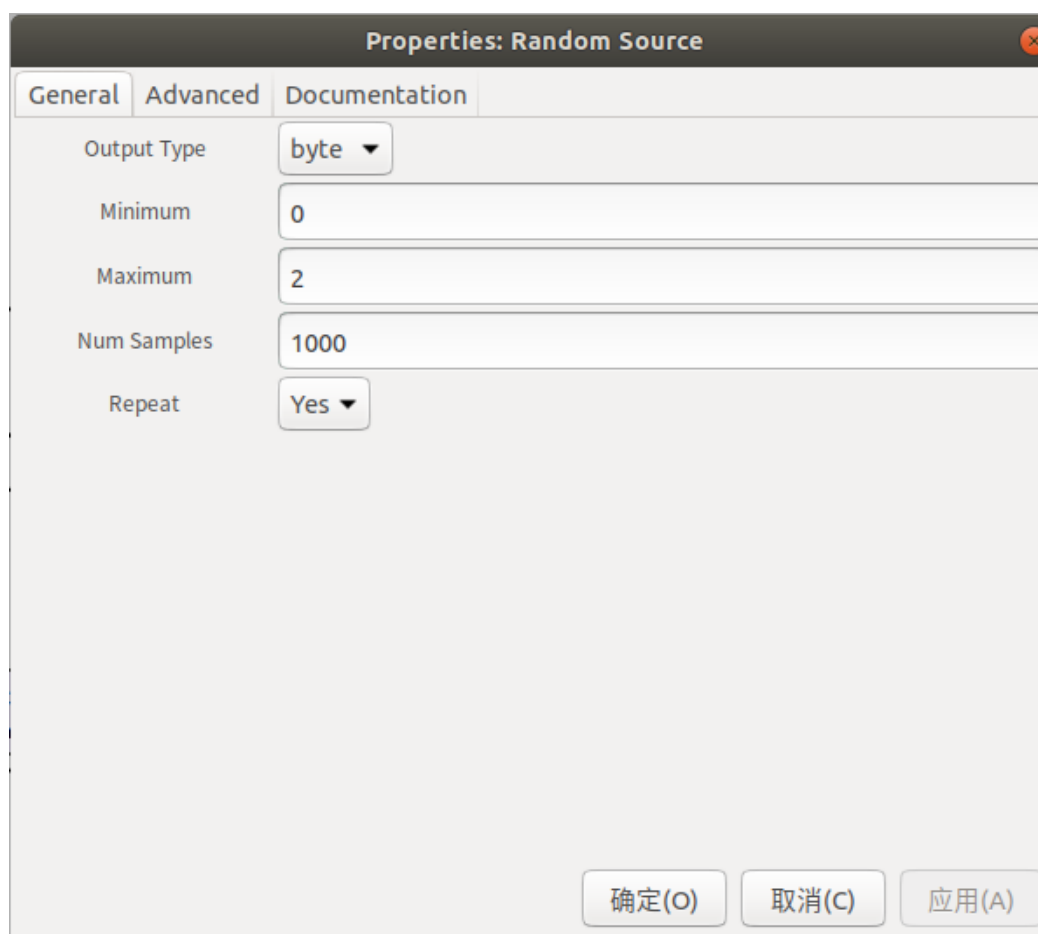


图 10-3

然后将序列进行重复 100 次，形成基带的符号，从这里可以看到，符号率是采样率的 $1/100$ 。

产生了对应的符号之后，需要和每个符号对应的载波进行相乘得到 FSK 调制信号。

点击运行仿真，可以观察经过 FSK 调制之后的结果如图 10-4 所示。

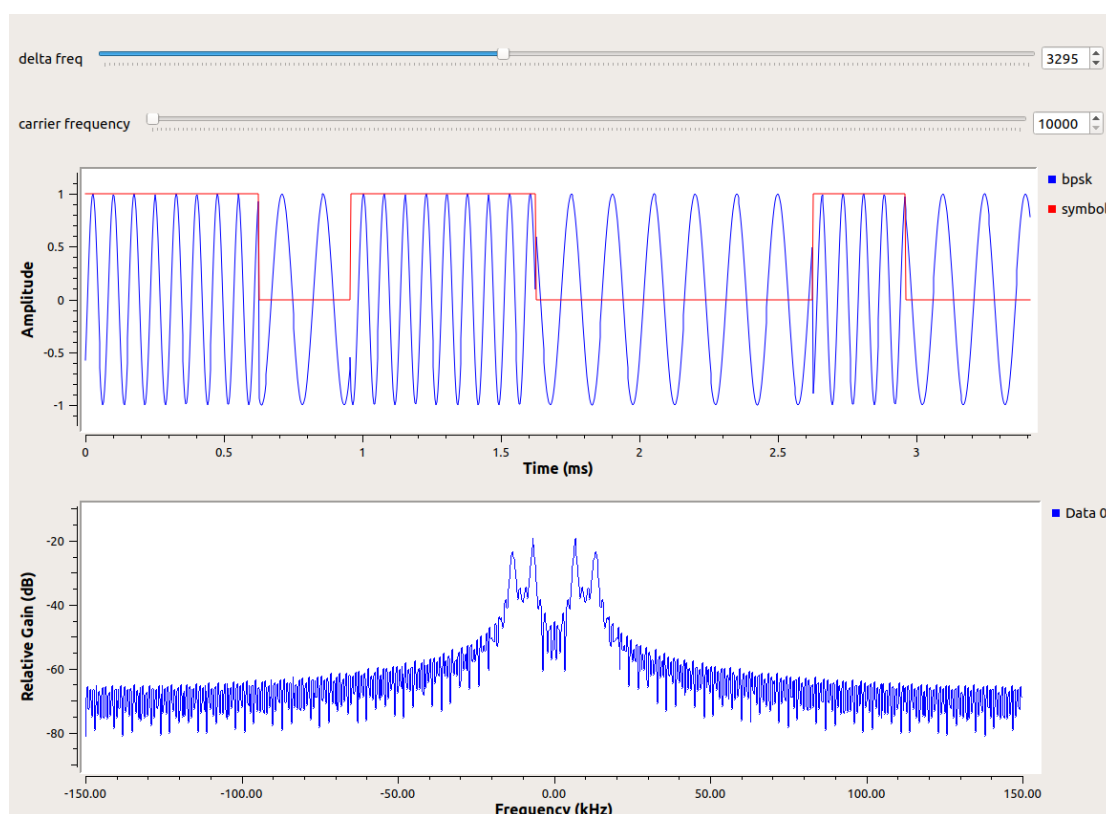


图 10-4 FSK 调制的结果

从时域波形上可以看到，不同的符号对应的载波信号的频率是不同的。由于采用的是 2FSK 信号，所以在频域上能够比较明显地观察到在两个载波频率的位置上有明显的尖峰。

10.3 FSK 信号的解调

新建一个 GRC 流图 `lab2_fsk_mod_demod.grc`。

在 GNU Radio 当中，常用来解调 FSK 信号的模块是 Quadrature Demod。如图 10-5 所示。该模块的输入是复数基带信号，输出的信号是信号的频率相对于采样率的归一化频率，然后再乘上一个增益。

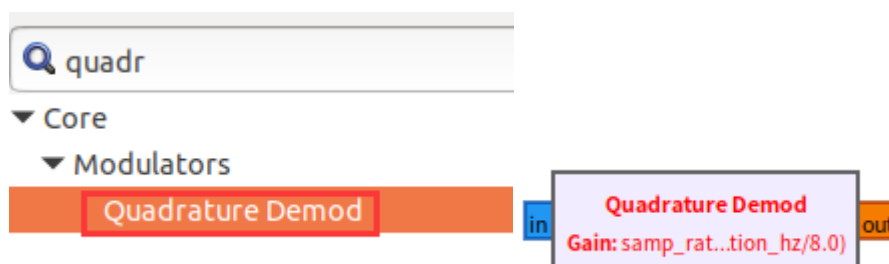


图 10-5 正交解调模块

该模块的工作原理，与之前再介绍 FM 的解调原理类似，采用复数延迟解调的方式来解调出消息信号。

由于 FSK 解调模块的输入是一个复数基带信号，因此需要得到一个复数基带信号，在本章的实验当中，会使用一个 Frequency Xlating FIR Filter 来获取基带信号。



图 10-6

该模块能够完成频率的搬移，信号的抽取和滤波操作。其参数如下：

- **Type:** 可以选择滤波器的类型，该滤波器可以进行数据类型的转换，在本例中使用 `float to complex`，抽头系数可以是实数类型，也可以是复数类型，在本例当中使用实数类型的抽头系数。
- **Decimation:** 是滤波器的抽取倍数，该滤波器可以进行抽取操作，实现下采样。
- **Taps:** 是滤波器的具体的系数，可以使用 `gnuradio` 当中提供的 `fir` 参数模块进行设置参数。
- **Center frequency:** 是指已调信号的中心频率，指定该模块的中心频率的时候，该模块会按照设定的中心频率的值对信号进行下变频。
- **Sample rate:** 系统的采样速率。

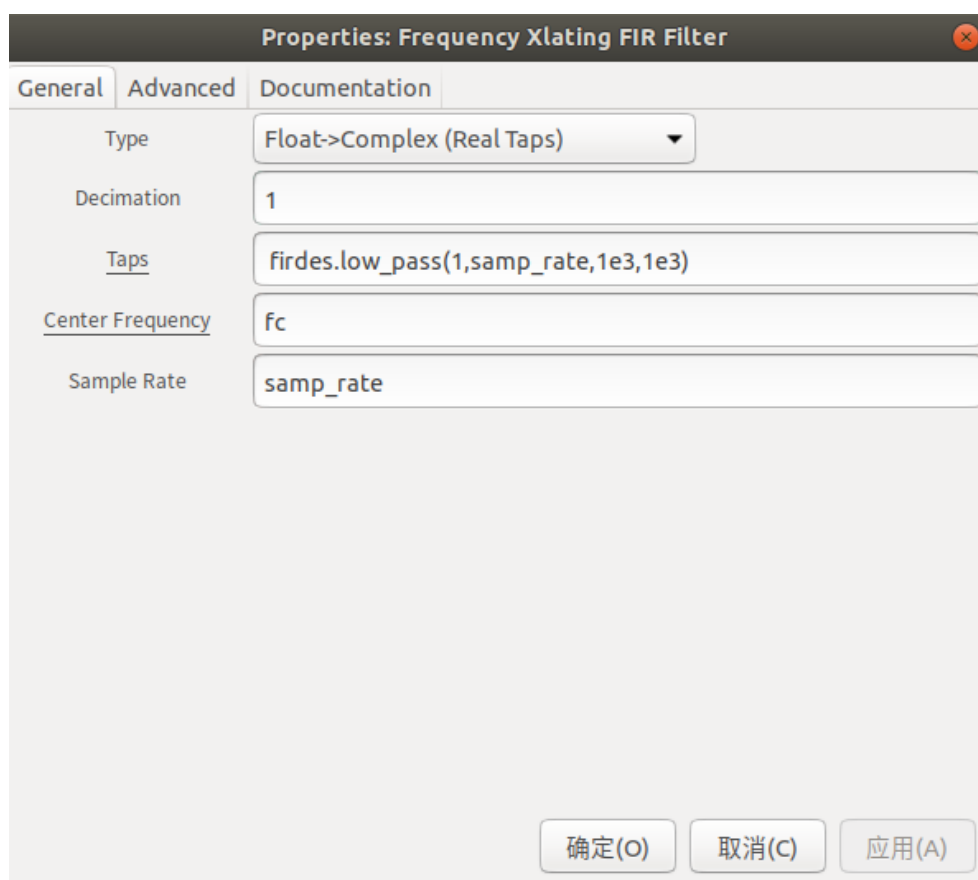


图 10-7

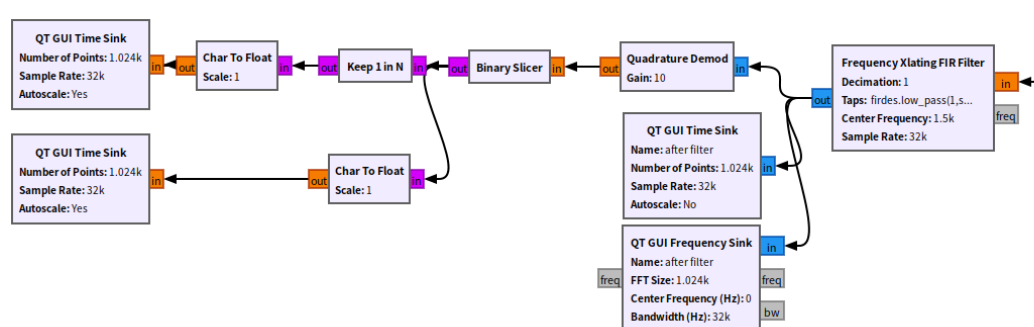


图 10-8 FSK 解调 GRC

点击运行可以运行这个 GRC 流图。首先我们观察经过 FSK 调制之后的信号的频域图和经过 Xlating 滤波器之后的频域波形，如图 10-9 所示。其中上面的信号是经过滤波之后的频域波形，下面是经过 FSK 调制的频域波形。

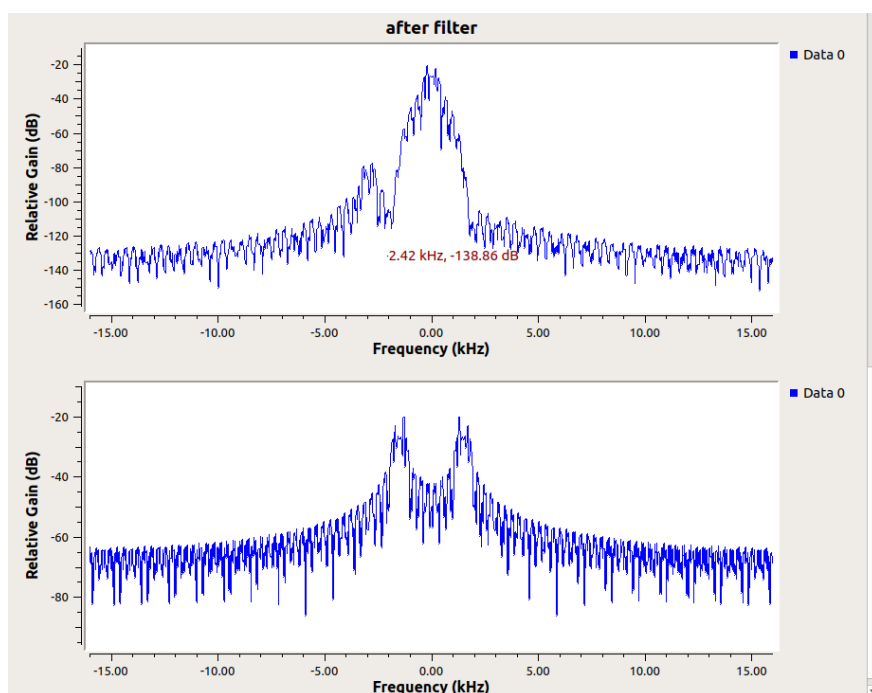


图 10-9 滤波前后信号对比

可以看到经过 FIR 滤波器之后，将经过 FSK 调制之后的信号进行了下变频，并且对下变频之后的信号进行了低通滤波，所以，我们在经过滤波之后的频域图像上能够看到一个较为明显的信号起伏。

最终我们可以观察经过 FSK 解调之后的数据，如图 10-10 所示。

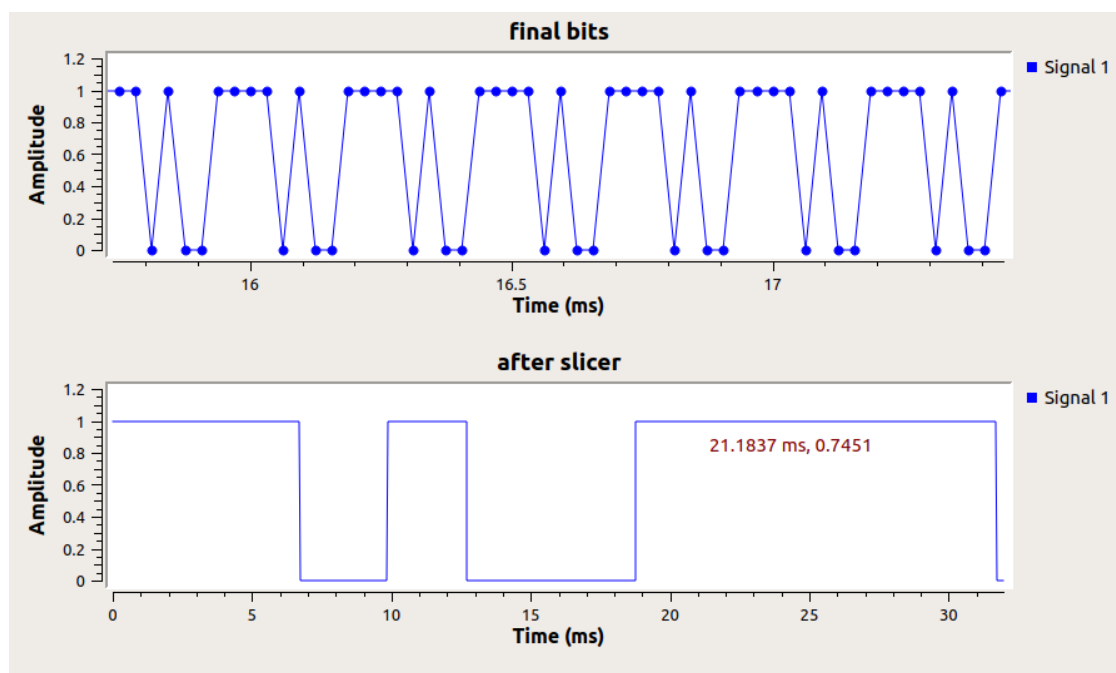


图 10-10 解调之后的数据

其中 after slicer 是得到的二进制数据，可以看到，得到的信号和在图 10-4 中的信号是一致的。

而 final bits 是最终得到的 bit，因为在发送数据时，对原始的 bit 使用 repeat 模块进行了 100 倍插值，最终需要将解调之后的符号进行抽取，从而恢复出原始的 bit。

可以将恢复出来的 bit 和数据源模块当中的 bit 进行比对，可以发现两者是一致的。

10.4 使用 ANTSDR 收发 FSK 信号

在经过上面的实验之后，我们已经能够实现 FSK 信号的收发，接下来我们会使用到 ANTSDR 来完成真实的 FSK 信号的发送和接收的工作。新建 GRC 流图 lab3_fsk_antcdr.grc。

本次实验的 GRC 流图和之前小节当中的实验是比较类似的，有区别的地方就在于我们在本章实验当中会使用到 PlutoSDR 收发模块来与 ANTSDR 进行数据的传输。

FSK 调制以及发送部分 GRC 流图如图 10-11 所示。

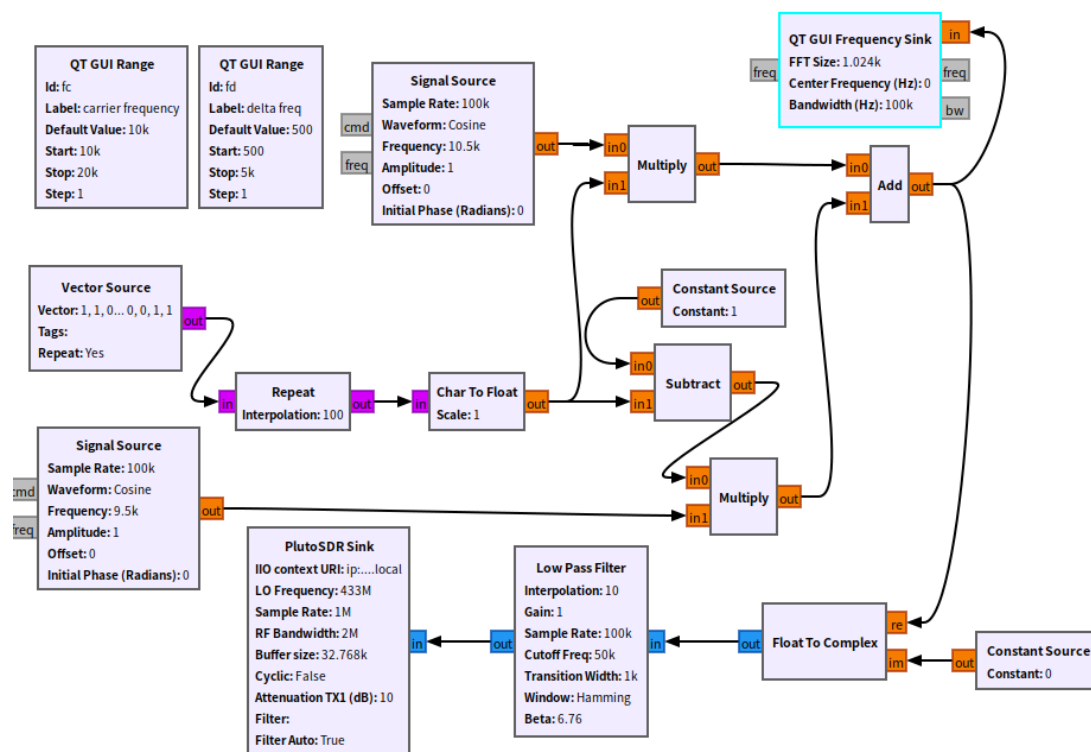


图 10-11 ANTSDR 发送 GRC 流图

其完成的工作和前面小节的实验类似，经由数据源产生的数据，进行 FSK 调制，调制完成之后进行上采样，让上采样之后的数据能够在 ANTSDR 的基带上进行传输。最终通过 PlutoSDR 模块，将信号发送到 433MHz 的频段上。

FSK 的接收以及解调 GRC 流程图如图 10-12 所示。

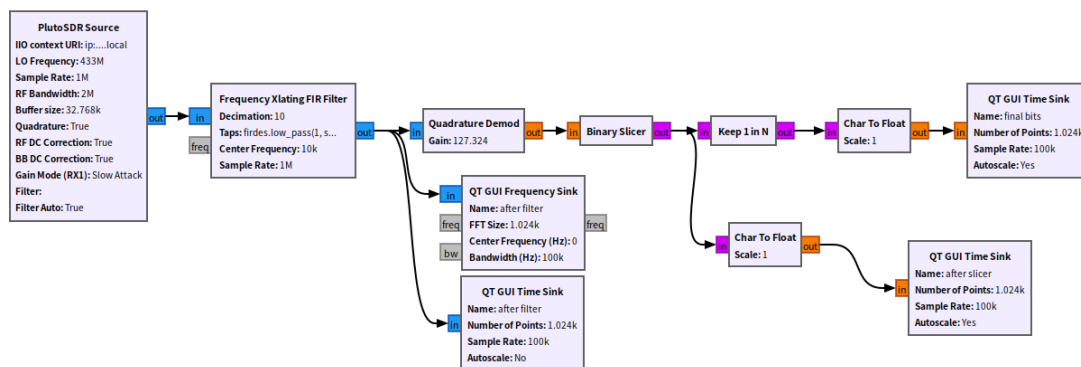


图 10-12 ANTSDR 接收 GRC 流图

在接收流图当中，对接收到的信号首先经过一个 FIR 滤波器，与此同时对数据进行 10 倍的抽取下采样。然后进行正交解调，最终能够得到解调之后的 bit。

点击仿真开始运行这个 GRC 流图。首先我们观察在发送端的信号的频域情况，如图 10-13 所示。可以看到在发送端，FSK 调制的时候的基带信号的载波频率为 10KHz，两个符号所对应的载波的频率偏差为 6KHz 左右。

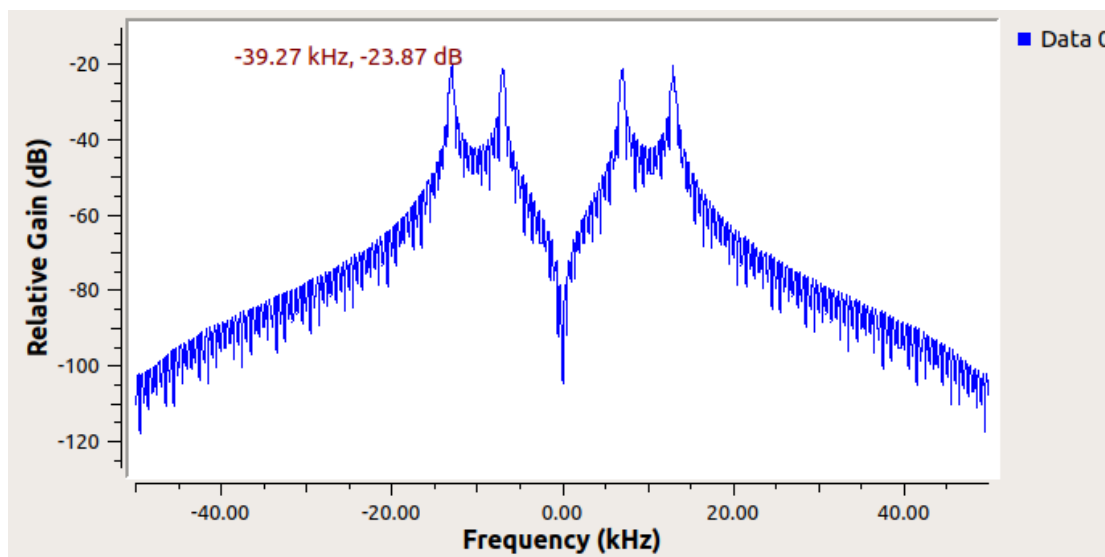


图 10-13 发送端 FSK 信号频域波形

然后我们可以观察在接收端当信号经过 Xlating FIR 滤波器之后的频域结果。可以看到此时信号已经被成功地变频到基带，并且消除了旁瓣的影响。

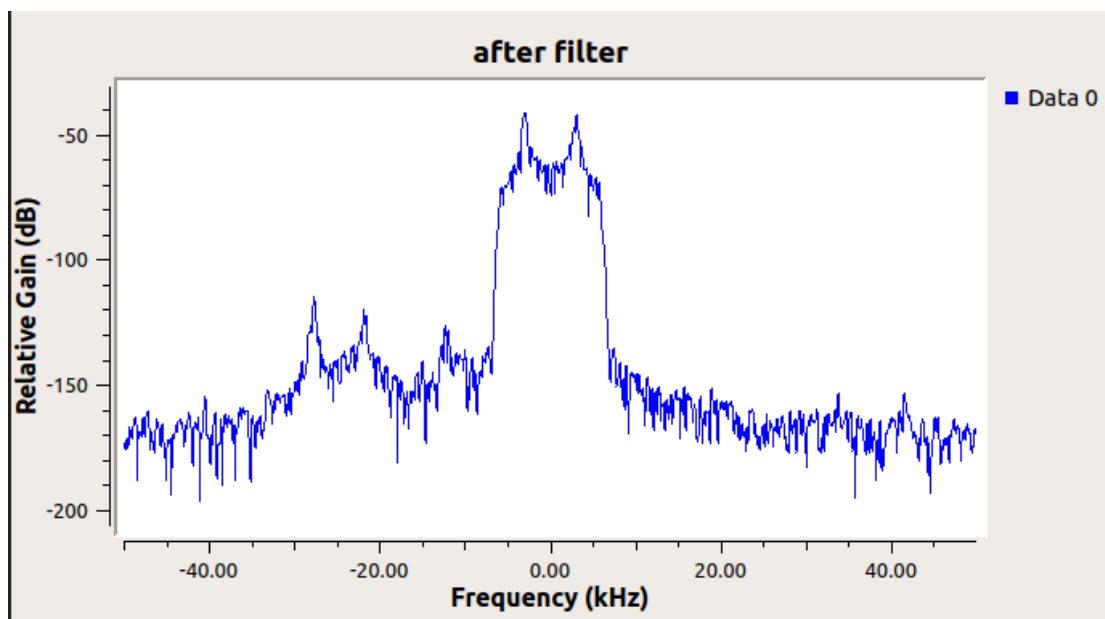


图 10-14 经过 Xlating 滤波器之后的结果

在之后就是进行和前面小节当中的实验类似的操作，对 FSK 信号进行解调，并将信号通过二进制转化模块，恢复出原始的数据，如图 10-17 图 10-15 所示。

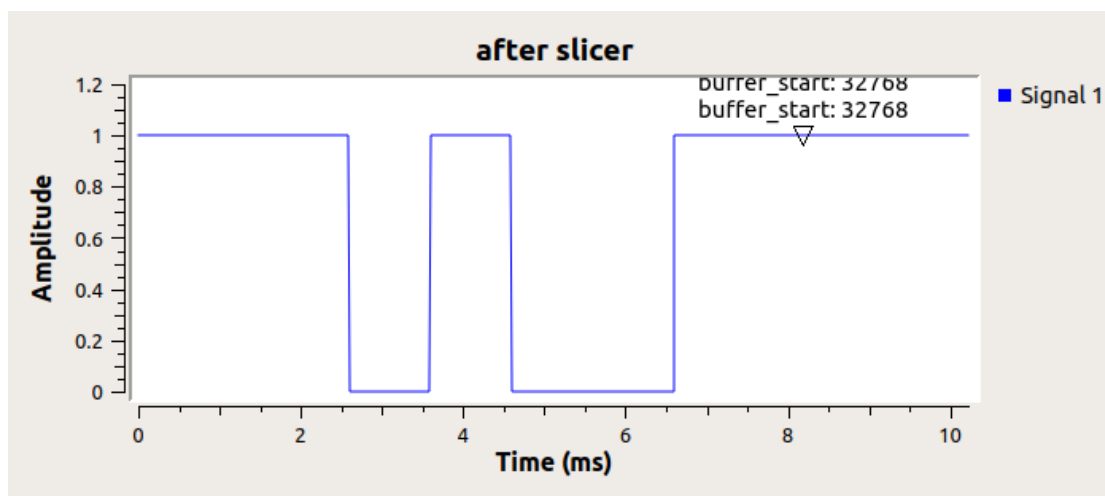


图 10-15 经过解调之后的结果

最终得到的 bit 如图 10-17 所示，可以看到，在发送端，数据源模块产生的数据流为 1, 1, 0, 1, 0, 0, 1, 1，如图 10-16 所示。在接收端，我们也能看到，解调出来的数据大致为 1, 1, 1, 1, 0, 1, 0, 0, 1, 1, 1, 1, 0, 1, 0, 0, 1, 1.....

可以看到，最终接收到的数据经过解调之后，是能够复原成原始的数据的。

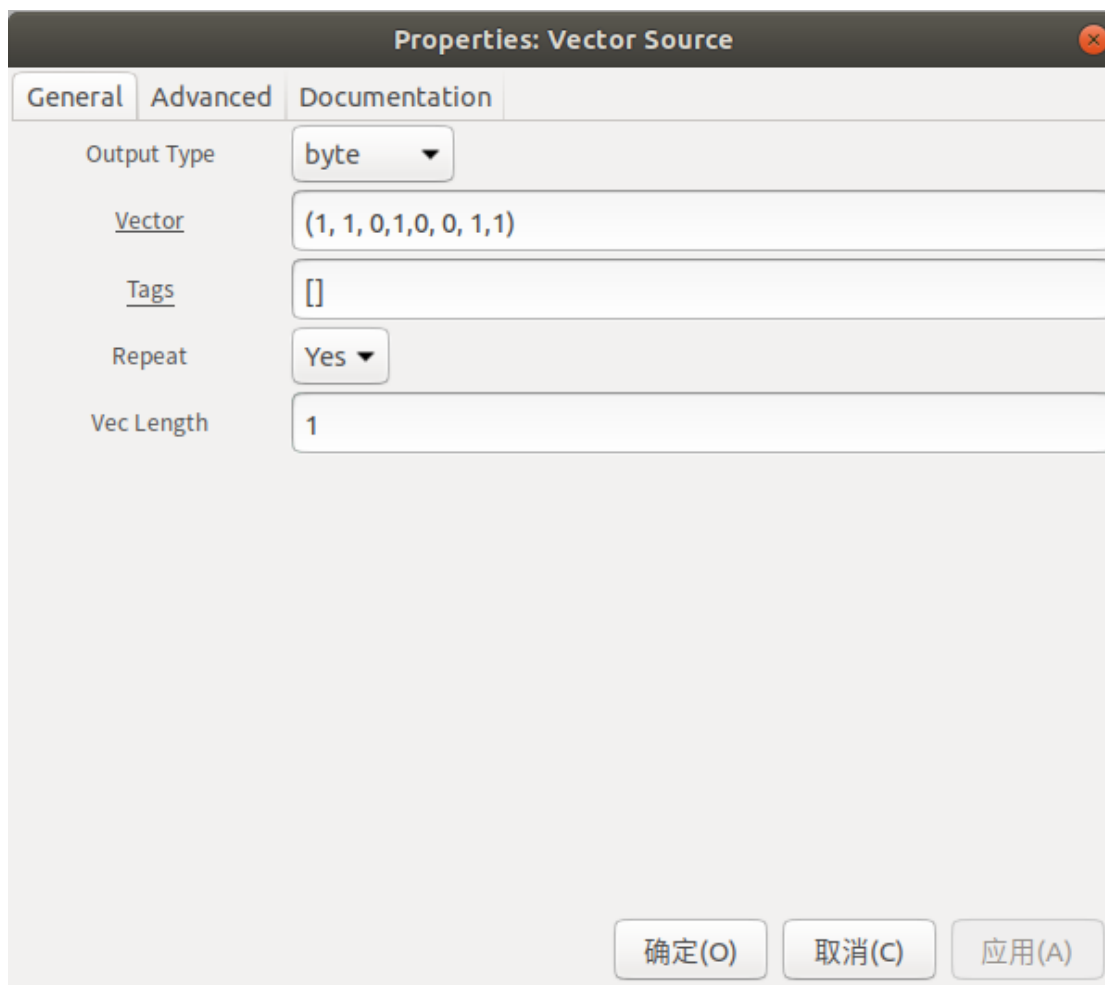


图 10-16 数据源模块产生的数据

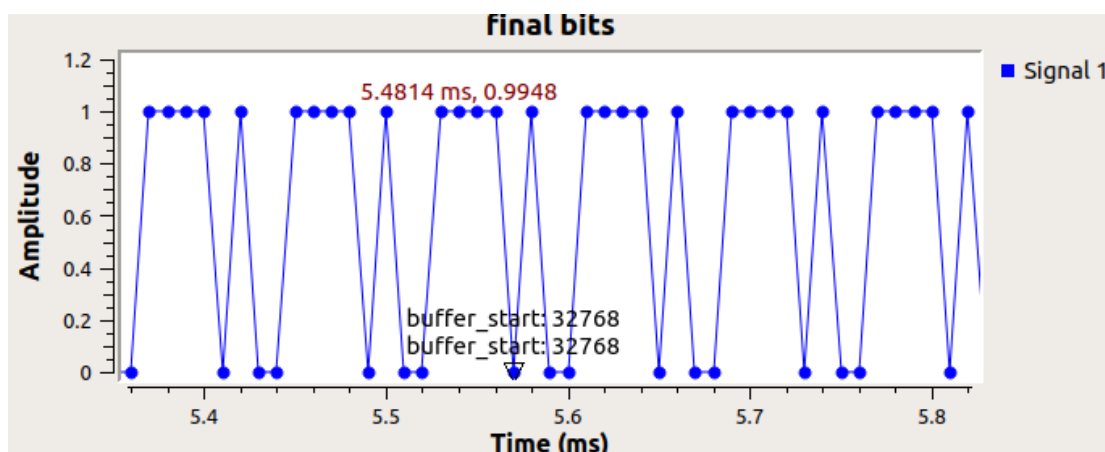


图 10-17 最终解调出来的数据

现在我们已经能够使用 FSK 进行简单的数据传输了，接下来的工作可以是构建自己的协议，来完成简单的通信了，由于在前面的章节当中，已经多次向读者介绍过如何来构造简单的协议，在本章当中就不再重复执行这些操作了，感兴趣

的用户可以自己来实现基于 FSK 实现文本或者图像的发送。

10.5 总结

在本章当中，我们简单地介绍了 FSK 地调制方法，并且在 GNU Radio 当中创建对应地流图实现了 FSK 的仿真，直观地演示了 FSK 调制之后的结果。并且最终借助 ANTSDR 完成了 FSK 信号的收发工作，能够成功地将信号接收并解调恢复出原始的数据。

第11章 使用 GNU Radio 发送视频

在前面章节的实验当中，我们介绍了无线通信当中常用的几种调制方式，前面的实验都是比较基础的，在本章的实验当中，我们会参照别人的开源工程，使用 GNU Radio 通过 ANTSDR 来发送一段视频，并在另外一台电脑上使用 SDRAngle 通过 ANTSDR 来对信号进行接收并解调。

11.1 DVB 系统

DVB(Digital Video Broadcasting, 数字视频广播)是广播电视传输的一些列标准。DVB 系统传输方式有如下几种：卫星(DVB-S 及 DVB-S2)；有线(DVB-C)；地面无线(DVB-T)；手持地面无线(DVB-H)。这些传输方式的主要区别在于使用的调制方式，因为它们应用的频率带宽的要求不同。利用高频载波的 DVB-S 使用 QPSK 调制方式，利用低频载波的 DVB-C 使用 QAM-64 调制方式，而利用 VHF 及 UHF 载波的 DVB-T 使用 COFDM 调制。

在本章的实验当中我们会使用到 DVB-S 和 DVB-S2。

GNU Radio 的示例代码中，提供了有关 DVB-S 和 DVB-S2 调制的例子，本章实验当中，我们基于这两个例子来使用 ANTSDR 完成视频信号的传输。

11.2 获取 GNU Radio 源码

在前面介绍 GNU Radio 安装的时候，我们是直接使用包管理工具来直接安装 GNU Radio 的，使用这种方式能够快速地在电脑上安装并使用 GNU Radio。GNU Radio 同样也可以由源码构建来进行构建。本次实验只需要使用使用到对应的两个例子就可以了。

首先到 GNU Radio 的 github 仓库当中将其克隆到本地。其仓库地址如下：

<https://github.com/gnuradio/gnuradio>

gnuradio / gnuradio

Watch 223Star 2.7kFork 1.5k

<> CodeIssues 404Pull requests 19DiscussionsActionsProjects 4

masterGo to fileAdd fileCode

ncorgan and willcode soapy: completed API related t... 2 days ago 14,439

.github/workflows	pkg: add packaging for debian/fedora in-tree	2 months ago
.packaging	pkg: add soapy for fedora build	5 days ago
cmake	soapy: add module gr-soapy	6 days ago
docs	gr-blocks: remove pdu blocks	2 months ago
dtools	dtools: tool to run clang-tidy on codebase, up...	3 months ago
gnuradio-runtime	soapy: added Python support for Soapy types...	4 days ago
gr-analog	analog: qa_sig_source use cmd port	18 days ago
gr-audio	gr-audio: Check CreateEvent() for failure	2 months ago
gr-blocks	blocks: remove tcp/udp qa	18 days ago
gr-channels	filter: remove deprecated mmse_interpolator	18 days ago
gr-digital	digital: remove corr_access_code_bb from gr...	18 days ago

GNU Radio – the Free and Open Software Radio Ecosystem

[gnuradio.org](#)

pythonradioc-plus-plusdspcybersecuritywireless sdr gnu

gnuradiohacktoberfest

Readme

GPL-3.0 License

Releases 130

Release v3.9.1.0 Latest on 23 Mar

+ 129 releases

图 11-1 GNU Radio 仓库

若用户不想使用 git 命令，可以采取通过对应的 Release 将 3.8.2 版本的 GNU Radio 下载到本地。

drmpeg and mbr0wn gr-dtv: Fix VL-SNR bugs based ... 3 days ago 13,569

.github/workflows

fixup! github: Add Centos 7 to GH-A, separate ...

3 months ago

cmake

CMake: Look for MPIR only when no GMP is fo...

2 months ago

docs

Revert "more qt4 qt5 updates": Can't uncere...

5 months ago

dtools

roll-tarball.sh: Extract version string from CMa...

16 months ago

gnuradio-runtime

pmt: remove tag_to_pmt method (unused)

last month

gr-analog

analog: signal source block accepts cmd mes...

9 months ago

gr-audio

gr-audio: Check CreateEvent() for failure

last month

gr-blocks

blocks: message_debug: mutex getting the st...

2 months ago

gr-channels

clang-format: Ordering all the includes

2 years ago

pythonradioc-plus-plusdspcybersecuritywireless sdr gnu

gnuradiohacktoberfest

Readme

GPL-3.0 License

Releases 130

Release v3.9.1.0 Latest on 23 Mar

+ 129 releases

图 11-2 查看当前的 Release

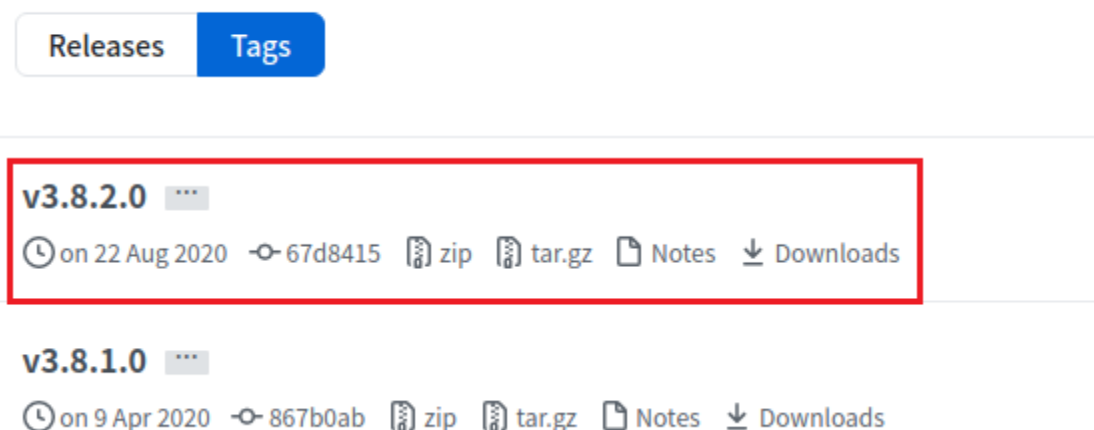


图 11-3

Assets 7

gnuradio-3.8.2.0.sha256	301 Bytes
gnuradio-3.8.2.0.sha256.sig	460 Bytes
gnuradio-3.8.2.0.tar.gz	3.4 MB
gnuradio-3.8.2.0.tar.xz	2.33 MB
gnuradio-3.8.2.0.tar.zst	2.53 MB
Source code (zip)	
Source code (tar.gz)	

图 11-4 GNU Radio 源码

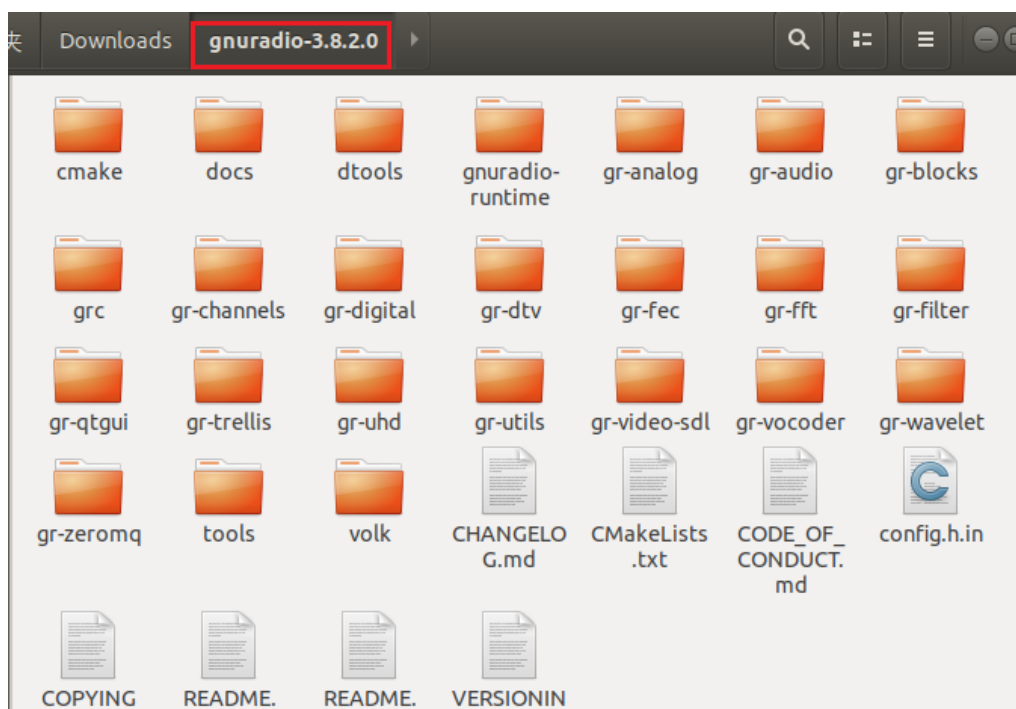


图 11-5 解压之后的目录

用户也可以直接通过 `git` 来对这个仓库进行克隆。首先使用一下命令将仓库克隆到本地。

```
git clone https://github.com/gnuradio/gnuradio.git
```

```
wcc@wcc-dev:~/Desktop$ git clone https://github.com/gnuradio/gnuradio.git
正克隆到 'gnuradio'...
remote: Enumerating objects: 171632, done.
remote: Counting objects: 100% (1020/1020), done.
remote: Compressing objects: 100% (489/489), done.
remote: Total 171632 (delta 558), reused 784 (delta 469), pack-reused 170612
接收对象中: 100% (171632/171632), 58.99 MiB | 4.24 MiB/s, 完成.
处理 delta 中: 100% (133838/133838), 完成.
wcc@wcc-dev:~/Desktop$
```

图 11-6 克隆到本地

然后进入到克隆之后的文件夹，跳转到 3.8.2.0 版本所对应的分支。

```
cd gnuradio
git checkout v3.8.2.0
```

```
wcc@wcc-dev:~/Desktop$ cd gnuradio
wcc@wcc-dev:~/Desktop/gnuradio$ git checkout v3.8.2.0
注意: 正在检出 'v3.8.2.0'。

您正处于分离头指针状态。您可以查看、做试验性的修改及提交，并且您可以通过另外的
检出分支操作丢弃在这个状态下所做的任何提交。

如果您想要通过创建分支来保留在此状态下所做的提交，您可以通过在检出命令添加
参数 -b 来实现（现在或稍后）。例如：

git checkout -b <新分支名>

HEAD 目前位于 67d8415f4 Release 3.8.2.0
```

图 11-7 跳转到对应的分支上

GNU Radio 中有关 DVB 的例子在 `gr-dtv` 目录下所对应的 `example` 目录下。

```
wcc@wcc-dev:~/Desktop/gnuradio$ cd gr-dtv/
wcc@wcc-dev:~/Desktop/gnuradio/gr-dtv$ ls -l
总用量 40
drwxrwxr-x 2 wcc wcc 4096 5月 10 09:35 apps
-rw-rw-r-- 1 wcc wcc 2475 5月 10 09:35 CMakeLists.txt
drwxrwxr-x 2 wcc wcc 4096 5月 10 09:35 docs
drwxrwxr-x 2 wcc wcc 4096 5月 10 09:35 examples
-rw-rw-r-- 1 wcc wcc 297 5月 10 09:34 gnuradio-dtv.pc.in
drwxrwxr-x 2 wcc wcc 4096 5月 10 09:35 grc
drwxrwxr-x 3 wcc wcc 4096 5月 10 09:34 include
drwxrwxr-x 8 wcc wcc 4096 5月 10 09:35 lib
drwxrwxr-x 3 wcc wcc 4096 5月 10 09:34 python
drwxrwxr-x 2 wcc wcc 4096 5月 10 09:35 swig
wcc@wcc-dev:~/Desktop/gnuradio/gr-dtv$
```

图 11-8 示例所在文件夹

我们本次实验需要使用到的就是 examples 下面的 dvbs_tx.grc 或者 dvbs2_tx.grc。

```
wcc@wcc-dev:~/Desktop/gnuradio/gr-dtv/examples$ ls
atsc_ctrlport_monitor.py  germany-g6.grc          vv007-16kfft.grc
catv_tx_256qam.grc        germany-g7.grc          vv008-16kfft.grc
catv_tx_64qam.grc        germany-g8.grc          vv009-4kfft.grc
CMakeLists.txt           germany-g9.grc          vv010-2kfft.grc
dvbs2_tx.grc             README.atsc             vv011-1kfft.grc
dvbs_tx.grc              README.catv             vv012-64qam45.grc
dvbt_rx_8k.grc           README.dvbs             vv013-64qam56.grc
dvbt_tx_2k.grc           README.dvbs2            vv014-64qam34.grc
dvbt_tx_8k.grc           README.dvbt             vv015-8kfft.grc
file_atsc_rx.grc         README.dvbt2            vv016-256qam34.grc
file_atsc_tx.grc         uhd_atsc_capture.grc   vv017-paprr.grc
germany-g10.grc          uhd_atsc_tx.grc        vv018-miso.grc
germany-g1.grc           uhd_rx_atsc.grc        vv019-norot.grc
germany-g2.grc           vv001-cr35.grc          vv034-dtg016.grc
germany-g3.grc           vv003-cr23.grc          vv035-dtg052.grc
germany-g4.grc           vv004-8kfft.grc        vv036-dtg091.grc
germany-g5.grc           vv005-8kfft.grc
```

图 11-9

然后我们就可以使用 GNU Radio 来打开这些示例工程了。为了便于管理，我们先将本次的实验的例子复制到我们自己的工程目录下。

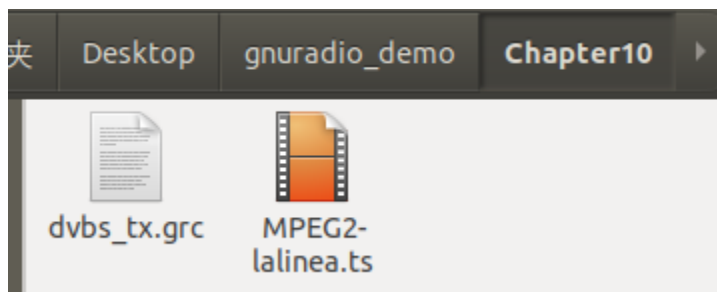


图 11-10 将工程复制到自己的目录下

11.3 DVB-S 视频发送

使用 GNU Radio 打开 dvbs_tx.grc 可以看大本次实验的一个示例工程，我们需要对其进行简单的修改。

首先我们打开原始的 GRC 流图，如图 11-12 所示：

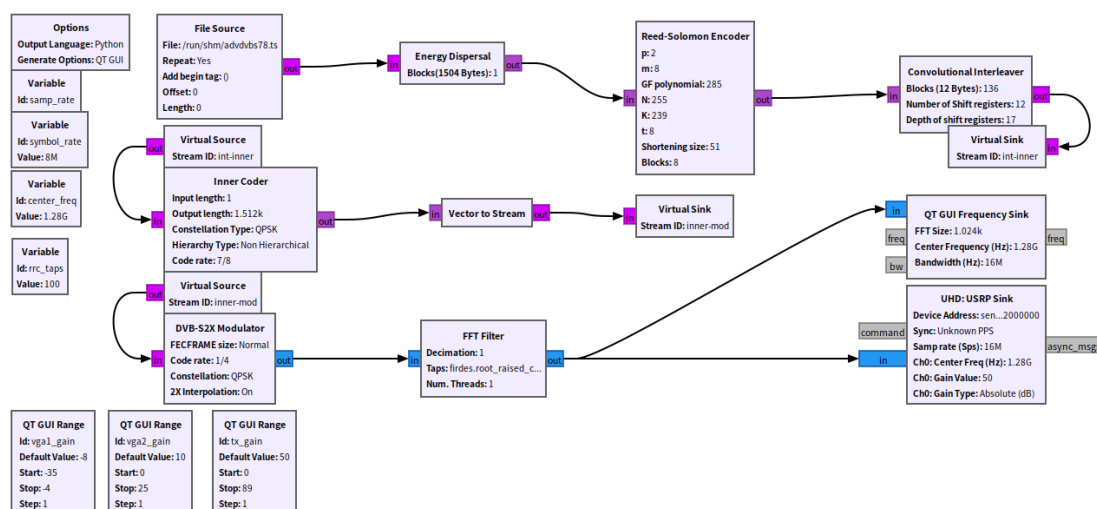


图 11-11 原始的 GRC 流程图

修改之后的整体的 GRC 流程图如下：

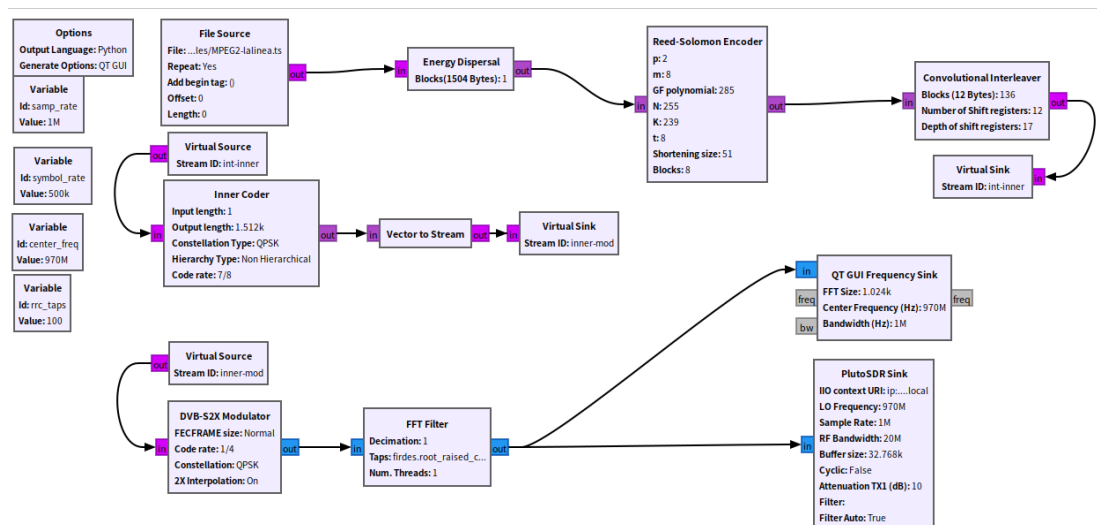


图 11-12 修改之后的整体 GRC 流程图

主要修改的地方是修改符号速率，替换视频源和替换 SDR 硬件平台，原始的例子是基于 USRP 进行设计的。

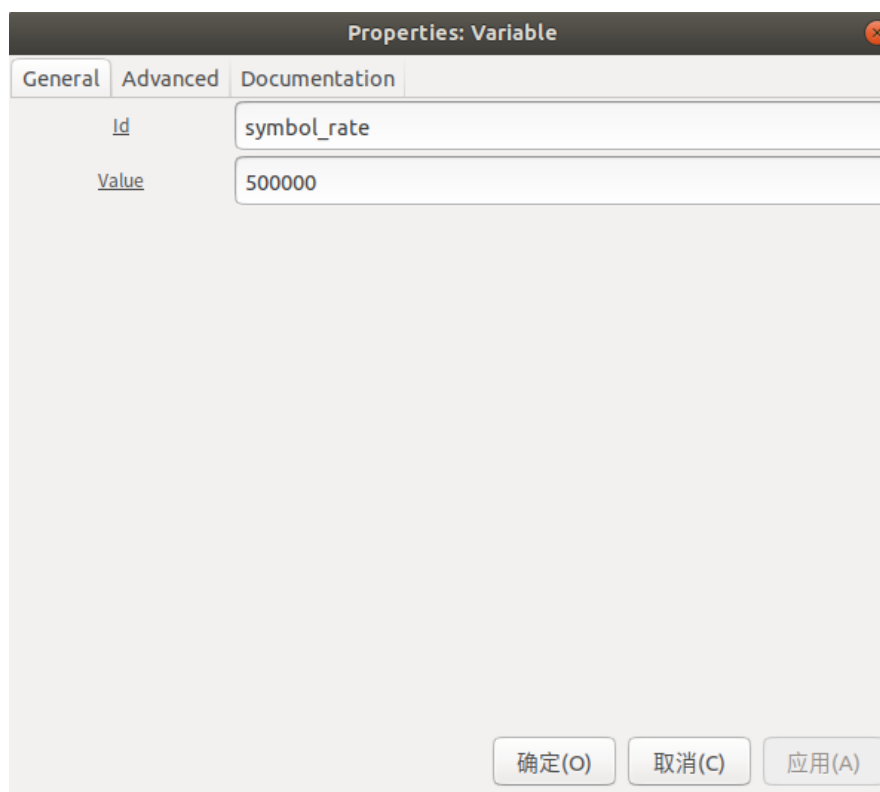


图 11-13 符号速率 500K

数字广播电视需要的图像的格式是多媒体格式 TS 格式，也就是 MPEG2-TS，因此我们在传输的时候首先需要准备一个 TS 格式的视频源，在 gnuradio 的示例 demo 文件夹下，由对应的 README 文件，里面提供了 ts 格式的文件的下载连接。


```

打开(O)  README.dvbt2  保存(S)
~/Desktop/gnuradio/gr-dtv/examples

Test streams are available here:

http://www.w6rz.net/vv001-cr35.ts 478,864,952 bytes md5sum = 34db2abd4f9745b7d241745d81604016
http://www.w6rz.net/vv003-cr23.ts 532,843,888 bytes md5sum = ced15bedb60d02a96256072c8c1ae1c4
http://www.w6rz.net/vv004-8kfft.ts 401,796,420 bytes md5sum = 39bfc2a3ded7fb4639e8fd4b4743188b
http://www.w6rz.net/vv005-8kfft.ts 441,103,648 bytes md5sum = 520fe0fd8a9cf5d5f101f33cb491230d
http://www.w6rz.net/vv007-16kfft.ts 231,303,732 bytes md5sum = 37bb0cde4ae5f3f8ead6d0037e945e4d
http://www.w6rz.net/vv008-16kfft.ts 614,272,704 bytes md5sum = f165d96060e5d6b0148493d0f655fdca
http://www.w6rz.net/vv009-4kfft.ts 365,549,644 bytes md5sum = 2883c89b9deba44486405b87147f15b3
http://www.w6rz.net/vv010-2kfft.ts 190,332,328 bytes md5sum = a11b62bd349882c0345eee7939530812
http://www.w6rz.net/vv011-1kfft.ts 232,390,560 bytes md5sum = 00066052e69661d6501c6bf715cbec69
http://www.w6rz.net/vv012-64qam45.ts 459,617,136 bytes md5sum = ab9569f46a2d08a51b3145c63d979653
http://www.w6rz.net/vv013-64qam56.ts 479,154,096 bytes md5sum = aedc4529fd7fbbbe3bf83df368bc4b96
http://www.w6rz.net/vv014-64qam34.ts 430,739,584 bytes md5sum = 481cb73003703867c1ff6e141af3fcff
http://www.w6rz.net/vv015-8kfft.ts 463,359,088 bytes md5sum = d10c005a4bbf794da1bd01d26f024949
http://www.w6rz.net/vv016-256qam34.ts 593,489,680 bytes md5sum = 3a054cfdcc04b85ad5459a330937ea11
http://www.w6rz.net/vv017-paprtr.ts 592,947,676 bytes md5sum = 10cebd26cd55695be08d294fadcbf1f3
http://www.w6rz.net/vv018-miso.ts 571,898,256 bytes md5sum = b432c64e40a2724eba2051a820733c76
http://www.w6rz.net/vv019-norot.ts 478,864,952 bytes md5sum = 34db2abd4f9745b7d241745d81604016
http://www.w6rz.net/vv034-dtg016.ts 397,372,592 bytes md5sum = 4550dfef45a5d31d8b91508aed1a9c35
http://www.w6rz.net/vv035-dtg052.ts 373,828,224 bytes md5sum = 8f31539a752fff44efcabedf5903b333
http://www.w6rz.net/vv036-dtg091.ts 334,481,140 bytes md5sum = 4d16c6fd3d929bb85e60eed76a7226a5
http://www.w6rz.net/germany-g1.ts 242,683,936 bytes md5sum = f6fc96bd2abed49f375285d45af31e3f
http://www.w6rz.net/germany-g2.ts 291,631,616 bytes md5sum = defa11f02511254e96fc8fbf59d47ff2
http://www.w6rz.net/germany-g3.ts 316,245,704 bytes md5sum = 2494f7b77eb1f11dad2e1c903ba6b389
http://www.w6rz.net/germany-g4.ts 345,044,672 bytes md5sum = da256d6828d97c458ffef27931c626ff
http://www.w6rz.net/germany-g5.ts 347,665,204 bytes md5sum = 48f72cef940f294b26247da91abb9f25
http://www.w6rz.net/germany-g6.ts 444,296,264 bytes md5sum = 605a76d12cce7d7009a08c87e97c70bc
http://www.w6rz.net/germany-g7.ts 351,893,888 bytes md5sum = 3ca71e2a35ab7be27ebd13ad97f00e85
http://www.w6rz.net/germany-g8.ts 365,656,052 bytes md5sum = 8cbd41ea1a6f6405f6662300f17a9104
http://www.w6rz.net/germany-g9.ts 260,004,376 bytes md5sum = a83d60366bca873cf30d18bd1d9e9f9f
http://www.w6rz.net/germany-g10.ts 229,421,288 bytes md5sum = 641a7e4c8a9db92f3020e19ec8d0b8a7

Note: The vv012-64qam45 profile may not run in real-time due
to the large number of PAPR iterations.

IMPORTANT: Because some of the blocks have to process entire
T2 frames, the shared memory requirement will be greater than
the Linux default on older kernels. The size of shared memory
can be increased with the following shell command:

```

图 11-14 TS 格式的测试视频源

本次实验我们也提供了一个原始的视频文件，如图 11-15 所示：

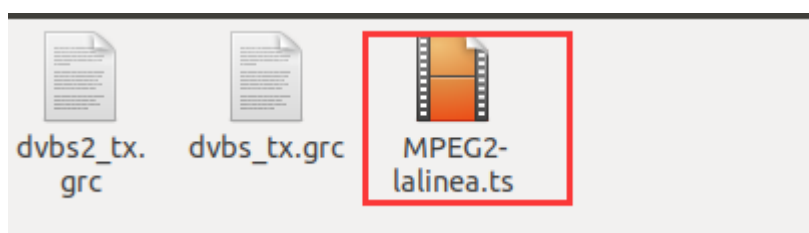


图 11-15

接下来我们就可以运行这个 GRC 流图，从而实现视频的发送了。

先使用一个 ANTSDR 设备连接到 Linux 实体机上。如下图所示：

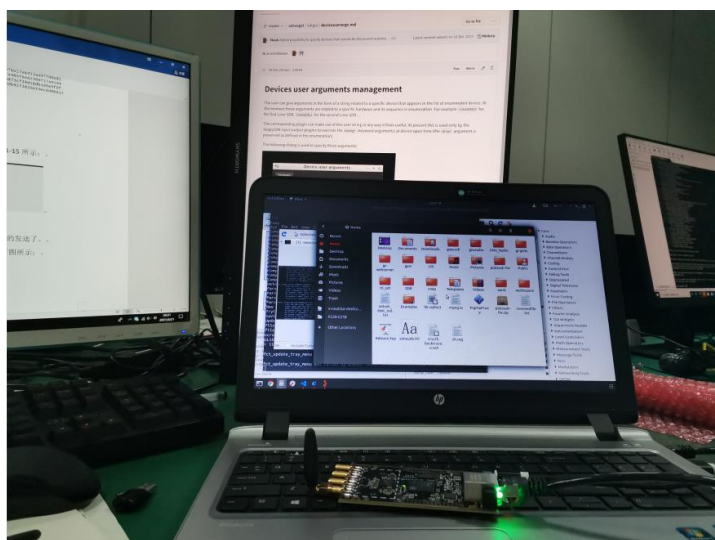


图 11-16 视频发射端连接

11.4 DVB-S 视频解调

本次实验，我们使用 SDRAngel 来实现视频的接收和解调。SDRAngel 是一个开源的 SDR 软件，使用该软件和相应的 SDR 硬件平台可以进行无线信号的收发。

SDRAngel 的 github 仓库地址如下：<https://github.com/f4exb/sdrangel>。我们直接下载 windows 版本的安装包即可。找到的 Release，然后下载对应的 exe 文件即可。

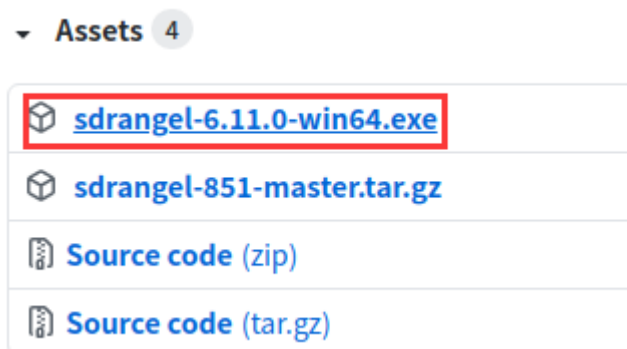


图 11-17 SDRAngel windows 平台安装包

下载完成之后安装对应的软件即可。安装完成之后，我们使用另外一个 ANTSDR 连接到对应的 windows 电脑上，等到 ANTSDR 上的系统启动完成之后，再打开 SDRAngel 设备。

SDRAngel 首次连接到 ANTSDR 时，需要先进行一些设置才能够正确地使用 ANTSDR 这个设备。

从 SDRAngel 的 Github 上的 Doc 可以知道，对于使用网络连接到 SDRAngel 的软件需要进行用户自定义的设置，才能够正确的访问设备。

Github Doc 连接如下：

<https://github.com/f4exb/sdrangel/blob/master/sdrgui/deviceuserargs.md>

首次打开 SDRAngel，先点击上方菜单栏的 Preference 然后选择 Device，然后选择 User arguments。

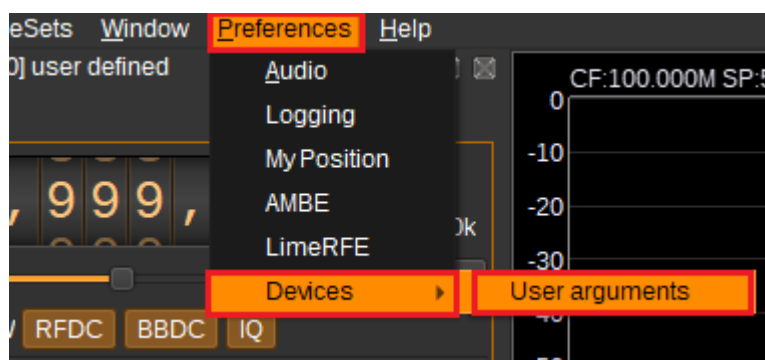


图 11-18 选择用户自定义参数

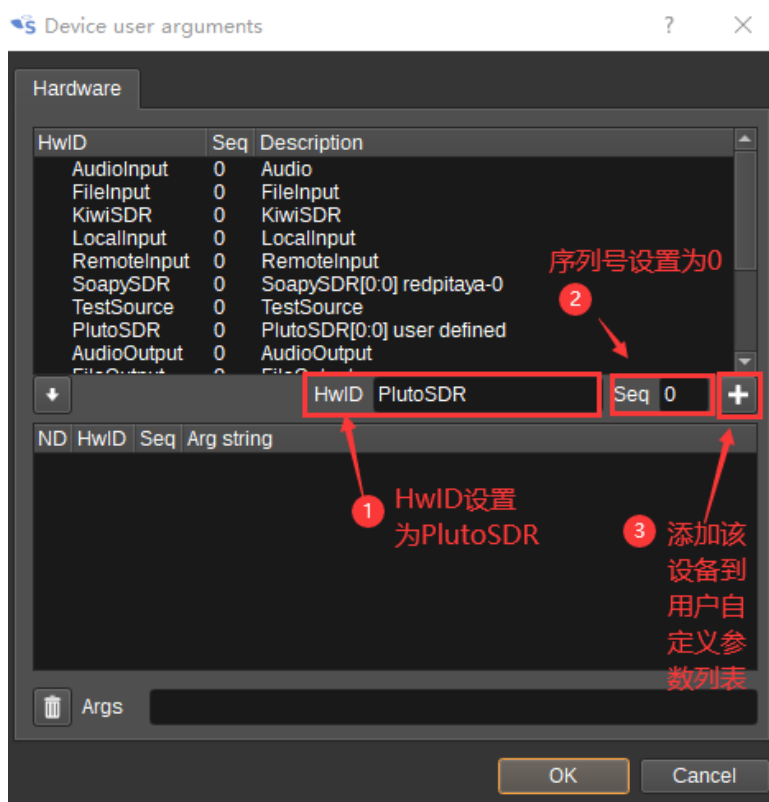


图 11-19 添加 PlutoSDR

添加完设备之后，还需要设置其参数，参数设置为

uri=ip:192.168.1.10

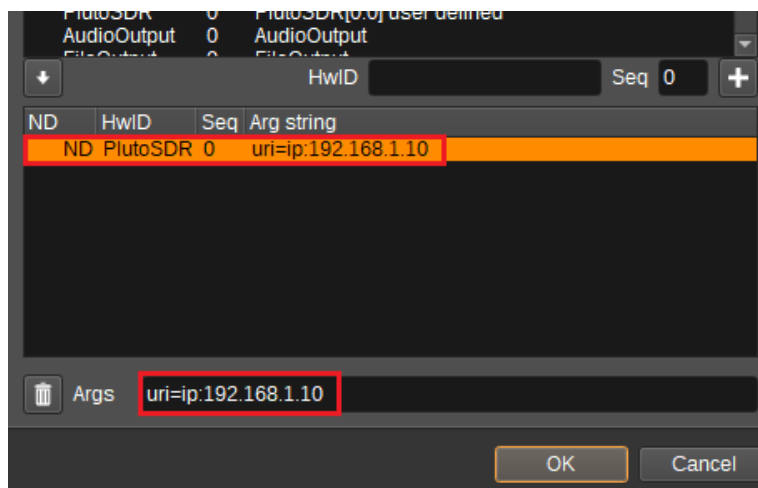


图 11-20 设置 IP 参数

然后再次打开这个用户自定义设备参数当中，我们就能够发现这个添加好的设备。

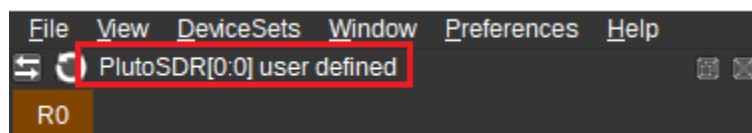


图 11-21

第一次设备添加完成之后，需要关闭 SDRAngel，重新进行启动。此时使用另外一个 ANTSDR 设备连接到一台装有 SDRAngel 的 windows 电脑上。

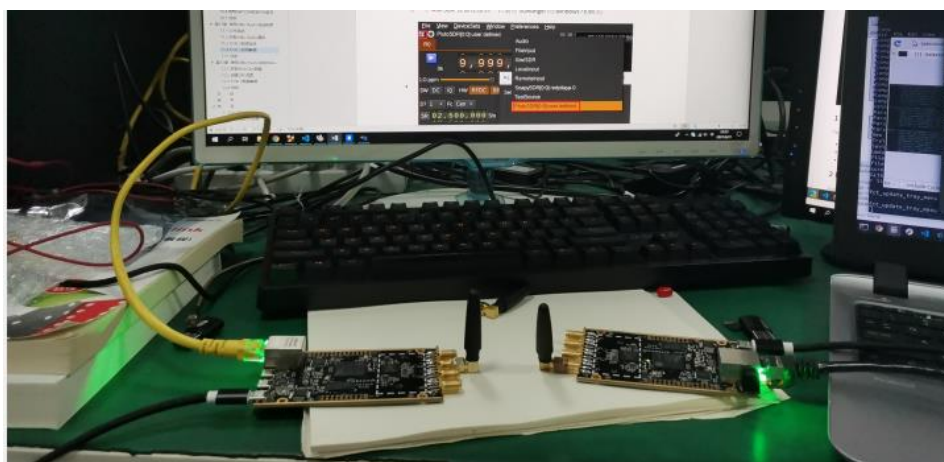


图 11-22 连接另一台 ANTSDR 到电脑

此时再次打开 SDRAngel，点击 File 下方的小箭头，在弹出窗口中选中自定义的 PlutoSDR 设备。

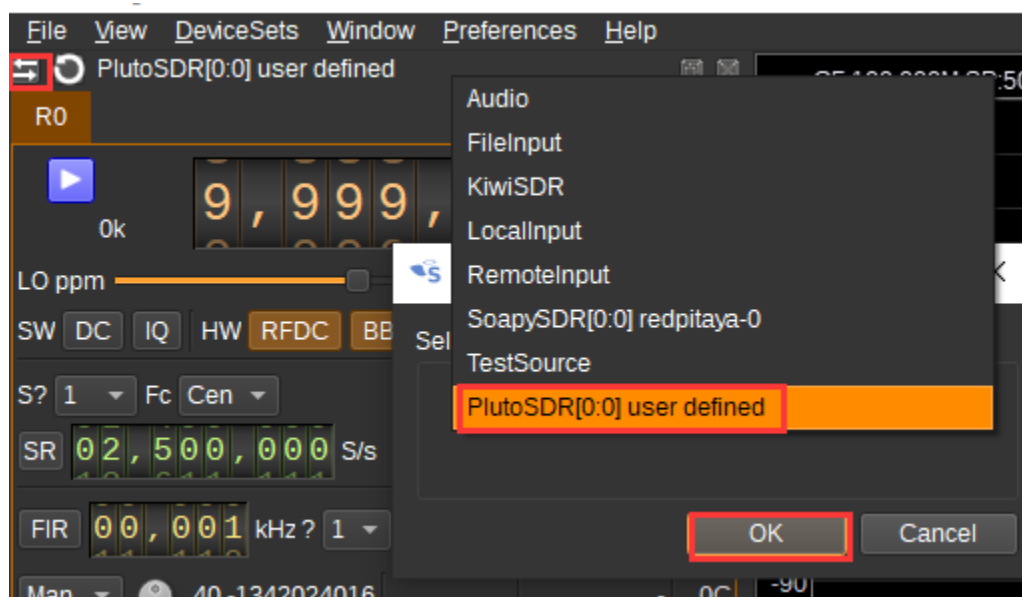


图 11-23

接下来就可以进行 DVB-S 信号的解调了。在 SDRAngel 的右边，添加一个用于解调 DVB-S 信号的解调器，点击 Channels 旁边的加号，添加一个 DATV 解调器。

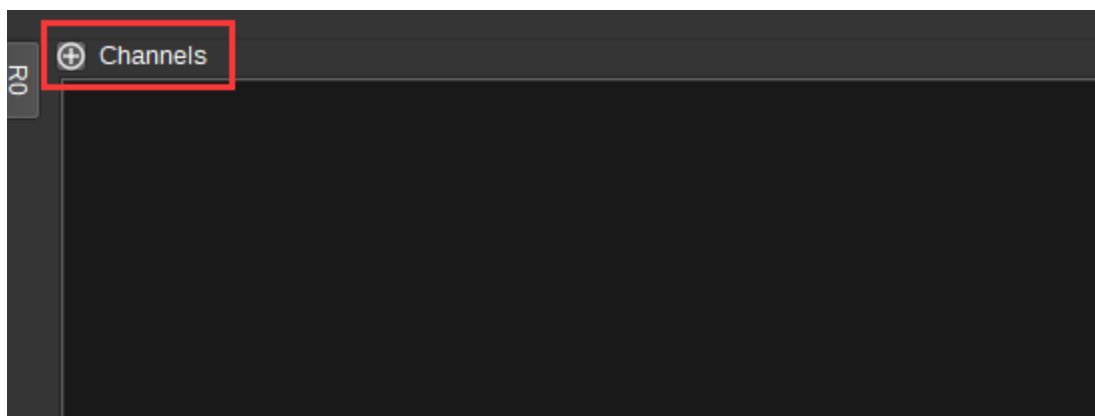


图 11-24

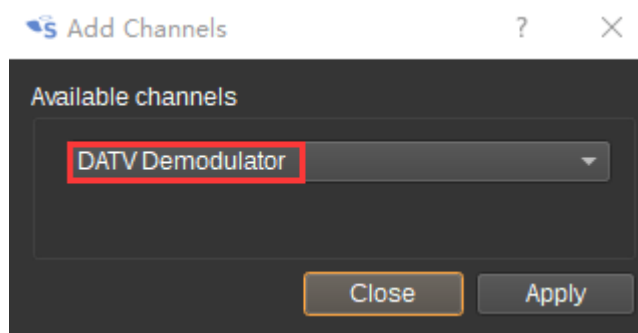


图 11-25 添加 DATV 解调器

DATV 解调器的设置如图 11-26 所示，选择调制传输方式为 DVB-S，调制方式为 QPSK，符号速率 500KHz，接收滤波器采用匹配滤波器,解调的参数按照如下当时进行设置。

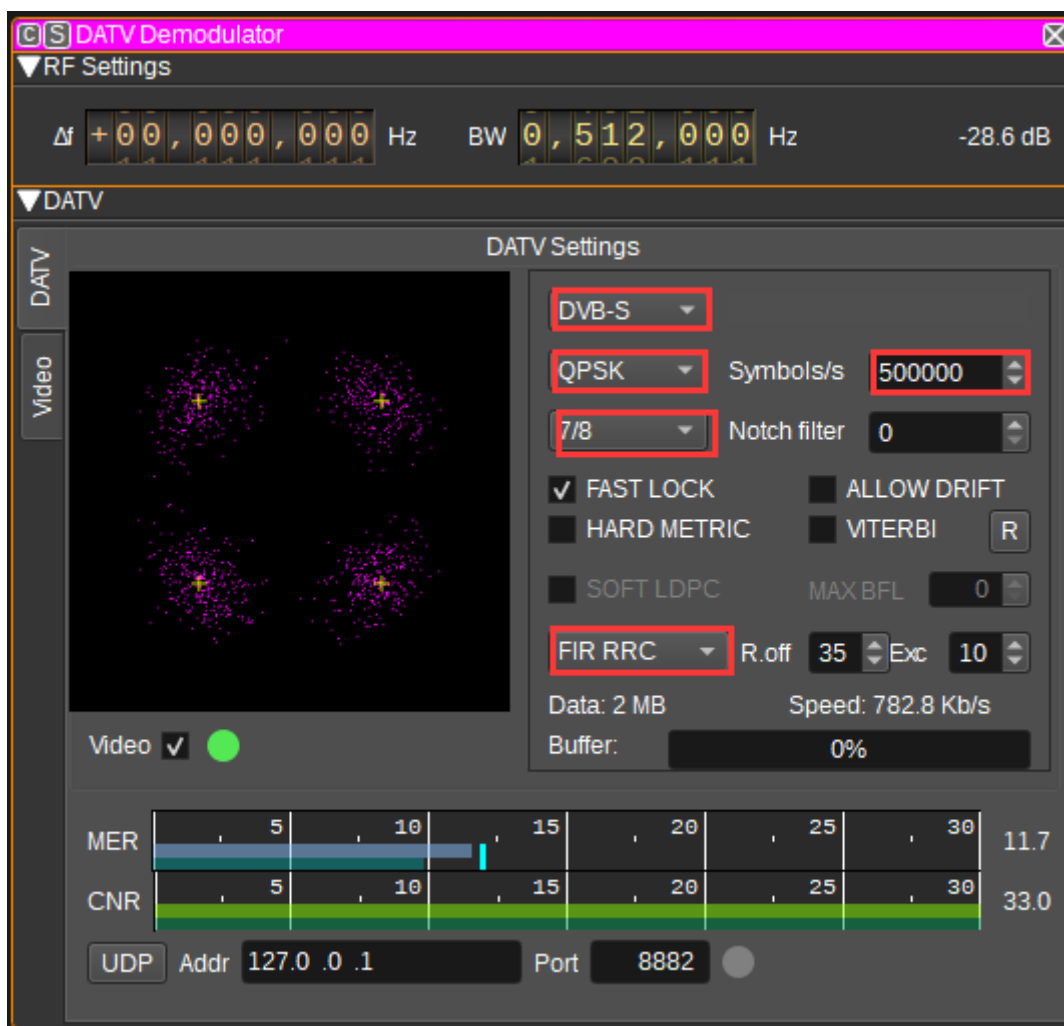


图 11-26 DATV 解调器设置

接下来，我们就可以将 SDR 接收设备的接收频率设置到 970MHz，然后点击

开始即可接收数据。

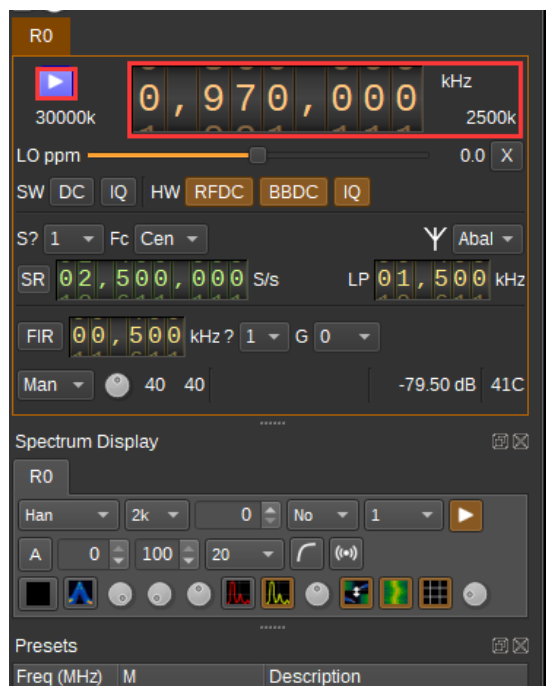


图 11-27 接收机设置

在 DATV 解调的下,就可以看到当前接收到的视频信号了,如图 11-28 所示。

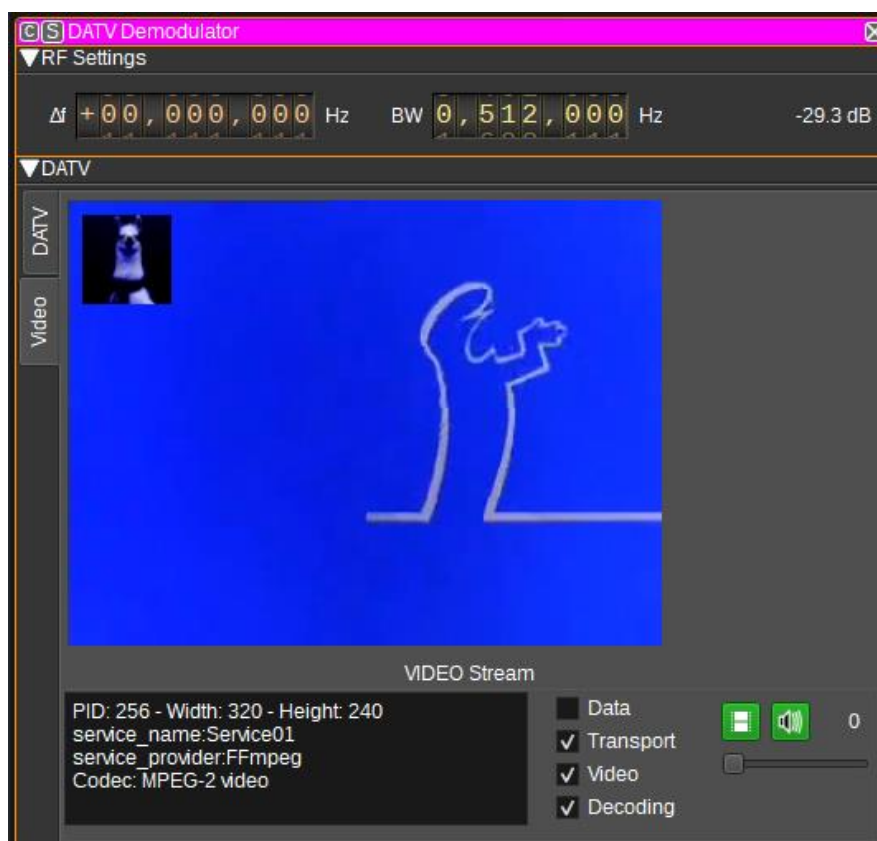


图 11-28 接收到的视频信号

11.5 总结

在本章实验当中，我们基于 GNU Radio 自带的例子，进行了简单的修改，使其能够使用 ANTSDR 来发送视频信号。并且在另外一台设备上，我们使用 SDRAngel 这个上位机软件成功地对信号进行了解调。

第12章 使用 GNU Radio 发送 WebCam 视频

在上一章实验当中，我们成功地使用 ANTSDR 发送了视频信号，只不过该视频信号是一个特定格式的文件。在本章当中，我们会在上一章实验的基础上，完成使用 ANTSDR 发送 webcamera 的图像。

12.1 获取 WebCam 数据

在本章当中，我们会使用 ANTSDR 来发送摄像头数据，那么我们就需要来获取 WebCam 的数据。经常使用的一种方法是通过 ffmpeg。

Ffmpeg 是一套可以用来记录、转换数字音频、视频，并能将其转化为流的开源计算机程序。采用 LGPL 或 GPL 许可证。它提供了录制、转换以及流化音视频的完整解决方案。它包含了非常先进的音频/视频编解码库 libavcodec，为了保证高可移植性和编解码质量，libavcodec 里很多 code 都是从头开发的。

本次实验的参考之一也是来源于一个 Pluto SDR 相关的开源项目，其 Github 的参考地址如下：<https://github.com/LamaBleu/Pluto-DATV-test>。该项目基于 pluto 是实现了视频的传输，提供了对应的 GRC 流图，以及获取 WebCam 数据的脚本还有一些用于测试的多媒体格式的视频，在上一章实验当中，我们使用的就是这个项目里面的视频源。本次实验我们会使用到其中的脚本。

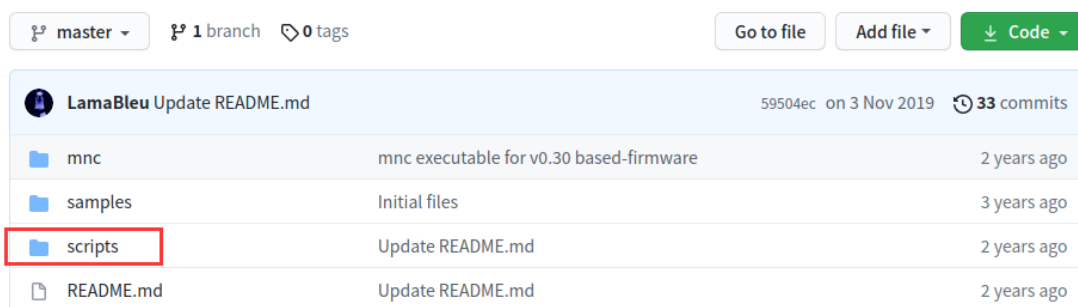


图 12-1 用于获取摄像头数据的脚本

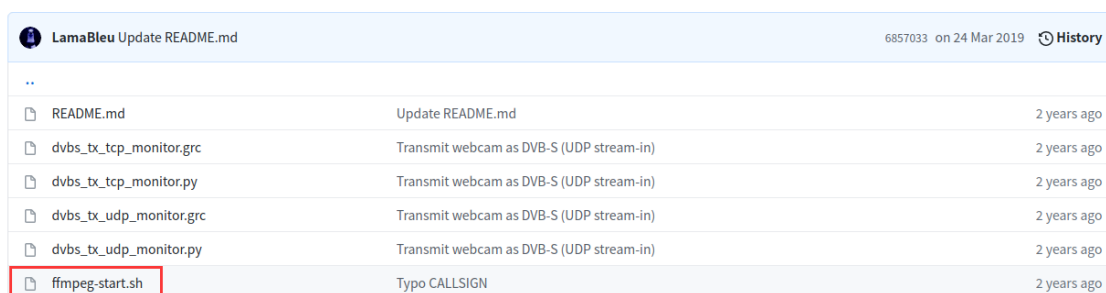


图 12-2

修改之后的脚本内容如下：

```
# Edit this file modify path to audio and TX (for OSD) file, change CALLSIGN

AUDIOFILE=/home/wcc/GNU_Radio/Lab/source_audio/music1_mono48kHz.wav
TEXTFILE=/home/wcc/GNU_Radio/Lab/source_audio/datv_text.txt
CALLSIGN=Microphase

ffmpeg -r 15 -i '/dev/video0' -i $AUDIOFILE -acodec mp2 -f mpegts -b:v 0.2M -s
320x240 \
-vf
"drawtext=textfile=$TEXTFILE:x=60:y=34:fontsize=40:shadowx=3:shadowy=3:fontc
olor=red:shadowcolor=white" \
-b:a 128k -ar 44100 -ac 2 -af asetpts=N/SR/TB \
-mpegts_service_id 1 -metadata service_provider=$CALLSIGN -metadata
service_name=$CALLSIGN \
-r 15 -ignore_unknown -pix_fmt yuv420p udp://127.0.0.1:58000
```

用户可以将其中的音频文件和传输的文本文件的路径替换为自己的路径即可。然后该脚本会使用 `ffmpeg` 来获取图像数据，并将前面的文本文件和音频文件组合，最终将视频文件转换成多媒体格式的文件，通过本地 `udp` 管道将数据送入本地回环的 5800 端口上，之后我们就可以在 GNU Radio 当中通过对应的端口来获取得到的图像数据了。

12.2 创建 GRC 流图

本次实验的 GRC 流图在上一章实验的基础上进行简单的修改即可。主要需要修改的地方就是将原始的图像数据源模块替换成 UDP 端口。

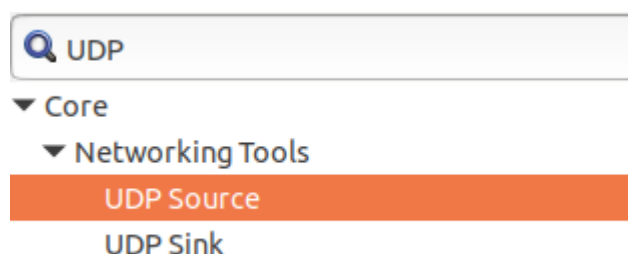


图 12-3 UDP 源模块

本次实验 UDP 源模块当中，需要指定对应的端口为 5800，也就是前面在脚本当中所设置的端口号，模块的设置如图 12-4 所示：

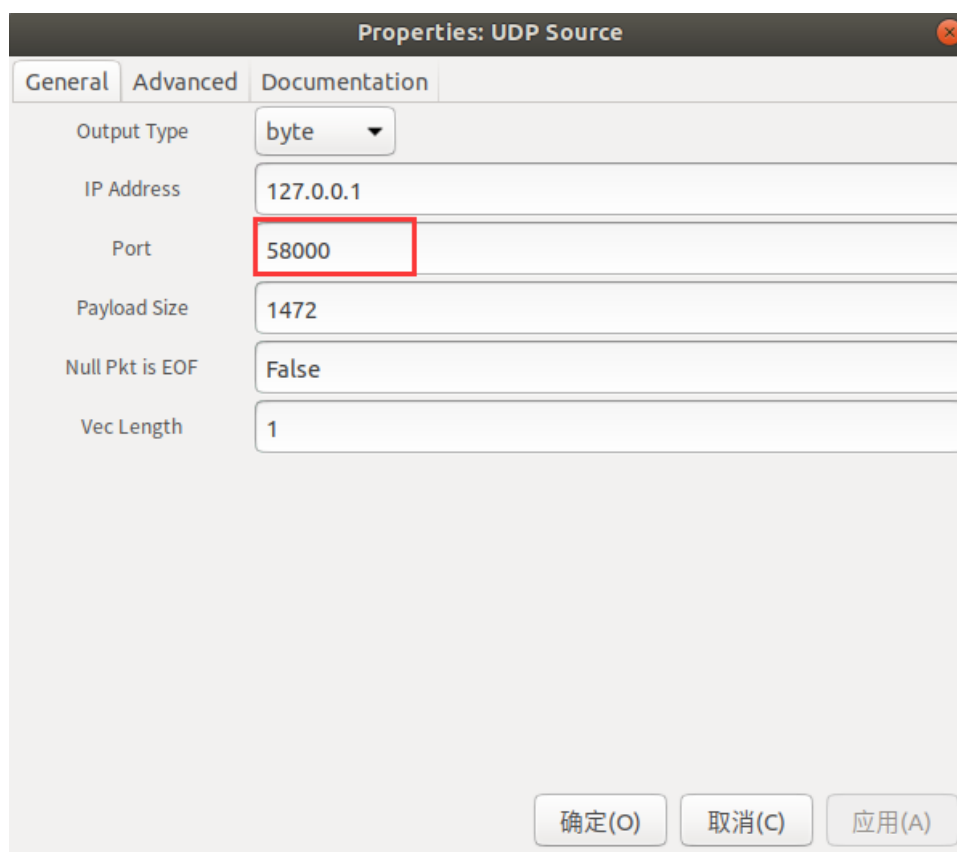


图 12-4

最终整个 GRC 流程图如图 12-5 所示，本次实验依然使用 DVB-S 进行数据的传输。

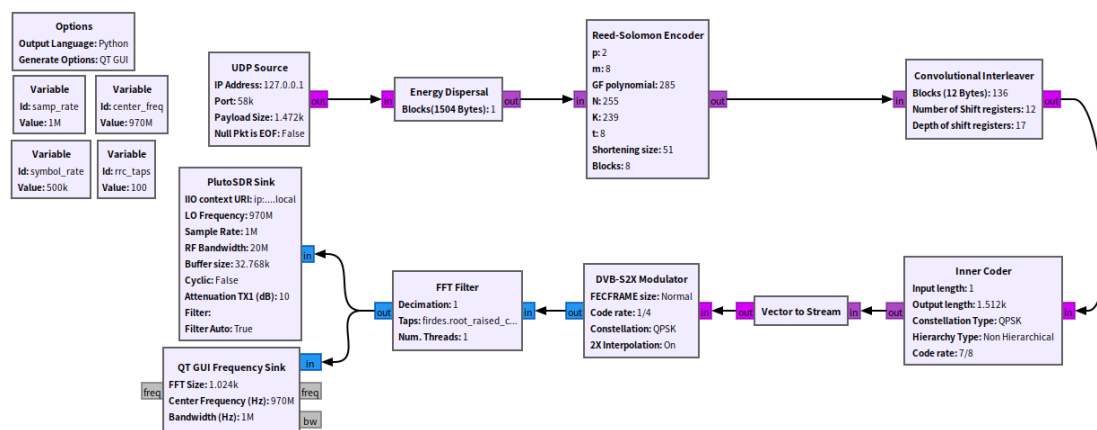


图 12-5 整个 GRC 流程图

接下来我们就可以开始尝试发送摄像头数据了，首先我们需要运行脚本，使用 `cd` 命令跳转到 GNU Radio 所在目录。

使用以下命令运行 shell 脚本：

```
source ffmpeg_start.sh
```

然后我们就可以看到当前已经开始产生摄像头数据了。

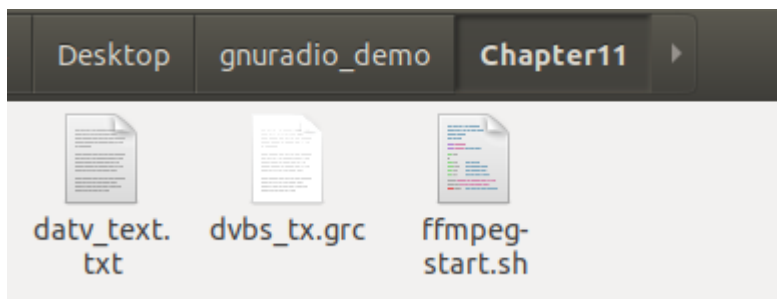


图 12-6 脚本所在路径

```
encoder : Lavc58.35.100 mp2
frame= 16 fps=0.0 q=2.4 size=      82kB time=00:00:00.87 bitrate= 768.1kbits/s
frame= 31 fps= 31 q=2.5 size=     151kB time=00:00:01.89 bitrate= 652.0kbits/s
frame= 46 fps= 30 q=2.5 size=     205kB time=00:00:02.91 bitrate= 576.4kbits/s
frame= 61 fps= 30 q=2.5 size=     263kB time=00:00:03.90 bitrate= 550.4kbits/s
frame= 77 fps= 31 q=2.5 size=     322kB time=00:00:04.93 bitrate= 535.0kbits/s
frame= 92 fps= 30 q=2.7 size=     378kB time=00:00:05.93 bitrate= 522.2kbits/s
frame= 107 fps= 30 q=2.5 size=     429kB time=00:00:06.93 bitrate= 506.6kbits/s
frame= 122 fps= 30 q=1.9 size=     496kB time=00:00:08.00 bitrate= 506.9kbits/s
frame= 137 fps= 30 q=2.7 size=     546kB time=00:00:08.97 bitrate= 498.0kbits/s
frame= 152 fps= 30 q=2.3 size=     600kB time=00:00:09.96 bitrate= 493.1kbits/s
frame= 168 fps= 30 q=2.4 size=     652kB time=00:00:11.00 bitrate= 485.5kbits/s
frame= 183 fps= 30 q=2.8 size=     712kB time=00:00:12.00 bitrate= 485.9kbits/s
frame= 198 fps= 30 q=2.6 size=     766kB time=00:00:13.07 bitrate= 479.6kbits/s
frame= 213 fps= 30 q=3.0 size=     817kB time=00:00:14.04 bitrate= 476.4kbits/s
frame= 228 fps= 30 q=2.8 size=     870kB time=00:00:15.03 bitrate= 474.0kbits/s
frame= 243 fps= 30 q=2.8 size=     928kB time=00:00:16.02 bitrate= 474.1kbits/s
```

图 12-7 产生图像数据

然后我们在 GNU Radio 当中运行之前的 GRC 流图，就能够将数据发送出去了。如图 12-8 所示，显示的是发送信号的频谱。

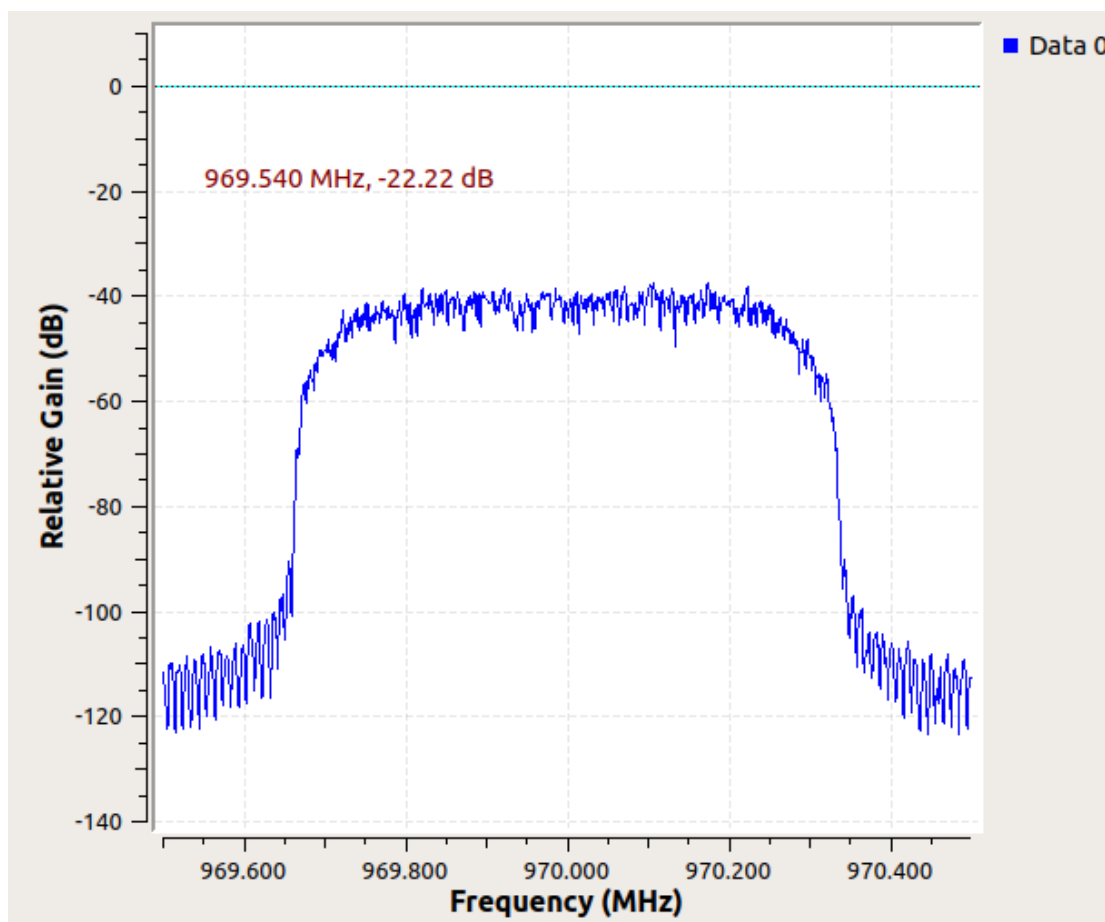


图 12-8 频域情况

12.3 DVB-S 数据解调

最后就可以使用 SDRAngel 进行 DVB-S 信号的接收和解调了。本次实验还是使用到两个 SDR 设备，一个用于在 Linux 实体机上使用 GNU Radio 进行信号的发送，一个用于在另外一台电脑上实现信号的接收。连接之后，将接收频率设置为 970MHz，然后就能够接收到解调出来的视频数据了。

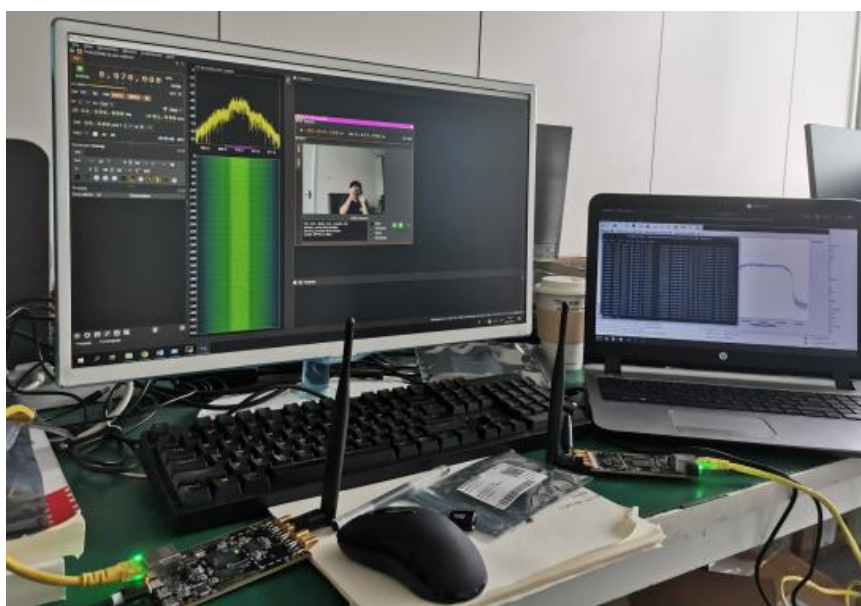


图 12-9

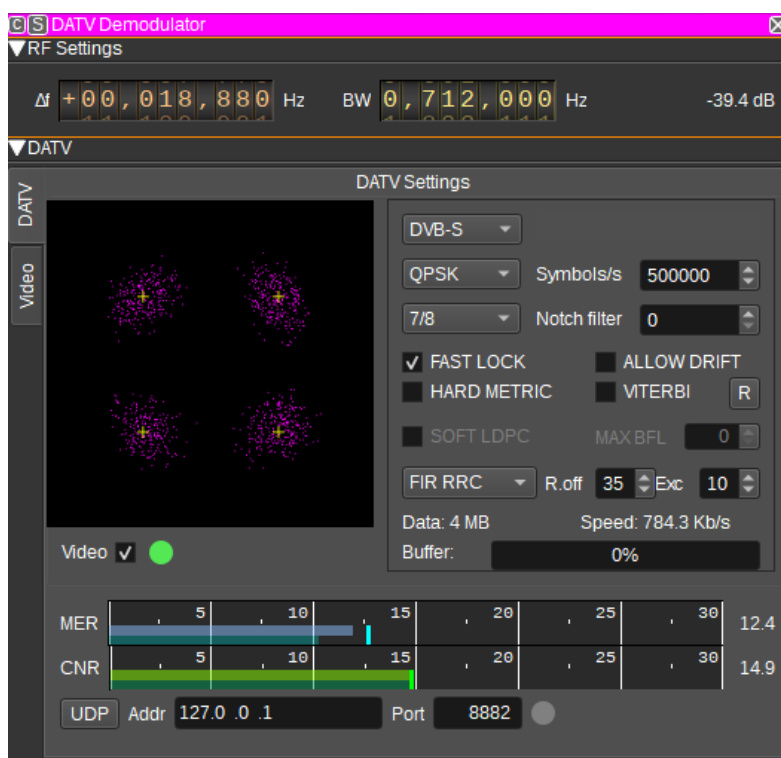


图 12-10 接收到的信号和星座图

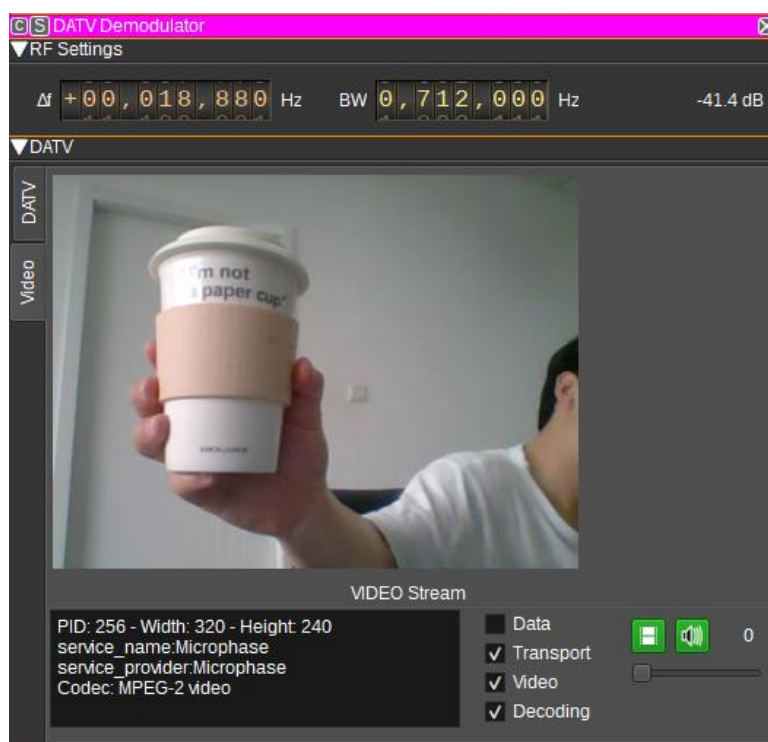


图 12-11 接收到的视频信号

12.4 总结

在本章的实验当中，我们使用了 `ffmpeg` 来获取 WebCam 的数据，并且基于上一章的 GRC 进行简单修改使其能够将摄像头数据通过 ANTSDR 进行发送，在接收端使用了 SDRAngel 和 ANTSDR 来进行了数据的接收和解调。

总 结

到这里基于 ANTSDR 的 GNU Radio 开发手册就完结了,最为初次编写的文档,由于本人水平有限,文档中难免会出现错误和疏忽,若您发现了文档当中出现的错误还请与我们联系,作为一款开源的 SDR 设备,您的反馈将会对我们提供莫大的帮助。

参 考

- Software Defined Radio using MATLAB & Simulink and the RTL-SDR
- Software-Defined Radio for Engineers
- 深入浅出通信原理
- 大话无线通信

附录

1. VirtualBox 安装

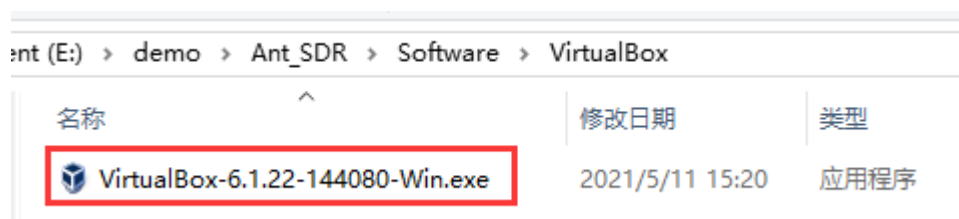
VirtualBox 是一款免费的虚拟机软件，在这里我们也以该软件为例来介绍 Ubuntu 虚拟机的安装。

VirtualBox 的下载地址如下：<https://www.virtualbox.org/wiki/Downloads>。在这里选择下载 Windows 版本的 VirtualBox。



附录-图 1

也可以使用资料当中的安装包。



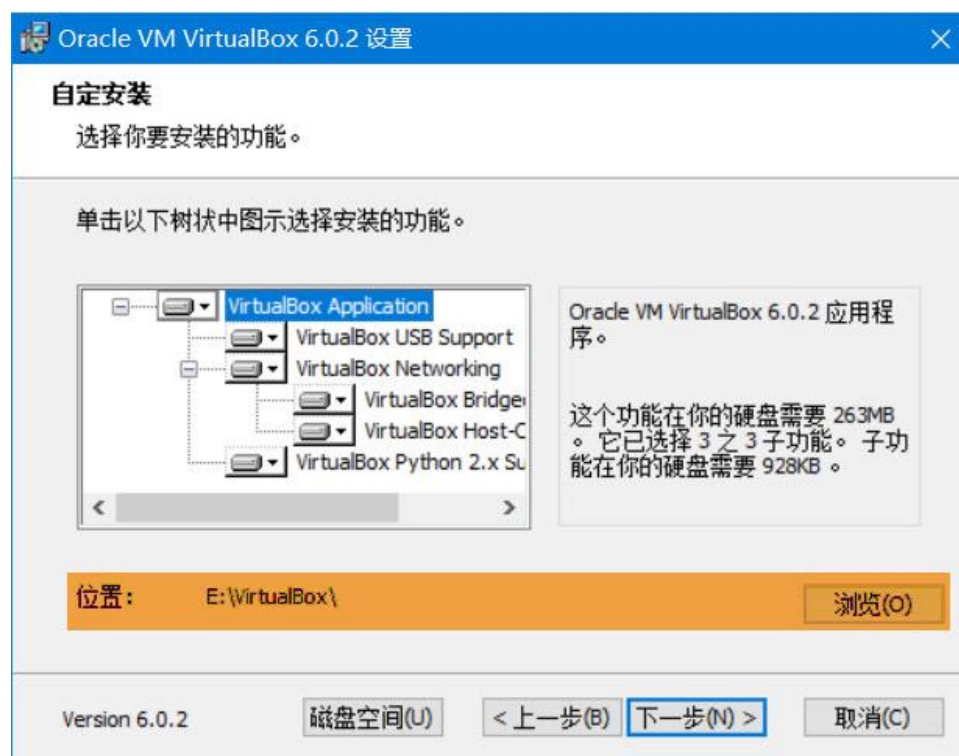
附录-图 2

然后双击这个软件进行安装：



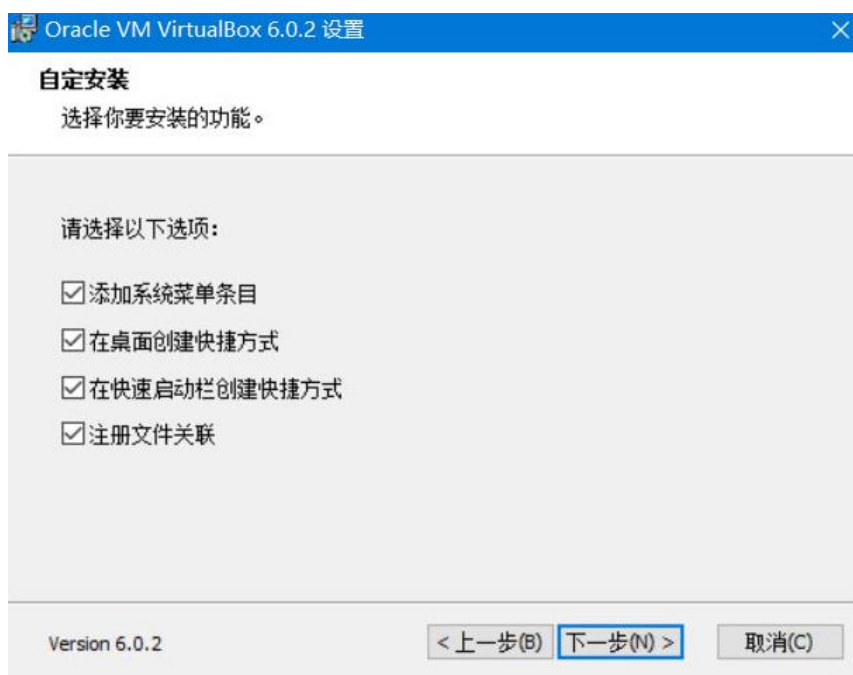
附录-图 3

需要安装的功能，选择默认即可。软件的安装路径，用户可以根据实际情况自由选择，本书选择安装在 E 盘，点击“下一步”，见下图。



附录-图 4

安装过程，选择软件自定义的功能，这里选择默认即可，点击“下一步”，见下图。



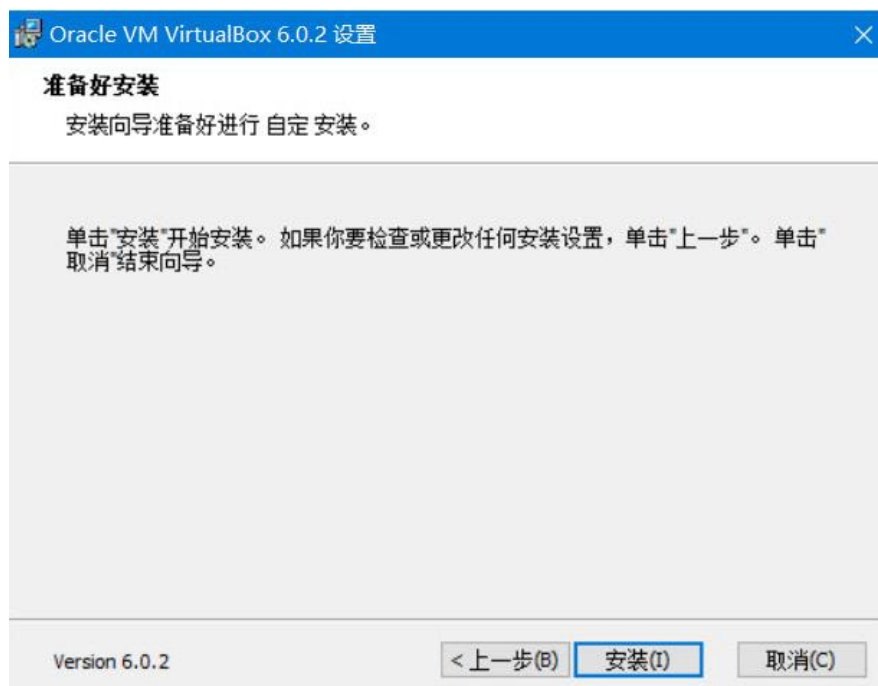
附录-图 5

在安装过程中，会提示一个警告，我们可以忽略该警告，点击“是”，继续安装，见下图。



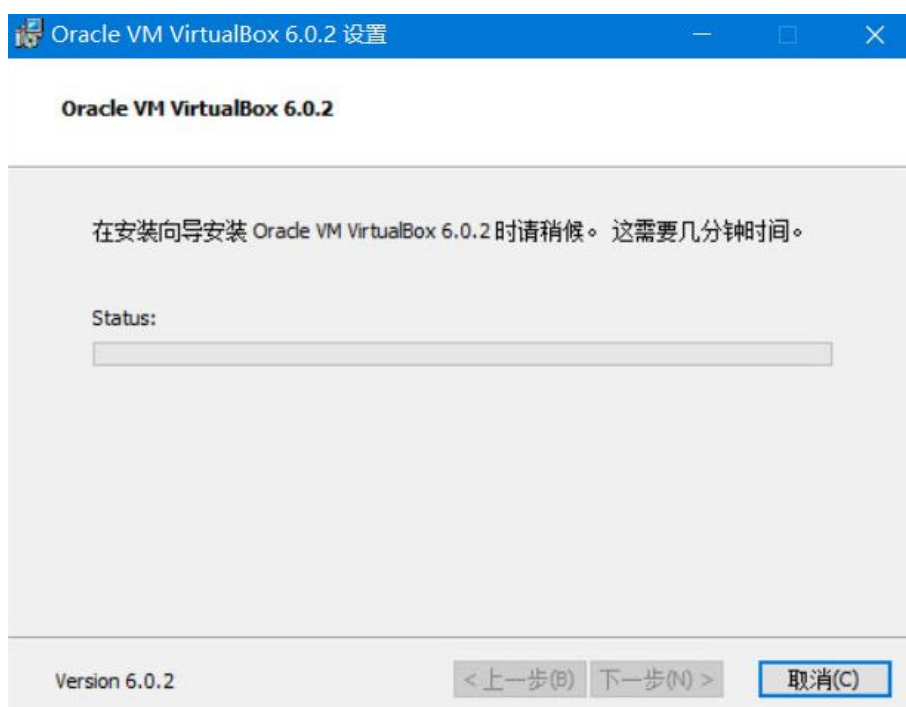
附录-图 6

下图中，点击“安装”按钮，开始安装软件。



附录-图 7

这样之后，便开始安装 VirtualBox 软件，静静地等待，软件安装完成。



附录-图 8

安装完成之后就可以运行软件了。



附录-图 9

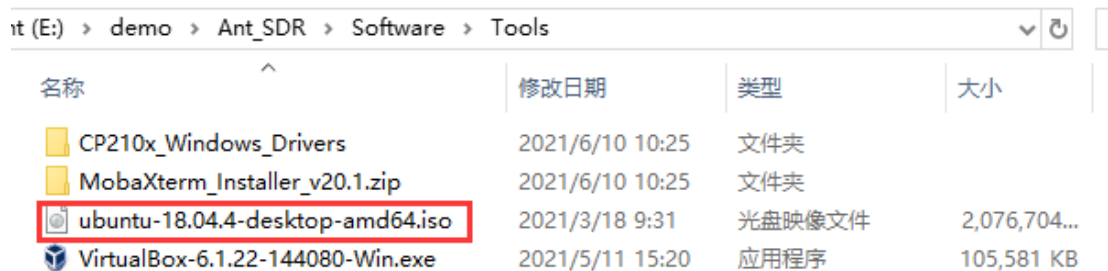
2. Ubuntu 虚拟机安装

在安装前需要先到 Ubuntu 的官网下载桌面版的系统镜像，可在如下址下载得到：<https://releases.ubuntu.com/18.04.5>，具体见下图。

	Parent Directory	-		
	MD5SUMS-metalink	2020-02-12 13:42	296	
	MD5SUMS-metalink.gpg	2020-02-12 13:42	916	
	SHA256SUMS	2020-08-13 15:39	202	
	SHA256SUMS.gpg	2020-08-17 12:28	833	
	ubuntu-18.04.5-desktop-amd64.iso	2020-08-06 22:59	2.0G	Desktop image for 64-bit PC (AMD64) computers (standard download)
	ubuntu-18.04.5-desktop-amd64.iso.torrent	2020-08-13 15:02	164K	Desktop image for 64-bit PC (AMD64) computers (BitTorrent download)
	ubuntu-18.04.5-desktop-amd64.iso.zsync	2020-08-13 15:02	4.1M	Desktop image for 64-bit PC (AMD64) computers (zsync metafile)

附录-图 10

在提供的资料文件夹当中，我们也为用户提供有虚拟机的安装镜像。



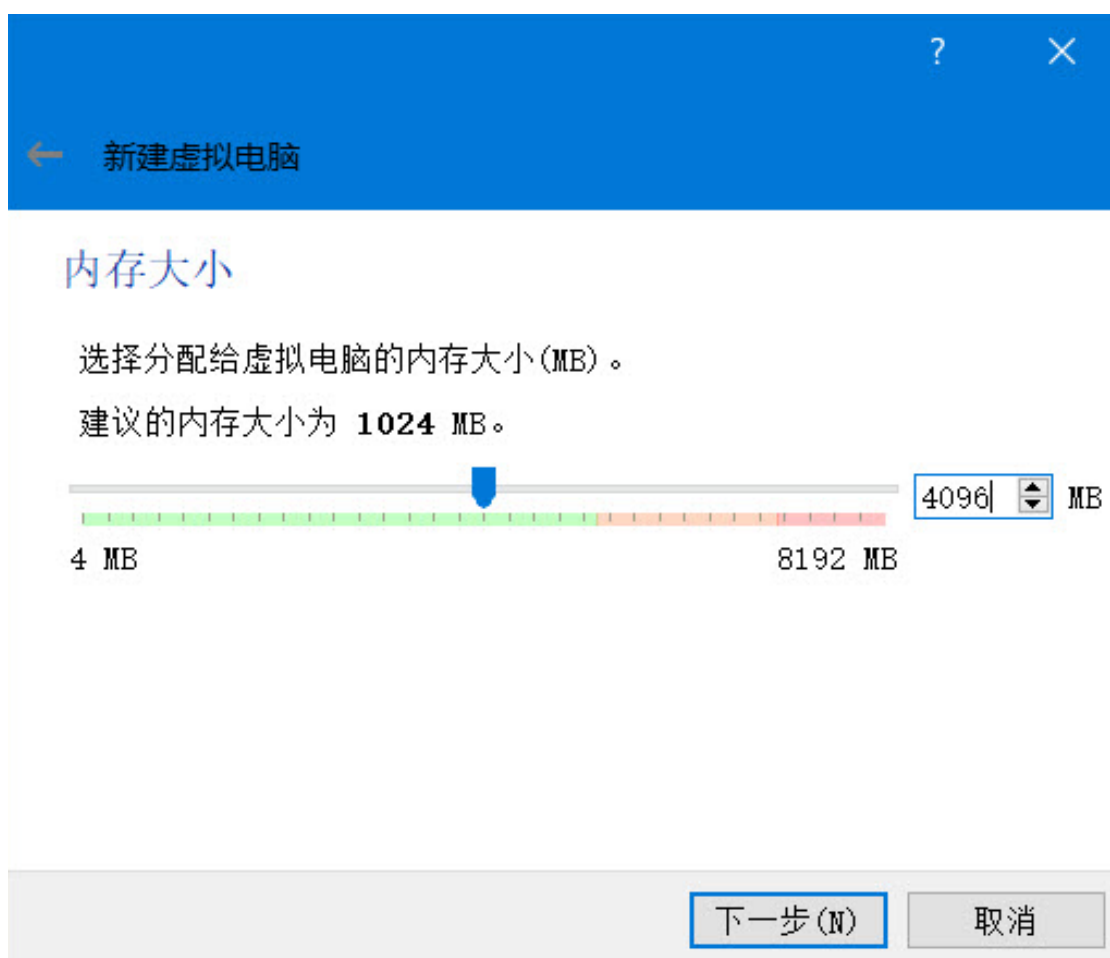
附录-图 11

镜像获取完毕之后，可以使用 VirtualBox 来安装虚拟机了。运行 VirtualBox 软件，点击“新建”按钮，来创建一个新的虚拟机，如下图所示。点击“下一步”。



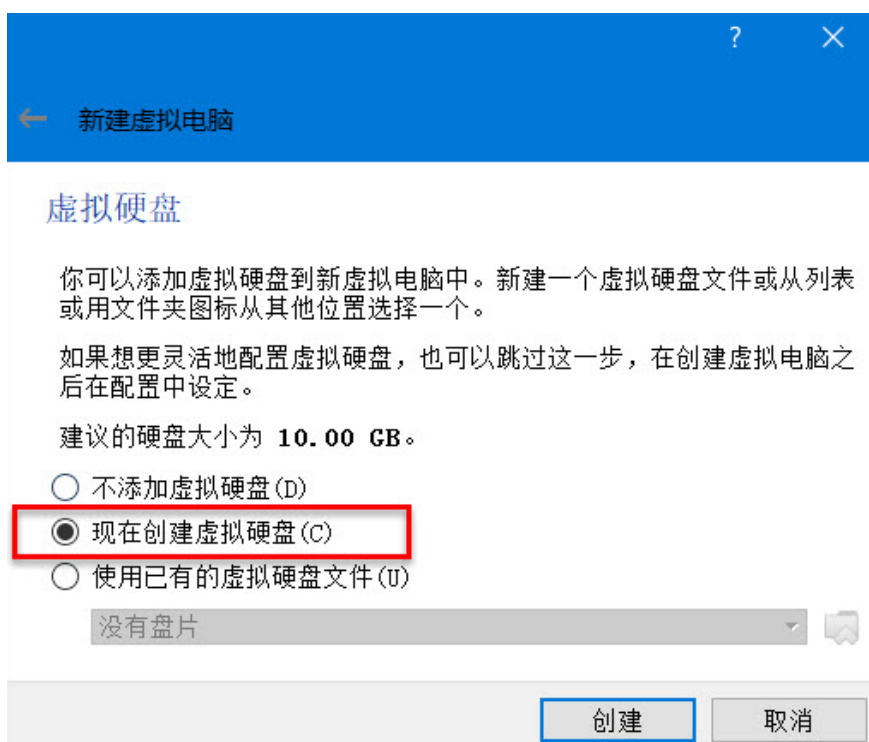
附录-图 12

选择内存大小，默认为 1024MB，可以根据自己电脑的配置选择，这里选择 4096MB，即 4GB 的内存大小，点击“下一步”按钮。

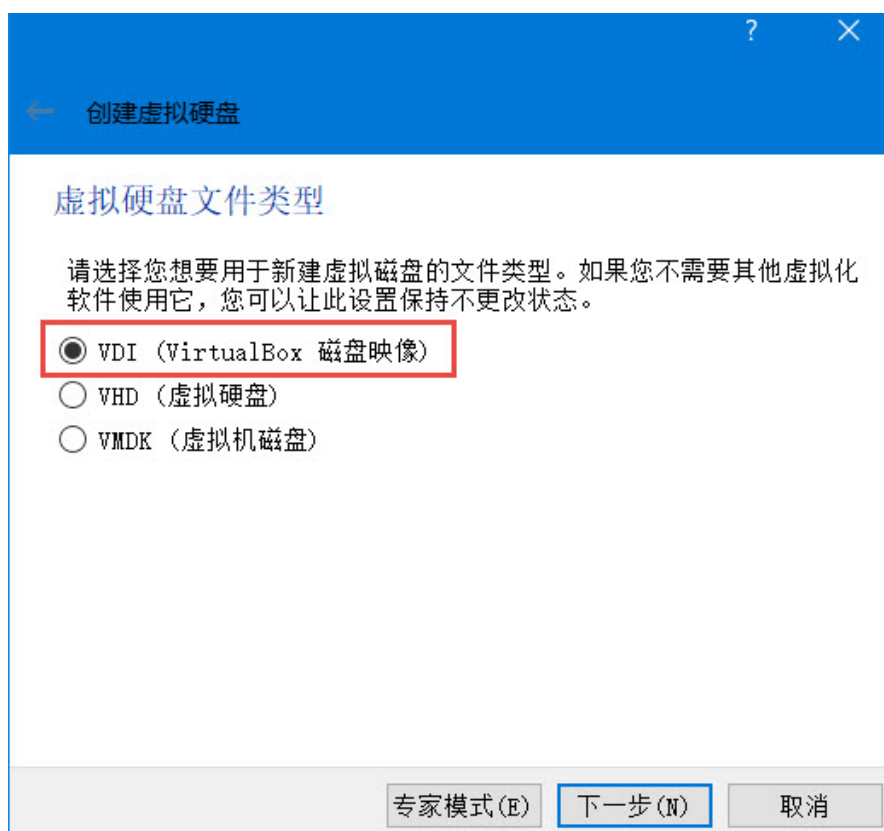


附录-图 13

选择虚拟硬盘，如附录-图 14 所示，选择“现在创建虚拟硬盘”，点击“创建”。出现如下所示的窗口，选择虚拟硬盘的类型，一般情况都选择 VDI，单击“下一步”。



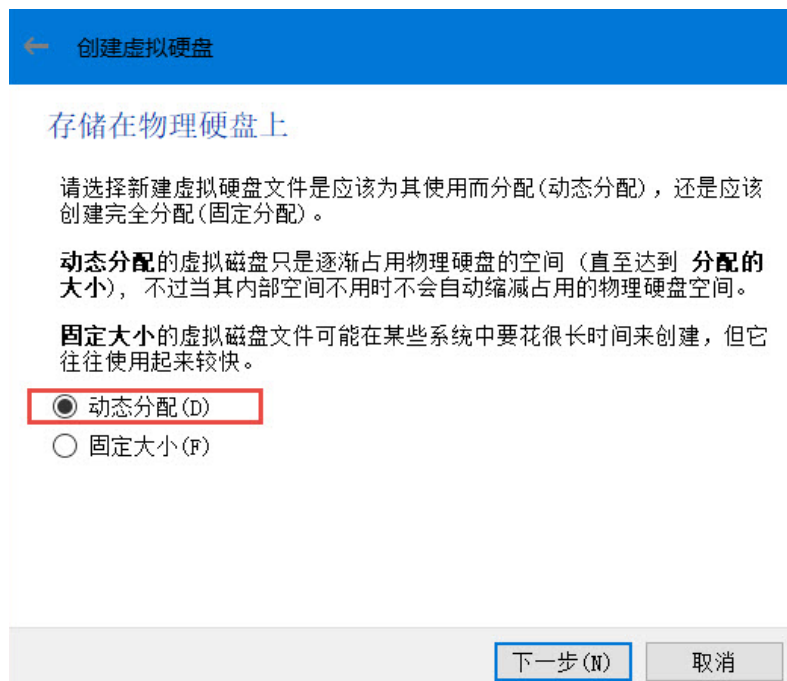
附录-图 14



附录-图 15

虚拟硬盘的分配方式有两种：一种是动态分配，需要多少用多少，一开始占

用小，随着空间需要而增长；另一种是固定大小，它会直接占用所分配的物理硬盘空间，但是访问速度快。这里我们选择“动态分配”，点击“下一步”之后，就需要我们选择虚拟硬盘大小，默认是 10GB，用户可以根据自己电脑配置和需要进行分配，这里选择 100GB（它不会立即占用这么多的空间，根据自己需求设置即可）。最后点击“创建”按钮，这样就创建好了。



附录-图 16

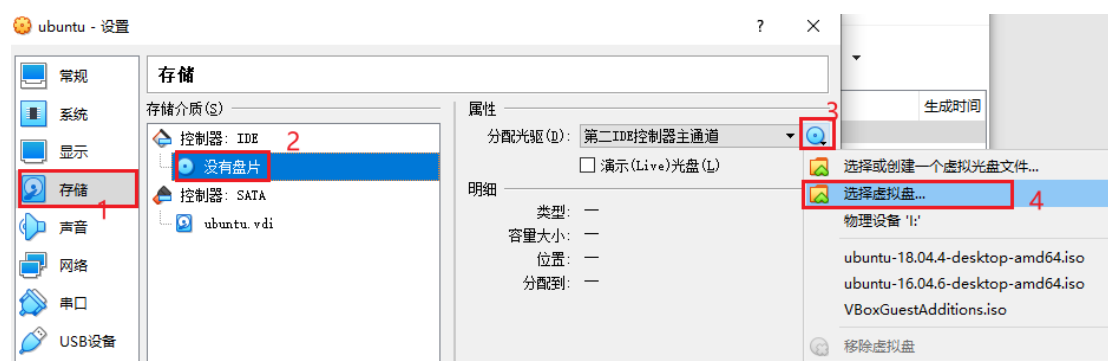


附录-图 17



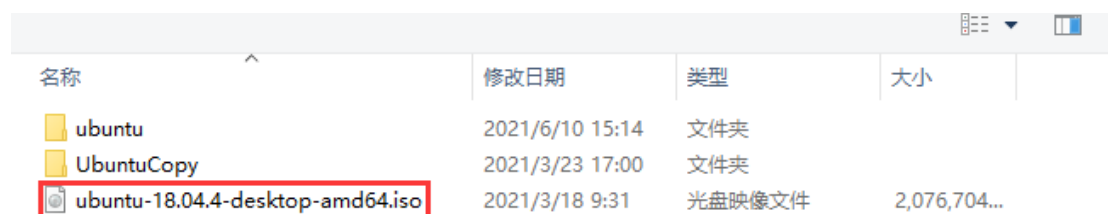
附录-图 18

点击右侧的“设置”按钮，如下图所示，选择“存储”栏目，然后选择处来设置系统镜像文件，通过点击处的按钮来选择我们 Ubuntu 镜像文件 (*.iso) 的路径，设置成功之后，如下图所示，点击“OK”，进入下一步。按照下图所示，依次点击进行选择 iso 进行。这里选择的 iso 镜像在前面所提供的 Ubuntu ISO 镜像。

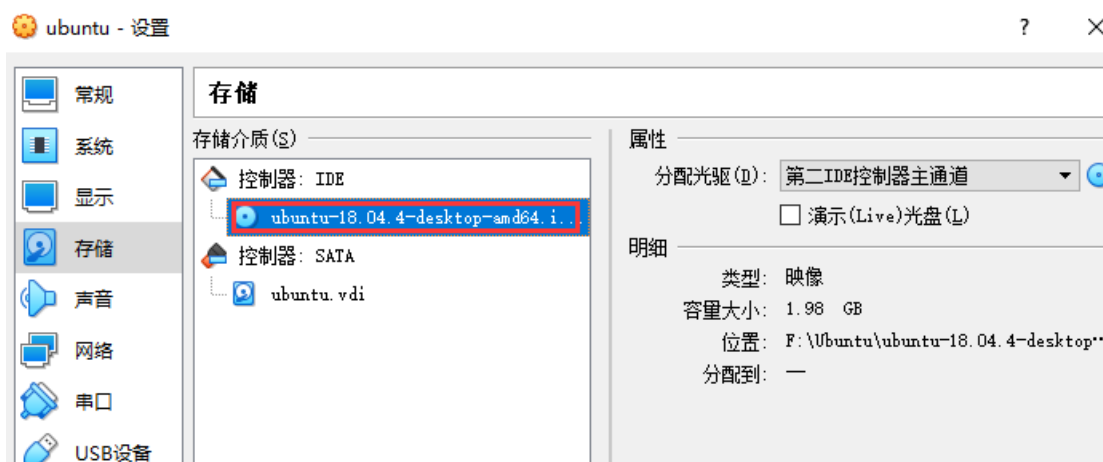


附录-图 19

这里需要定位到 iso 文件。

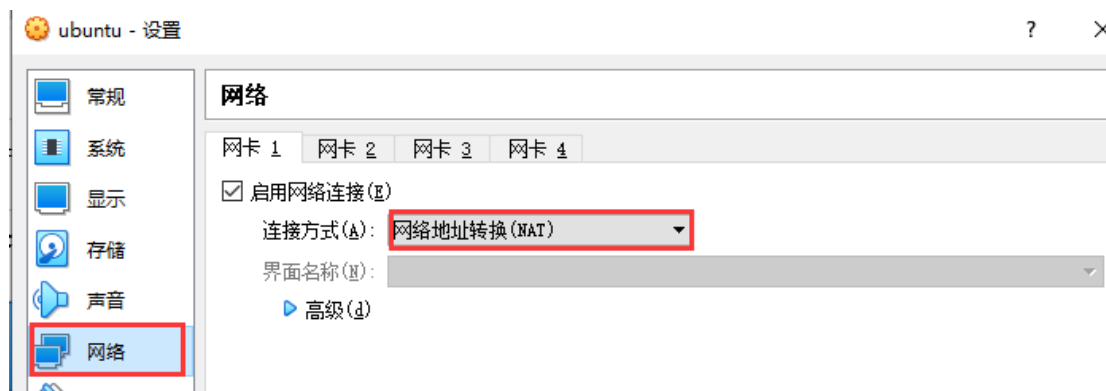


附录-图 20



附录-图 21

接下来选择“网络”栏目，将网卡连接方式配置为桥接网卡，如下图所示。



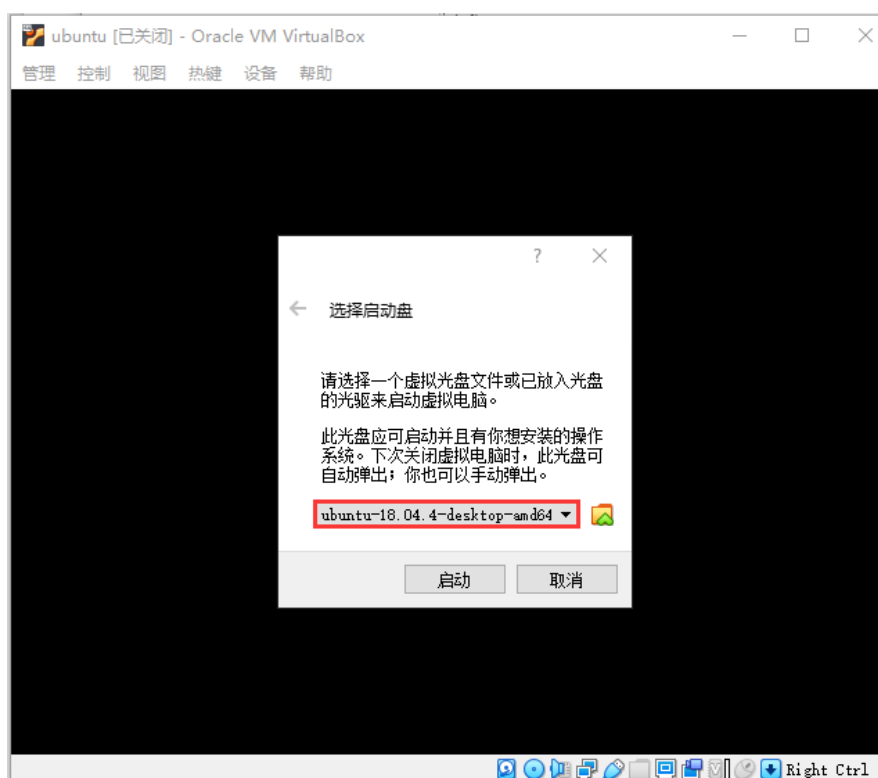
附录-图 22

紧接着就可以启动 Ubuntu 进行虚拟机的安装了。



附录-图 23 启动虚拟机

若在启动的时候，提示需要选择启动盘，选择我们之前为该虚拟机准备的 ISO 镜像即可，如下图所示。



附录-图 24

接下来选择安装系统的语言，可以选择英文或者中文，用户可以根据自己喜好安装，然后点击 install Ubuntu 进行系统的安装。



附录-图 25

接下来是键盘布局，保持默认选择继续。



附录-图 26

接下来保持默认，点击继续。



附录-图 27

继续保持默认，点击现在安装，进行安装系统。



附录-图 28

出现是否写入磁盘的时候，选择继续即可。



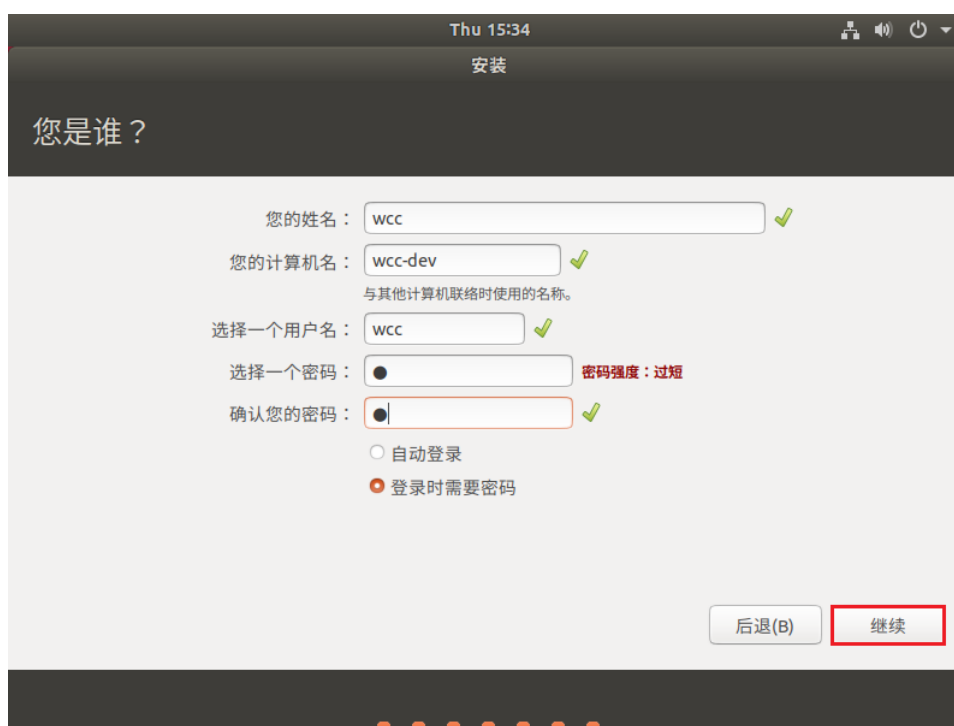
附录-图 29

接下来是时区的选择，可根据用户自己所在的位置进行选择。



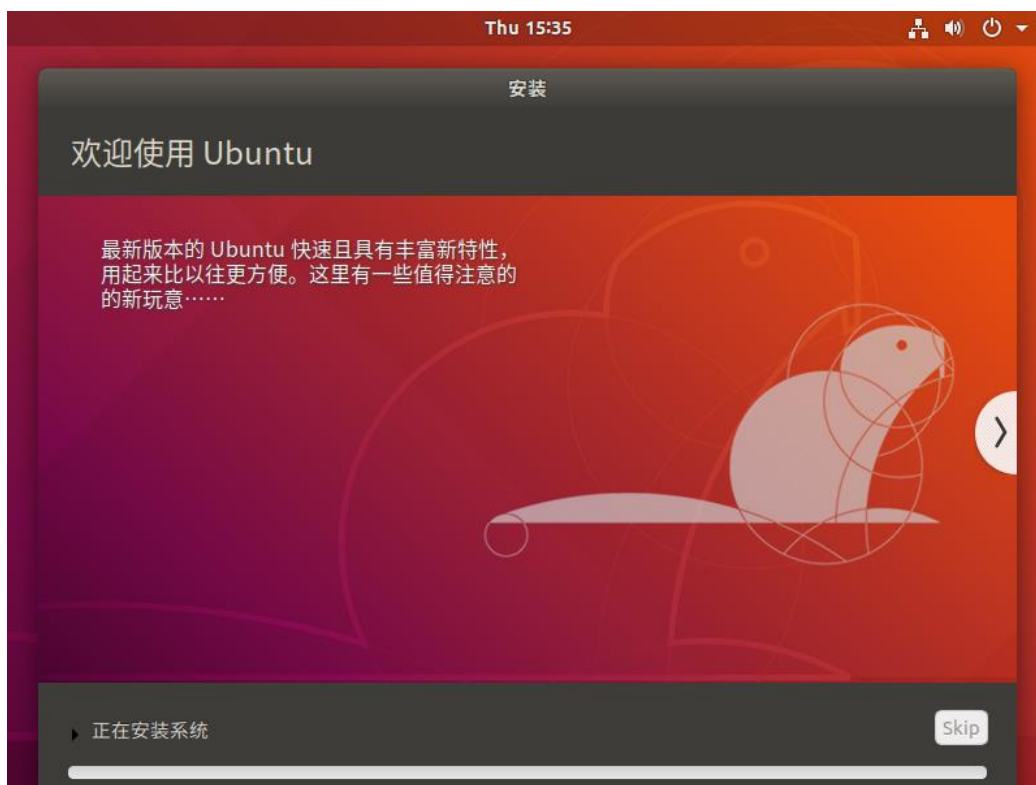
附录-图 30

然后需要设置用户名和密码，用户根据自己的情况进行设置即可。然后就可以进行安装系统了。



附录-图 31

根据主机的性能，安装虚拟机会持续一段时间，请耐心等待。



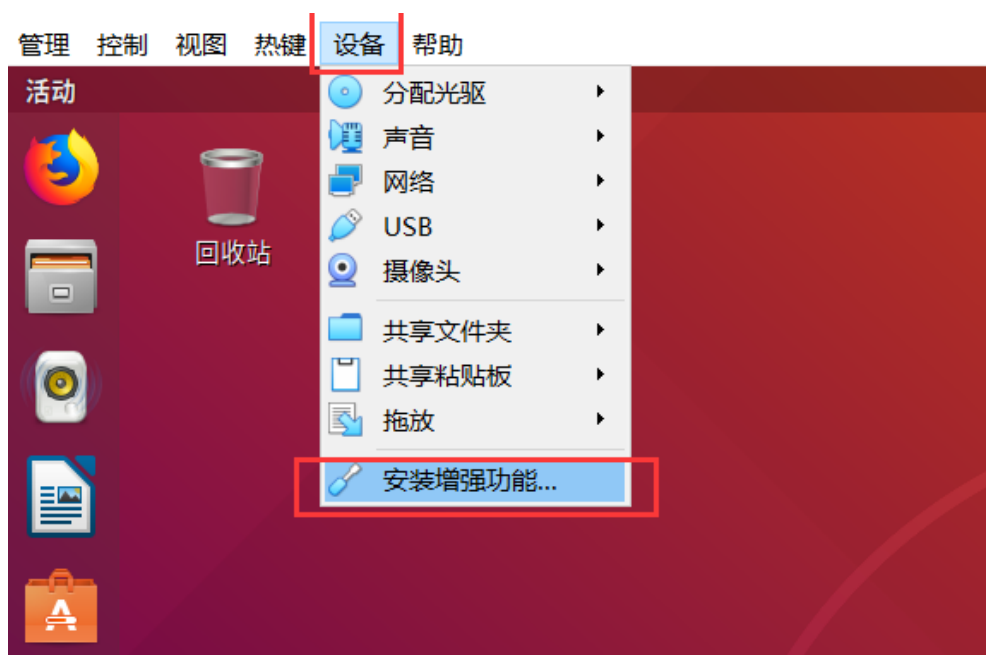
附录-图 32

虚拟机安装完成之后的界面如下图所示：



附录-图 33

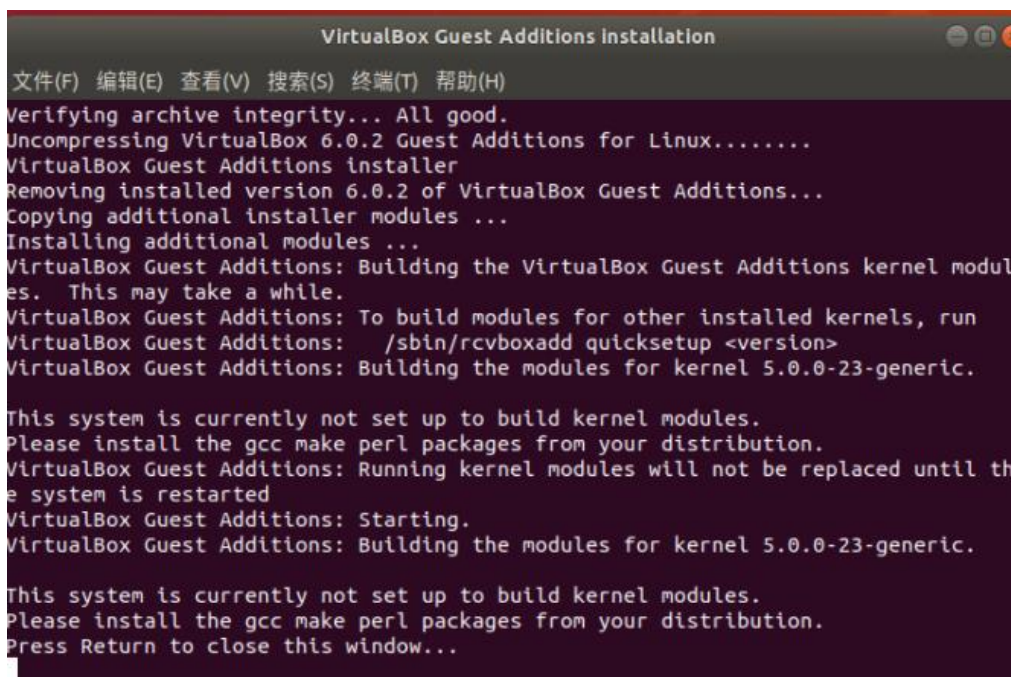
接下来我们可以给虚拟机安装一下增强功能，能够为用户的开发提供便利。点击 VirtualBox 左上角安装增强功能。



附录-图 34

点击安装增强功能后，虚拟机中会插入一个光盘，并会自动启动安装程序，见下图，在弹出的 界面点击运行。若没有自动启动，可手动双击该光盘进行安装。

点击运行后会出现终端界面输出运行提示，最后提示“press return to close this window”时，按回车键退出即可。



附录-图 35

接下来重启虚拟机，就能够完成整个增强功能的安装。重启虚拟机之后进入系统当中能够更改虚拟机显示大小，虚拟机的分辨率也会随之发生变化则说明增强功能安装成功。